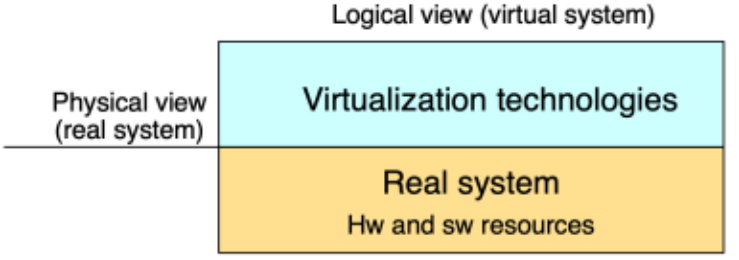
Virtualization

- Provides a high-level abstraction that hided the details of the underlying implementation
  - Hardware and software resources
- Offers a logical view of computing resources that differs from the physical ones



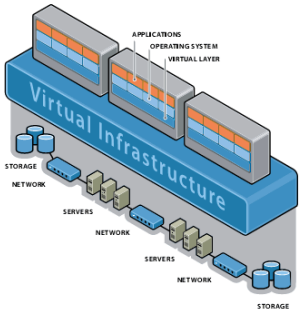
- How? Decouples user-perceived architecture and behavior of hw and sw resources from their physical implementation
- Goals:
  - Portability, efficiency, reliability, security, …

Virtualization of resources

- System virtualization
  - Virtualizes hw and sw resources
  - Virtual machines, containers, unikernels, …

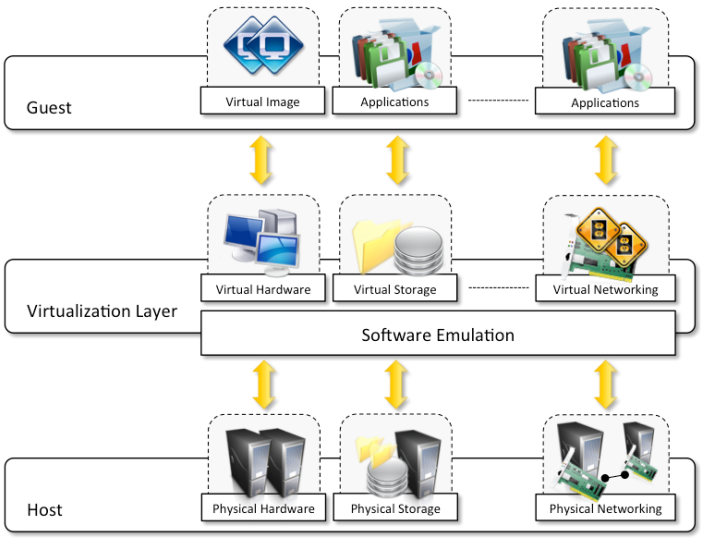
Our focus

- Storage virtualization
  - Abstracts storage resources
  - Storage Area Network (SAN), virtual disks, …
- Network virtualization
  - Virtual LAN (VLAN), Virtual Private Network (VPN), …
- Data center virtualization
  - Software-Defined Data Center (SDDC)



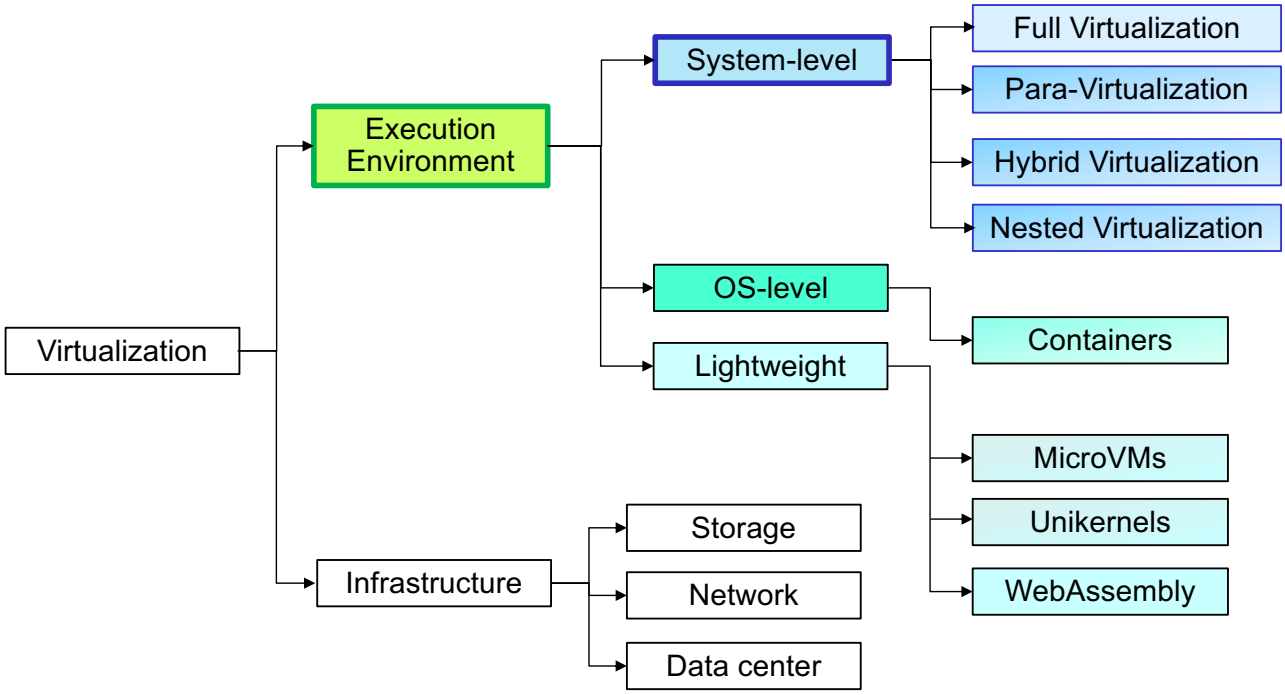
Major components of virtualization

- **Guest**:
  - The virtualized environment
  - Interacts with the virtualization layer, not directly with host
- **Host**:
  - The physical system
  - Provides the original hw and sw resources



- **Virtualization layer**
  - Responsible for creating the virtual environment
  - Manages resource allocation, isolation, and interaction between guest and host

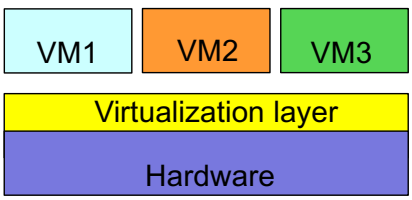
Taxonomy of virtualization techniques



- **Execution environment virtualization:** the oldest, most popular and most developed area ⇒ our focus

Virtual Machine

- **Virtual Machine** (**VM**): a complete compute environment with its own isolated processing capabilities, memory, and communication channels
- Represents hw/sw resources of a physical machine differently from their physical reality
  - E.g., VM hw resources (CPU, network card, ...) can differ from physical machine's hw resources
  - E.g., VM sw resources (OS, applications) can differ from physical machine's sw resources
- A single physical machine can host multiple independent VMs



Virtualization: a brief history

- An “old” idea in computer science
  - Originates in the 1960s in centralized computing
  - Goals: let legacy software run on expensive mainframes and transparently share scarce physical resources
  - Example: IBM System/360-67 mainframe

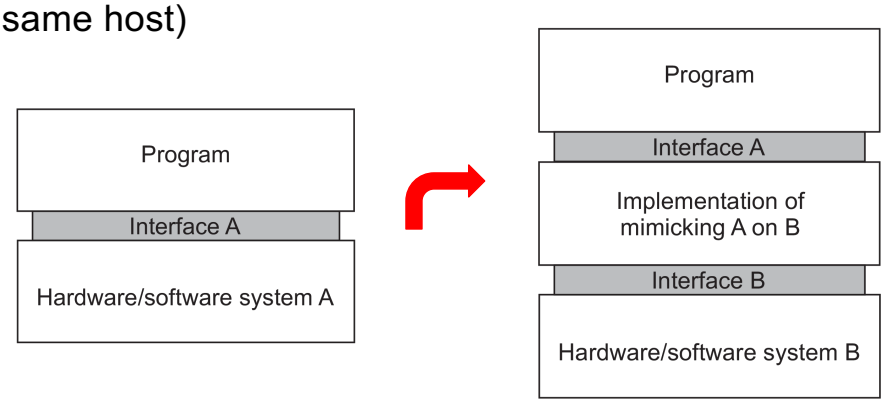
- Shift in the 1980s with the rise of PCs
  - OS multitasking to transparently share computing resources
  - Virtualization became less relevant for a while

Virtualization: a brief history

- Late 1990s: virtualization interest returns
  - Needed to simplify programming on special-purpose hardware
  - VMware founded in 1998
- Why the comeback?
  - Hardware evolves faster than software → higher management costs
  - Poor application portability
  - Significant under-utilization of hardware resources
  - Need to share resources efficiently and reduce infrastructure costs
- Today, virtualization is a core technology in cloud computing

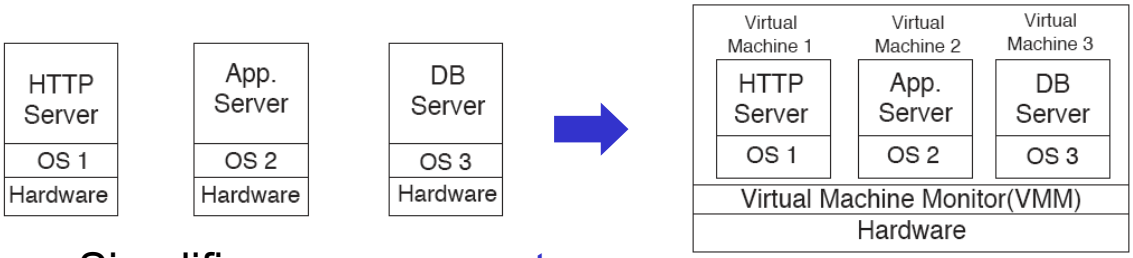
Virtualization: benefits

- Improves portability, compatibility, interoperability, and migration
  - Hardware independence: create once, run everywhere
  - Legacy support: run old OSes/apps on modern hardware
  - Support heterogeneous environments (different OSes on same host)



Virtualization: benefits

- Enables server consolidation and efficiency
  - VMs share one physical machine (multiplexing)
  - Reduces number of servers → lower cost, energy, and space usage
  - Better utilization of otherwise idle hardware



- Simplifies management
  - Easier maintenance and upgrades
  - Rapid provisioning: quickly create, clone, or destroy VMs
  - Snapshots and rollback simplify testing and recovery
  - Per-VM monitoring and resource accounting (important for cloud billing)

Virtualization: benefits

- Improves reliability and security
  - Strong isolation: faults or attacks in one VM do not affect others
  - Malware, bugs, and crashes remain contained within the VM
  - Micro-segmentation and secure virtual networks
  - Disaster recovery: VM images can be replicated and restored elsewhere
- Provides performance isolation
  - Controlled scheduling of shared resources ensures predictable performance
  - Reduces noisy-neighbor effects

Virtualization: benefits

- Improves load balancing and high availability
  - Improves load distribution across physical hosts
  - Live migration enables moving VMs seamlessly across physical hosts and reduces downtime
  - Enhances availability in cloud and data-center environments
- Better development and testing
  - Safe sandbox environments
  - Easy creation of testbeds and multi-VM setups
  - Useful for CI/CD pipelines and experimentation

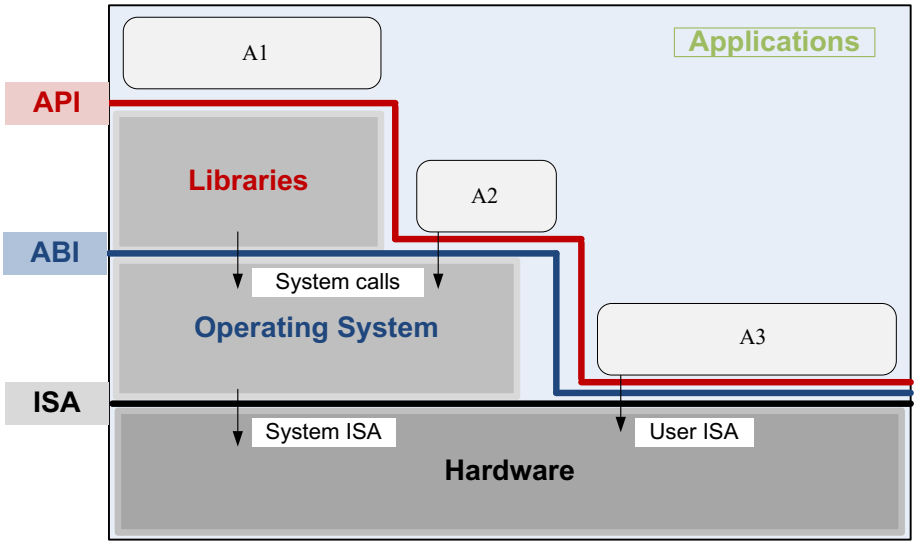
Reasons to use virtualization

- Personal and educational
  - Learn and experiment with multiple OSes
  - Develop, test, and debug applications in isolated environments
  - Simulate distributed systems on a single machine

- Enterprise and professional
  - Support DevOps workflows and continuous integration
  - Encapsulate entire systems in VM images for replication, migration, or redeployment
  - Ensure business continuity and disaster recovery readiness
  - Run legacy applications or OSes on modern platforms

Un computer offre generalmente 4 tipi diversi di interfacce su 3 livelli diversi: un interfaccia tra hardware e software ovvero l'instruction set architecture (ISA), che forma l'insieme delle istruzioni macchina che si suddivide in istruzioni privilegiate e istruzioni generiche. L'Application binary interface (ABI) che fornisce le system call come interfaccia con in sistema operativo. E un'interfaccia per le funzioni di libreria ovvero l'application programming interface (API)

Interfaces in computer system



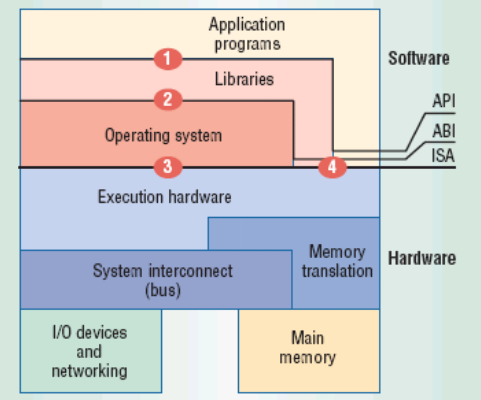
Applications:

- use library functions (A1)
- make system calls (A2)
- execute machine instructions (A3)

Interfaces in computer system and virtualization

Levels where virtualization can be realized

- Key idea: virtualization is strictly related to computer system interfaces → mimics their behavior
  - Hw/sw interface (**system ISA**): manages system resources, privileged instructions executed only by OS [interface 3]
  - Hw/sw interface (**user-level ISA**): focused on computation, *unprivileged* instructions executed by any program [interface 4]
  - System calls: controlled access to OS services from user applications [interface 2]
  - **ABI** (Application Binary Interface): ensures binary compatibility [interfaces 2 + 4]
  - Library calls (**API**): high-level interface for application software to use system services [interface 1]



Smith and Nair, The architecture of virtual machines, *IEEE Computers*, 2005

Implementation levels of virtualization

- Virtualization can be implemented at various operational levels
  - Primary involved interface is indicated within [ ]
- ISA level [interface 4]
- **Hardware level** (aka *system VMs*) [interface 3]
- **Operating system level** (aka *containers*) [interface 2] ← Our main focus
- Library level [interface 1]
- User application level (aka *process VMs*) [interfaces 4 + 1]

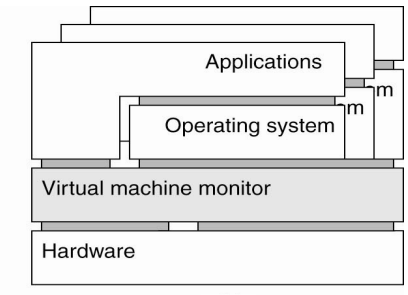
Implementation levels of virtualization

- ISA level
  - Goal: emulate a given ISA (unprivileged instructions) on the host machine for portability
  - Methods
    - Code interpretation: interpret each source instruction to host ISA instructions
    - Dynamic binary translation: convert code in blocks for faster execution
  - Example: run MIPS binary code on x86 host

Implementation levels of virtualization

- Hardware level (aka *system VMs*)
  - Goal: virtualize host resources (CPUs, memory, I/O) via **Virtual Machine Monitor** (**VMM**), aka *hypervisor*
    - VMM manages hw resources and shares them among multiple VMs
    - VMM intercepts privileged instructions and enforces isolation
  - Provides complete environment where multiple VMs coexist
  - Examples: VMware, KVM, Xen, Parallels, VirtualBox
  - Examples: microVMs
    - Lightweight VMs with minimal OS (e.g., Firecracker, Kata Containers)

Multiple instances of combinations <applications, OS>



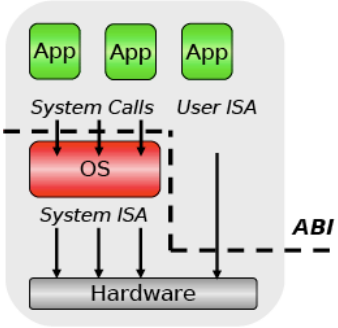
Implementation levels of virtualization

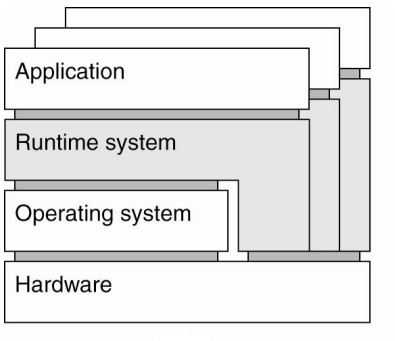
- Operating system level (aka *containers*)
  - Goal: create multiple isolated containers sharing the OS kernel
  - Containers rely on OS kernel to provide system calls
  - Examples: Docker, LXC, Podman
- Library level
  - Goal: run apps on host OS that does not natively support them
    - Translate high-level API calls from the application and translate them to host OS API rather than emulating full OS
  - Examples:
    - Wine: Windows apps on top of Linux https://www.winehq.org
    - Cygwin: "Get that Linux feeling – on Windows" https://cygwin.com

Implementation levels of virtualization

- User application level (aka *process VMs*)
  - Virtual platform for a single process
  - Provides virtual ABI/API to user app
  - Executes portable intermediate code (e.g., Java bytecode)
  - Examples: JVM, .NET CLR, WebAssembly runtimes

Multiple instances of combinations <application, runtime system>





System-level virtualization: terminology

- Let's focus on **system-level virtualization** via VMM/hypervisor

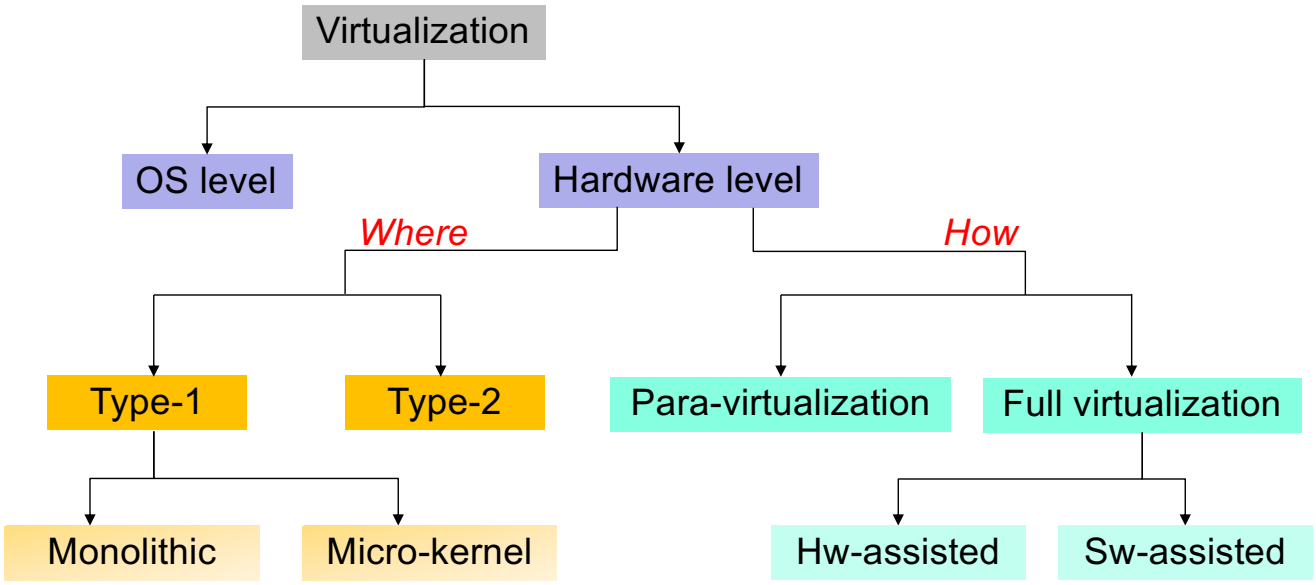
- **Host**: base platform on top of which VMs run; consisting of:
  - Physical machine
  - *Optional* host OS
  - VMM/hypervisor
- **Guest**: everything inside the VM
  - Guest OS
  - Applications executed inside the VM

System-level virtualization: taxonomy

- System-level virtualization classification
  1. Where VMM is deployed
     - **System VMM**
     - **Hosted VMM**
  2. How to virtualize instruction execution
     - **Full virtualization**
       - *Software-assisted*
       - *Hardware-assisted*
     - **Paravirtualization**

studiamo 3 soluzioni per virtualizzare l'esecuzione delle istruzioni

System-level virtualization: taxonomy

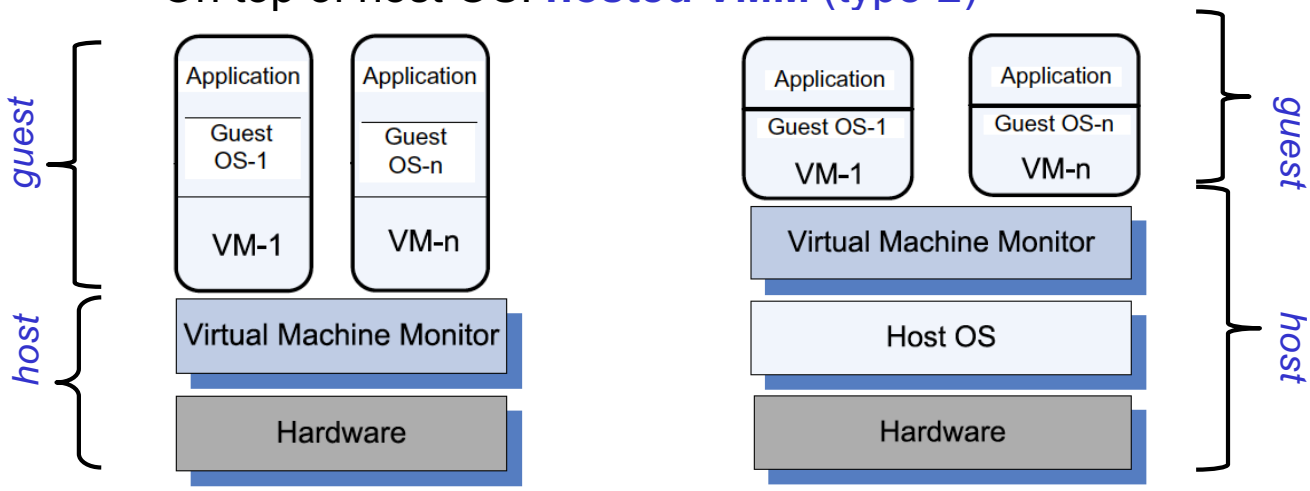


Comparison of platform virtualization software
https://en.wikipedia.org/wiki/Comparison_of_platform_virtualization_software

System vs hosted VMM

VMM deployment level in system architecture

- Directly on hardware: **system VMM** (type-1 / native / bare-metal)
- On top of host OS: **hosted VMM** (type-2)



System VMM Hosted VMM

System vs hosted VMM

- **System VMM** (*type-1*): runs directly on hw and provides virtualization features integrated into a minimal OS
  - Can use either *microkernel* (only basic functions, no device drivers) or *monolithic* (integrated drivers) architecture design
  - Examples: Xen, VMware ESXi, Xen, Microsoft Hyper-V, KVM, Nutanix AHV
- **Hosted VMM** (*type-2*): runs on top of host OS and uses host OS drivers and system calls
  - Interacts with the host OS via the ABI and presents virtualized hw (emulated or hardware-assisted) to the guest OSes
  - ✓ Leverages host OS for device management and low-level services (e.g., resource scheduling)
  - ✓ No guest OS modifications required
  - ✗ Lower performance with respect to system VMM due to extra sw layer
  - Examples: VirtualBox, Parallels Desktop

Privileged instructions

- Instructions that can only be executed by OS kernel (or hypervisor) nel nostro caso dal hypervisor
- They access or control critical system resources such as:
  - CPU modes (switching between user and kernel)
  - Memory management (page tables, MMU)
  - I/O devices (disk, network, peripherals)
  - Interrupt handling

sarà importante virtualizzare queste istruzioni. N.B. siamo nel caso in cui il VMM è sul bare-metal e non on top del host OS. in quel caso l'host OS si occupa delle istruzioni a livello di kernel

- Purpose: prevent user programs from directly manipulating hw or compromise system stability
- Examples:
  - Changing the CPU mode (user → kernel)
  - Modifying control registers (x86 `CR3` for page tables)
  - Direct access to I/O ports

Full virtualization vs paravirtualization

How to manage execution of privileged instructions in VMs

- **Full virtualization** da l'illusione di non avere alcun intermediario
- **Paravirtualization** il guest OS è opportunamente modificate per eseguire nel contesto di virtualizzazione

- Full virtualization
  - VMM exposes to each VM simulated hw interfaces *identical* to the underlying physical machine l'hypervisor deve possedere un meccanismo trasparente per eseguire istruzioni kernel level
  - VMM intercepts attempts to perform privileged instructions (e.g., I/O, TLB update) and emulates their behavior
  - Examples: KVM, VMware ESXi, Microsoft Hyper-V
- Paravirtualization
  - VMM exposes to each VM simulated hw interfaces that are *similar* but not identical to the underlying physical machine
  - Hardware is not fully emulated; a minimal software layer (Virtual Hardware API) manages VMs and ensures isolation
  - Examples: Xen, Oracle VM, PikeOS

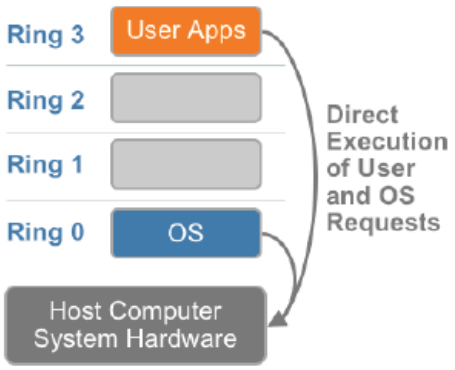
Full virtualization vs paravirtualization

- Full virtualization: pros
  - ✓ Runs unmodified guest OSs
  - ✓ Strong isolation between VMs: improves security and allows emulating different architectures independently
- Full virtualization: cons
  - ✗ VMM is more complex
    - It must intercept and emulates all privileged instructions
  - ✗ Requires processor support to make virtualization more efficient: why?

Issues to address for system-level virtualization

- Non-virtualized processor architectures use at least 2 protection rings: supervisor and user
  - Ring 0: most privileged (unrestricted access to system resources)
  - Ring 3: least privileged

x86 architecture w/o virtualization



vediamo le differenze tra un processore non virtualizzato ed uno virtualizzato

- With virtualization
  - VMM operates in supervisor mode (ring 0)
  - Guest OS and applications run in less privileged rings (guest OS in ring 1 or 3)
  - **Ring deprivileging** problem: guest OS executes in a ring that is not its own → cannot execute privileged instructions directly
  - **Ring compression** problem: since both guest OS and applications run at the same level, guest OS must enforce protection

più importante →

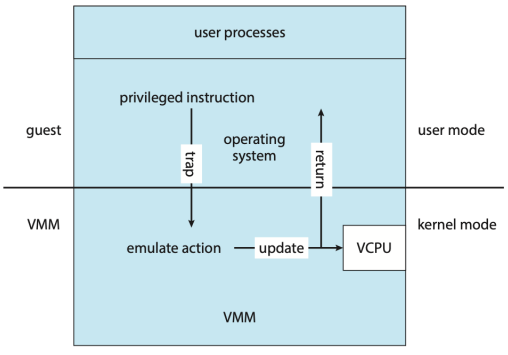
dovrebbe essere nel ring 0 ma non lo è

di fatti servono meccanismi aggiuntivi per il guest OS

Addressing ring deprivileging

- **Trap-and-emulate**
  - When the guest OS executes a privileged instruction (e.g., `lidt`), an exception (trap) is raised and control is transferred to the VMM; the VMM performs a safety check on the requested operation, executes ("emulates") its behavior, and returns the result to guest OS
  - Non-privileged instructions, instead, run by guest OS do not trap and are executed directly

se ad esempio il guest OS vuole aggiornare la tabella degli interrupt, la system call avvisa l'hypervisor che la intrappola e si fa carico di eseguire l'istruzione virtualizzata e di ritornare il risultato al guest OS



- Issue: on some architectures, certain instructions do not trap, requiring more complex solutions

Popek and Goldberg virtualization requirements

- Popek and Goldberg requirements (1974) define when an architecture can support *efficient* virtualization
- Conditions:

Equivalence
producono istante per istante lo stesso risultato

1. A program running under the hypervisor should exhibit a behavior essentially identical to that demonstrated when running on an equivalent machine directly.

Resource control

2. The hypervisor should be in complete control of the virtualized resources.

Efficiency

3. A statistically significant fraction of machine instructions must be executed without the intervention of the hypervisor.

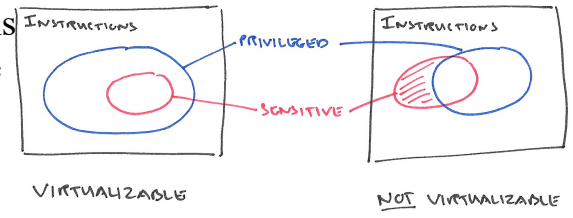
Formal requirements for virtualizable third generation architectures, 1974
https://dl.acm.org/doi/pdf/10.1145/361011.361073

sulla base di questi requisiti possiamo classificare le istruzioni ISA

Instruction types and virtualization theorem

- ISA instruction classification
  - Privileged instructions: execute only in supervisor mode; trap in user mode
  - Sensitive instructions: affect or reveal CPU/system state classe importante
    - Control-sensitive: modify CPU configuration/state (e.g., interrupt/paging tables)
    - Behavior-sensitive: reveal CPU state
  - Innocuous instructions: neither sensitive nor privileged
- **Popek and Goldberg theorem**: *a VMM can be constructed if all sensitive instructions are privileged*

dunque le sensitive instructions devono essere un sottoinsieme delle istruzioni privilegiate



c'è solo un intersezione quindi non è virtualizzabile

https://blog.acolyer.org/2016/02/19/formal-requirements-for-virtualizable-third-generation-architectures

Problem and implications

- Problem: some architectures have sensitive non-privileged instructions, e.g.:
  - x86: many sensitive non-privileged, such as:
    - `pushf`, which pushes the flags register on the stack
  - MIPS: mostly virtualizable, but `$k0`, `$k1` registers are accessible in user mode
  - ARM: mostly virtualizable, but some instructions undefined in user mode
- Must virtualize:
  - Privileged instructions → trap to VMM and emulate
  - Sensitive non-privileged instructions → must also be virtualized

pushf è push del registro dei flag sullo stack

ma sono i registri utilizzati per salvare e caricare lo stato

Solutions for virtualizing sensitive instructions

- Trap-and-emulate
  - Privileged instructions trap → VMM emulates instruction behavior
  - Non-privileged instructions run directly on CPU
  - Limitation: some sensitive non-privileged instructions do not trap on certain CPUs → which solutions?

1. **Hardware-assisted virtualization** (hw level)
   - VMM relies on hardware to intercept privileged and sensitive instructions automatically
   - All sensitive instructions trap automatically to VMM
2. **Fast binary translation** (sw level)
   - VMM rewrites sensitive instructions on the fly
3. **Paravirtualization**
   - Modify guest OS to avoid or neutralize sensitive non-privileged instructions

abbiamo il supporto del hw per la virtualizzazione

soluzione disponibile dal 2005 e oggigiorno la più usata

prima soluzione ideata a fine anni 90

l'overhead era però elevato

soluzione ideata nei primi anni 2000

realizzata per la prima volta a Cambridge per l'hypervisor Xen, per ridurre l'overhead della fast binary translation

Hardware-assisted CPU virtualization

- Idea: introduce a new privilege level for the hypervisor
  - Hypervisor privilege > OS/kernel privilege
  - All sensitive instruction trap to hypervisor level
    avviene quindi una trap per le sensitive instruction
- Provides two new CPU operating modes
  - **Root mode** for the VMM
  - **Non-root mode** for guest OSs
- Each mode supports all 4 x86 protection rings

grazie a queste modalità si possono utilizzare appunto tutti e 4 i ring, risolvendo il problema del ring deprivileging e del ring compression

Hardware-assisted CPU virtualization

- VMM runs in Root-Ring 0, while guest OSs run in guest mode in Non-Root Ring 0 at their original privilege levels
  - Eliminates ring deprivileging and ring compression issues

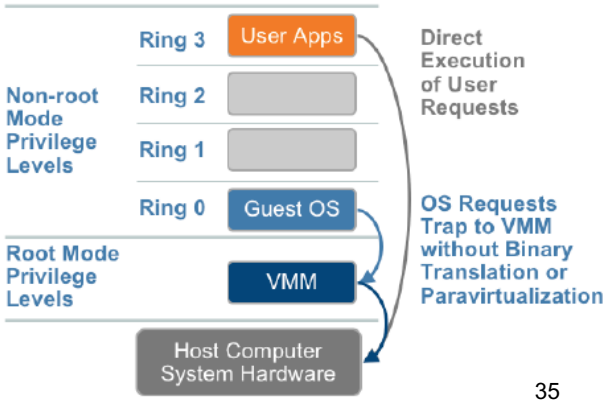
la trap è implementata a livello hardware

- When a guest OS executes a privileged or sensitive instruction in Non-Root mode, the CPU automatically stops guest execution (VM exit) and switches control to the VMM, which runs in Root mode
  - VMM can control guest execution through VM control data structures in memory

VM-exit è simile ad un context switch. il contesto viene salvato prima di cambiare mode

- Implemented in modern CPUs (e.g., Intel VT-x and AMD-V, since 2005)

x86 architecture with with full virtualization and *hardware-assisted CPU virtualization*



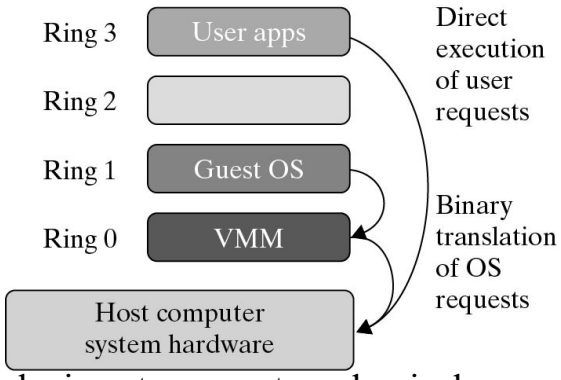
Fast binary translation

- Software-based and the oldest solution
- VMM scans code before execution
- Replaces blocks containing privileged instructions with functionally equivalent blocks that notify the VMM when executed
  - Translated blocks run directly on hw and are cached for future reuse

la trap in questo caso è implementata a livello software

✗ Higher complexity and lower performance compared to hw-assisted virtualization

x86 architecture with full virtualization and fast binary translation



soluzione trasparente e che risolve il ring decompression

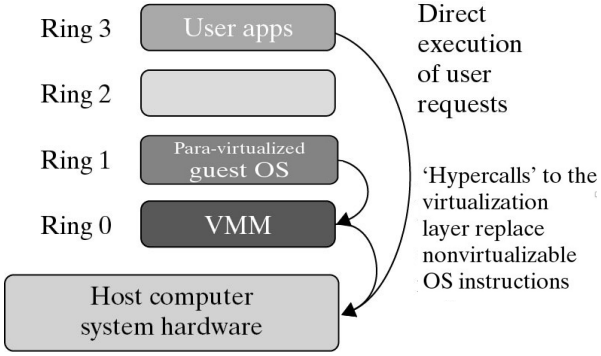
Paravirtualization

- Non-transparent virtualization
  - Guest OS kernel must be modified to use the virtual API exposed by the VMM
- Non-virtualizable instructions are replaced with *hypercalls* that communicate directly with the hypervisor
- Hypercall: *software trap* from the guest OS to the hypervisor

ecco la modifica del guest OS

hypercall : hypervisor = syscall : kernel

x86 architecture with *paravirtualization*



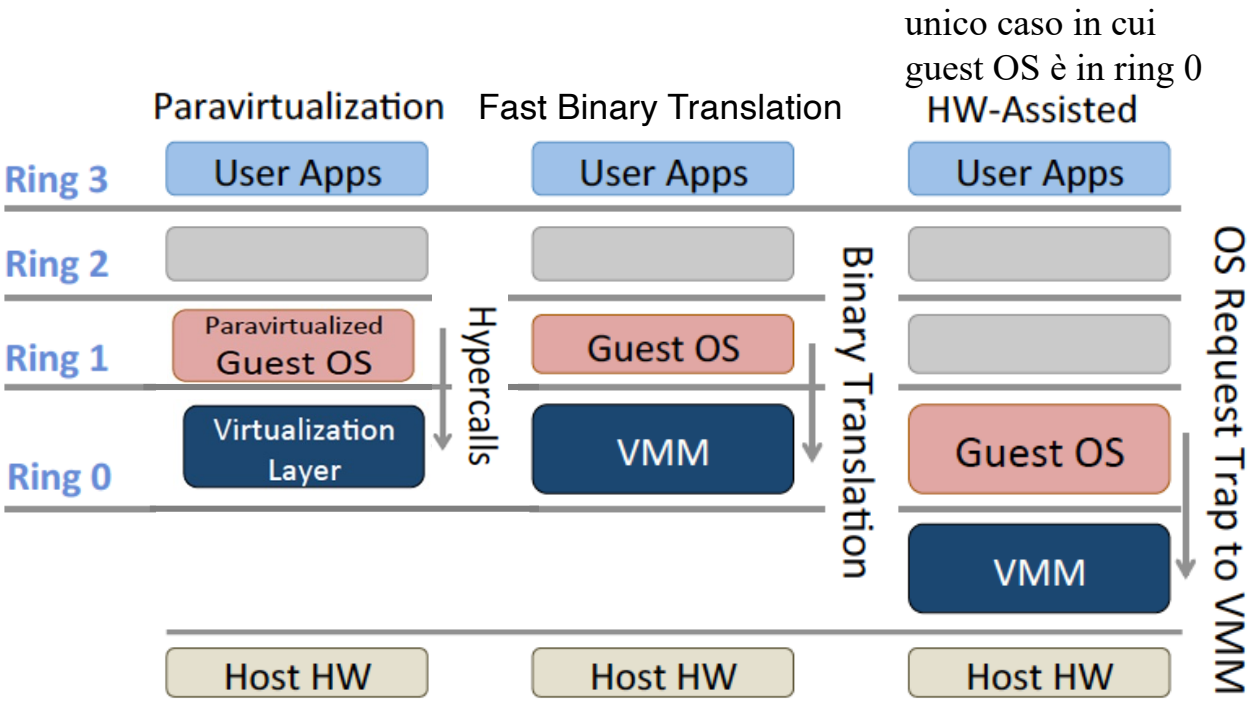
- Hypercall execution
  - Control jumps from guest OS to hypervisor
  - Hypervisor performs the requested operation
  - After handling, control returns to guest OS kernel, which then resumes the application

Paravirtualization

- Pros:
  - ✓ Easier and more practical to implement than full virtualization on non-virtualizable hardware
  - ✓ Lower runtime overhead compared to fast binary translation
  - ✓ Does not require virtualization extensions (unlike hw-assisted virtualization)

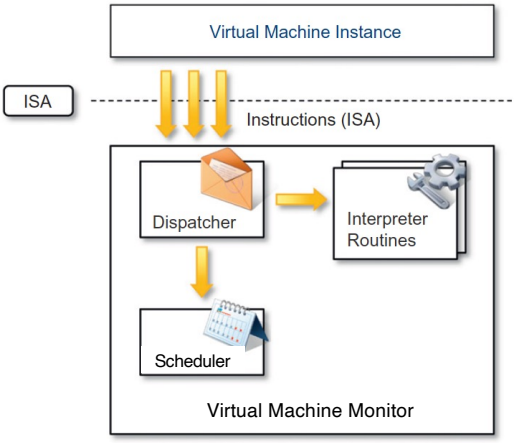
- Cons:
  - ✗ Requires source code access to modify the guest OS and make it paravirtualized windows non approva
  - ✗ Maintenance overhead: each paravirtualized OS version must be maintained separately
    - Paravirtualized OS cannot run directly on hardware

Summing up different approaches



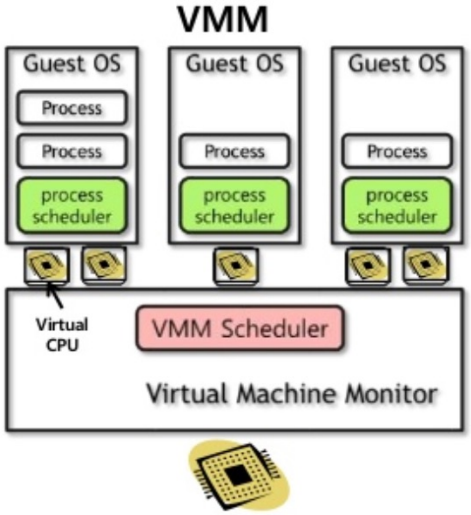
VMM reference architecture

- Main modules
  - *Dispatcher*: VMM entry point that reroutes privileged instructions issued by VMs to the appropriate module
  - *Scheduler*: invoked by the dispatcher when a VM requests system resources; decides resource allocation among multiple VMs componente fondamentale
  - *Interpreter*: executes the appropriate routine for each privileged instruction



VMM CPU scheduler

- Guest OS sees virtual CPUs
- Physical CPUs on host machine are multiplexed among VMs
- How to schedule virtual CPUs on physical CPUs?



Case study: Xen

- What Xen is https://www.xenproject.org
  - Open-source type-1 (system VMM) hypervisor with microkernel design
  - Originated at Cambridge University in the late 1990s
  - Foundation for XenServer, Oracle VM, and used in cloud and embedded systems (e.g., AWS, Alibaba, ARM)
  - Supports paravirtualization (**PV**), hardware-assisted virtualization (**HVM**) and PV-HVM hybrids
- Key ideas for paravirtualization (no hw support yet)
  - Guest OS interacts with the hypervisor via a **hypercall API**
  - Paravirtualization replaces:
    - Privileged instructions → hypercalls
    - Page table updates
    - I/O device access
    - Interrupt & timer operations
  - Requires PV-enabled OSs and drivers (e.g., Linux)

prodotto che ha posto le fondamentale per gli hypervisor moderni

appunto bare metal

oggi AWS usa il suo VMM basato su KVM

usato da AWS fino al 2017.

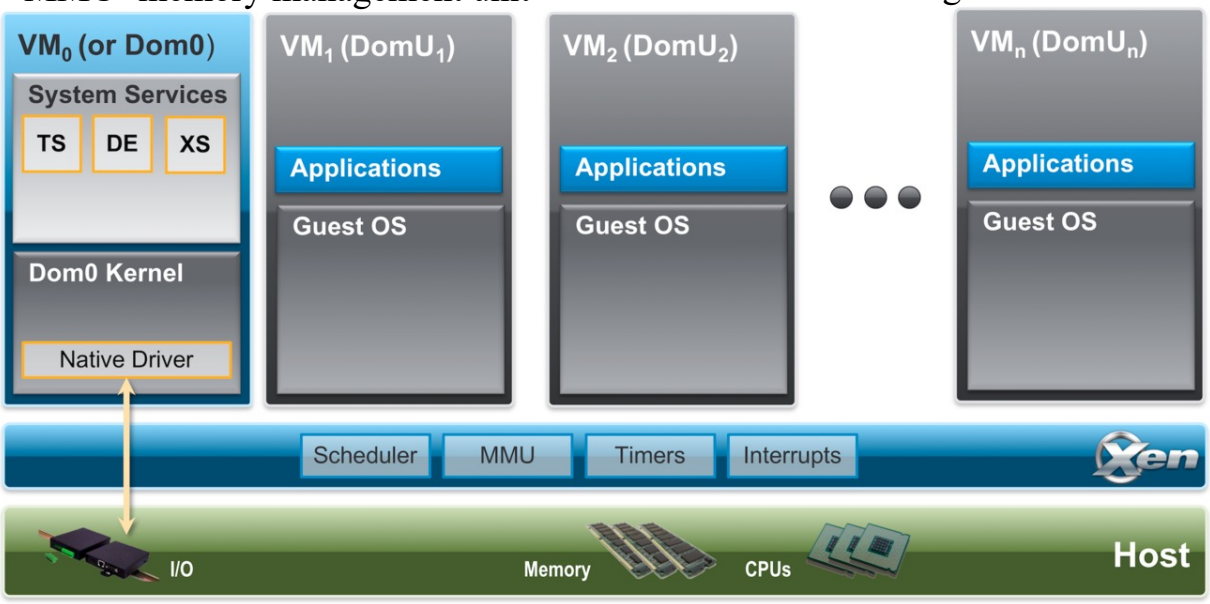
repac utili di cosa si fa carico la paravirtualizzazione

Xen: architecture

andiamo ad osservare nel dettaglio i componenti dell'architettura di Xen

MMU=memory management unit

da notare che Xen non gestisce l'I/O



https://wiki.xenproject.org/wiki/Xen_Project_Software_Overview

la VM Dom0, mette a disposizione dei driver native e dei servizi di sistema per gestire l'I/O in Xen.

questa separazione è presente anche in soluzioni PV-HVM hybryd

Xen: architecture

- Thin, micro-kernel hypervisor
  - 300K LoC on x86, 65K on ARM
- Hypervisor manages vCPU scheduling, memory, timers, and interrupts
- **DomU** (unprivileged guest domains / VMs)
  - Run guest OSs
  - Have no privilege to access hardware or I/O functionality
- **Dom0** (privileged control domain)
  - Contains the device drivers and multiplexes I/O for DomU
  - Manages direct hardware access
  - Runs management services
    - XenStore (XS): shared key–value config store
    - Toolstack (TS): VM lifecycle management and configuration
    - Device Emulation (DE)
  - Mandatory, started on boot

XenStore contiene TUTTI i metadati del contesto virtualizzato. è capace anche di notificare le VM i cambiamenti di configurazioni

linee di codice

la migrazione avviene anche live

servizio che interagisce con i driver per la gestione del I/O

il disaccopiamento Xen e Dom0 consente di limitare il numero di righe di codice di Xen, riducendo la superficie d'attacco possibile

Xen and paravirtualization

- In modern deployments Xen no longer uses PV to virtualize the CPU, HVM is preferred
  - PV guests are still used when hardware lacks virtualization extensions
- Paravirtualization remains important for I/O
  - Xen uses virtual-only devices with PV drivers to avoid slow hardware emulation, enabling faster and more efficient I/O

i dettagli non saranno trattati

https://wiki.xenproject.org/wiki/Understanding_the_Virtualization_Spectrum

- Summarizing Xen impact
  - First hypervisor to popularize paravirtualization
  - Influenced modern cloud hypervisors (AWS, Google)
  - Clear example of a micro-hypervisor
    - Reduces attack surface, increases stability and security, and keeps the hypervisor small and lightweight
  - Still used in security-sensitive and embedded environments

Xen: Credit scheduler

- VMM scheduler selects which virtual CPUs (vCPUs) of all VMs run on the physical CPUs (pCPUs)
  - Adds a scheduling layer below the guest OS scheduler
- Default Xen scheduler: **Credit** scheduler
  https://wiki.xenproject.org/wiki/Credit_Scheduler
- Goals: alcuni condivisi con lo scheduling tradizionale di un OS
  - Proportional fair share: each domain receives a fair share of pCPU proportional to its weight ciascun Dom riceve una quota proporzionale al peso del Dom (anche Dom0)
  - Work-conserving: no pCPU stays idle if runnable vCPUs exist
  - Low latency: fast switching among runnable vCPUs
- Key parameters:
  - Weight: relative share of CPU time (default = 256) più importante. contatore di crediti decrementato con l'uso della CPU
  - Cap: optional limit on maximum CPU share parametro di configurazione
    - cap = 0 → vCPU can use extra CPU time (work-conserving) default value
    - cap > 0 → upper bound (% of one pCPU)

Xen: Credit scheduler

- Each vCPU receives credits proportional to its weight
- Running consumes credits → vCPU becomes:
  - UNDER (credits ≥ 0) il weight, con il passare del tempo viene resettato
  - OVER (credits < 0)
- Each pCPU keeps a runqueue of vCPUs
  - UNDER vCPUs first, then OVER
  - OVER vCPUs run only if no UNDER exist
  - Round-robin within each group della pCPU
  - Time slice: 30 ms
- Placement pCPU di riferimento
  - vCPU is inserted into its home pCPU runqueue when runnable
  - May be migrated to another pCPU by load balancing
- Load balancing favorisce sempre le vCPU UNDER
  - Before going idle, a pCPU checks others for runnable UNDER vCPUs; ensures no pCPU is idle while work exists

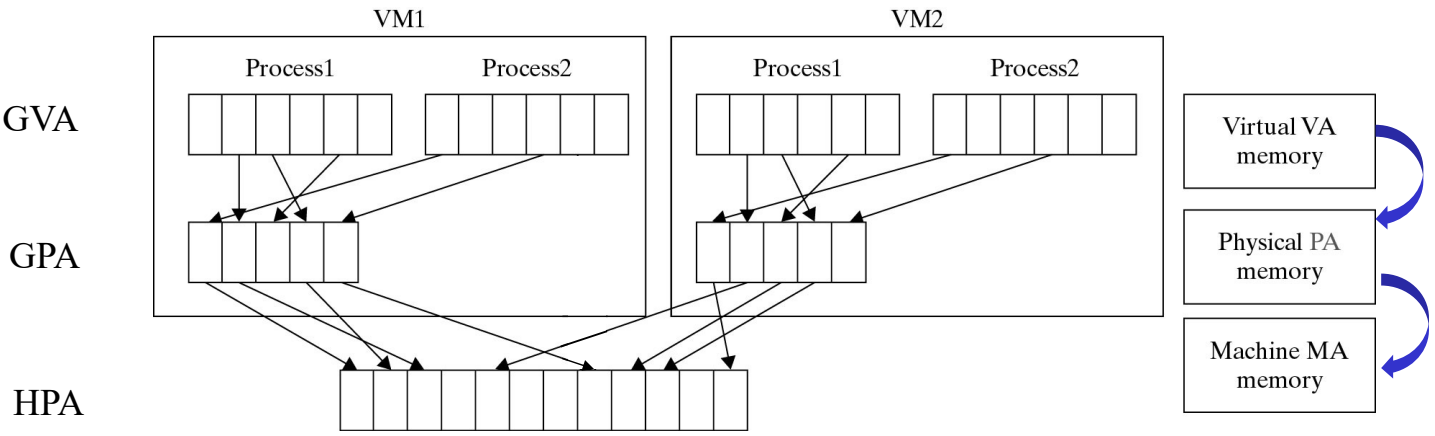
Memory virtualization gestita dalla MMU

- *Non-virtualized* environment
  - Single-level memory mapping: virtual address (VA) → physical address (PA) via page tables
  - MMU and TLB accelerate translations
- *Virtualized* environment
  - Multiple VMs share machine memory → VMM must partition it
  - Requires **two-level mapping:** guest virtual address (**GVA**) → guest physical address (**GPA**) → host physical address (**HPA**)

un OS si aspetta che lo spazio di memoria inizi da 0 ma se partiziono la memoria ciò non accade ecco perché 2 livelli di mapping

- Terminology
  - Guest virtual memory: visible to apps
  - Guest physical memory: visible to guest OS
  - Host physical memory: actual machine memory visible to VMM

Two-level memory mapping

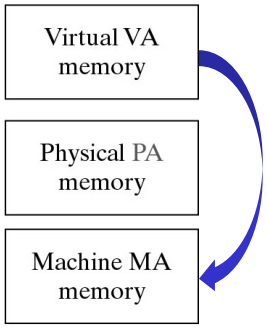


- **GVA → GPA → HPA**
- GPA ≠ HPA: why?
  - Because each guest OS assumes it owns *contiguous, zero-based* physical memory
  - VMM must preserve this illusion

meccanismo che evita il doppio processo di mapping

Shadow page tables (SPT) gestite da hypervisor

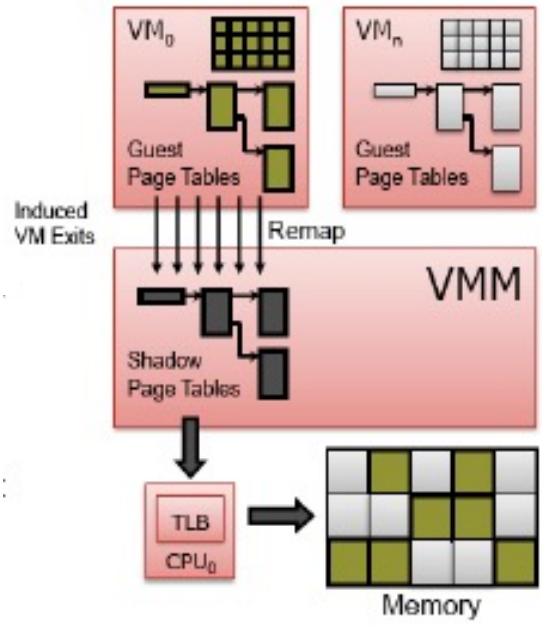
- Goal: avoid performing two translations on every access by giving hardware a direct **GVA → HPA** mapping
- Key mechanisms:
  - VMM maintains shadow page tables (**SPTs**) **which** directly maps GVA to HPA and uses them to accelerate
  - Guest OS builds its own page tables normally, but MMU uses SPTs created by the VMM
  - VMM keeps SPTs consistent with guest page tables
  - Guest page tables are mapped read-only
  - Any guest update → trap to VMM, which updates both guest PT and SPT ("memory tracing")



le page table del guest sono solo read only, il VMM gestisce la write sia SPT che page table del guest

SPT drawbacks

- High overhead: frequent traps and VM exits
  - Every guest PT write → VM exit → high overhead
  - VM exit: similar to a very expensive context switch
- Large memory footprint for maintaining SPTs
- Complex software logic
  - VMM intercepts paging operations and constructs copy of PTs



soluzione ai drawback

Hw support for memory virtualization

- More efficient hardware solution: Second Level Address Translation (SLAT)
- Hardware performs the second translation (GPA → HPA)
- Removes need for SPT maintenance
- Dramatically reduces traps and VM exits
- Provides significant speedup (~50%) for memory-intensive workloads
- Examples: Intel EPT, AMD RV

VM portability

- **VM image**: a single file for VM containing the bootable OS, data files, and applications
- VM images come in different formats dipendono dal hypervisor
- How to import and export VM images and avoid vendor lock-in? Soluzione, formato standard
- Open Virtualization Format (OVF)
  - Open industry standard (ISO 17203) for packaging and distributing VMs
    - Virtual-platform agnostic
  - Image stored in .ova file (Open Virtual Appliance)
  - VM configuration specified in XML format within a .ovf file
  - Supported by many hypervisors, including Hyper-V, VMware, VirtualBox, XenServer

VM resizing and migration

- Techniques for deploying and managing large-scale virtualized environments
  - **Dynamic resizing** for vertical scaling
    - Scale VMs up or down
  - **Live migration** of VMs
    - Move VMs between different physical machines (or even data centers) with shutting them down, ensuring operation during migration

VM dynamic resizing usato per lo scaling VERTICALE

è sempre possibile, spegnere una VM, riconfigurarla e poi riavviarla. ma non è sempre conveniente

- Fine-grain mechanism for adjusting VM resources (CPU, memory) on-the-fly
  - Example: app running on a VM consumes excessive resources, thus VM runs out of RAM and CPU
  - Solution: dynamically resize VM (aka *warm* resizing) to allocate more CPU and memory

✓ More cost-effective and faster than rebooting the VM

✗ Not all virtualization products and guest Oss support it

- What can be resized *without* stopping and rebooting the VM?
  - Number of virtual CPUs
  - Memory

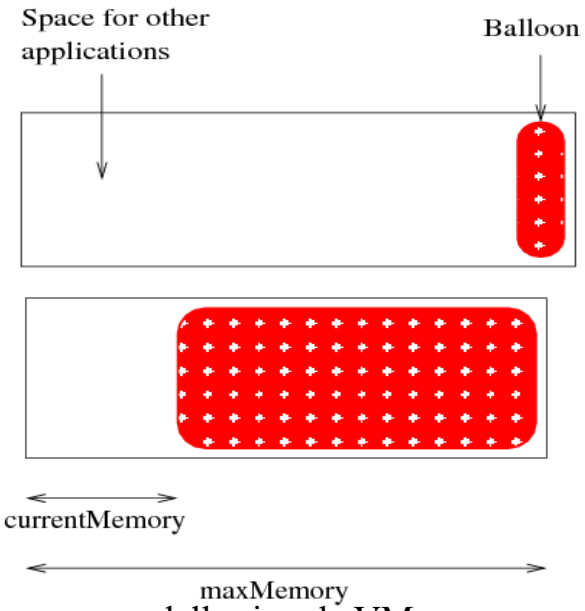
VM dynamic resizing: CPU

- Add or remove virtual CPUs (without turning off VM)
- Linux supports CPU hot-plug/hot-unplug
  https://www.kernel.org/doc/html/latest/core-api/cpu_hotplug.html
  - Relies on virtual file system `sysfs` (processor info is in `/sys/devices/system/cpu`)
    - `/sys/devices/system/cpu/cpuX` for cpuX (X = 0, 1, 2, …)
  - To turn on cpu #5:
    `echo 1 > /sys/devices/system/cpu/cpu5/online` direttamente da shell
  - To turn off cpu #5:
    `echo 0 > /sys/devices/system/cpu/cpu5/online`
- Manage VM CPU resizing using `virsh` letteralmente virtualshell
  - `virsh`: command line tool to configure and manage VMs, available with some hypervisors (KVM, Xen)
  - E.g., to set the number of vCPUs

```
virsh setvcpus <vm_name> <vcpu_count> --current
```


VM dynamic resizing: memory

- Based on *memory ballooning*
  - Mechanism used by hypervisors (e.g., KVM, Xen, VMware) to pass memory back and forth between hypervisor and guest OSs
- Balloon inflation: hypervisor takes memory from the VM
  - May swap to disk
- Balloon deflation: more memory is allocated to the VM when needed
  - Cannot exceed maxMemory limit



della singola VM

passiamo alla migrazione

VM migration: pros and cons

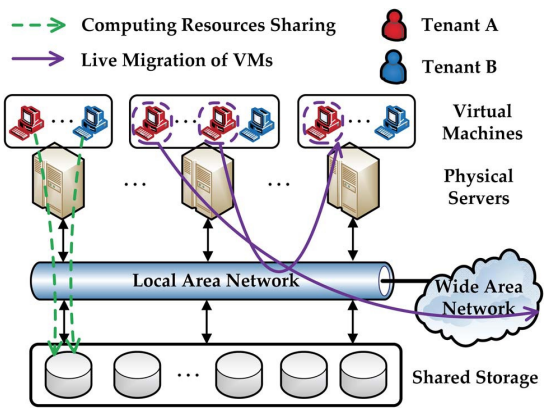
- Useful in clusters and virtual data centers to:
  - ✓ Consolidate infrastructure
  - ✓ Enable failover flexibility
  - ✓ Balance load
  - ✓ Improve maintenance

- Cons
  - ✓ Requires VMM support
  - ✓ Migration overhead ovvero l'aumento del downtime, dato che spengo la VM
  - ✓ WAN migration limited

non consideremo il caso di migrazione tra data center differenti

VM migration

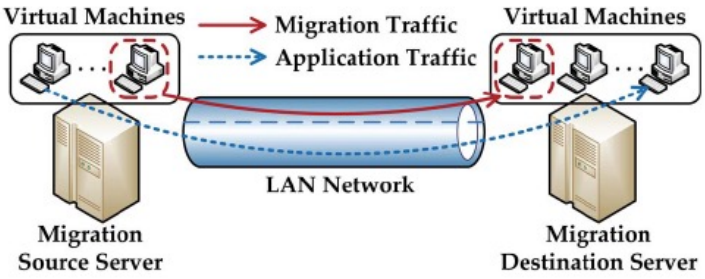
- Approaches to migrate VM instances:
  - Cold migration: shutdown the source VM, transfer its image to to the destination host, then restart it
    - ✗ VM downtime can be too long il downtime, comprende lo shutdown, il trasferimento e il boot della VM. (una VM è grande svariati GB)
    - ✗ VM image can be large and network bandwidth limited
  - **Live migration**: the source VM is running during migration
    - ✓ Minimal downtime
    - ✓ Widely used by large-scale environments (e.g., Google: > 1M migrations per month)



VM live migration

- Preliminary steps before VM live migration
  - Setup phase: determine source host, destination host, and VM to migrate (goals: load balancing, energy efficiency, server consolidation) by means of a selection algorithm
- What to migrate? Memory, storage, network connections
- How to migrate? Transparent migration for applications running inside the VM
  - Challenges: full migration transparency is difficult, application downtime is still possible: how to limit it?

qui entra in gioco il MAPE



VM live migration: storage

- We consider VM migration within a cluster environment

- To migrate storage
  - With shared storage
    - Use a SAN (Storage Area Network), NAS (Network Attached Server), or distributed file system (e.g., HDFS, Ceph) to store the VM disk image
  - Without shared storage
    - The source VMM stores the VM disk data on the source host local storage and the disk image is copied to the destination host

fisico o virtuale

occorre quindi aspettare il tempo per la copia

VM live migration: network

- To migrate network connections
  - Source VM has a virtual IP address and virtual MAC address, which can be distinct from the IP address and MAC address of source host
    - VMM maintains a mapping between virtual IP addresses and MAC addresses to their corresponding VMs
  - If source and destination hosts on a single switched LAN, network migration happen transparently grazie al protocollo ARP
    - The source host sends an unsolicited ARP reply to notify the network that the VM's IP address has moved to a new location (the destination host)
    - Possible packet loss limited to a few in-flight packets inteso come delay
  - Otherwise, various alternatives:
    - Use network layer forwarding tunneling
    - Update DNS records (if VM uses a domain name)
    - Use a load balancer in front of the VM

le prime due funzionano anche in data center sparsi geograficamente

debbo però aggiornare anche i proxy

VM live migration: memory

include anche i registri della CPU e dei driver

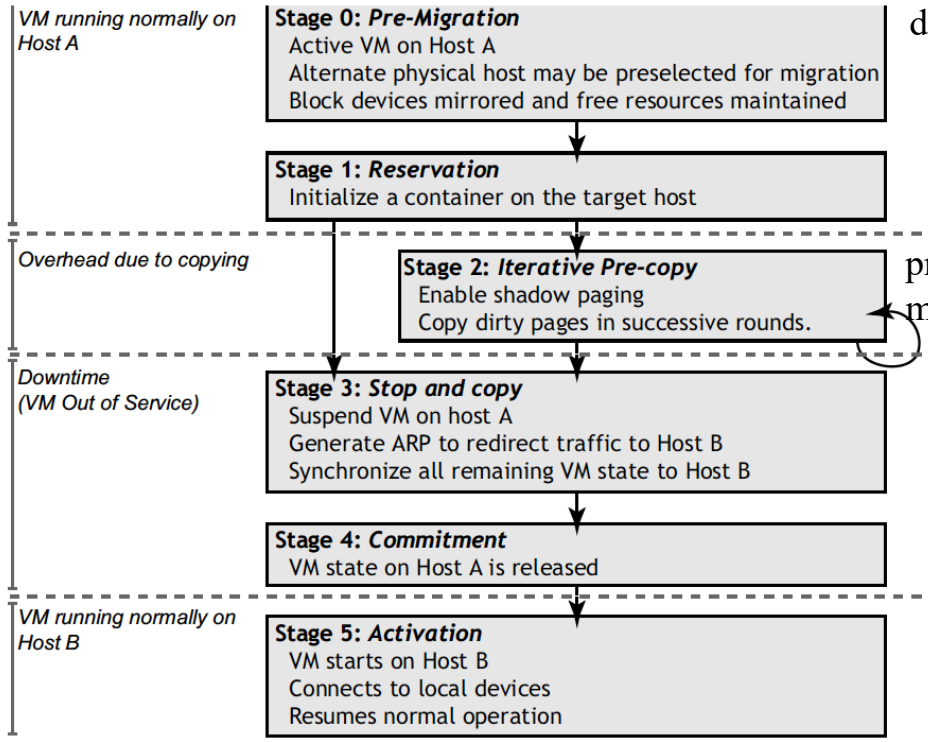
- To migrate memory (including CPU and device state):
  1. Pre-copy phase: VMM iteratively copies the memory pages from source VM to destination VM while source VM is running
     - During iteration *n* those pages dirtied during iteration *n*-1 are copied
  2. Stop-and-copy phase: source VM is suspended and the last dirty pages are copied, along with CPU state and device driver state; VM applications do not run
     - Downtime: can range from msec to sec, depending on memory size, application memory workload, network bandwidth
  3. Commitment and reactivation phases: destination VM is activated and recovers application execution; source VM is removed (and source host may be turned off)

n è arbitrario, oppure si usa un timer di esecuzione

il downtime è influenzato molto da applicazioni memory-intensive

- Known as **pre-copy** approach
  - Memory is copied *before* VM execution resumes at destination
  - Most commonly used technique, e.g., by KVM, VMWare, Xen, Google CE

VM live migration: pre-copy



dettagli del processo

prima storage, poi memory

Clark et al., Live Migration of Virtual Machines, NSDI 2005
https://www.usenix.org/legacy/event/nsdi05/tech/full_papers/clark/clark_html/

VM live migration: alternatives for memory

- Pre-copy struggles with memory-intensive apps
  - Especially write-intensive ones, because it cannot migrate memory faster than the rate at which memory is dirtied by running apps
- Two alternative approaches
  - Post-copy
  - Hybrid
- **Post-copy** approccio ottimale per applicazioni che sono memory-intensive
  - CPU state and device state: immediately transferred
  - VM execution control: transferred to destination host
  - Memory transfer: fetched on-demand as needed by the running VM (*pull* approach) le pagine arrivano tramite pull
  - ✓ Reduces downtime and overall migration time
  - ✗ Incurs app degradation due to page faults that must be resolved over the network i page fault sono più costosi

pessimo per applicazioni read-intensive

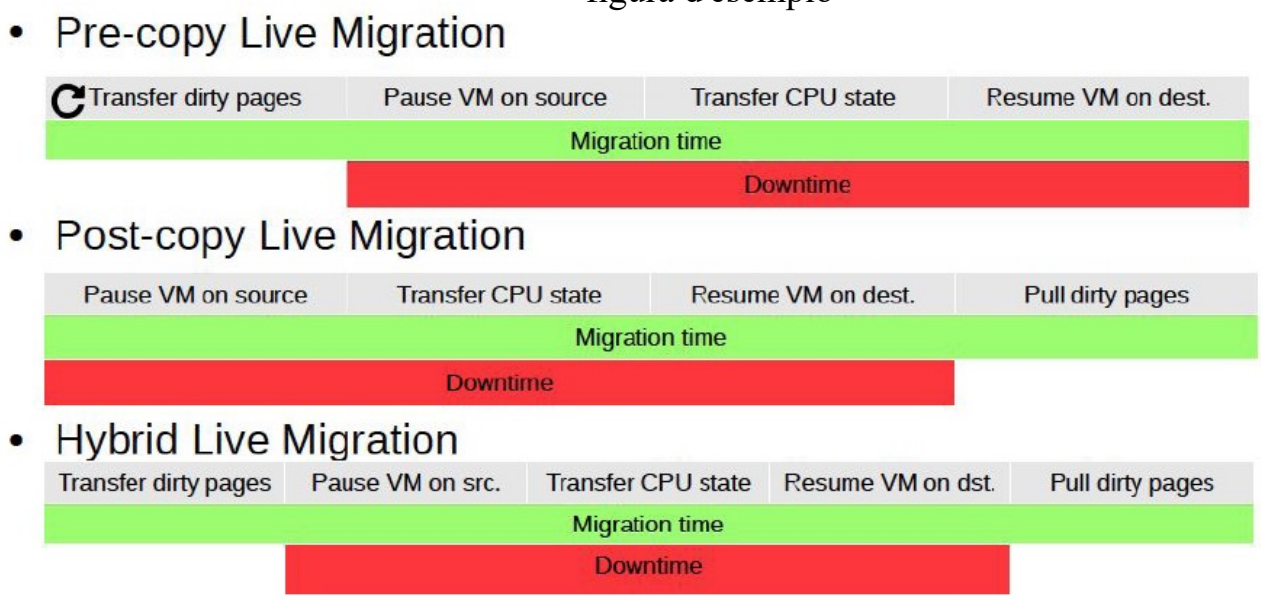
VM live migration: alternatives for memory

- **Hybrid**
  - Combines pre-copy and post-copy to balance migration efficiency and minimize application downtime
  - Special case of post-copy migration: post-copy preceded by a bounded pre-copy stage
  - Idea: before switching VM execution to destination host, transfer a subset of the most frequently accessed memory pages, so to reduce app performance degradation by minimizing the number of page faults after VM is resumed
  - ✓ Pre-copy reduces the number of future network-bound page faults

- Pre-copy still dominates in terms of support and usage
  - `virsh migrate` command

VM live migration: alternatives for memory

figura d'esempio

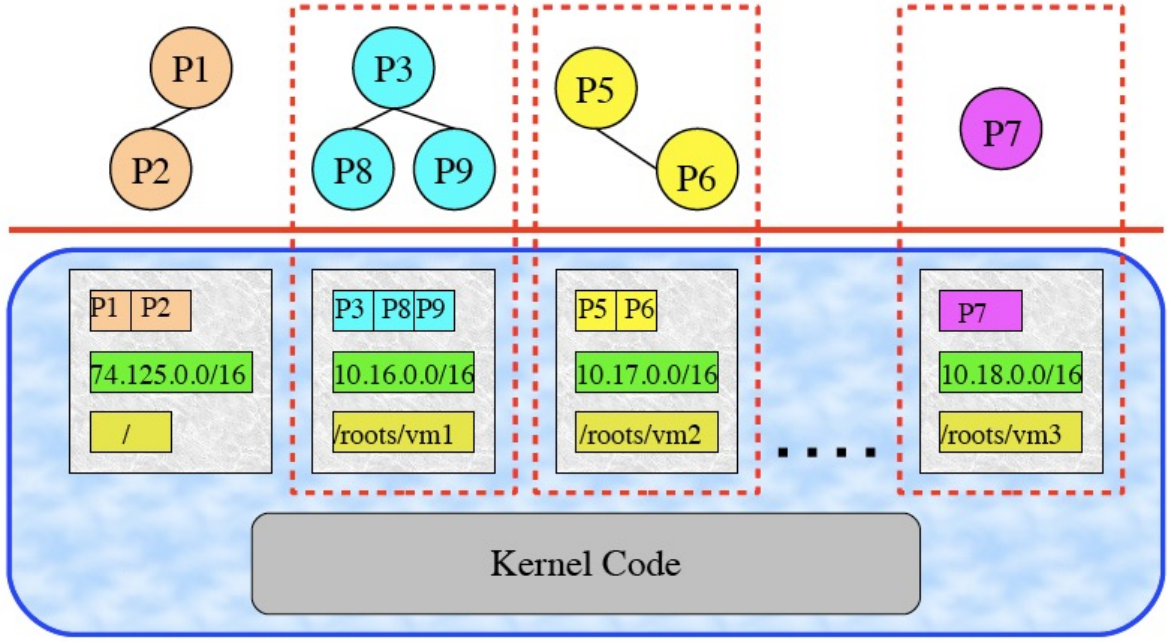


OS-level virtualization

- Let's consider **operating system (OS) level virtualization** (or ***container-based virtualization***)
- It allows running multiple isolated (*sandboxed*) user-space instances on top of a single OS attraverso la condivisione del kernel, che pone però delle limitazioni
  - Such instances are called:
    - **containers**
    - jails
    - zones nasce su Free-BSD

OS-level virtualization

- OS kernel allows the existence of multiple isolated user-space instances, called containers sinteticamente
- Each container has:
  - Its own set of processes, file systems, users, network interfaces with IP addresses, routing tables, firewall rules, …
- Containers share the same OS kernel (e.g., Linux)



OS-level virtualization: mechanisms

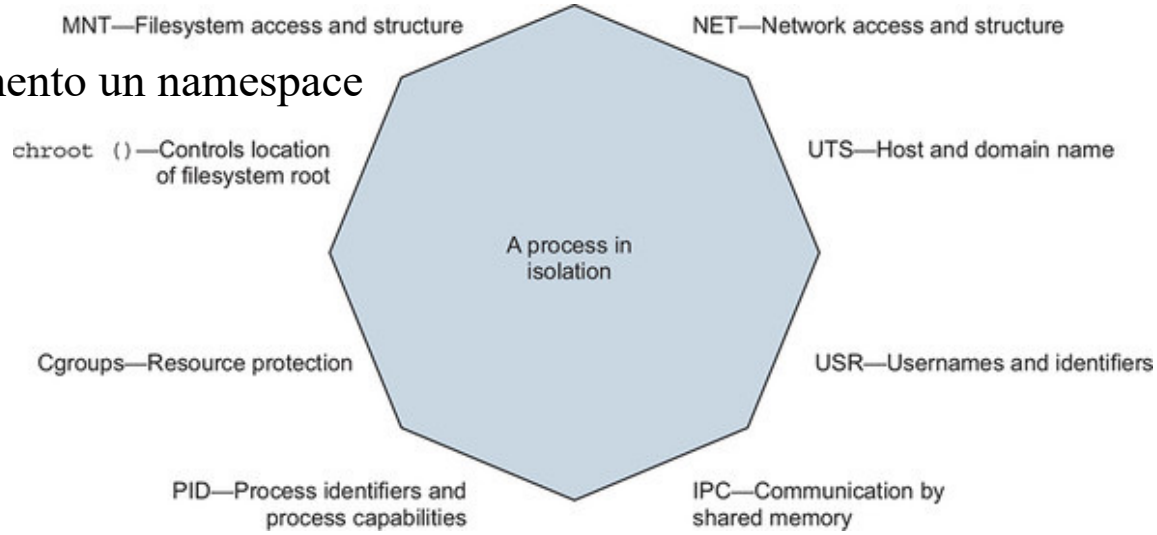
- Which OS kernel mechanisms are used to manage containers? più altri meccanismi di sicurezza che non saranno tratti nel corso
  - Containers need to isolate processes from each other in terms of sw and hw (CPU, memory, …) resources
- Main mechanisms offered by Unix-like OS kernels
  - chroot (change root directory) presente su qualsiasi Unix-based OS
    - Allows changing the apparent root folder for the current running process and its children
  - cgroups (Linux-specific)
    - Manage resources for groups of processes, such as CPU and memory allocation
  - namespaces (Linux-specific)
    - Per-process resource isolation, ensuring that each container has its own isolated environment

da considerare con cautela data la possibile problematica di sicurezza che può generare

Mechanisms: namespaces

- Feature of Linux kernel that allows to isolate what a set of processes can see in the operating environment
  - Includes resources such as processes, ports, files, …
- Kernel partitions resources so that one set of processes can see a specific set of resources, while another set of processes sees a different set of resources
- 6 different types of namespaces

i sei tipi sono anche a cosa fa riferimento un namespace



Mechanisms: namespaces

- mnt: isolates mount points seen by a container
  - Virtually partitions the file system, so processes running in separate mount namespaces cannot access files outside of their mount point
- pid: isolates PID space, ensuring that each process only sees itself and its children (PID 1, 2, 3, …)
- network: allows each container to have its dedicated network stack
  - Its own private routing table, set of IP addresses, socket listings, firewall rules, and other network-related resources
- user: isolates user and group IDs
  - E.g., allows a non-root user on host to be mapped to root user within the container, without granting actual root access to host

Mechanisms: namespaces

- uts (Unix timesharing): provides dedicated host and domain names
  - Allows processes to think they are running on servers with different names, even though they share the same host
- ipc: provides dedicated shared memory for IPC
  - E.g., separate Posix message queues for different containers

interprocesscomunication

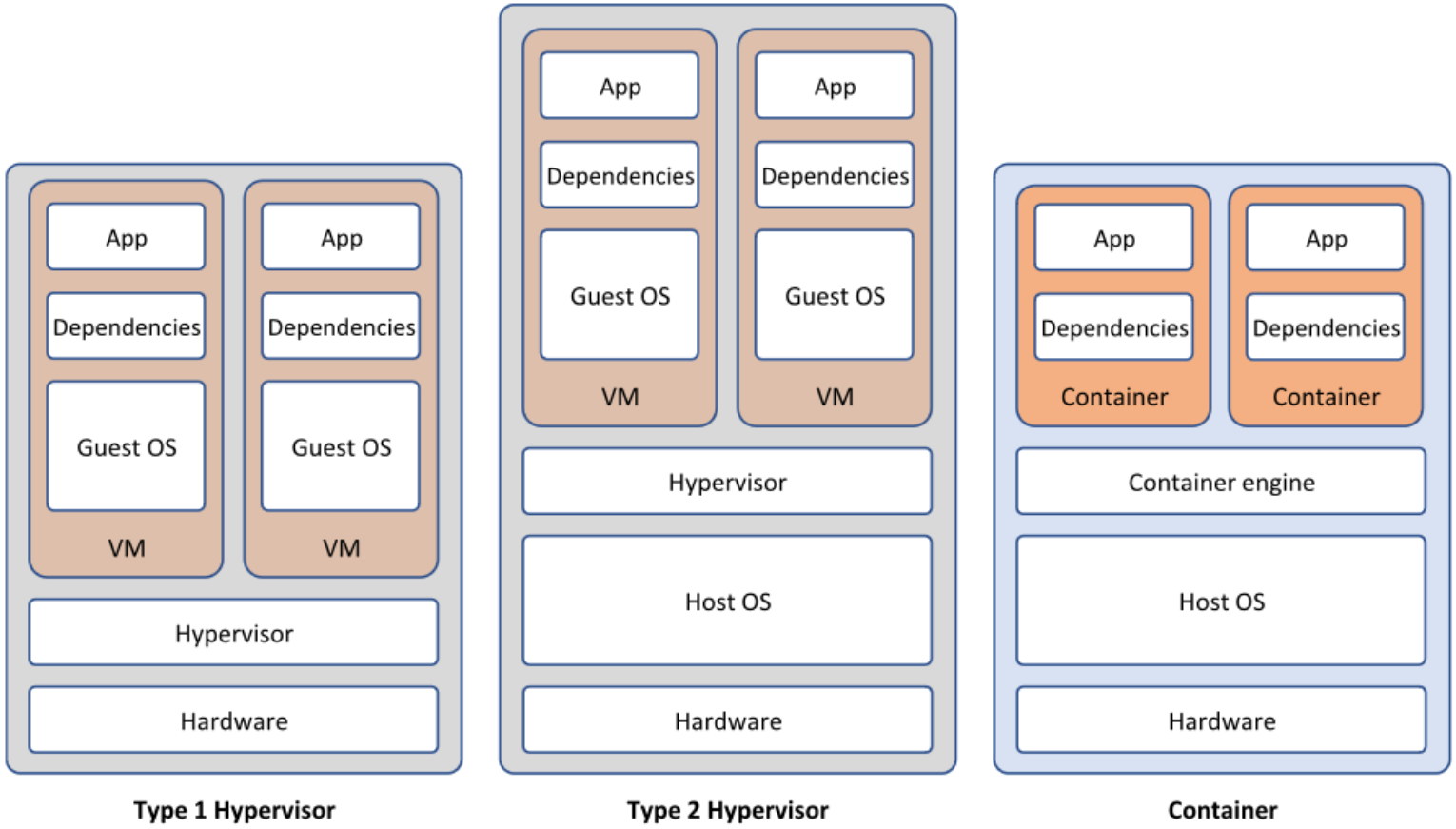
Mechanisms: cgroups

- cgroups = control groups
- Limit, measure and isolate the use of hw resources (CPU, memory, I/O, network) for a group of processes
- Exposed via a filesystem interface (similar to `sysfs` and `procfs`)
  - Default mount point: `/sys/fs/cgroup/`

- In a nutshell:
  - namespaces implement *information isolation*: what a container can see
  - cgroups implement *resource isolation*: how many resources a container can use

OS-level virtualization: pros

- VMM-based vs container-based virtualization



In a nutshell: lightweight vs. heavyweight

OS-level virtualization: pros

vs. VMM-based virtualization (type-1)

Non è presente Guest OS, dato che il kernel è condiviso

- ✓ Near-native performance nessun intermediario per syscall
  - No VMM indirection for system calls
- ✓ Fast startup and shutdown
  - Seconds (even msec) per container vs. minutes per VM
- ✓ High density
  - Hundreds of containers per physical machine (PM)
- ✓ Small footprint
  - Container images are smaller since they exclude the OS kernel a differenza di poche VM grosse svariati GB.
- ✓ Memory efficiency
  - Containers can share memory pages on the same PM
- ✓ Portability and interoperability
  - Apps run across environments

OS-level virtualization: cons

vs. VMM-based virtualization (type-1)

✗ Less flexible

- Only supports native apps for the OS kernel (e.g., no Windows container on Linux host)
- Cannot run different OS kernels on same PM; however, can run multiple Linux distributions (e.g., Ubuntu, CentOS)

✗ Weaker isolation

- Process-level isolation leads to higher performance interference on shared resources

✗ Higher security risks

un container compromesso permette di attaccare l'host sottostante

- A kernel vulnerability affects the entire system
- Since containers share the kernel, one compromised container can impact other containers and the host

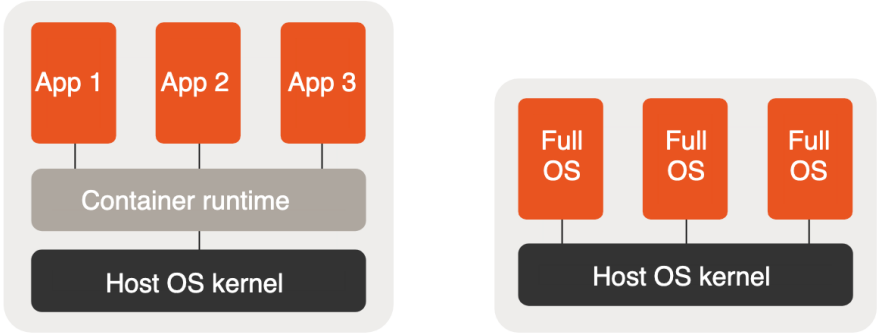
✗ Reduced hardware/device isolation

il container può avere accesso diretto al hardware. nvidia fornisce il passthrough tramite VGPU

- Device passthrough is more difficult and less secure

OS-level virtualization: products

- Docker
  - Most popular container engine
  - Provides *application containers*
    - Package and run *a single application* with its dependencies
  - Supports Open Container Initiative (OCI) standards https://opencontainers.org
- LXC (LinuX Containers) https://linuxcontainers.org/lxc/
  - Supported by mainline Linux kernel
  - Provides *system containers*
    - Run a *whole OS user space with multiple processes*, but share the host kernel



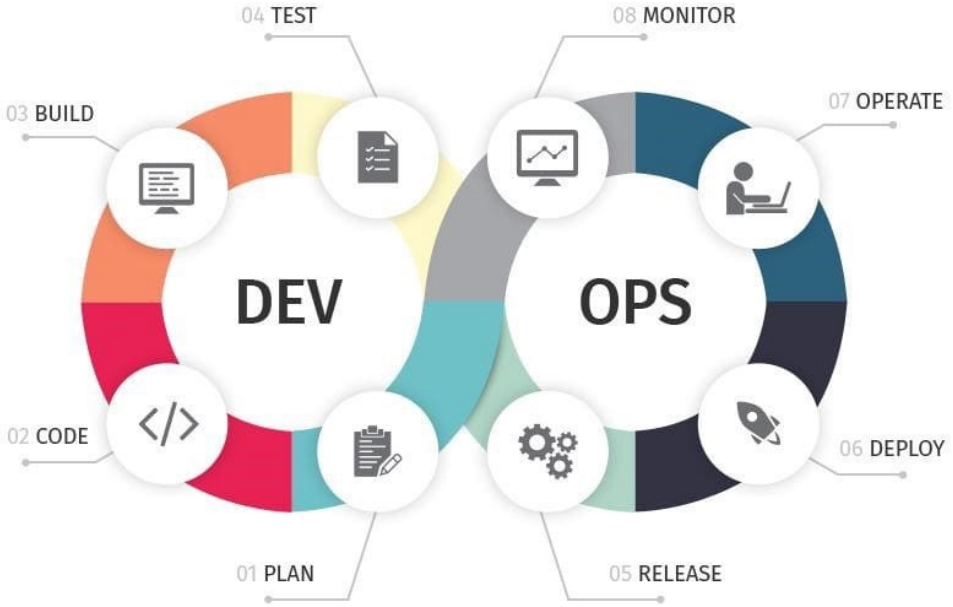
OS-level virtualization: products

- Podman https://podman.io
  - Supports OCI; Docker compatible CLI
  - Daemonless, rootless operation for improved security
- FreeBSD Jail
  - Strong process and filesystem isolation
- OpenVZ / Virtuozzo https://openvz.org
  - For Linux; primarily for system containers
- Non-Linux platforms
  - Windows and macOS support containers (e.g., Docker Desktop)
- Alternative approach
  - Install a Linux VM as a guest OS
  - Run container engines (e.g., Docker) inside the VM si ha nested virtualization insomma
  - Performance Impact: nested virtualization leads to reduced performance

Containers, DevOps and CI/CD

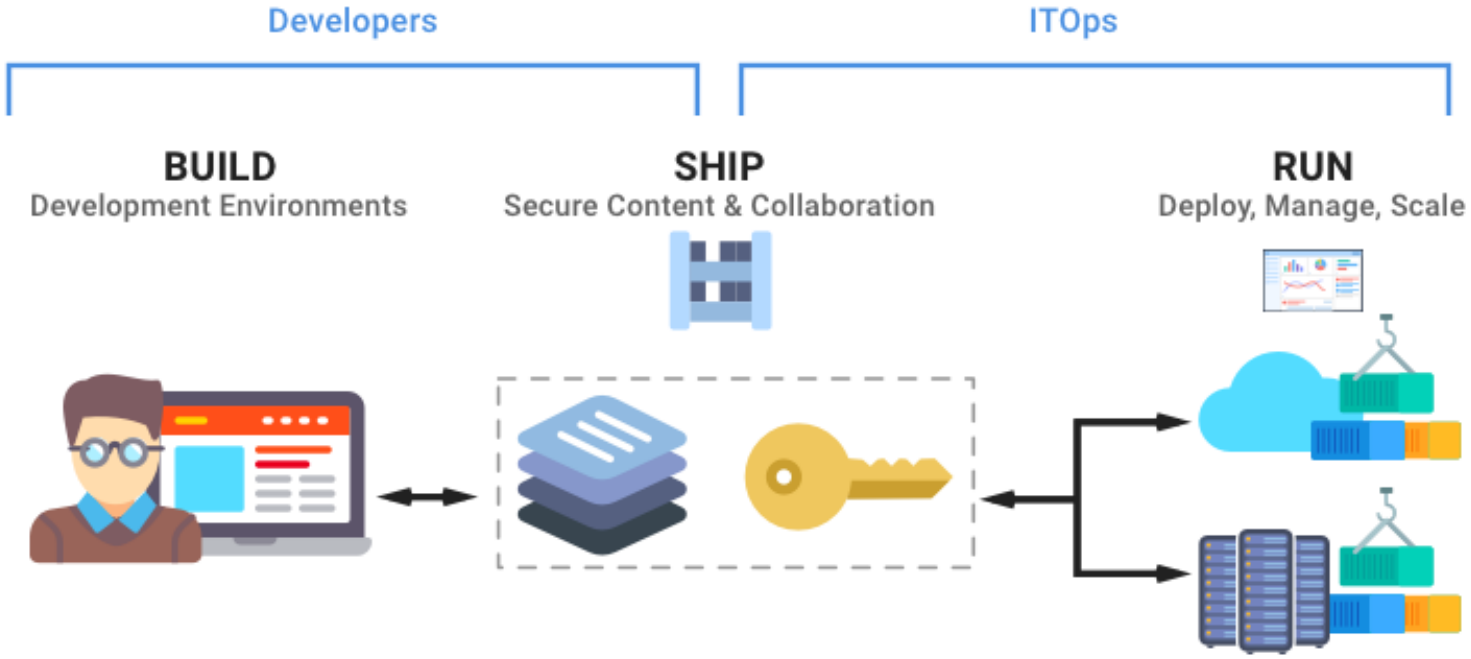
- DevOps: development methodology that bridges the gap between Development and Operations, focusing on collaboration, continuous integration, and automated delivery
- CI/CD:
  - Continuous Integration (CI): merges developers' work into a shared codebase
  - Continuous Delivery (CD): ensures frequent and reliable releases

mentre prima DEV e OPS erano separati e svolti da due team. oggigiorno c'è una stretta collaborazione tra i membri che sviluppano e gestiscono lo sviluppo e la manutenzione del software.



Containers, DevOps and CI/CD

- Containers: simplify building, packaging, sharing, and deploying apps with all dependencies
- Enable collaboration by sharing images, and streamline deployment across environments without extra configuration



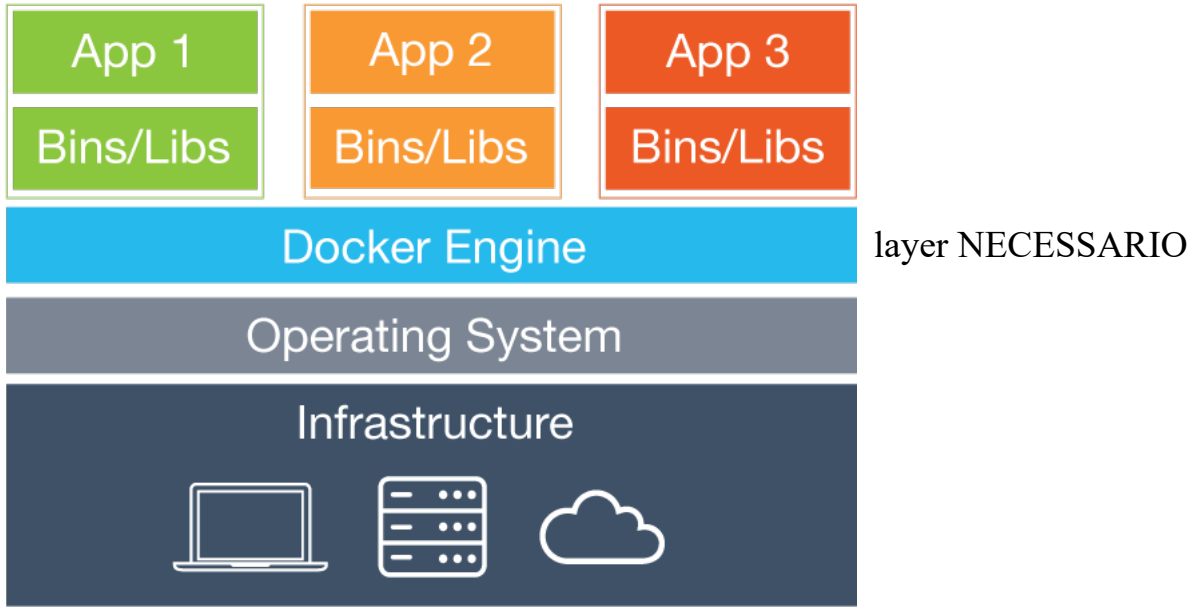
Containers, microservices, and serverless

- Using containers
  - Package apps and all dependencies into a single unit that runs almost anywhere
  - Use fewer resources than traditional VMs

    I container sono anche più facili da manutenere
- Containers enable
  - Microservices: break down apps into small, independent services
  - Serverless: package functions into containers for efficient execution assieme alle micro VM

Case study: Docker

- Lightweight and secure container-based virtualization
  - Application containers: contain the application and its dependencies, but share OS kernel with other containers
  - Isolation: containers run as isolated processes in user space on host OS
  - Portability: containers are infrastructure-agnostic and can run anywhere qualche dipendenza però, dall'architettura della CPU c'è in realtà



layer NECESSARIO

Docker: features

- Portable deployment
  - Easily deploy across different machines and environments
- Versioning
  - Git-like version control for container images
- Component reuse
  - Reuse components (e.g., libraries, services) via Docker images aumenta la portabilità e interoperabilità
- Shared libraries
  - Access to pre-built images on Docker Hub https://hub.docker.com
- OCI support
- Scalability
  - Works seamlessly with Kubernetes for scaling applications

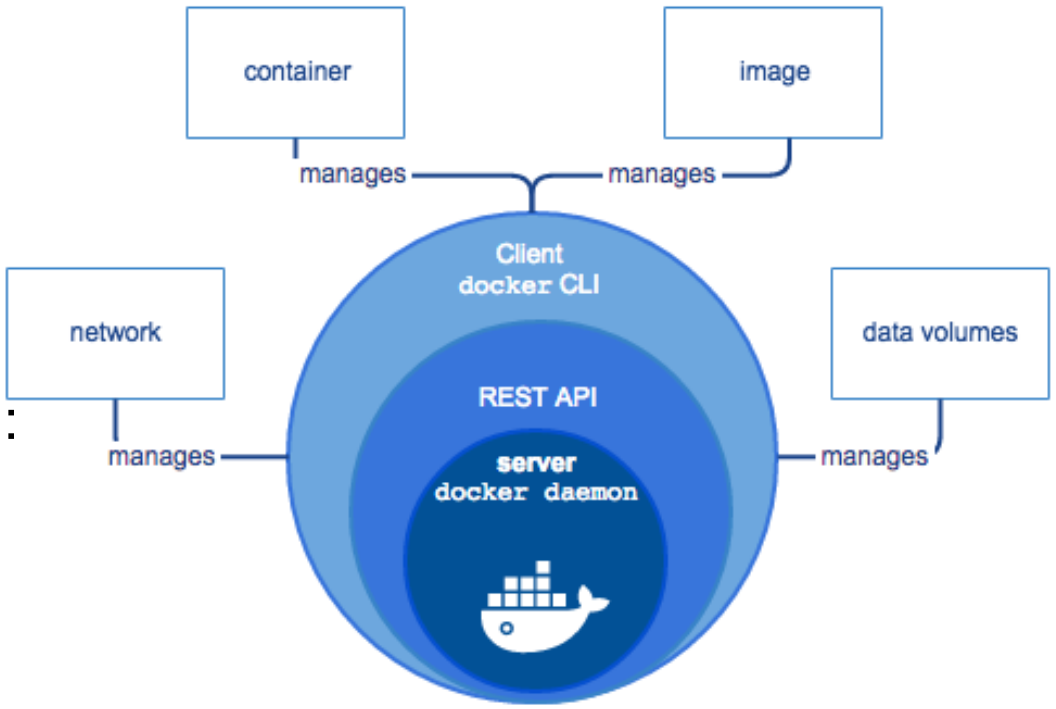
Kubernetes permette di eseguire la stessa applicazione su più server. mentre docker compose no anche con docker compose che però non scala dinamicamente le repliche a differenza di Kubernetes

Docker: internals

- Written in Go
- Exploits Linux kernel mechanisms for resource management and isolation (cgroups and namespaces)
  - Early versions were based on Linux Containers
  - Then transitioned to libcontainer, a Go-based container runtime: provides tools for managing containers with namespaces, cgroups, capabilities, and filesystem access controls
    https://pkg.go.dev/github.com/opencontainers/runc/libcontainer
  - runc: libcontainer is now part of runc, the CLI tool for spawning and running containers based on OCI specification
    https://github.com/opencontainers/runc

Docker Engine: architecture

- Docker Engine: core component of Docker that enables containerization
- Client-server application composed by
  - **Docker daemon (dockerd)**: server component that listens for API requests and manages Docker objects like images, containers, networks, and volumes



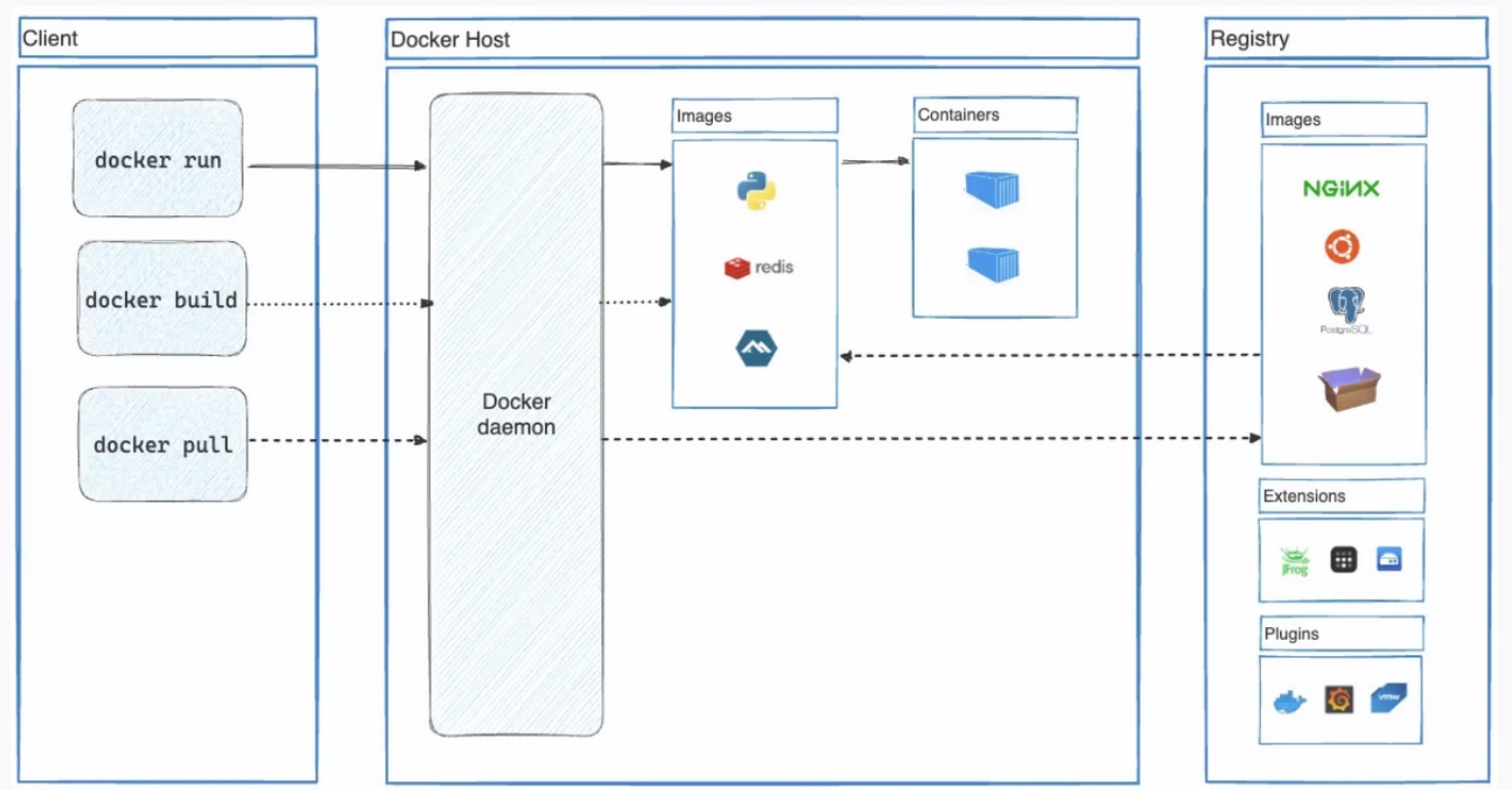
  - **REST API**: specifies the interfaces that programs can use to interact with Docker daemon for operations like creating, managing, and querying Docker objects
  - **CLI client**: allows users to send commands to Docker daemon via REST API

https://docs.docker.com/get-started/docker-overview/#docker-architecture

Docker: client-server architecture

- Docker client
  - Interface through which users interact with Docker
  - Sends commands to Docker Daemon to build, run, and distribute Docker containers
  - Client and daemon communicate through sockets or the REST API

sul server si può creare una rete interna che viene usata dai container per collaborare



Docker: images

- Read-only template A container image is a standardized package that includes all of the files, binaries, libraries, and configurations to run a container
  - Used to create Docker containers, containing everything needed to run app (code, dependencies, configurations)
- Build component
  - Docker images enable distribution of apps with their runtime environments, removing the need to manually install packages
  - Target machine must be Docker-enabled
- **Dockerfile** cosa sono? Template in particolare:
  - Text file with simple instructions that Docker uses to build images automatically
- **Image registry** in remoto
  - Images can be pulled and pushed to/from public or private registries
- Image naming
  - Format: `[registry/][user/]name[:tag]` opzionali sono gli elementi nelle parentesi
  - Default tag is latest

Docker image: Dockerfile

- Docker images are created from a Dockerfile and a context
  - Dockerfile: text file containing instructions to assemble the image potrebbe essere minimo
  - Context: set of files (e.g., app code, libraries) used during the image build process
  - Images often build on parent images (e.g., Alpine, Ubuntu)
- Dockerfile syntax

  `# Comment` (for comments)

  `INSTRUCTION arguments` (commands like RUN, COPY, etc.)
- Instructions in the Dockerfile run sequentially

Docker image: Dockerfile

- Common Dockerfile instructions
  - **`FROM <image>`**: specifies the parent image (mandatory unless you want to start from **`scratch`**) FROM scratch (per partire da zero)
  - **`WORKDIR <path>`**: sets the working directory inside the container
  - **`COPY <host-path> <image-path>`**: copies files from host to container image
  - **`RUN <command>`**: executes the specified command during image build
  - **`ENV <name> <value>`**: sets an environment variable
  - **`EXPOSE <port>`**: exposes a network port for the container
  - **`CMD ["<command>", "<arg1>"]`**: defines the default command to run when the container starts

https://docs.docker.com/get-started/docker-concepts/building-images/writing-a-dockerfile/

Docker image: Dockerfile

- Example: Dockerfile to build the image of a container that will run as application a simple todo list manager written in Node.js

```
# syntax=docker/dockerfile:1


FROM node:lts-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
EXPOSE 3000
```

Directory with app code

```
├── getting-started-app/
│   ├── package.json
│   ├── README.md
│   ├── spec/
│   ├── src/
│   └── yarn.lock
```

https://docs.docker.com/get-started/workshop/02_our_app/

Docker image: build

- Building Docker image from Dockerfile and context
  - Context: set of files located in the specified PATH or URL
- Build command

```
$ docker build [OPTIONS] PATH | URL | -
```

- Example: to build an image for Node.js app (slide 24)

```
$ docker build -t getting-started .
```

  - The `-t` flag is used to tag the image with a name (and optionally a version)
  - If Dockerfile is named something different from Dockerfile, use the `-f` flag:

```
$ docker build -t getting-started -f myDockerfile .
```

https://docs.docker.com/reference/cli/docker/build-legacy/

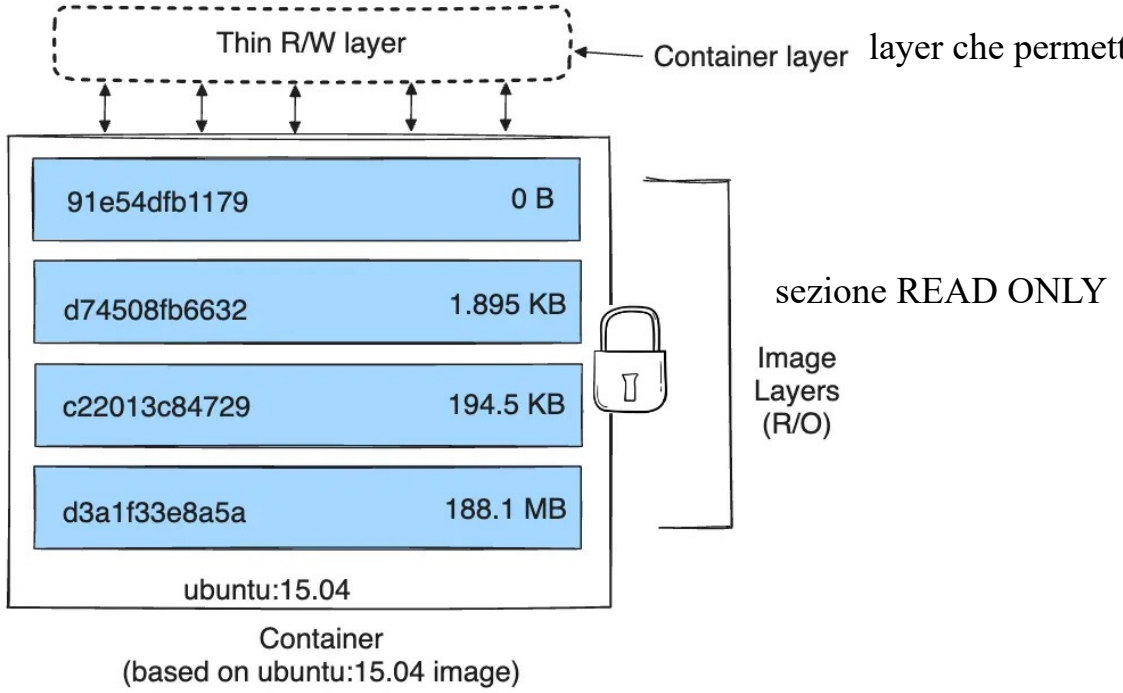
Docker image: layers

- Each Docker image consists of a series of layers
- Docker uses *union file systems* to combine these layers into a single unified view

dando l'illusione che i layer siano un singolo file

  - These layers are stacked to form a base the base of a container's root file system
  - Docker leverages copy-on-write (CoW) strategy to optimize performance

i layer sono codificati con un hash perché due container possono condividere layer, il che permette di risparmiare spazio.

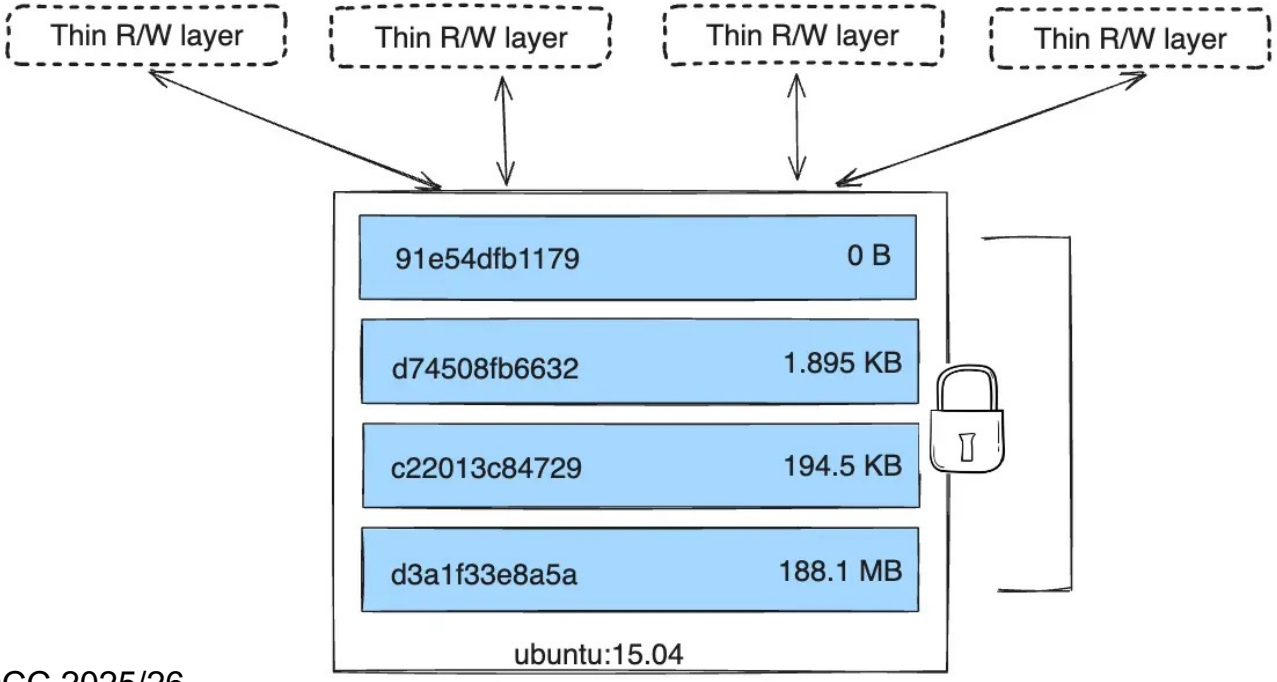


Docker image: layers

- Layering pros
  - ✓ Efficient layer sharing and reuse: common layers are installed only once, saving bandwidth and storage space
  - ✓ Separation of concerns: allows better management of dependencies grazie anche al versioning
  - ✓ Facilitates software specialization

https://docs.docker.com/storage/storagedriver

ciò che è stato scritto, viene cancellato. ad esempio se creo un file nel container sarà cancellato



l'ultimo layer è quindi dinamico rispetto ai restanti che si sono "statici" e immutabili. per dati persistenti si utilizzeranno i volumi

Docker image: layers

- Dockerfile instructions that modify the filesystem create new layers
  - Examples: `FROM`, `RUN`, `COPY`, `ADD`
- Instructions that only modify the image's metadata do not create new layers
  - Examples: `CMD`, `LABEL`
- All image layers (except the top one) are read-only
  - To enable efficient layer sharing across images
- When a container is started, a writable layer (aka *container layer*) is added on top
  - Changes made by a running container (e.g., creating or modifying a file) are written to the writable layer
  - Unique to each container
  - Ephemeral: deleted when the container is removed
  - Use it only for temporary/runtime data, not persistent data

attenzione quindi ad utilizzare RUN per singole istruzioni!
si va a sprecare spazio se si usa in maniera sconsiderata

Docker container storage

- Containers are usually stateless
- Why? Easier to:
  - Scale: start quickly new replicas
  - Restart: from failure
  - Migrate: move between hosts
- Very little data is written to container's writable layer
- Data is typically stored in **Docker volumes**
- However, some workloads require writing data to container's writable layer

Docker: storage backends

- How Docker daemon stores image layers and container writable layers on disk
  - Storage drivers
  - containerd image store
- Storage drivers (legacy): common options
  - Overlay2: file-level, preferred for all Linux distros
  - Btrfs: supports snapshotting
  - Zfs: block-level
- Storage driver considerations
  - Driver choice affects container performance
  - Drivers are optimized for space efficiency
  - Write performance speeds may be lower than native file system performance due to CoW

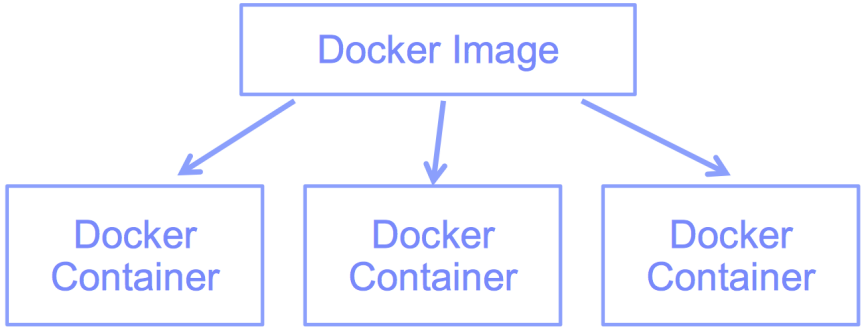
https://docs.docker.com/storage/storagedriver/select-storage-driver

Docker: storage backends

- New default backend: containerd image store (Docker Engine 29.0+)
- Uses content-addressable blob store
- Pros and cons:
  - ✓ Cleaner architecture
  - ✓ Unifies Docker and containerd ecosystem
  - ✗ Uses more disk space than storage drivers (no deduplication at driver level)

Docker: containers and registry

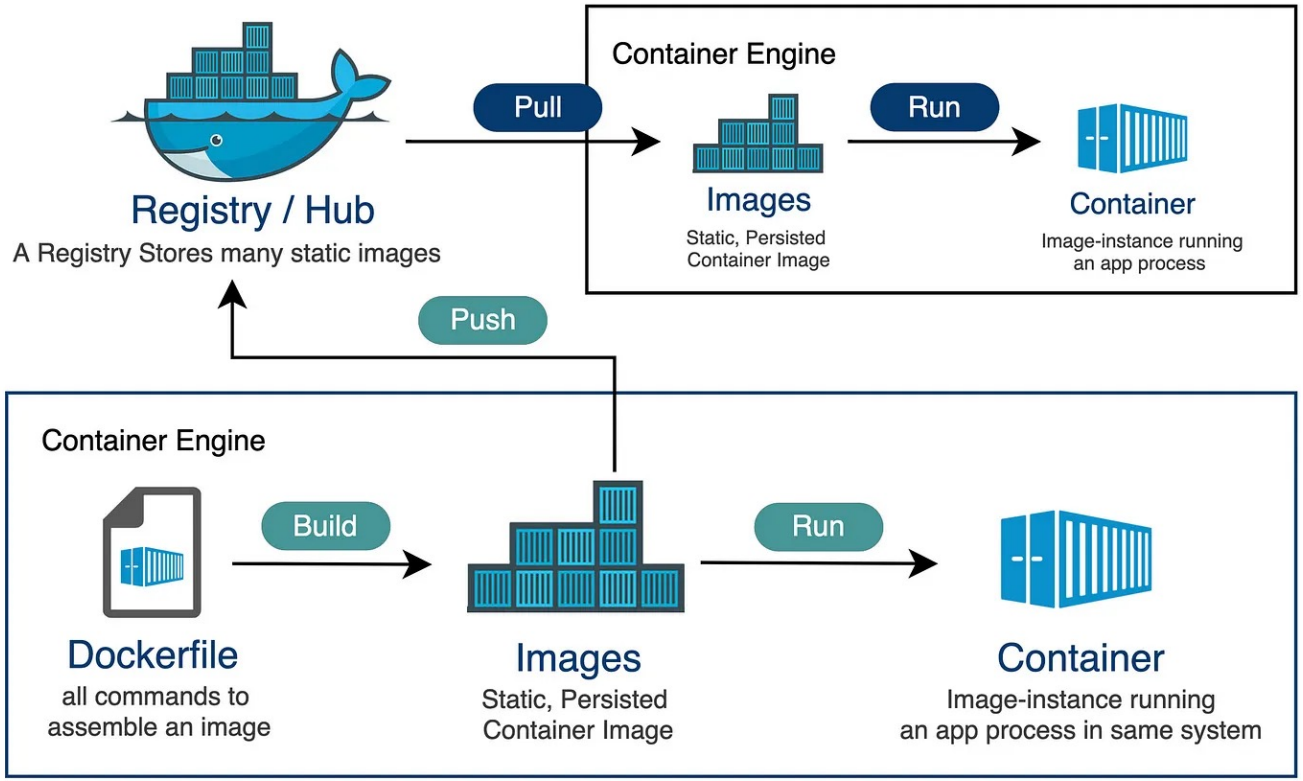
- Docker **container**: runnable instance of a Docker image
  - Containers are the ***run*** component of Docker
  - Run, start, stop, move, or delete a container using Docker API or CLI commands
  - Stateless nature of containers: when a container is deleted, any data written outside of data volumes is lost



- Docker **registry**: stateless server-side application that stores and distributes Docker images
  - Registry is the ***distribute*** component of Docker
  - Public and private registries
    - Docker-hosted registries: Docker Hub (official one), Docker Store (for open-source and enterprise-verified images)
  - Open library of images

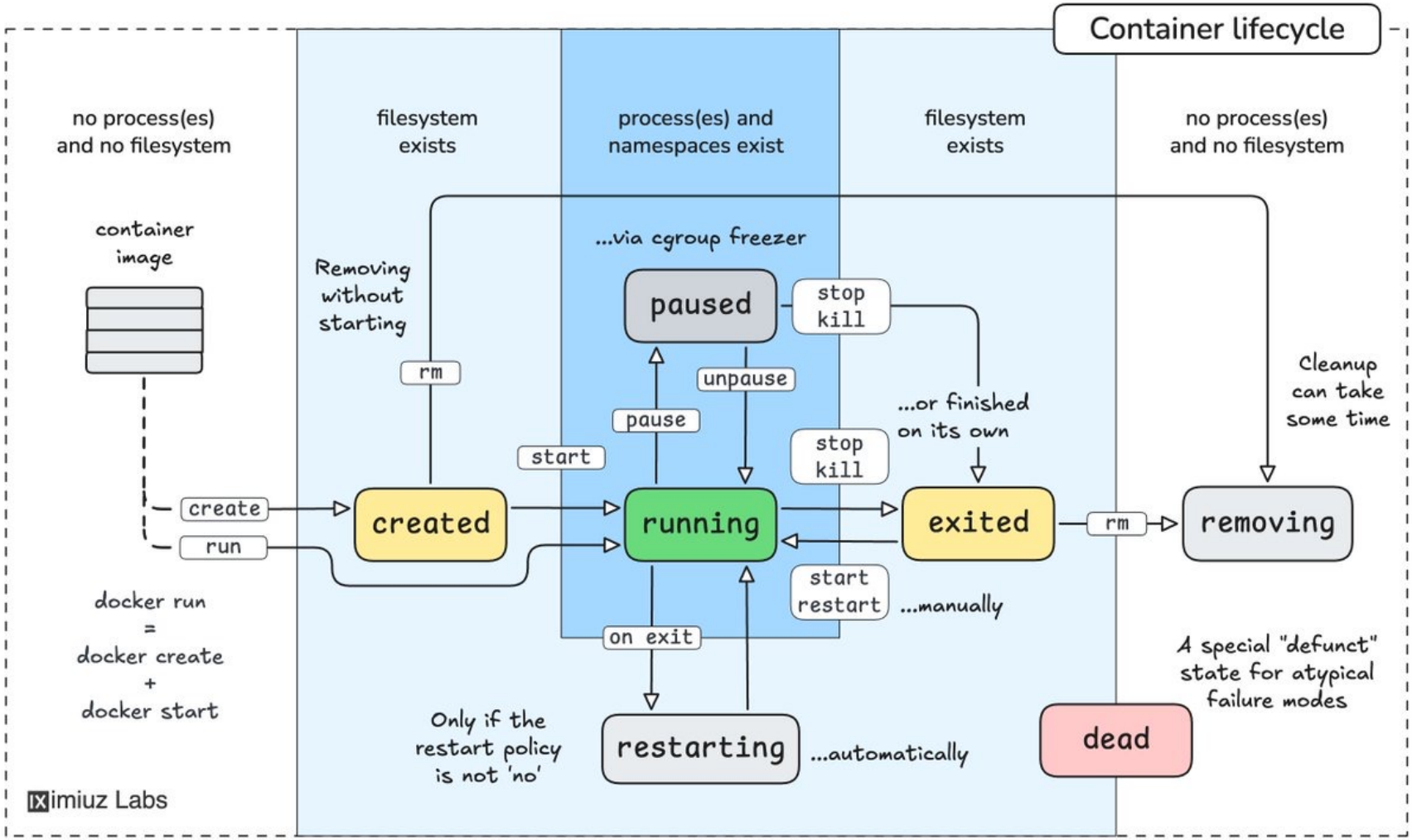
Docker: run container

- When you run a container from an image that is not yet installed locally but is available on Docker Hub, Docker will pull the image from the registry before starting the container



Docker container: states and transitions

- Different states a container can go through during its lifecycle and actions that trigger transitions



Ciclo di vita del container

Docker commands: info

- Get system-wide info on Docker installation

  ```
  $ docker info
  ```

  including:

  - Number of images and containers, along with their status
  - Storage driver
  - Operating system, architecture, total memory
  - Docker registry information

Docker commands: image handling

- List all images on host

  `$ docker images` or `$ docker image ls`

- List all images, including intermediate image layers

  `$ docker images -a` or `$ docker image ls -a`

- You can filter and format the output to list images by name, tag, image digests (sha256) or image that meet specific conditions
  - E.g., list unused images (<none>) that are no longer associated with any tagged images but consume disk space

  `$ docker images --filter "dangling=true"`

- Remove an image

  `$ docker rmi imageid`

  alternatively, `$ docker image rm imageid`

or use *imagename* instead of *imageid*

Docker commands: image handling

- Remove dangling images

```
$ docker image prune
```

- Inspect an image, including layers and image metadata

```
$ docker inspect imageid
```

Docker commands: run

```
$ docker run [OPTIONS] IMAGE [COMMAND] [ARGS]
```

- Common options
  - `--name` set container name
  - `-d` detached mode (background)
  - `-i` interactive (keeps STDIN open)
  - `-t` allocate a pseudo-tty, usually with `-i`
  - `--expose` declare port(s) inside container
  - `-p` or `–publish` map container port(s) to host
  - `-v` mount volume
  - `-e` set environment variables inside container
  - `--rm` automatically remove container when it exits
  - `--restart` set restart policy (e.g., `always`, `on-failure`)

https://docs.docker.com/reference/cli/docker/container/run/

Docker commands: containers management

- List containers
  - Only running containers: `$ docker ps`
    alternatively, `$ docker container ls`
  - All containers (including stopped or killed containers):
    `$ docker ps -a`
- Manage container lifecycle
  - **Stop** a running container
    `$ docker stop containerid`
  - **Start** a stopped container
    `$ docker start containerid`
  - **Kill** a running container
    `$ docker kill containerid`
  - **Remove** a container (after stopping it)
    `$ docker rm containerid`

- Stop and remove a container

```
$ docker ps
$ docker stop containerid
$ docker ps -a
$ docker rm containerid
```

or use *containername* instead of *containerid*

Docker commands: containers management

- Stop all containers

```
$ for i in $(docker ps -q); do docker stop $i; done
```

- Run a command inside a running container

```
$ docker exec [OPTIONS] CONTAINER [COMMAND] [ARGS]
```

Example: `$ docker exec -it mycontainer /bin/bash`

-it è per specificare una shell interattiva

- Inspect a container
  - To get the most detailed view of a container environment

```
$ docker inspect containerid
```

- Copy files between host and container

```
$ docker cp containerid:path localpath
$ docker cp localpath containerid:path
```

la copia sul container mette i dati nel layer effimero, ricordalo

Docker: networking

- Containers communicate with each other or with external systems
- Published ports nel comando docker run
  - Use `-p hostPort:containerPort` (e.g., `-p 8080:80`) in `docker run` to expose a container port externally usato per mappare le porte SUL host
  - Security issue: exposing container ports can be insecure
    - Restrict access to host only by binding to localhost (e.g., `-p 127.0.0.1:8080:80`)
  - Work only on host network: containers can communicate with each other via internal Docker network!
- IP address and hostname
  - Containers get an IP address from Docker's dynamic subnet
  - Docker daemon manages subnetting and IP allocation
  - Default container hostname = container ID (can override with `--hostname`)

Docker: network drivers

- Docker networking is pluggable

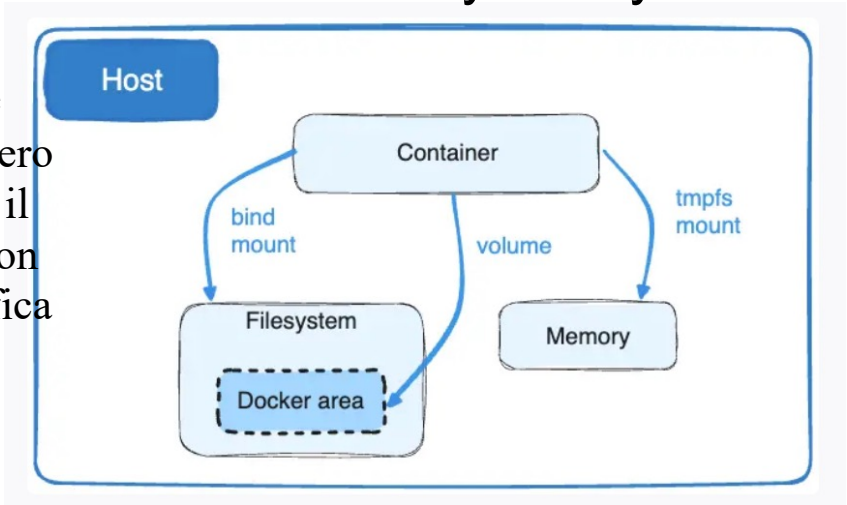
docker utilizza diversi driver per usare la rete interna che collega i container.

  - Docker supports multiple network drivers that provide different networking behaviors, including:
- **`bridge`** (default) utilizzato da docker compose
  - Default network driver when no network is specified
  - Use when containers need to communicate on the same host
  - Implements a software bridge
    - Containers on the same bridge network can communicate with each other
    - Containers on different bridge networks are isolated
- **`host`**
  - Removes network isolation between container and host
  - Container uses host's network stack directly (no private IP assigned)

Docker: volumes

- Preferred mechanism for persisting data generated or used by containers
  - Volume content persists outside the container lifecycle: use volumes over container writable layer for persistent data
  - Does not increase container image size
- How volumes work
  - Docker creates a directory inside its storage directory to manage volume content
  - Default location on Linux: `/var/lib/docker/volumes/`
  - Volumes are created automatically if they do not already exist

esistono anche le soluzioni di bind mount e tmpfs mount per avere memoria persistente il mount bind, non mette a dispoizione un intero file system persistente per il container, ma fa il binding per uno specifico file del container con un file su volume in questo modo ogni modifica sul file persistente si riflette per il container



Docker: volumes

- Mounting a volume: use `-v` or `--volume` in `docker run`

  `$ docker run -v source:destination:[options] imageid`

  - If the volume does not exist, Docker automatically creates it
  - `[options]`: optional flags, e.g., `ro` for read-only volume
- Managing volumes
  - Create a volume: `$ docker volume create volumename`
  - List all volumes: `$ docker volume ls`
  - Inspect a volume: `$ docker volume inspect volumename`
  - Remove a volume: `$ docker volume rm volumename`
- Can also be declared in a Dockerfile using

  ```
  VOLUME ["/localpath"]
  ```
- Working with volume data: pre-populate or load data using `docker cp`

  ```
  docker cp /localpath containerid:/path
  ```

 https://docs.docker.com/engine/reference/commandline/

Docker volume: pros

rendere i container statefull

- ✓ Fully managed by Docker
- ✓ Easy to back up or migrate
- ✓ Accessible through Docker CLI or API
- ✓ Work on both Linux and Windows containers
- ✓ Shareable across multiple containers
- ✓ Can store encrypted content
- ✓ Suitable for pre-populated or write-heavy workloads (e.g., database, logging service)

Docker hands-on

- Download and install Docker
  - Available on multiple platforms

  https://docs.docker.com/get-started
- Test Docker installation

  ```
  $ docker --version
  ```
- Run the default hello-world container

  ```
  $ docker run hello-world
  ```
- Run a “Hello World” message using Alpine Linux

  ```
  $ docker run alpine /bin/echo 'Hello world'
  ```
  - alpine: lightweight Linux distro with very small image size
- Use commands to:
  - List containers and container images
  - Stop and remove containers, remove container images

Docker hands-on: networking

- Run nginx Web server inside a container
  - Bind container port 80 to host port 80

```
$ docker run -dp 80:80 --name web nginx
```

Flag `-p`: publish container port (80) to host port (80)

Flag `-d`: run in detached mode

1. Send HTTP request through Web browser
   - First retrieve hostname of host machine (e.g., localhost)
2. Send HTTP requests to nginx from another interactive container using a custom bridge network

```
$ docker network create -d bridge my_net
$ docker run -dp 80:80 --name web --network=my_net nginx
$ docker run -it --network=my_net --name web_test busybox
/ # wget -O - http://web:80/ contattiamo il container nginx per riceve il file html
/ # exit
```

Docker hands-on: from Dockerfile

- Running Apache web server with minimal index page
  1. Define the container image using Dockerfile
     - Start from Ubuntu, install and configure Apache
     - Declare incoming port 80 using EXPOSE

```
FROM ubuntu:22.04

# Install dependencies
RUN apt-get update -y && \
    apt-get install -y apache2 && \
    apt-get clean

# Add simple web page
RUN echo "Hello World!" > /var/www/html/index.html

# Expose port and run Apache in the foreground
EXPOSE 80
CMD ["apache2ctl", "-D", "FOREGROUND"]
```

da notare che sono 3 comandi in una sola RUN. questo perché ogni RUN crea un layer, quindi facendo così risparmiamo spazio di memoria!

anche se manca sudo, i comandi non sono eseguiti come root!

Docker hands-on: from Dockerfile

2. Build the image

```
$ docker build -t hello-apache .
```

3. Run the container and bind ports

```
$ docker run -dp 127.0.0.1:8080:80 hello-apache
```

4. Execute an interactive shell in the running container

```
$ docker exec -it hello-apache /bin/bash
```

- To reduce container image size, avoid unnecessary layers
  - E.g., in Dockerfile combine `apt-get update` and package installation into a single RUN instruction (see slide 48)

Docker hands-on: volumes

- Run nginx container with a volume

```
$ docker volume create my-vol
$ docker volume ls
$ docker volume inspect my-vol
$ docker run -d \
    --name devtest \
    -v my-vol:/app \
    nginx:latest
```

  - `my-vol` is the source volume, `/app` is the target path inside container

```
$ docker inspect devtest
```

  - Check that Docker has created and mounted the volume correctly

Docker: optimize Docker images

- Fewer layers → smaller images → faster builds and deployments
- Why optimize Docker images?
  - Essential for DevOps engineers at every stage of CI/CD process
  - Reduces image size and disk usage
  - Speeds up image transfer, deployment, and startup times
  - Improves security by reducing the attack surface
  - Best practice used by Google and other major tech companies slim tool di Google per ridurre la dimensione. ad esempio per nginx crea una image di 12.4 MB!
  - Best practice employed by Google and other tech giants

https://devopscube.com/reduce-docker-image-size

Docker: optimize Docker images

- Techniques
  1. Use minimal base images (e.g., alpine, minideb) or distroless base images
     - Distroless images:
       - contain only application and its runtime dependencies
       - include package managers, shells, or other common utilities
       - More secure, smaller, harder to tamper with

       https://github.com/GoogleContainerTools/distroless
  2. Minimize the number of image layers
     - Combine related commands in a single RUN instruction
     - Avoid unnecessary COPY, ADD, or repeated RUN steps

Docker: optimize Docker images

- Techniques
  3. Multistage builds
     - Use intermediate images (build stages) to
       - compile code per utilizzare gli eseguibili direttamente
       - install dependencies, and package files
     - Final image contains only the files and libraries needed to run app

i layer non referenziati da alcuna immagine vengono eliminati una volta rimossa l'iimmagine di build

```
# Build stage
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go mod init myapp
RUN go mod tidy
RUN go build -o myapp

# Final stage
FROM alpine:3.20
COPY --from=builder /app/myapp /usr/local/bin/myapp
CMD ["myapp"]
```

supposto di avere il sorgente GO nella stessa directory del dockerfile. nel container andiamo a fare il build dell'applicazione copiando il sorgente in /app

l'immagine del build stage pesa 850MB mentre l'immagine final pesa solo 10.7MB!

qui creo l'immagine che sarà utilizzata, banalmente copiando l'eseguibile creato nell'immagine precedente nel container che utilizzerò

Docker: optimize Docker images

- Techniques
  4. Exploit layer caching condivisione dei layer. questo però necessità costruire layer riutilizzabili
     - Place instructions that change infrequently (like installing dependencies) before COPY commands
     - Docker reuses cached layers for faster builds when source code changes.
  5. Use a `.dockerignore` file
     - Specify files and directories to exclude from the build context
     - Common exclusions: `node_modules`, `.git`, `*.log`, `__pycache__/`
  6. Keep application data in a volume
     - Avoid storing persistent data inside the container.
     - Use Docker volumes to store databases, logs, uploaded files

Docker: sizing containers

- By default, containers have no resource constraints
  - Can use as much CPU, memory, and I/O as the host's kernel scheduler allows

- Control resources by setting runtime configuration flags of `docker run`
  - Docker uses cgroups to manage resource limits

https://docs.docker.com/engine/containers/resource_constraints

Docker: sizing containers - memory

- Avoid running out of memory (OOM)
  - Containers may be killed
  - Docker daemon has lower OOM score, so less risk than containers
- Enforce hard or soft memory limits
  - Hard limit: container cannot use more than the specified limit; use `--memory` flag
  - Soft limit: container can use more memory if needed, unless certain conditions are met (e.g., kernel detects contention or low memory on host machine)
  - Example: hard limit (500 MB) and soft limit (300 MB)

```
$ docker run –it --memory-reservation="300m" \
  --memory="500m" ubuntu /bin/bash
```

Docker: sizing containers - CPU

- Options to limit CPU usage

  `--cpus=<value>`: limit container to a specific number of CPUs (hard limit)

  `--cpu-quota=<value>:` set Completely Fair Scheduler (CFS) CPU quota on container

  `--cpuset-cpus`: restrict container to specific CPUs/cores, example `--cpuset-cpus="0,1"` (use only CPU 0 and 1)

  `--cpu-shares`: set relative CPU weight (soft limit)

- Example: limit container to use at most 50% of CPU every second

```
$ docker run -it --cpus=".5" ubuntu /bin/bash
```

Alternatively,

```
$ docker run -it --cpu-period=100000 \
            --cpu-quota=50000 ubuntu /bin/bash
```

Resizing containers

- Containers can be resized and migrated, just like VMs

- Resizing allows changing CPU, memory, and I/O limits dynamically
  - Note: on Docker, dynamic resizing is not supported on Windows

```
$ docker update [OPTIONS] CONTAINER [CONTAINER...]
```

  - Examples

```
$ docker update --cpu-shares 512 containerID
$ docker update --cpu-shares 512 -m 300M containerID
```

il resizing non è disponibile su Windows

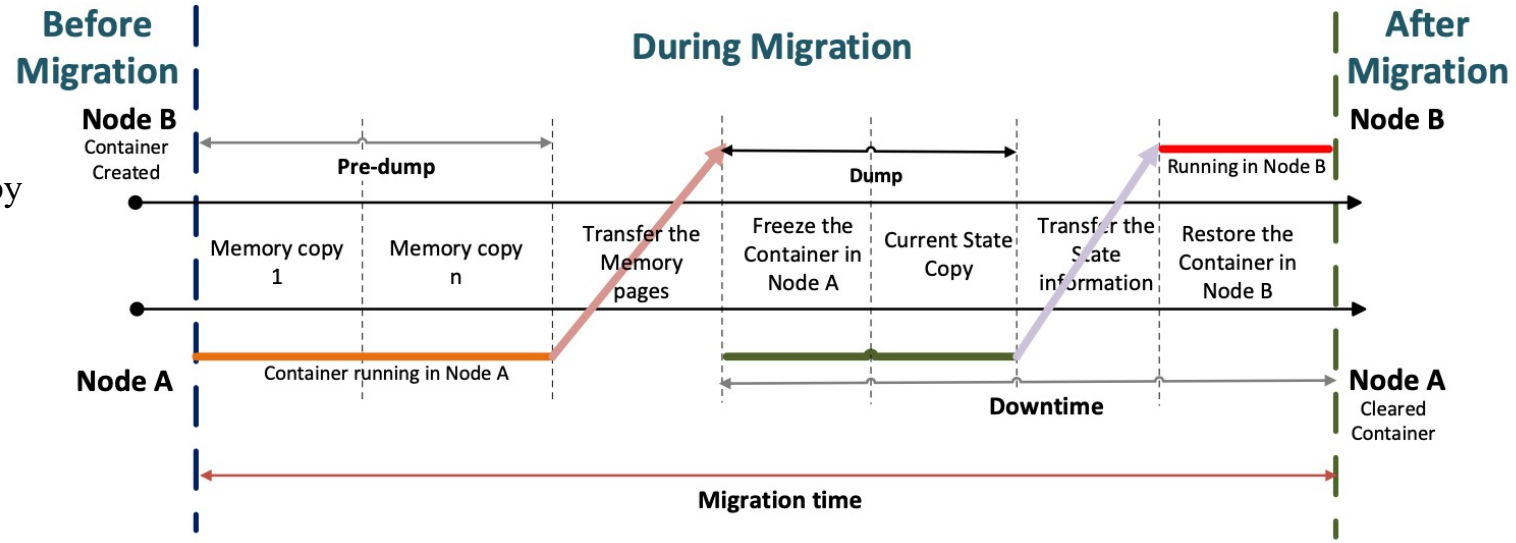
Container live migration

- As for VM migration, we need to:
  - Save state
  - Transfer state
  - Restore state
- State saving, transferring and restoring happen with frozen app: migration downtime
  - Use memory pre-copy or memory post-copy
- No native support in container engines, additional tools required la live migration non è supportata nativamente dal docker engine
- We also need to migrate container image, volumes, and network connections

Container live migration

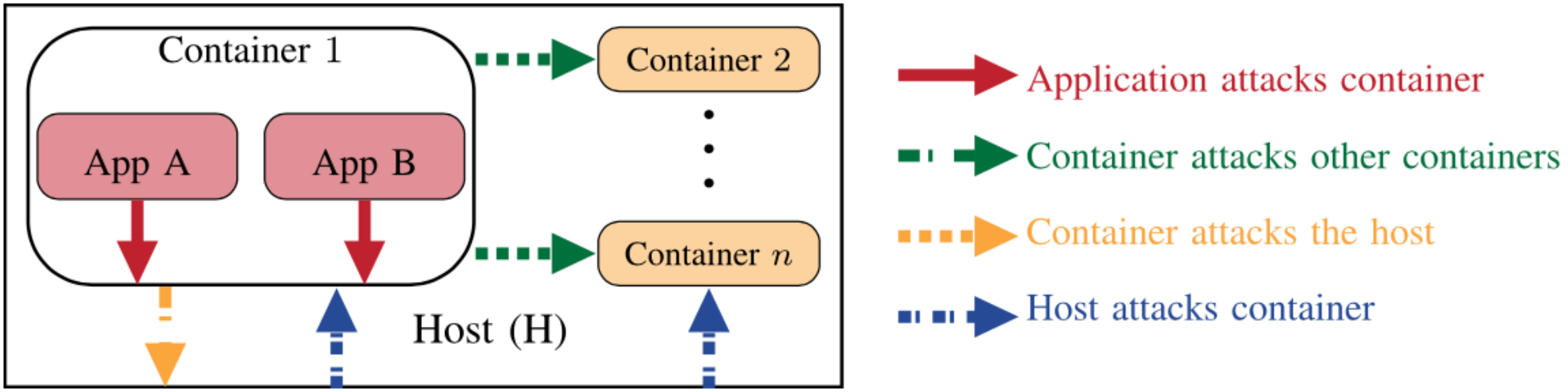
- CRIU (Checkpoint/Restore in Userspace) tool for live migration through checkpointing and restoration (Docker and other engines) https://criu.org
  - Checkpoint: freeze running container on source host and collects information about its CPU state, memory content, and process tree
  - Transfer and restore: transfer collected information to destination host, restore container's state and resume execution https://docs.docker.com/reference/cli/docker/checkpoint

la figura mostra il pre-copy



Container security

- Where attacks come from in containerized environment?



- Attack origins
  - Vulnerabilities in containerized apps
  - Container runtime/kernel flaws

Container security

- Types of attacks
  - Container escape
    - Exploitation of container vulnerabilities to break isolation and access the host system
  - Privilege escalation
    - Once on the host, attackers can escalate privileges to:
      - Access other containers
      - Run malicious code on host system

podman non consente di eseguire in maniera privilegiata il container a differenza di docker

- Consequences
  - Compromised host system
  - Cross-container attacks

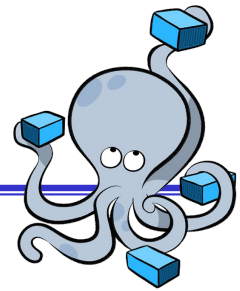
Docker: useful tools

- To manage images
  - Reduce image size: Slim https://slimtoolkit.org
  - Explore image layers: Dive https://github.com/wagoodman/dive
  - Automate image builds: Packer https://www.packer.io
- To monitor containers
  - cAdvisor https://github.com/google/cadvisor
- To check fo vulnerabilities
  - Docker Scout https://docs.docker.com/scout
  - Trivy https://trivy.dev
  - Static analysis: Clair https://github.com/quay/clair
- Many other tools: https://github.com/veggiemonk/awesome-docker

Container orchestration

- Sw platforms for managing multi-container apps
- Functionalities: configure, provision, deploy, monitor, and dynamically control containerized apps
  - Designed to integrate and manage containers in large-scale environments across multiple hosts
  - Can include autoscaling, load balancing, networking, monitoring and logging, fault tolerance and self-healing mechanisms
- Examples per applicazioni multi-host
  - Docker Swarm
  - **Kubernetes** che aiuta a scalare le repliche, il carico di lavoro e gestire politiche di rete oltre ad altre funzioni come ad esempio il self healing
  - Nomad https://developer.hashicorp.com/nomad
- Also available as fully managed Cloud services
- For single-host deployment of small-scale apps
  - **Docker Compose**

Docker Compose



- Tool for defining and running multi-container Docker applications https://docs.docker.com/compose
  - Included with Docker Desktop https://docs.docker.com/compose/install
- How it works invece di fare tutto manualmente con diversi dockerfile e configurazione di rete
  - Configuration-as-code
  - Define services: users specify the containers (services) to be instantiated, their configuration and relationships in a YAML file compose.yaml
- Single host, multiple containers o meglio li chiameremo servizi
  - Orchestrates multiple containers on a single host (single Docker engine) definendo magari dipendenze per l'avvio
- Network setup
  - Compose automatically creates a network and attaches all containers to it, enabling easy communication between them

Docker Compose: how to use

- Define containers in a YAML file named `compose.yaml` (or `compose.yml`)
  - Specifies the containerized services
  - Defines how containers interact and their configurations
- Start Docker composition (background `-d`):

```
$ docker compose up -d
```

  - By default, Docker Compose looks for `compose.yaml` in working directory
    - Use `-f` flag to specify a different YAML file

```
$ docker compose -f composefile up -d
```

- Stop running containers:

```
$ docker compose stop
```

- Bring composition down, removing everything

```
$ docker compose down
```

Docker Compose: Compose file

container

- To configure Docker application's services, networks, volumes, and more
  - Different versions of Compose file format
  - Compose V2: implements format defined by Compose Specification https://compose-spec.io/ and includes support for legacy formats (2.x and 3.x)
- What is inside `compose.yaml`
  - `version`, `services`, `networks`, `volumes`, `configs`, `secrets`
  - Only `services` is required, others are optional

https://docs.docker.com/reference/compose-file

https://github.com/docker/awesome-compose

Docker Compose: Compose file

unica sezione obbligatoria

- Service: abstract representation of computing resources within app, that can be scaled, updated or replaced independently from other components
  - Defines a set of containers
  - Compose file must include the `services` top-level element
- Within each service
  - `build`: defines how to create service image (e.g., from Dockerfile)
  - `container_name`, startup and shutdown dependencies between services (`depends_on`), exposed containers `ports`, CPU and memory limits, `volumes` that are accessible to service containers
  - and other settings, see https://docs.docker.com/reference/compose-file/services/

Docker Compose: example

- Simple Python web app running on Docker Compose
  - 2 containers: Python web app and Redis
  - Use Flask framework and maintain a hit counter in Redis
  - Redis: in-memory, key-value data store

  See https://docs.docker.com/compose/gettingstarted

- Steps:
  1. Write Python app
  2. Define Python container

```
# syntax=docker/dockerfile:1
FROM python:3.10-alpine
WORKDIR /app
ENV FLASK_APP=app.py
ENV FLASK_RUN_HOST=0.0.0.0
RUN apk add --no-cache gcc musl-dev linux-headers
COPY requirements.txt requirements.txt
RUN pip install -r requirements.txt
EXPOSE 5000
COPY . .
CMD ["flask", "run", "--debug"]
```

Docker Compose: example

- Steps (cont'd):
  3. Define services in Compose file
     - 2 services: web (image built from Dockerfile) and redis (official image pulled from Docker Hub https://hub.docker.com/_/redis)

compose.yaml

```
services:
  web: è il primo container
    build: .
    ports:
      - "8000:5000"
  redis: è il secondo container
    image: "redis:alpine"
```

  4. Build and run app with Compose

     ```
     $ docker compose up -d
     ```

  5. Send HTTP requests using curl or browser (counter is increased)
  6. Stop Compose and bring everything down

     ```
     $ docker compose down
     ```

Docker Compose: example

- Specify restart policy for containers nella sezione service
  - Options: `on-failure[:max-retries]`, `always`, `unless-stopped` numero di tentativi per il restart; utile per sapere informazioni sul crash docker logs; occhio comunque ai crash delle applicazioni che rendono uno "zombie" il container che comunque rimane up & running, ma l'applicazione crashata NON riparte
- Start multiple replicas of same service using `deploy` specification
  - Scale out or in manually the number of replicas

```
$ docker compose -f compose_v2.yaml up --scale web=4 -d
```
4 repliche

```
$ docker ps
$ docker compose -f compose_v2.yaml up --scale web=1 -d
$ docker ps
```

✗ Docker Compose only supports manual scaling

- To experience autoscaling, learn Kubernetes

Docker Compose: full example

compose_v2.yaml

vedi anche compose_v3.yaml con load balancer nginx

```
services:
  web:
    build: .
    ports:
      - "5000"
    restart: always
    deploy:
      replicas: 3
    environment:
      - FLASK_APP=app.py
    depends_on:
      - redis
  redis:
    image: "redis:latest"
    container_name: redis
    restart: always
    volumes:
      - redis-data:/data

volumes:
  redis-data:
```

repliche di default se non specificate

volume per mantere persistente il valore del contatore

in questo caso lascio a docker la scelta del mapping della porta del host, in quanto con le repliche ognuna di essi avrà una porta host dedicata non esiste un meccanismo nativo per NON esporre la distribuzione delle repliche. dunque docker compose non supporta la trasparenza di distribuzione. sempre per le repliche non scelgo il nome del container per evitare conflitti.
questa soluzione dunque espone le repliche, dato che per utilizzare ognuno debbo ogni volta cambiare la porta del host nella connessione http. per evitare ciò posso creare un load balancer tramite nginx in un nuovo container che comunicherà con le repliche di flask attraverso il network. infine esporrò solo il load balancer per l'applicazione

Example of Dockerized distributed system

- Kafka cluster using Docker Compose

- Multiple pre-configured options are available, e.g.,
  - https://bitnami.com/stack/kafka/containers includes both single container and Docker Compose setup with Zookeeper or Kraft mode
  - https://medium.com/@darshak.kachchhi/setting-up-a-kafka-cluster-using-docker-compose-a-step-by-step-guide-a1ee5972b122 cluster of 3 brokers (Kraft mode) and UI, Docker network for inter-broker communication, and persistent volume storage

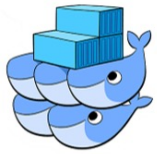
Docker Compose: pros and cons

- ✓ Simplified control
- ✓ Efficient collaboration
- ✓ Rapid application development
- ✓ Reproducible environments
- ✓ Portability across environments (development, staging, production)

esponiamo le repliche senza load balancer

- ✗ Single host limitation
- ✗ Lack of elasticity
- ✗ Not production-grade orchestration almeno su larga e media scala
- ✗ Only basic security features

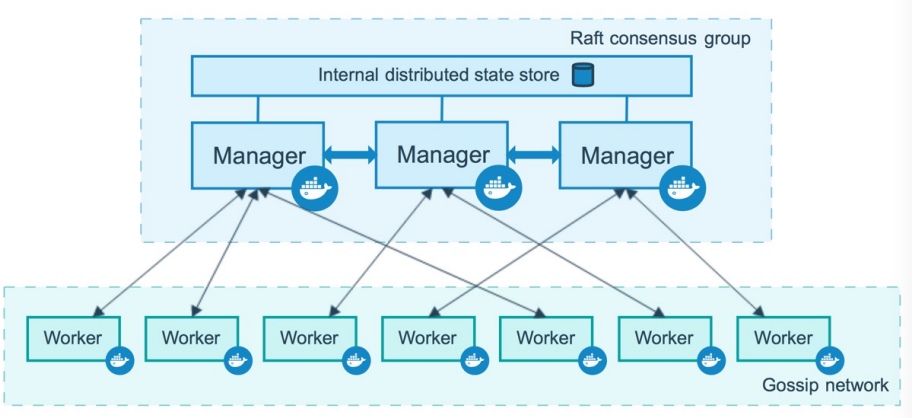
Docker Swarm



- Swarm mode: advanced feature of Docker to natively manage a cluster of Docker engines called a *swarm* https://docs.docker.com/engine/swarm/
- Swarm: multiple Docker engines running in swarm mode swarm è l'orchestratore multi-host di default di docker
  - Swarm mode helps you orchestrate containers across multiple machines sistema distribuito con pattern master-worker
- Composed of Manager nodes and Worker nodes
  - Manager nodes: control the swarm and handle the orchestration of services possone esserci più manager
  - Worker nodes: run containers (tasks) as assigned by manager nodes
  - Tasks: containers running in a service
    - Task: smallest unit of work, typically one container
  - Services: how tasks (containers) should run on the swarm; provide an abstraction for deploying and managing tasks

Docker Swarm: architecture

- Node: single instance of Docker engine in a swarm
  - Manager nodes: handles cluster management, including task scheduling
    - Multiple managers to improve fault tolerance
    - Raft as consensus algorithm to ensure consistency
  - Worker nodes execute tasks
    - Workers use a gossip protocol to exchange information about their state



Docker Swarm: features

- Cluster management integrated with Docker
- Decentralized: distributed decision-making
- Declarative service model
- State reconciliation
  - Swarm monitors cluster state and reconciles any differences wrt desired state (e.g., a node crashes)
- Scaling
  - Easily scale services but lacks auto-scaling
- Multi-host networking
  - Use overlay networks to enable communication between services across nodes
- Load balancing
  - Can expose service ports to an external load balancer
- Secure: TLS authentication, encryption, role-based AC
- Easy to use and lightweight

Container-as-a-Service (CaaS)

termine coniato recentemente dai cloud provider

- Cloud-based platform for managing and deploying containerized applications si possono racchiudere nei servizi PaaS
  - Combines containerization with cloud benefits
- Features
  - Container orchestration and management: container lifecycle, scheduling, load balancing, and fault tolerance
  - Configuration for resource optimization (e.g., auto-scaling)
  - Security and access control
  - Integration with other cloud services (e.g., monitoring)
- Examples
  - Amazon Elastic Container Service https://aws.amazon.com/ecs
  - Azure Container Instances https://azure.microsoft.com/products/container-instances
  - Google Cloud Run https://cloud.google.com/run

Virtualization and IaaS providers

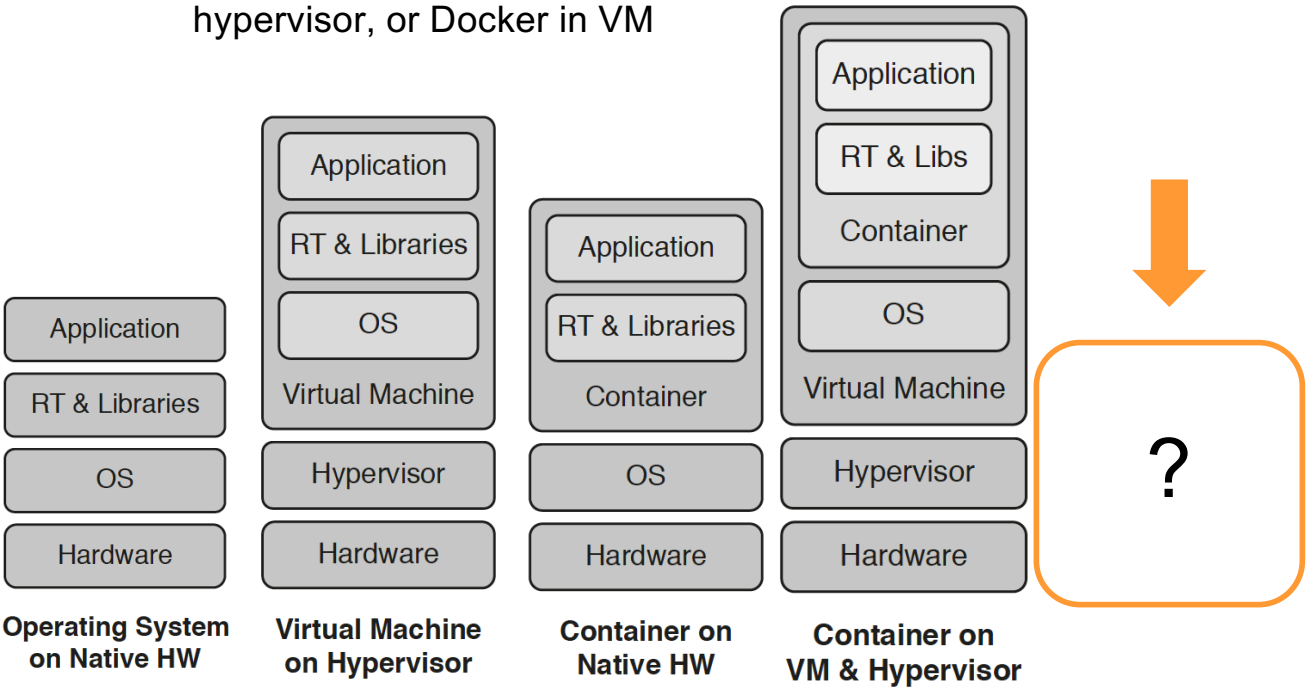
- Which virtualization technology for IaaS providers?
  - ✓ Hypervisor-based virtualization: greater security and isolation, flexibility (different OSs on same PM)
  - ✓ Container-based virtualization: smaller deployment size and higher density, faster startup/shutdown
- Questions
  - Containers on top of bare metal or inside VMs?
    - Performance and density vs. isolation and security
  - Are containers replacing VMs?
    - Not entirely: lightweight vs. isolation and security

più propriamente nel host OS

docker engine ad esempio su Windows e MacOs usa una VM per eseguire i container

New lightweight virtualization approaches

- Deployment approaches examined
  - Plus nested virtualization: hypervisor inside VM on top of an hypervisor, or Docker in VM



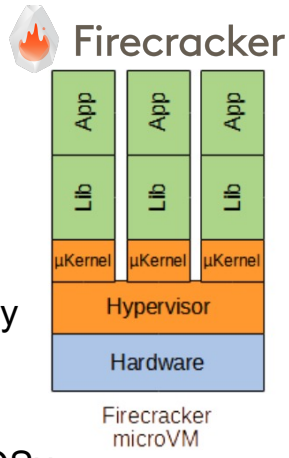
New lightweight approaches to virtualization

- Microservices, serverless computing, cloud-edge continuum demand for
  - Low-overhead (or lightweight) virtualization techniques, even lighter than containers
  - Better security
  - Portability across OSs and architectures (e.g., Arm, Intel)
- Technologies enabling lightweight virtualization: **MicroVM**, **lightweight OSs**, **unikernels** and **WebAssembly**
- MicroVMs, lightweight OSs, and unikernels: reduce OS overhead and attack surface obiettivo condiviso dalle tecniche
  - OS overhead: many common OS services (shells, editors, core utils, package managers) are unnecessary
  - Attack surface: images contain only the essential code needed to run the app, reducing potential attack vectors

MicroVM runtimes

- Tiny, specialized VMM that run lightweight VMs (called microVMs)
- Goal: reduce memory footprint and improve security of virtualization layer
- Firecracker: minimalist VMM purpose-built by Amazon for secure, efficient and multi-tenant microVMs
  https://firecracker-microvm.github.io
  - Why? To enable AWS Lambda and AWS Fargate for serverless and containerized workloads
  - Based on KVM but with minimalist design (no unnecessary devices and guest functionality)
  - Open source, written in Rust
  - microVM: <125 ms startup time and <5 MB memory footprint
  - Scales to thousands of multi-tenant microVMs
  - Supported OS guests inside microVM: Linux and OSv

Google usa gVisor



OSv è un sistema operativo ad-hoc realizzato per le VM

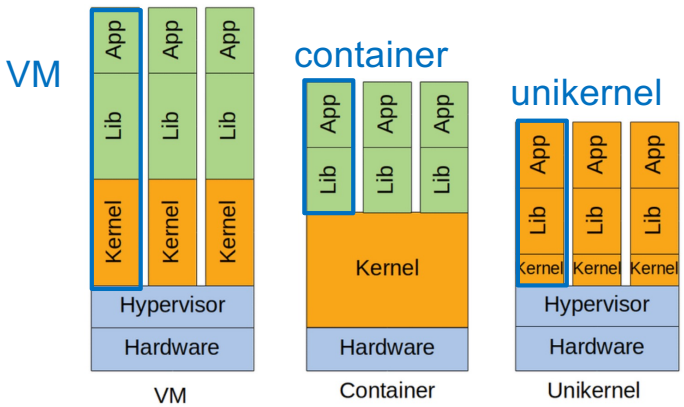
Lightweight operating systems

- Minimal, special-purpose OSs to run containerized apps
- Fedora CoreOS https://fedoraproject.org/it/coreos/
  - Minimal, monolithic, and compact Linux distribution designed for running containers
    - Only components for container deployment, together with built-in tools for service discovery, container management, and configuration sharing
  - Designed for scale and security
  - Fast bootstrap and small memory footprint
  - Can be installed directly on hardware or on hypervisor
  - Includes Docker and podman
- Other products: Ubuntu Core, balenaOS
  - Designed for edge and IoT devices

Unikernels

- Specialized, single-purpose OS designed to run a single application with minimal overhead
  - Single application + OS into a single executable (aka library OS): monolithic process that runs entirely in kernel mode
  - Single address space: app and OS share the same memory space
  - Built by compiling high-level language directly into specialized machine image that runs on hypervisor
  - Goal: isolation benefits of hypervisor without overhead of guest OS

applicazione e kernel condividono lo stesso spazio di indirizzamento eliminando cosi tutti i context switch da user mode a kernel mode. si ha solo la kernel mode



Unikernels: pros and cons

- Pros (specialized → high performance)
  - ✓ Lightweight: less resource-intensive (memory and CPU)
  - ✓ Minimal footprint
  - ✓ Reduced attack surface
  - ✓ Fast execution (no context switching)
  - ✓ Fast boot (measured in ms)
  - ✓ Strong isolation
- Cons
  - ✗ Less flexible than VMs and containers
  - ✗ Limited ecosystem support, including debugging and monitoring tools che rende appunto debugging e monitoraggio difficile
  - ✗ Scalability is more challenging (e.g., multi-instance deployments or load balancing)
  - ✗ Unikernel orchestration is not as easy as container orchestration

Unikernels: frameworks

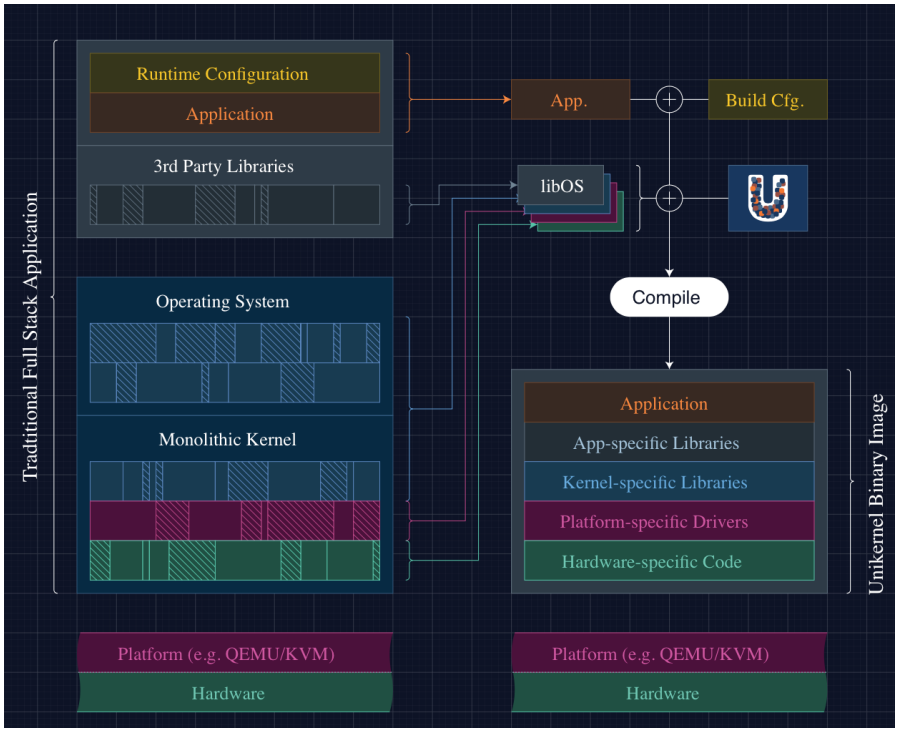
- Frameworks
  - MirageOS (Ocaml language) https://mirage.io
  - OSv https://github.com/cloudius-systems/osv
  - Nanos https://nanos.org/
  - Unikraft
- OSv
  - Cloud-native unikernel optimized for high performance and low overhead in virtualized environments
  - Linux ABI compatibility allows running Linux applications with minimal changes
  - Open-source and fast
    - Can boot in ~5 ms on Firecracker using 11 MB of memory

Unikernels: Unikraft

- Fast, secure, and open-source Unikernel Development Kit https://unikraft.org
  - Designed to simplify unikernel creation without requiring deep expertise
  - Build, run and package: similar to Docker
  - Modular: supports a wide range of components (e.g., networking, storage), allowing to create custom unikernels
  - Multi-language support (e.g., C, C++, Rust, Go)
  - Application compatibility
    - Can run complex apps (e.g., Redis, Nginx, Memcached)
    - POSIX compliant: compatibility with a broad range of Unix-like applications
  - Architecture compatibility
    - Works with multiple hypervisors (e.g., Xen, KVM) and supports various CPU architectures, allowing deployment on both virtualized and bare-metal environments
  - Active development

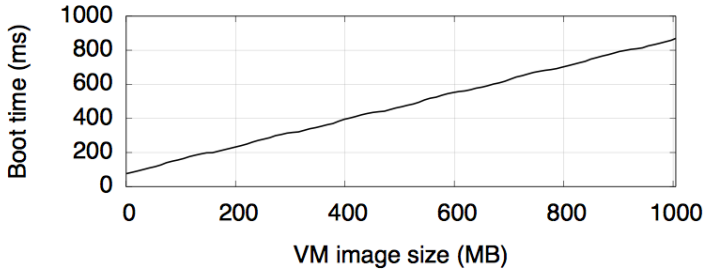
sistema di storage key-value alternativa a redis

Unikernels: Unikraft

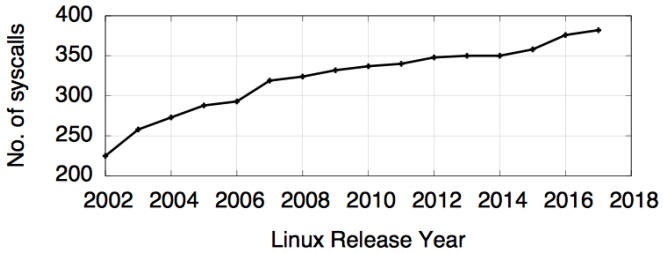


Performance of virtualization approaches

- VM boot times grow linearly with VM size



- Difficulties in securing containers due to growth of Linux syscall API



My VM is lighter (and safer) than your container, SOSP 2017
https://dl.acm.org/doi/pdf/10.1145/3132747.3132763

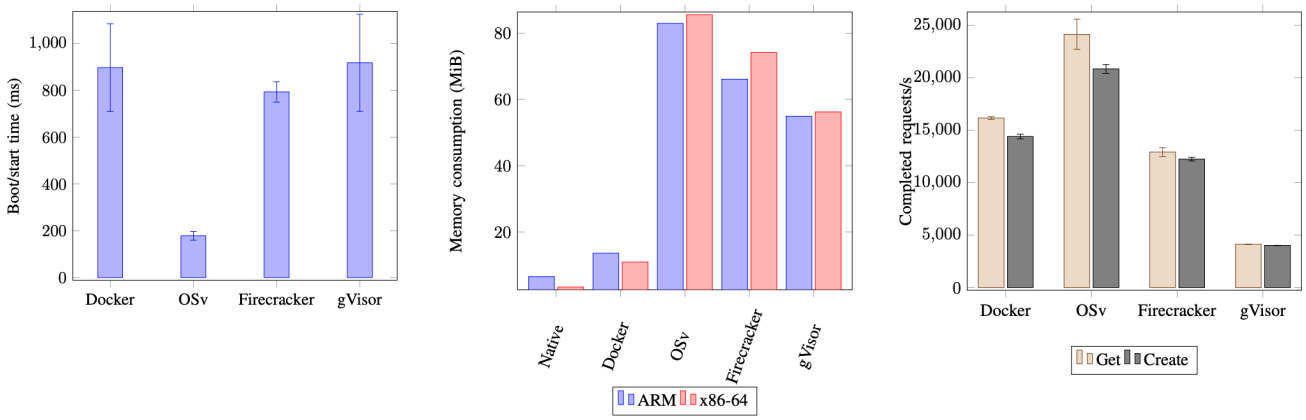
Performance of virtualization approaches

- Performance comparisons of hypervisor vs. lightweight virtualization
- Key findings:
  - Overhead introduced by containers is almost negligible
  - Fast instantiation time of containers
  - Small per-instance memory footprint
  - High density
  - But security tradeoffs: containers offer less isolation

Virtualization	Boot time	Image size	Memory footprint	Programming language dependance	Live migration
VM	~5/10 sec	~1 GB	~100 MB	No	Yes
Container	~0.8/1 sec	~50 MB	~5 MB	No	Non-native
Unikernel	<10 msec	<20 MB	~10 MB	Partially	No

Performance of virtualization approaches

- Comparing lightweight virtualization approaches
- Overall result: no clearly superior solution, each one has its own strengths and weaknesses



A functional and performance benchmark of lightweight virtualization platforms for edge computing, EDGE 2022 https://ieeexplore.ieee.org/document/9860335

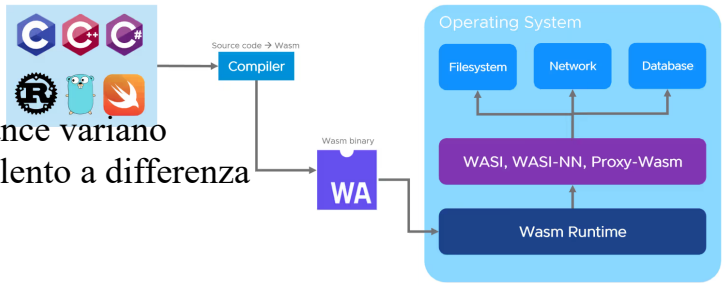
WebAssembly (Wasm)

- Safe, portable, binary code format designed for efficient execution and compact representation https://webassembly.org
  - Safe: runs in a sandboxed environment, preventing untrusted code from harming the host system
  - Portable: the same Wasm binary can run on any platform that supports the Wasm runtime (browsers, servers, embedded)
  - Binary format: designed to be compact and fast to load and execute
- Other features
  - Open standard https://www.w3.org/TR/wasm-core-2
  - Portable compilation target for many programming languages
    - Originally built to safely execute JavaScript code in browsers
  - Memory-safe, sandboxed execution
  - Computational model based on stack VM come la JVM

WebAssembly: features

- Wasm code is validated and executed in a memory-safe, sandboxed environment → strong isolation and protection from unsafe operations
  - Wasm interacts with the host system through WebAssembly System Interface (WASI), which provides a standardized set of capability-based APIs
  - A Wasm module cannot directly perform OS systems calls, instead imports host-provided WASI functions
- Development workflow: write code in one of many supported languages, compile it to Wasm, and run it inside a Wasm runtime

nel caso della virtualizzazione



a seconda del linguaggio le performance variano terribilmente, con python che risulta lento a differenza di C, Rust o Go

WebAssembly: features

- Wasm uses stack-based VM to execute instructions https://en.wikipedia.org/wiki/Stack_machine
  - Code is composed of sequences of instructions executed in order
  - The operand stack is used to store values for computation
  - Instructions:
    - Pop argument values from the stack and push back onto it
    - Each operation manipulates values on the stack
    - Example: `i64.add`
      - Takes two i64 values from the stack
      - Add them
      - Pushes the result back onto the stack
    - Control instructions alter control flow
- JVM is a famous example of stack-based VM

che appunto produce bytecode

WebAssembly: example

- Factorial function written in C and its corresponding Wasm code after compilation
  - In `.wat` text format (human-readable textual representation of Wasm)

```c
int factorial(int n) {
  if (n == 0)
    return 1;
  else
    return n * factorial(n-1);
}
```

```
(func (param i64) (result i64)
  local.get 0             # put arg[0] on stack
  i64.eqz                 # compare top of stack to zero
  if (result i64)         # if it is zero
      i64.const 1         # put 1 on stack
  else
      local.get 0         # put arg[0] on stack
      local.get 0         # put arg[0] on stack
      i64.const 1         # put 1 on stack
      i64.sub             # subtract the top 2 values in stack
      call 0              # Call function #0 (return value is on the stack)
      i64.mul             # Multiply
  end)'
```


WebAssembly: example

- Factorial function written in C and its corresponding Wasm code after compilation
  - In `.wasm` binary format

```c
int factorial(int n) {
  if (n == 0)
    return 1;
  else
    return n * factorial(n-1);
}
```

```
00 61 73 6D 01 00 00 00
01 06 01 60 01 7E 01 7E
03 02 01 00
0A 17 01
15 00
20 00
50
04 7E
42 01
05
20 00
20 00
42 01
7D
10 00
7E
0B
0B
```

WebAssembly: pros and cons

- ✓ Efficient: near-native execution speed
- ✓ Secure: memory-safe, sandboxed execution, which prevents data corruption and security breaches
- ✓ Language-, platform-, and hardware-independent
  - Does not favour any particular language
  - Can run as a standalone VM
  - Can be compiled for all modern architectures, including desktop, mobile devices, and embedded systems
- ✗ In development
- ✗ Support varies by language
- ✗ Multiple runtimes (e.g., Wasmtime, Wasmer, WasmEdge) with different features: choice is complex

Kubernetes oggi supporta il Wasm

non c'è uno standard! e i differenti runtime non sono completi

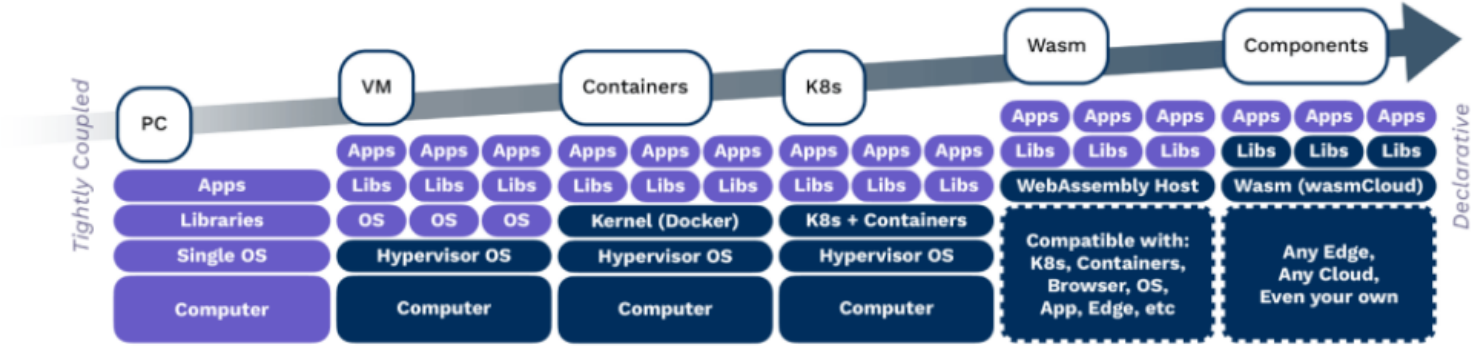
non è supportata nativamente la comunicazione tra due applicazioni di wasm!

WebAssembly

- How to try: Wasm applications with Linux containers in Docker (beta)
  https://docs.docker.com/desktop/features/wasm/
  - Enable it on Docker Desktop by checking *Enable Wasm* on the *Features in development* tab under *Settings* (requires containerd image store)

```
$ docker run \
  --runtime=io.containerd.wasmedge.v1 \
  --platform linux/arm64 \
  secondstate/rust-example-hello
```

The full scenario



FORMAT	PC	CLOUD	CONTAINER	K8S	WASM	COSMONIC
EXECUTION	*Image (Datacenter)*	*VM (Public Cloud)*	*Container (Docker)*	*Containers (K8s / Cloud)*	*WASM (Everywhere)*	*Distributed WASM (Everywhere)*
Dev Responsibility	Full	OS, App, Lib	App, Lib	App, Lib	Wasm	Business Logic
Abstraction	-	CPU	Linux Kernel	K8s	Secure Sandbox	Sandbox + Capabilities
Compatibility	All	Most	Most	Most	Most	Most
Size	Large	Med	Small	Small	Tiny	Minuscule
Portability	-	Low	Med (CPU, Linux)	Med (CPU, Linux)	High	Highest
Security	System	OS	Process Boundary	Process Boundary	Capability	Component
Location	On Prem & Co-location	Proprietary Cloud & Edge	Dev, Edge, Cloud, K8s	Dev, Edge, Cloud, K8s	Dev, App, Edge, Cloud, K8s, Browser, Devices	Dev, App, Edge, Cloud, K8s, Browser, Devices

Legend: **Developer Provided** **Service Provided** **Flexible**

Definizione di microservizi di Martin Fowler:
In short, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around business capabilities and independently deployable by fully automated deployment machinery.
There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.

Microservices

come visto nella fase iniziale del corso

- Architectural style for distributed applications
- Structures an application as a collection of loosely coupled services

system oriented architecture

- Not entirely new: derives from **SOA** and **Web services**
  - But with some significant differences
- Focuses on how to build, manage, and evolve architectures composed of small, self-contained units
- Key characteristics
  - Modularization: application decomposed into a set of independently deployable services, that are loosely coupled and cooperating

si passa da applicazioni monolitiche a collezioni di servizi indipendenti che collaborano

implementati come visto tramite container e sfrutta RPC o il pattern pub-sub per comunicare e collaborare

  - Rapid deployment and scalability
  - Data management: services typically own their memory persistence layer (e.g., relational DBs and NoSQL data stores)

MongoDB, redis ecc.

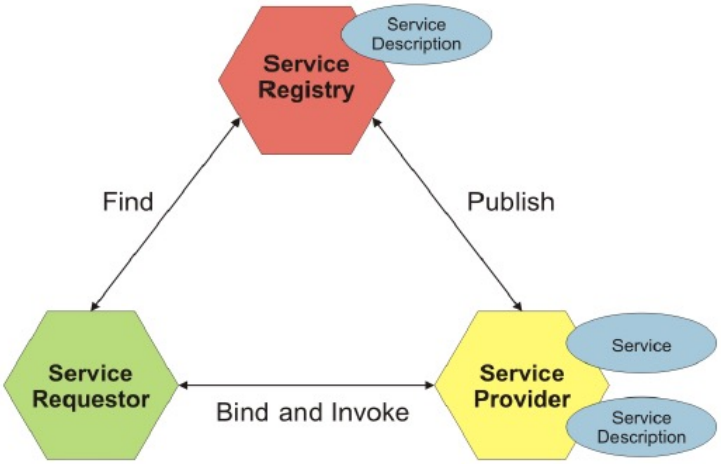
Service Oriented Architecture (SOA)

- Architectural paradigm for designing loosely coupled distributed sw systems
- Definition https://docs.oasis-open.org/soa-rm/v1.0/soa-rm.pdf

  SOA is a paradigm for organizing and utilizing distributed capabilities that may be under the control of different ownership domains. It provides a uniform means to offer, discover, interact with and use capabilities to produce desired effects consistent with measurable preconditions and expectations
- Properties of SOA https://www.w3.org/TR/ws-arch/
  - Logical view of services
  - Message-oriented and description-oriented
  - Service granularity and network orientation
  - Platform neutral

SOA: Core entities

- 3 interacting entities
  1. *Service requestor* (or *consumer*): requests service execution
  2. *Service provider*: implements and exposes the service
  3. *Service registry*: publishes the service and enables discovery



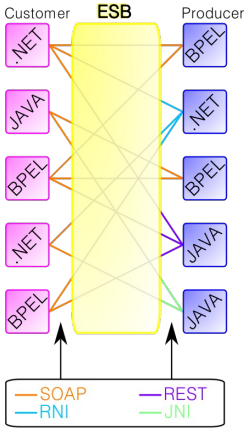
Web services

- Web services: implementation of SOA
- Definition https://www.w3.org/TR/ws-arch/
  - Web service: software system designed to support interoperable machine-to-machine (M2M) interaction over a network
  - Web service interface described in a machine-processable format tramite un ecosistema complesso
  - Other systems interact with web service in a manner prescribed by its description using SOAP messages, typically conveyed using HTTP

la complessità del ecosistema è stato anche il tallone d'achille della tecnologia

Web services

- Large ecosystem of standards and specification (more than 60), among which: nei microservices avremo le API
  - Service description : **WSDL** (Web Service Description Language) basato su xml
  - Communication: **SOAP** (Simple Object Access Protocol) che sta al di sopra di http o ftp
  - Service registry: **UDDI** (Universal Description, Discovery and Integration)
  - Business process definition: **BPEL** (Business Process Execution Language), **BPMN** (Business Process Model and Notation)
  - SLA: **WSLA**
- Wide variety of technologies
  - Including **ESB** (Enterprise Service Bus): integration platform for routing, mediation, transformation, and communication; commonly used in enterprise SOA systems



SOA vs. microservices

- Heavyweight vs. lightweight
  - SOA: heavyweight middleware (e.g., ESB)
  - Microservices: lightweight technologies
- Protocols
  - SOA: web services protocols
  - Microservices: RESTful APIs and HTTP, often JSON-based
- Architectural view
  - SOA: integration solution in ambiti enterprise
  - Microservices: to build complete applications in ambiti enterprise
- Data ownership
  - SOA: often shared databases or centralized data models
  - Microservices: each service owns its own data store
- Deployment and lifecycle
  - SOA: services often deployed together or coordinated
  - Microservices: independent deployment and versioning

Microservices and containers

- Microservices are an ideal complement to container-based virtualization
  - "One microservice instance per container" il dogma da ricordare
    - Each microservice packaged as a container image
    - Each instance deployed as a container
  - Containers enable runtime management (scaling, migration)
- Pros and cons:
  - ✓ Scale out/in by changing the number of container replicas
  - ✓ Scale up/down by adjusting container resources
  - ✓ Isolation of microservice instance
  - ✓ Resource limits per service
  - ✓ Fast build and startup
  - ✗ Require container orchestration to manage multi-container applications che nel caso semplice del singolo host è docker compose mentre nel caso di multipli host docker swarm o meglio kubernetes

Microservices: benefits

- Increased software agility
  - Each microservice is an independent unit of development, deployment, operation, versioning, and scaling
  - Encapsulation via APIs: interaction occurs through well-defined APIs that hide implementation details
  - Container-based virtualization
- Improved scalability and fault isolation
- Increased reusability across business domains
- Improved data security
- Faster development and delivery
- Greater team autonomy

Microservices: concerns

- Increased network traffic
  - Remote service calls introduce network latency
- Higher system complexity
- Increased operational complexity
  - Deployment, monitoring, and management
- More difficult testing and debugging
  - End-to-end and distributed testing are harder

Microservices and scalability

- How to achieve scalability of microservices?
  - Run multiple instances of the same microservice
  - Load balance requests across instances

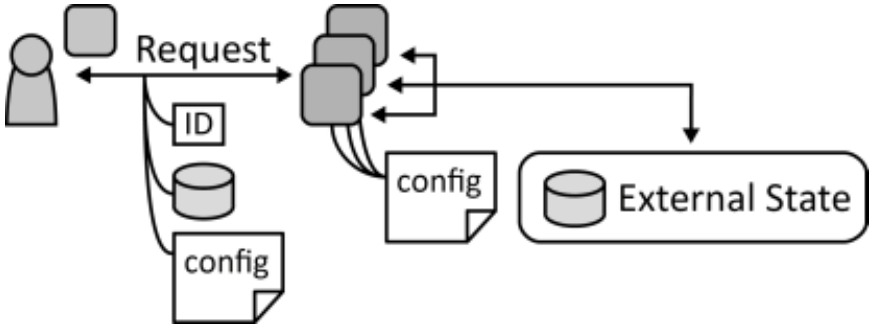
IMPORTANTE

- Prefer **stateless services**
  - No state stored in the service instance
  - Scale faster and more easily than stateful services

anche perché docker come visto ha uno strato di write non persistente al suo interno
ma gestisce la persistenza con i volumi

Stateless service

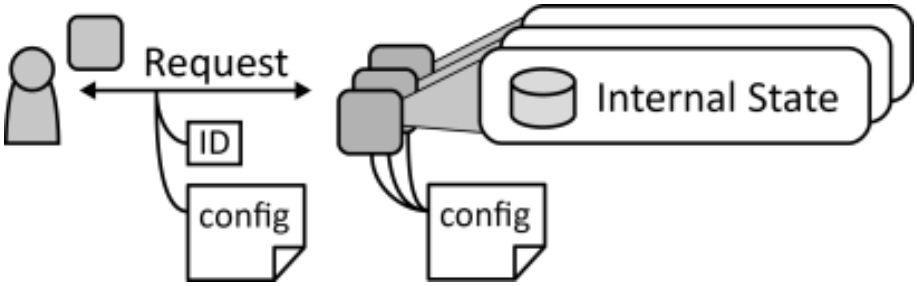
- Does not store client or session state internally
- State handled externally (e.g., database, cache)
- Easier to scale out
- More tolerant to service failures



https://www.cloudcomputingpatterns.org/stateless_component/

Stateful service

- Stores state internally
- Multiple instances of a scaled-out service must synchronize state problema nei DB come si ben sa
- Required to provide consistent, unified behavior
- State management adds complexity and latency
- How can multiple service instances maintain a synchronized internal state while scaling?



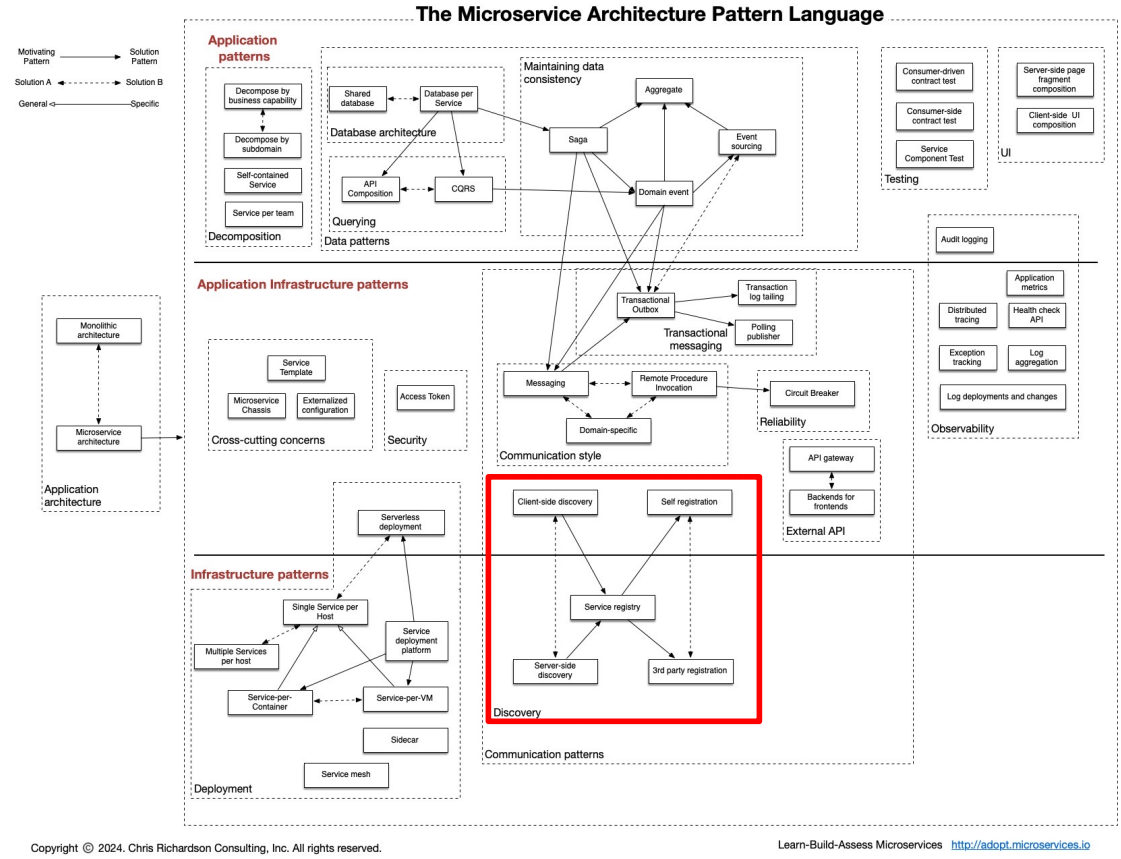
https://www.cloudcomputingpatterns.org/stateful_component/

Stateful service: scaling approaches

- Centralized storage
  - All instances read/write state to a shared database or cache
- State replication
  - Replicate state across instances
- Event sourcing / CQRS patterns
  - Store changes as events; rebuild state per instance
- State partitioning (or sharding)
  - Divide state into independent partitions
  - Each instance handles a subset of the state
- Sticky sessions (less used)
  - Route requests from the same client to the same instance
- Tradeoffs
  - Adds operational complexity
  - Can limit elastic scalability compared to stateless services

l'architettura a microservizi consiste in vari pattern che vengono implementati

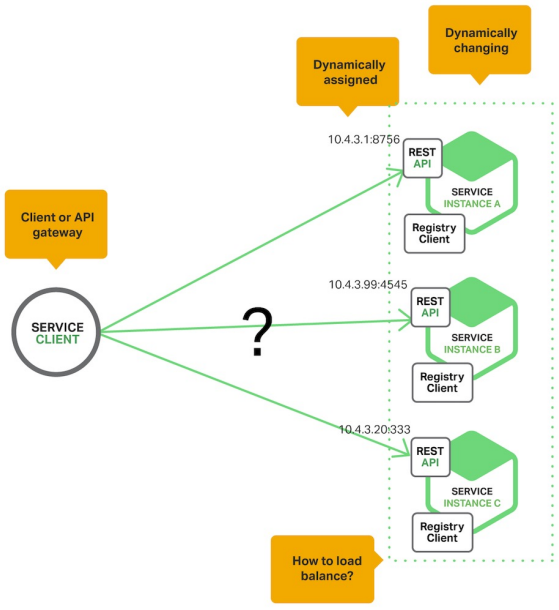
Microservice patterns



a noi ci interessa la discovery

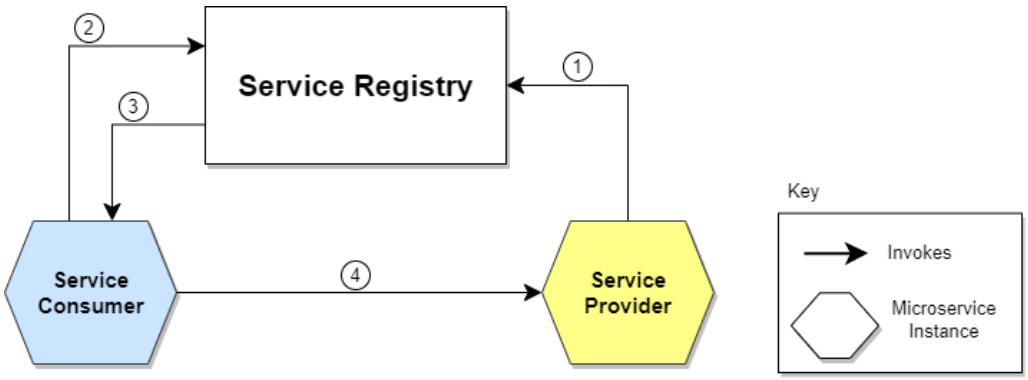
Service discovery

- Microservice clients need to locate service instances
  - Instances have dynamic network locations (IP address, port)
  - The set of instances changes due to auto-scaling, failures, upgrades
- Service discovery provides
  - Registration: microservice instances register themselves when they start
  - Lookup / resolution: clients can find service instances dynamically



Service discovery: patterns

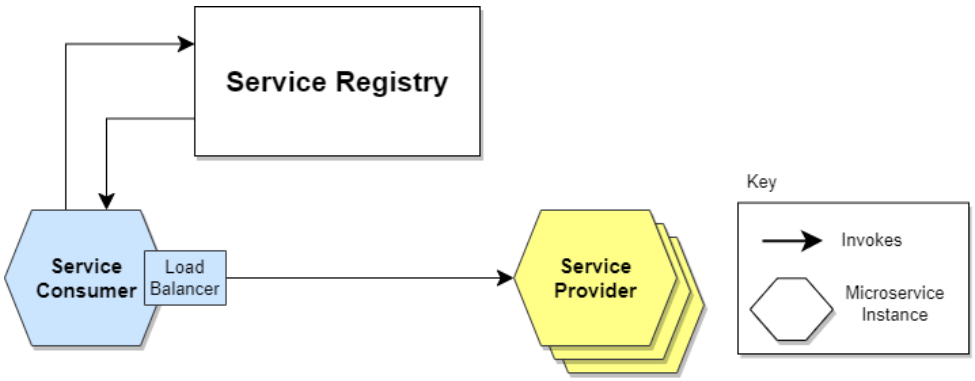
1. Service registry come visto in RPC per SunRPC
   - A database of services, instances and network locations
   - Instances register at startup and deregistered at shutdown
   - Clients query the registry to find available instances



Service discovery: patterns

2. Client-side service discovery
   - Client determines service network location and load balances requests among them
   - Client queries Service Registry, uses a load-balancing algorithm to pick an instance, and sends request directly to chosen instance

   https://microservices.io/patterns/client-side-discovery.html

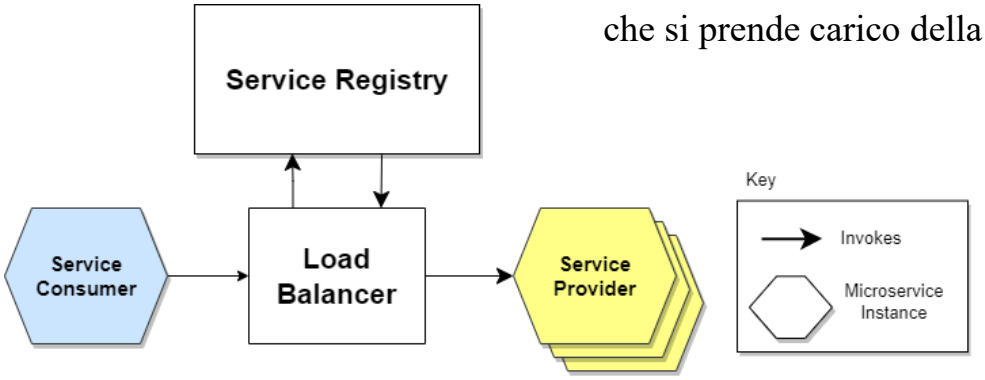


Service discovery: patterns

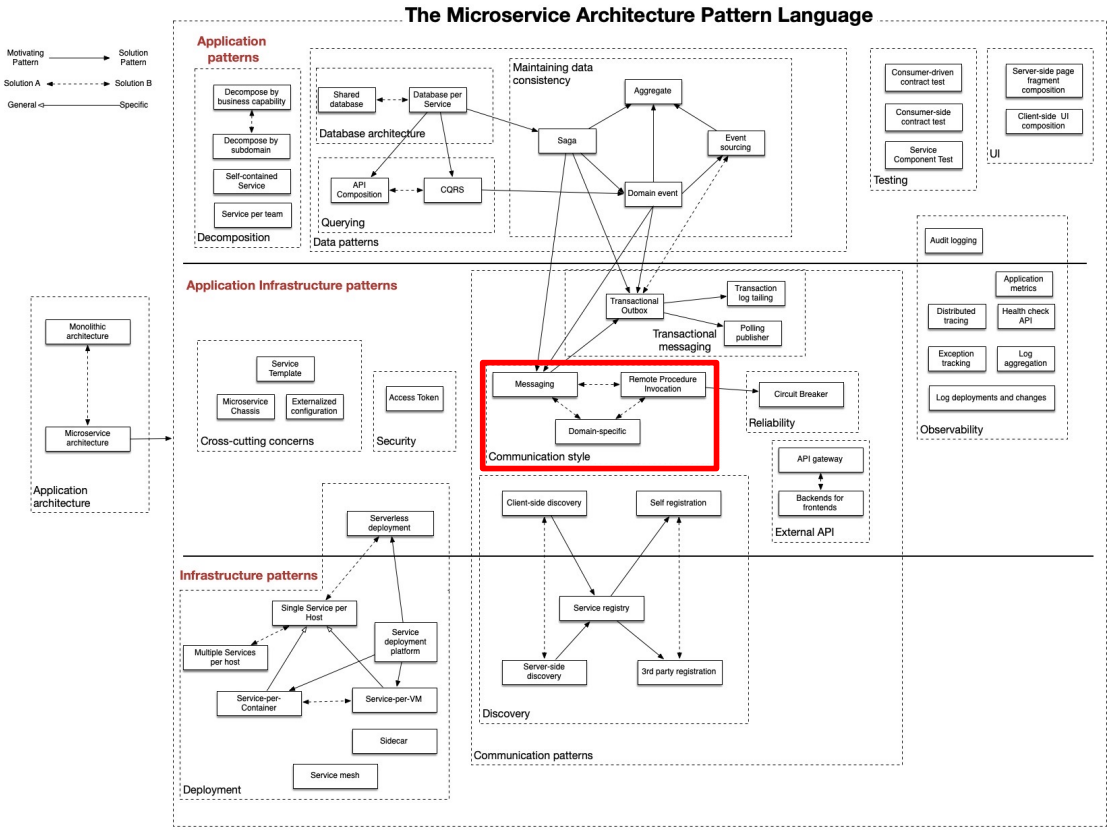
3. Server-side service discovery
   - Client sends request to load balancer at a known location
   - Load balancer queries Service Registry and routes request to an available instance

   https://microservices.io/patterns/server-side-discovery.html

load balancer come intermediario che si prende carico della richiesta



Microservice patterns



Copyright © 2024. Chris Richardson Consulting, Inc. All rights reserved.

Learn-Build-Assess Microservices http://adopt.microservices.io

c'è poi lo stile di comunicazione tra i microservizi

Communication styles

- Synchronous communication
  - Request/response style
  - Mechanisms: HTTP/REST, RPC
  - Typically, one-to-one interaction
  - ✗ Can reduce availability if service is slow or down

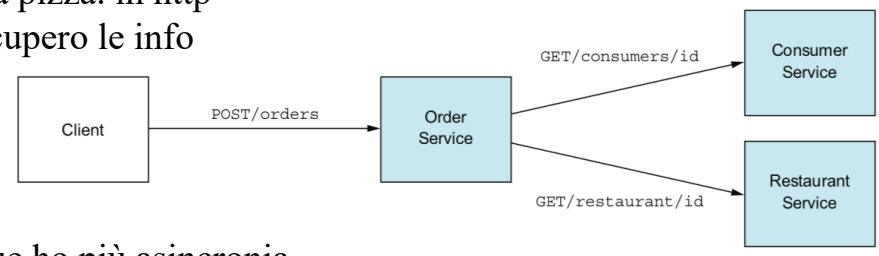
- Asynchronous communication
  - Event-driven or message-based
  - Mechanisms: pub/sub systems, message queues, related protocols
  - Supports one-to-one or one-to-many interaction
  - ✓ Improves resilience and decoupling

già visti tramite Kafka ad esempio

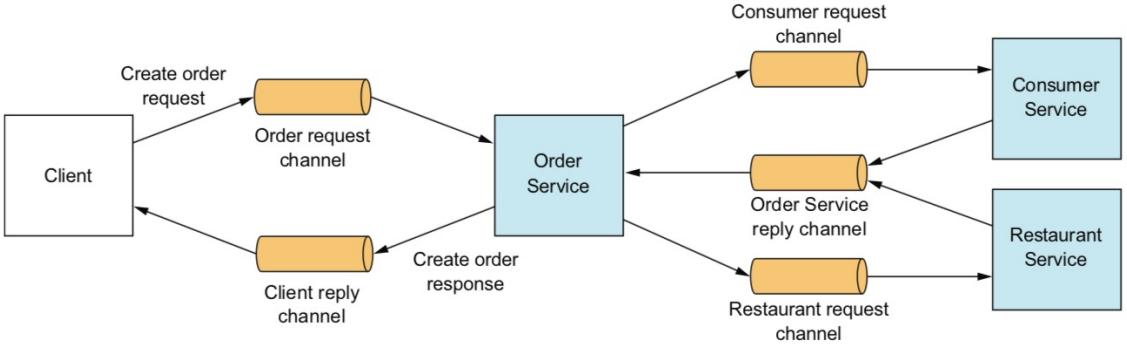
Communication styles

- Example of synchronous vs. asynchronous communication

client che ordina una pizza. in http faccio una post e recupero le info tramite le get

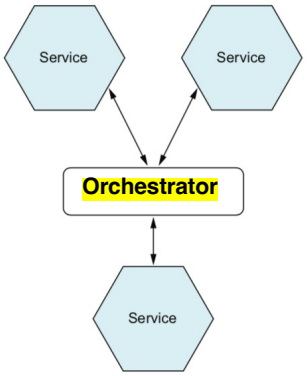


con un message-queue ho più asincronia



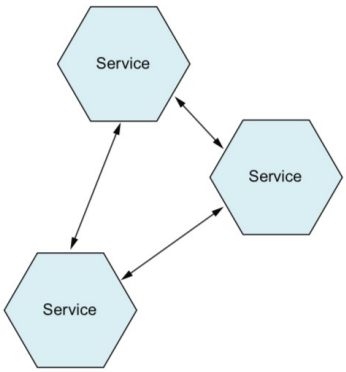
Service interaction

- Microservices can interact according to 2 patterns:
  - Orchestration
  - Choreography
- **Orchestration**: centralized approach
  - A single centralized process (*orchestrator*, *conductor* or *message broker*) coordinates interactions
  - Orchestrator invokes and combines services, which may be unaware of the composition



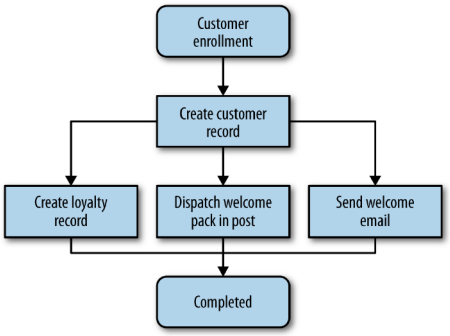
Orchestration and choreography

- **Choreography**: decentralized approach
  - Interaction defined by exchange of messages, rules, and agreements between services
  - Services react to events/messages directly, without a central coordinator



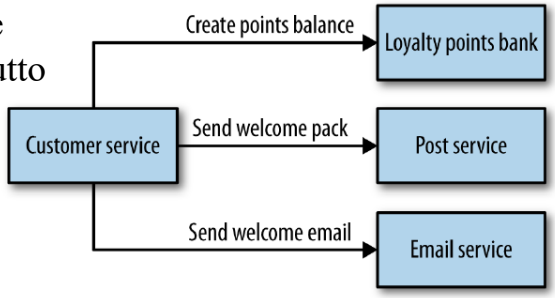
Orchestration and choreography: example

- Workflow for customer creation



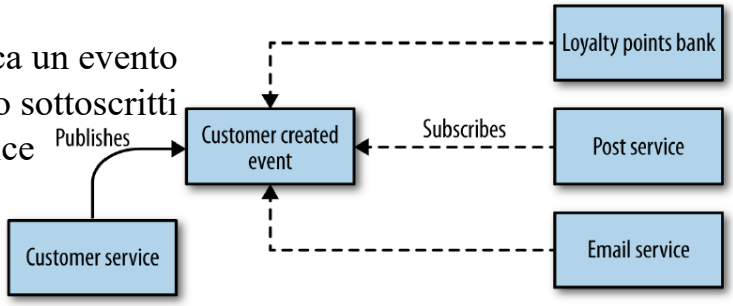
Orchestration

un service che si occupa di tutto



Choreography

si pubblica un evento a cui sono sottoscritti altri service



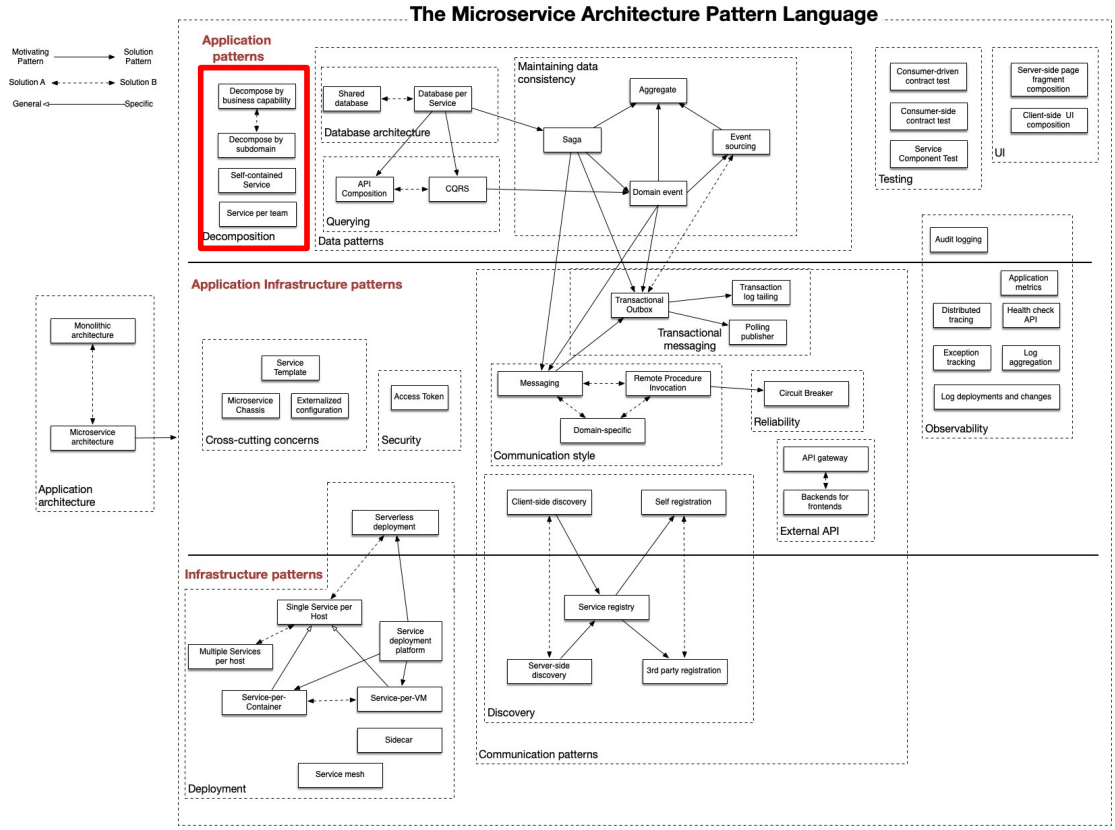
Source: S. Newman, "Building Microservices", O'Really, 2015

Orchestration vs choreography

- Orchestration:
  - ✓ Simpler and more popular
  - ✗ SPoF and potential performance bottleneck
  - ✗ Tight coupling between orchestrator and services
  - ✗ Higher network traffic and latency

- Choreography
  - ✓ Lower coupling and operational complexity
  - ✓ Increased flexibility and easier to change individual services
  - ✗ Services must be aware of each other's locations
  - ✗ Harder to observe and debug
  - ✗ Implementing guarantees like reliable delivery is more challenging

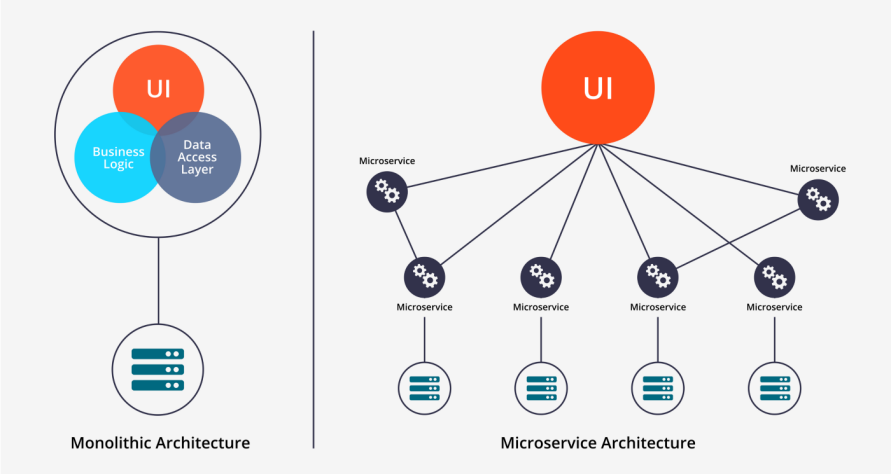
Microservice patterns



c'è poi la decomposizione. come decomponiamo le applicazioni monolitiche legacy?

Decomposition patterns

- Monolithic application: built and deployed as a single unit
- Decomposing into microservices is mostly an art
  - No single “best” strategy
  - Multiple decomposition patterns exist
    https://microservices.io/patterns



Decomposition patterns

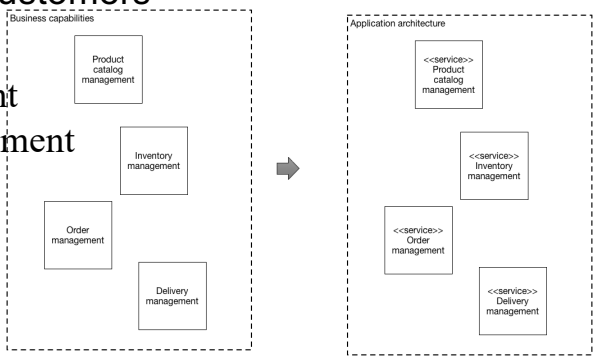
- Design considerations
  - Stable architecture
    - Decomposition boundaries should not change frequently
  - High cohesion
    - Each service implements a small set of strongly related functions
  - Common Closure Principle (CCP)
    - Things that change together should be packaged together
    - A change should ideally affect only one service
  - Loose coupling
    - Each service exposes an API that encapsulates its implementation
    - Internal changes should not affect clients
  - Testability
    - Each service should be independently testable
  - Team size and ownership
    - Service small enough for a “two-pizza team” (≈ 6–10 people)
    - Teams should be autonomous, with minimal coordination

Main decomposition patterns

- Example: e-commerce app that takes orders from customers, verifies inventory and available credit, and ships them

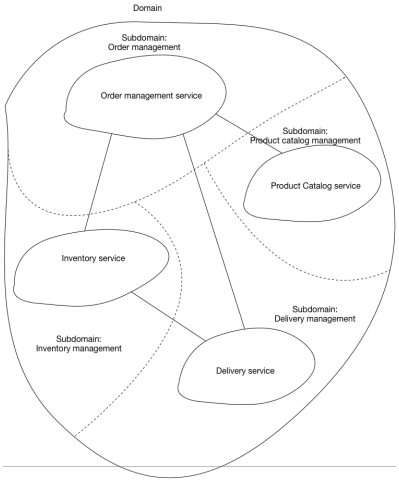
1. Decompose services based on business capability
   - Business capability: a function the business performs to generate value
   - Organizational/business view
   - Good for initial decomposition and team alignment
   - E.g., *Order Management* is responsible for orders, *Customer Management* for customers

Quindi ogni management possiede un servizio. Perciò Customer Manegement porta a Customer service e Order management porta a Order service e cosi via...

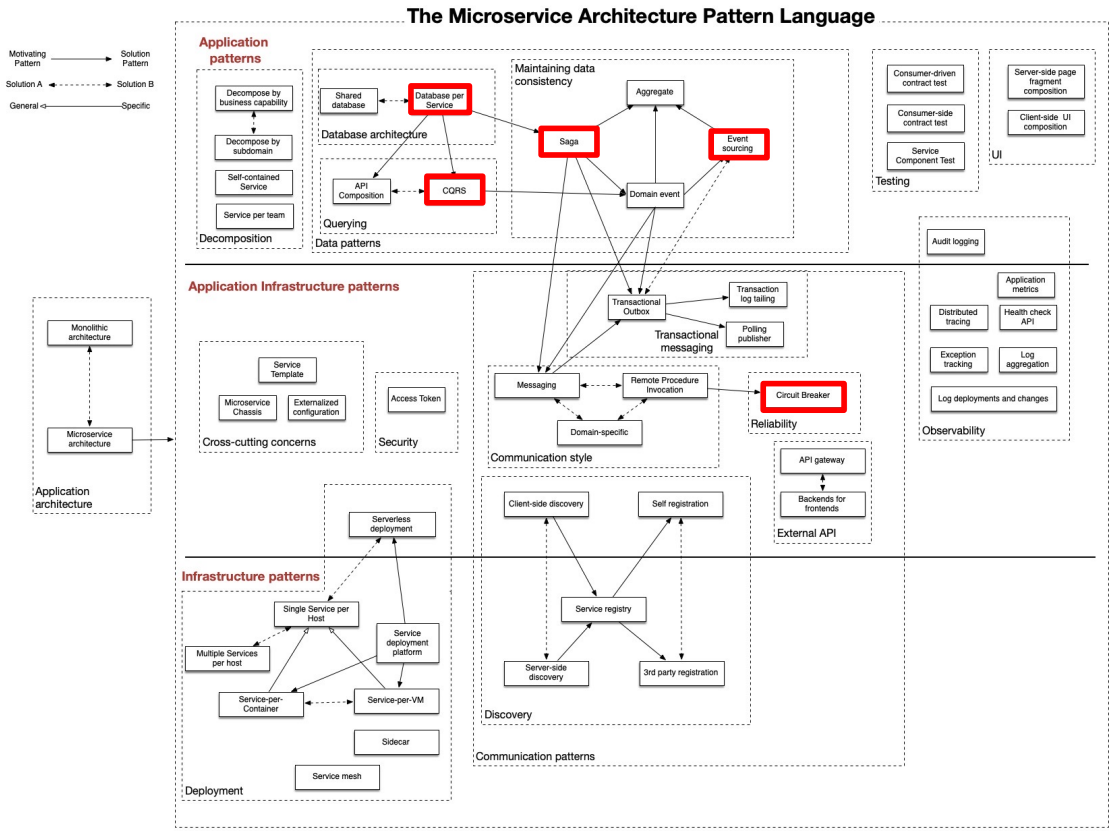


Main decomposition patterns

2. Decompose by domain-driven design (DDD) subdomain
   - A domain is divided into multiple subdomains, each one corresponding to a different part of the business
   - Domain-modeling view
   - Good for complex domains with rich business rules
   - E.g., *Order Management*, *Inventory*, *Product Catalogue*, *Delivery*



Microservice patterns

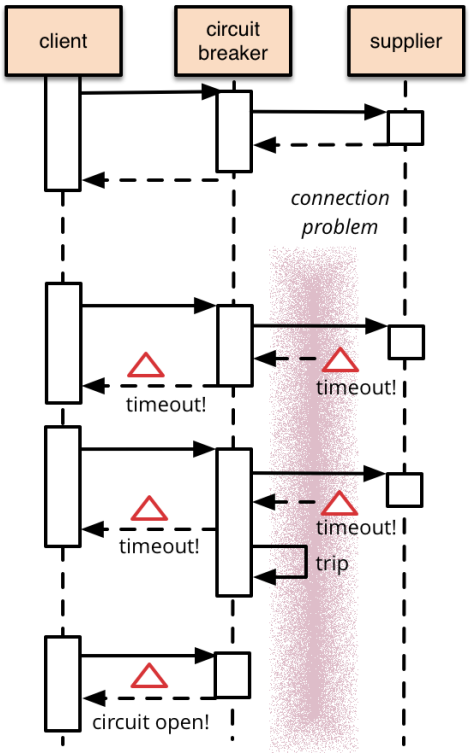


Copyright © 2024. Chris Richardson Consulting, Inc. All rights reserved.

Learn-Build-Assess Microservices http://adopt.microservices.io

Reliability patterns: Circuit breaker

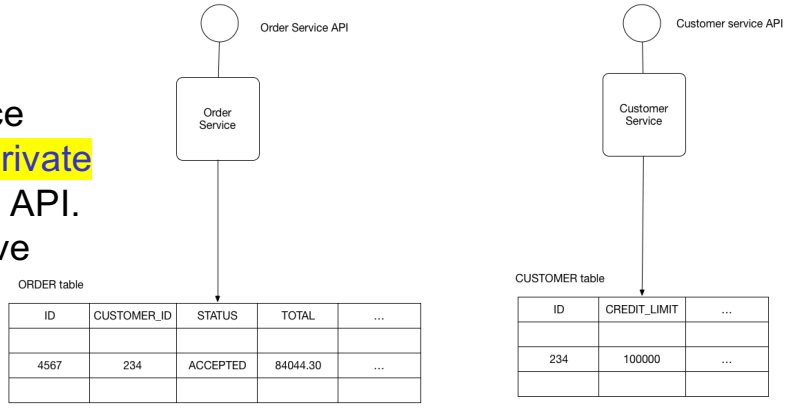
- Problem: how to prevent a network or service failure from cascading to other services?
- Solution: a service client invokes a remote service via a proxy that behaves like an electrical circuit breaker
  - Requests flow normally while the circuit is closed
  - When failures exceed a threshold, the circuit opens and requests fail immediately (optional fallback response)
  - After a timeout, the circuit enters half-open state and allows limited test requests
  - On success, it closes again; on failure, it reopens and restarts the timeout



https://microservices.io/patterns/reliability/circuit-breaker.html

Data patterns: Database per service

- Problem: which database architecture?
- Solution: each microservice keeps its persistent data private and accessible only via its API. Service transactions involve only that service's DB

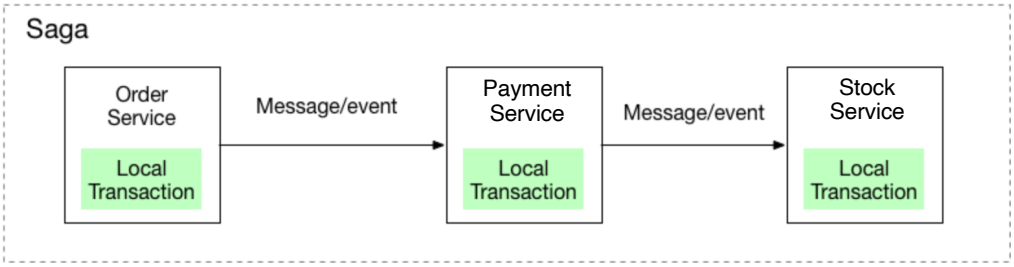


- Pros and cons
  - ✓ Helps ensure loose coupling among services
  - ✓ Each service can use the most appropriate DB type (e.g., KV data store, graph database)
  - ✗ Transactions spanning multiple services are more complex
  - ✗ Managing multiple DBs increases operational complexity
    - A dedicated DB server per service is not required
- Options: private tables per service, schema per service, database server per service

https://microservices.io/patterns/data/database-per-service.html

Data patterns: Saga

- Problem: each service has its own DB, but some transactions span multiple services: how to maintain data consistency across services without using distributed transactions (e.g., two-phase commit)?
- Solution: implement each cross-service transaction as a saga
- **Saga:** a sequence of local transactions
  - Each local transaction updates its DB and publishes a message/event to trigger the next transaction
  - If a local transaction fails, the saga executes compensating transactions to undo changes made by preceding transactions (rollback)

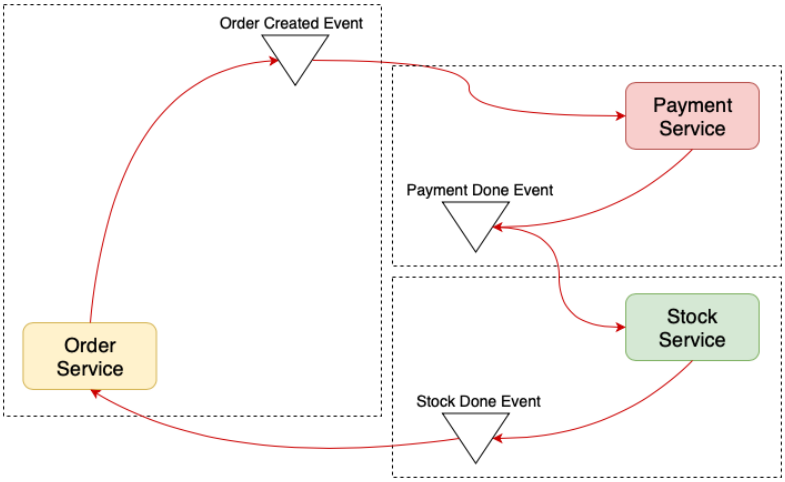


https://microservices.io/patterns/data/saga.html

Data patterns: Saga

- 2 ways to coordinate a saga:
  - Choreography: each local transaction publishes events that trigger local transactions in other services
  - Orchestration

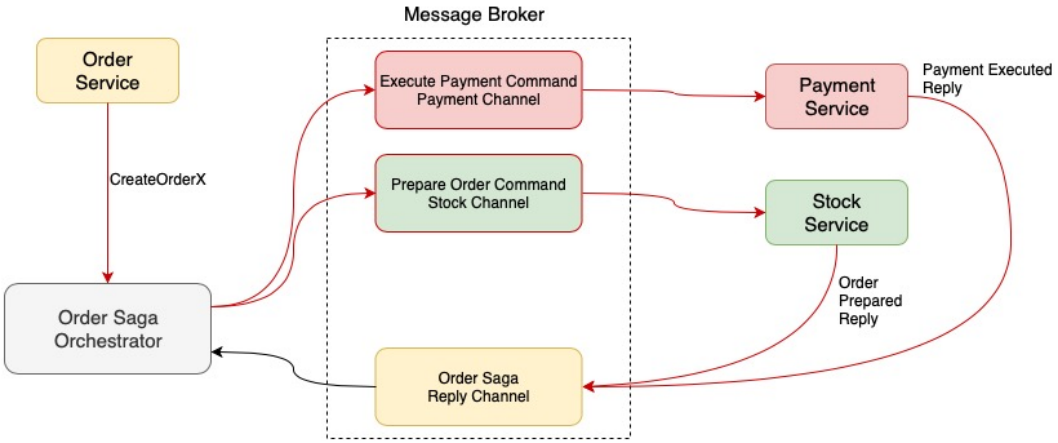
Choreography



Data patterns: Saga

- 2 ways to coordinate a saga:
  - Choreography
  - Orchestration: a central orchestrator tells each service which local transaction to execute

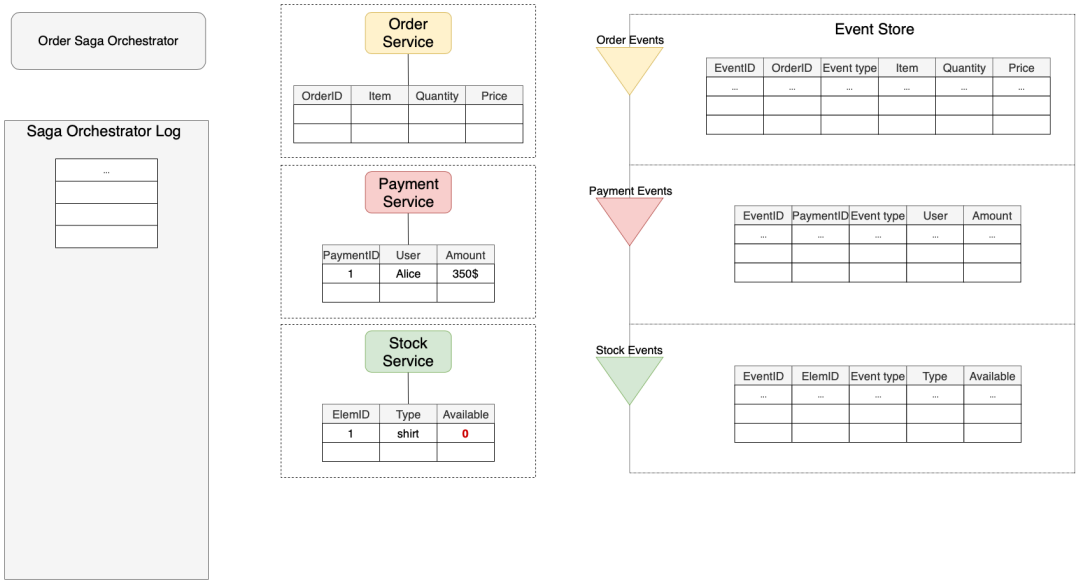
Orchestration



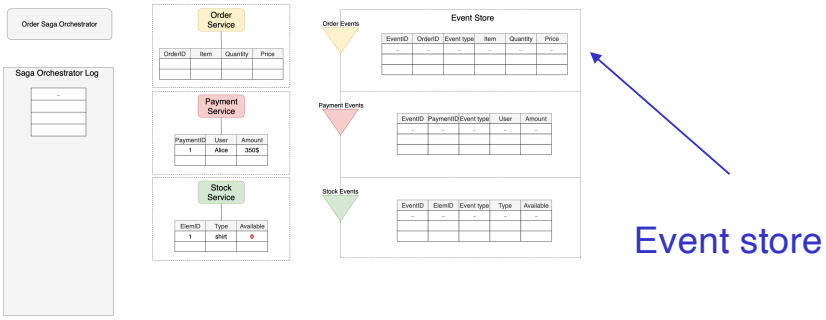
Data patterns: orchestration-based Saga

- Let's consider orchestration-based saga
  - Source: MSc thesis by Andrea Cifola

http://www.ce.uniroma2.it/courses/sdcc2122/slides/Microservice_SAGAexample.pdf



Data patterns: event sourcing



- In the saga, we also use another pattern: event sourcing https://microservices.io/patterns/data/event-sourcing.html
  - Problem: a service participating in a saga must atomically update its DB and publishes messages/events to the orchestrator to avoid data inconsistencies
  - Solution: persist a sequence of domain events representing state changes; store events in an *append-only* event store (a DB of events); service state can be reconstructed by replaying events

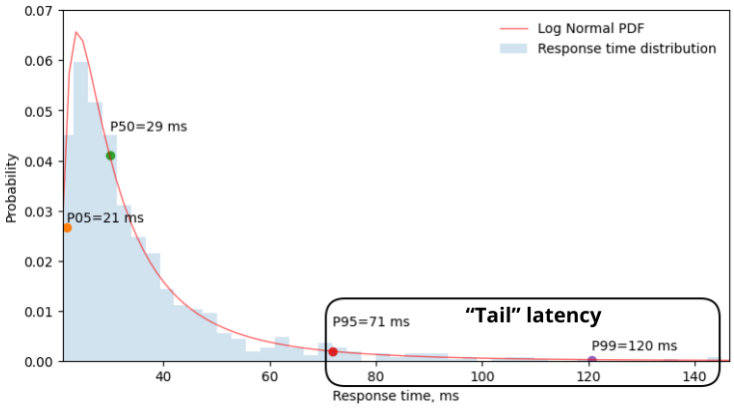
Data patterns: CQRS

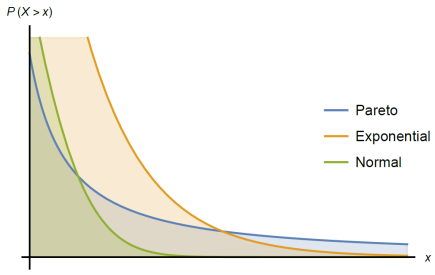
- Problem: how to query data from multiple services in a microservice architecture? How to separate read and write load to scale each independently?
- Solution: use a view DB, a read-only replica designed to support queries
  - Application keeps the view updated by subscribing to domain events published by the service that owns the data https://microservices.io/patterns/data/domain-event.html
- Known as Command Query Responsibility Segregation (CQRS), i.e., separate commands (writes) from queries (reads) for a data store https://microservices.io/patterns/data/cqrs.html

Microservices observability challenge

- Service distribution, even at large scale: difficult to monitor and capture causal and temporal relationships among microservices
- Need for monitoring perché è necessario: per debug e analisi di performance
  - To debug the application
  - To analyze performance and latency, including tail latency

nella tail latency quindi si ha poca probabilità di avere latenza alta, ma quando c'è è un problema

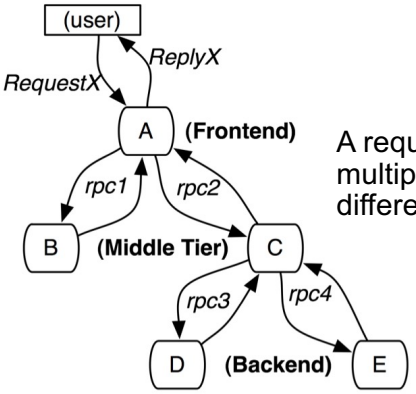




Microservices observability challenge

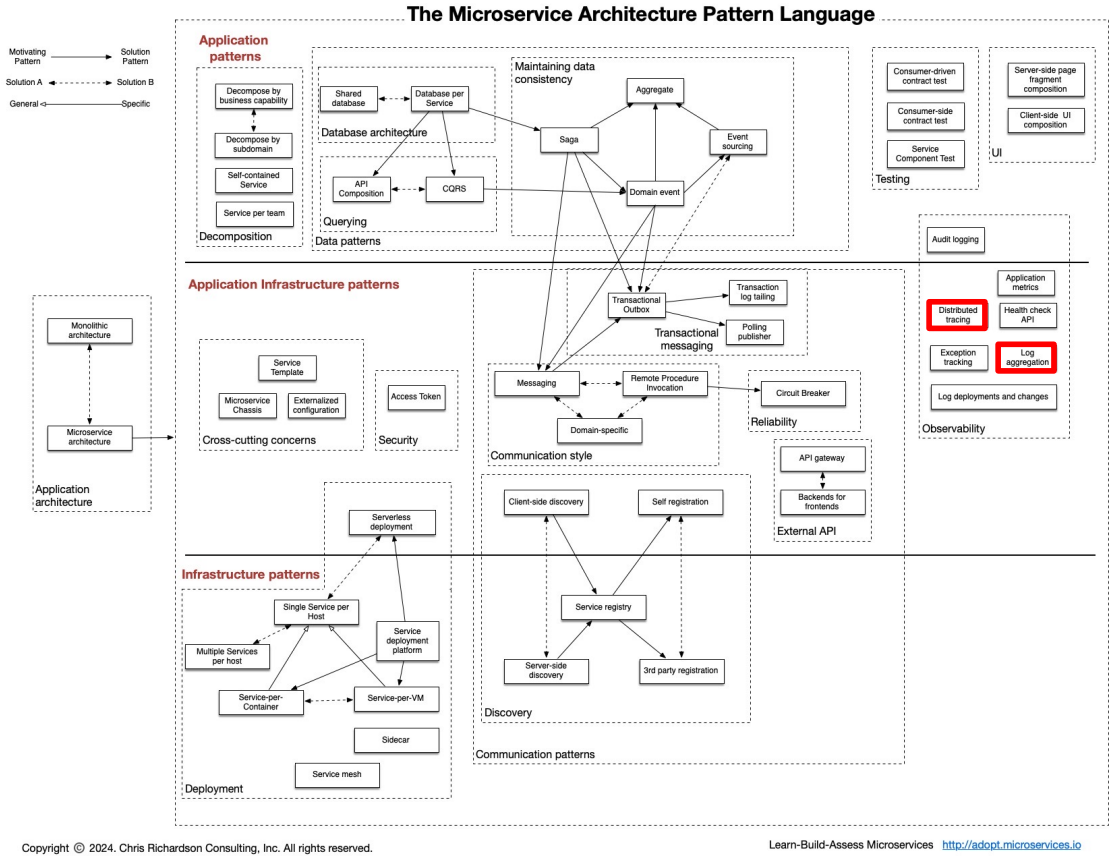
- Need for monitoring
  - To analyze service dependencies
  - To identify the root cause of anomalies, which requires:
    - Constructing a service dependency graph that shows the sequence of invoked microservices
    - Localizing the root cause microservices using the graph, traces, logs, and KPIs

senza il monitoraggio, in una catena di microservizi non si saprà mai dove e perché avviene un fallimento



A request is passed through multiple microservices with different functionalities

Microservice patterns



in "soccorso" ci vengono i pattern di osservabilità

Observability patterns: Log aggregation

- Problem: how to understand application behavior and troubleshoot problems?
- Solution: use a centralized logging service that aggregates logs from all microservice instances
  - DevOps team can search and analyze logs and configure alerts triggered by specific log messages
  - E.g., AWS CloudWatch, Splunk

✗ Centralized (if physical, not only logical)

✗ Handling large volumes of logs requires substantial infrastructure

https://microservices.io/patterns/observability/application-logging.html

Observability patterns: Distributed tracing

serve anche tracciabilità!

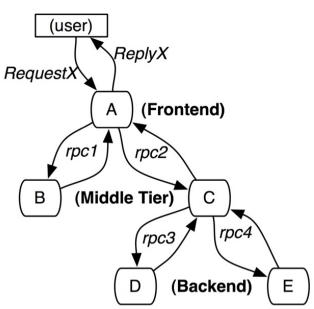
- Problem: how to understand complex app behavior and troubleshoot problems?
- Solution: instrument microservices with code to
  - Assign a unique request ID (*trace ID*) to each user request
  - Pass the trace ID through all microservices involved in handling the request
  - Include the trace ID in log messages
  - Record trace context (e.g., start time, operation, duration) in a distributed store
- ✗ Storing and aggregating traces may require significant infrastructure

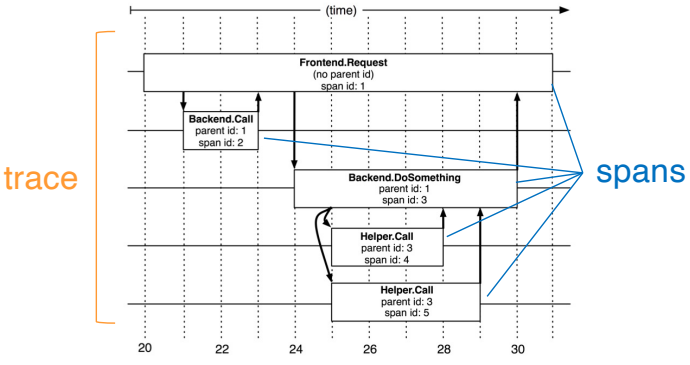
https://microservices.io/patterns/observability/distributed-tracing.html

ancora una volta, serve spazio di archiviazione

Monitoring microservices: tools

- Dapper
  - Google's distributed tracing system
  - Based on *spans* and *traces*
    - Span: individual unit of work in an application(e.g., HTTP request, call to DB); includes operation name, start time, and duration
    - Trace: collection of spans in a parent/child relationship (can be seen as a DAG); shows how requests propagate through services and other components





Barroso er al., Dapper, a large-scale distributed systems tracing infrastructure, 2010

Monitoring microservices: tools

- Dapper trace sampling and storage
- Traces are sampled using an adaptive rate
  - Why sample traces? andiamo a campionare le traces
    - Storing all traces consume excessive storage, generate high network traffic, and introduce significant application overhead
  - Sampling rate adjusts dynamically based on traffic volume and system behavior
- Trace collection and storage
  - Span data is written to local log files by each service
  - Dapper daemons pull the logs and send them through a collection infrastructure
  - Traces are stored in BigTable, with one row per trace ID

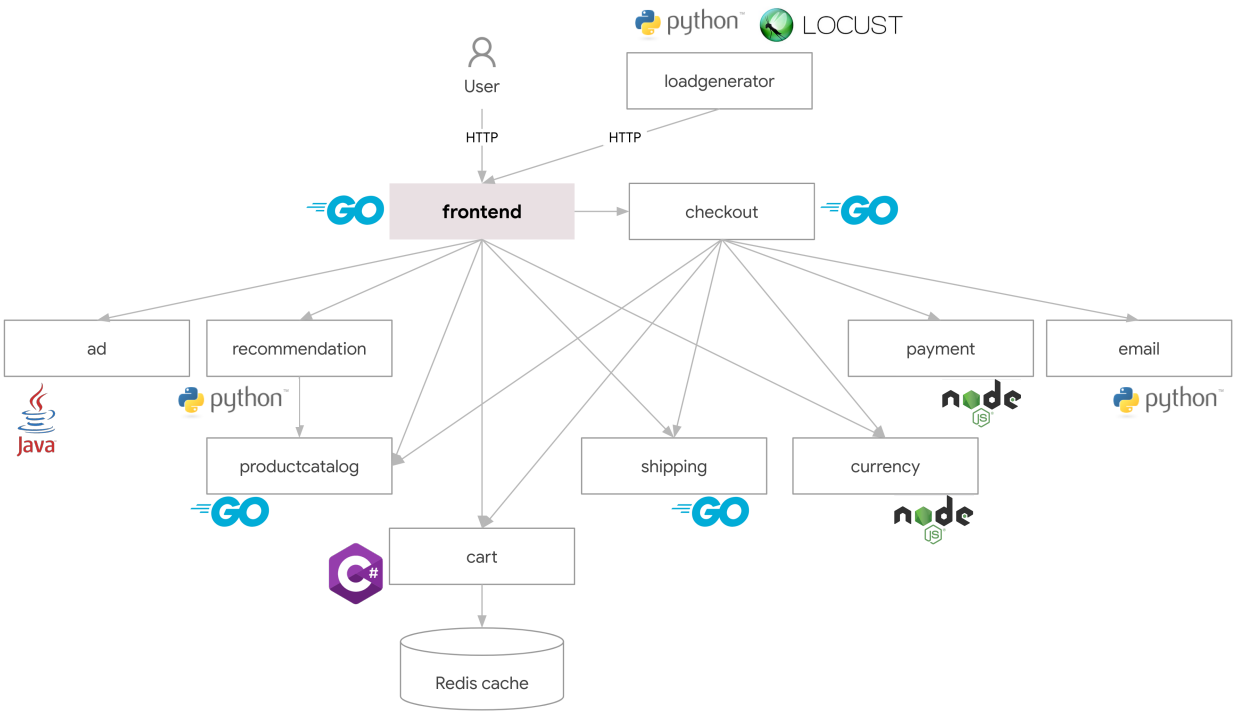
Monitoring microservices: tools

- Open-source distributed tracing tools
  - Jaeger https://www.jaegertracing.io
    - Inspired by Dapper, supports large-scale trace analysis (optionally via Spark/Flink)
  - Zipkin https://zipkin.io
    - Lightweight distributed tracing system
  - OpenTelemetry https://opentelemetry.io
    - Industry-standard instrumentation framework
    - Broad language support
    - Integrated with popular frameworks and libraries
    - Exports traces to multiple backends (e.g., Jaeger, Zipkin)
- Need for standards to support interoperability across tracing tools
  - W3C Trace Context defines a standardized format for propagating tracing data https://www.w3.org/TR/trace-context-2/

Example microservices app

- Google's Online Boutique
  https://github.com/GoogleCloudPlatform/microservices-demo
  - Online store where users can browse items, add them to the cart, and purchase them
- Composed of 11 microservices written in different languages
  - Polyglot microservices: a "renaissance" in programming language diversity
- How to build a polyglot application?
  1. REST and JSON as message interchange format
  2. gRPC and protocol buffers as IDL and message interchange format: chosen approach in Online Boutique

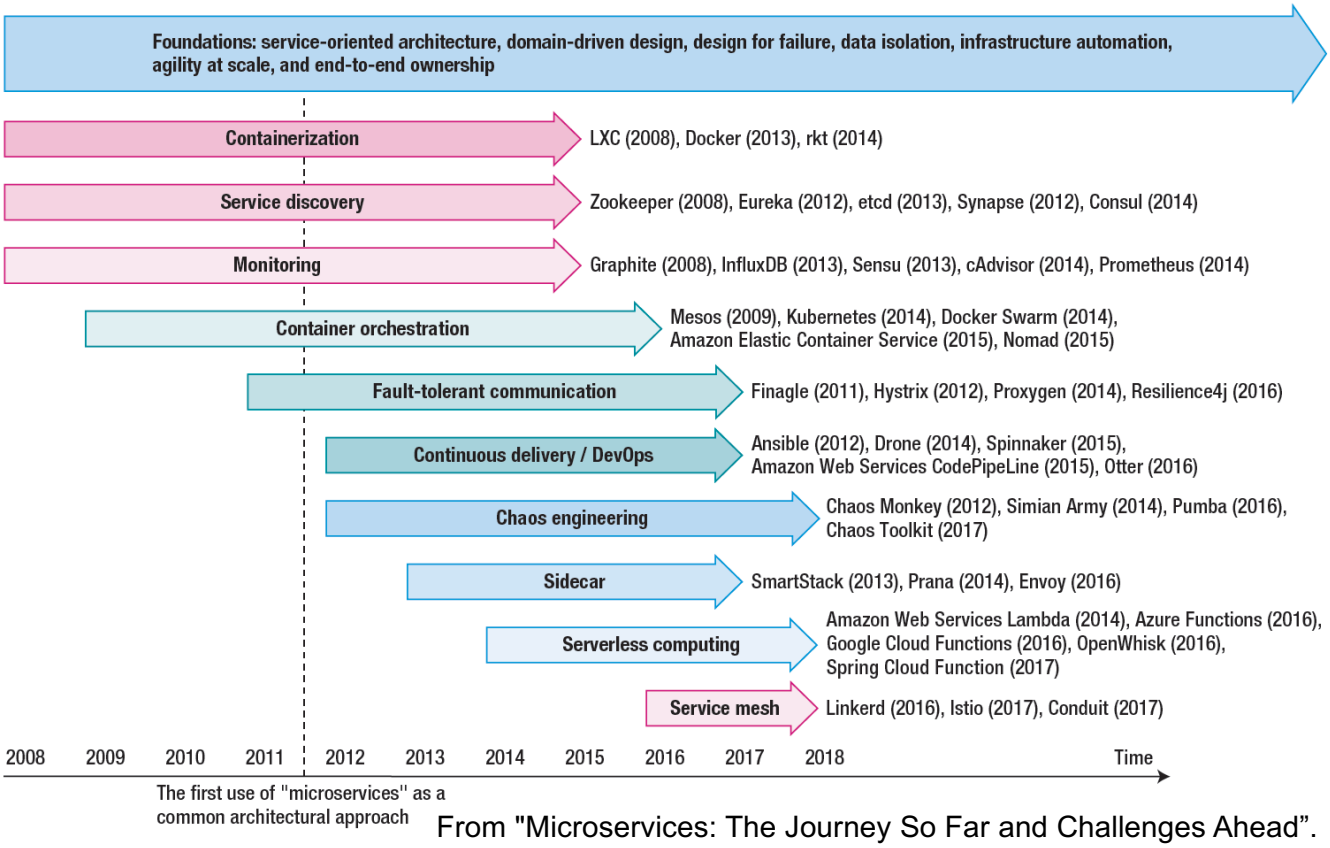
Online Boutique: architecture



Online Boutique: features

- Composed of 10 microservices (plus a load generator), written in different languages and communicating using gRPC
- Technologies demonstrated by Google Online Boutique:
  - Kubernetes and Google Kubernetes Engine (GKE): container orchestration che vedremo
  - gRPC: we know it ☺
  - Istio / Cloud Service: service mesh for traffic management, security and observability
  - Google Cloud Observability: monitoring, logging, and tracing on Google Cloud https://cloud.google.com/products/observability
  - Locust: load testing tool https://locust.io
  - Skaffold: command line tool for Kubernetes and containers development https://skaffold.dev

Microservice technologies timeline



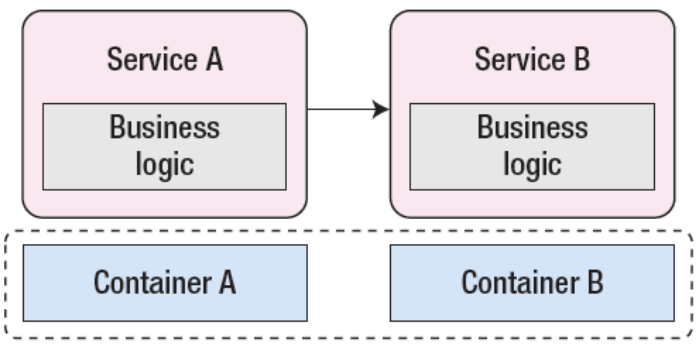
From "Microservices: The Journey So Far and Challenges Ahead".

Generations: at the beginning

- 4 generations of microservice architectures
- 1st generation: containers and orchestration
  - **Container-based virtualization** (e.g., Docker)
  - **Service discovery**, e.g.,
    - etcd https://etcd.io: distributed reliable key-value store
    - Zookeeper
  - **Monitoring** tools: enable runtime monitoring and analysis of microservice resources behavior at different levels of detail
    - Graphite https://graphiteapp.org
    - InfluxDB https://www.influxdata.com
    - Prometheus https://prometheus.io/

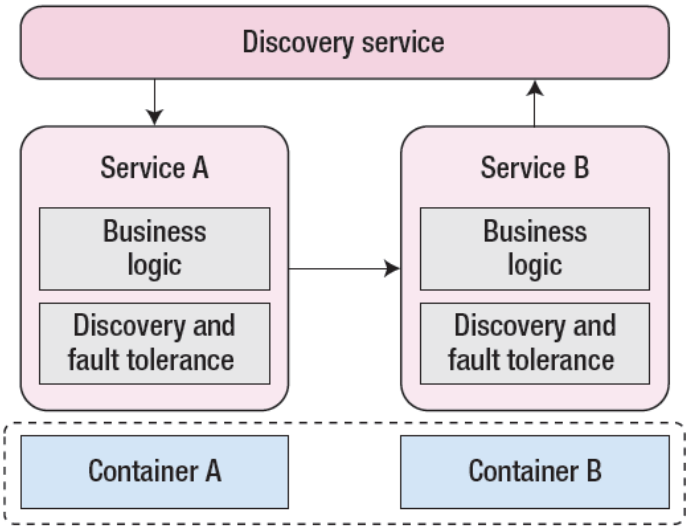
Generations: container orchestration

- **Container orchestration**
  - E.g., Kubernetes, Docker Swarm
  - Automates container deployment, scaling, and management
  - Abstracts physical/virtual infrastructure from developers
- Limitation
  - Application-level fault tolerance implemented inside microservice code non vi è quindi un sistema per la fault tolerance



Generations: service discovery and fault tolerance

- 2nd generation service discovery tools and fault-tolerant (FT) communication libraries
  - Goal: enable more efficient and reliable service-to-service communication
  - Use FT communication libraries implementing resiliency patterns: circuit breaker, fallback, retry/timeout

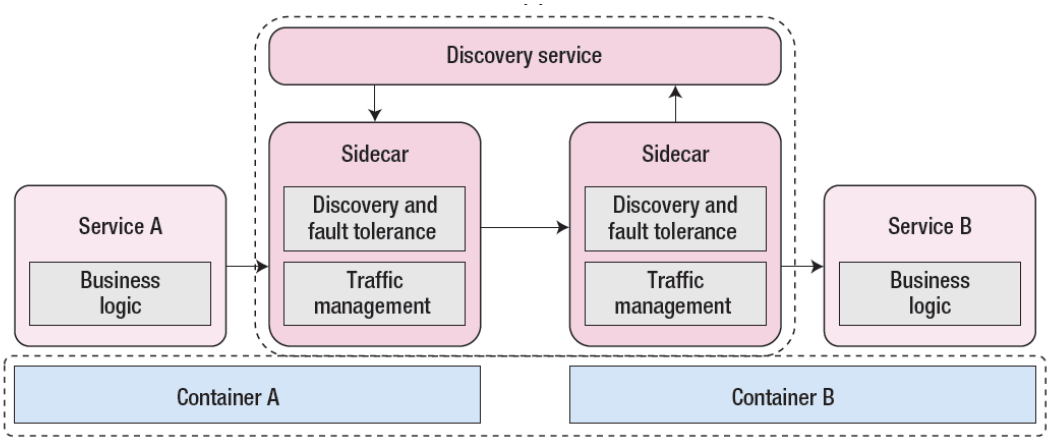


Generations: service discovery and fault tolerance

- Examples:
  - Consul: initially service discovery, now service mesh https://developer.hashicorp.com/consul
  - Finagle: protocol-agnostic FT RPC library https://github.com/twitter/finagle
  - Resilience4j: Java FT library with multiple resiliency patterns https://resilience4j.readme.io
- Limitation
  - Developers must explicitly integrate FT libraries into application code

Generations: service mesh

- 3rd generation: service mesh and sidecar proxies
  - Encapsulate communication concerns: service discovery, load balancing, security, fault tolerance, observability
  - Benefits
    - Abstract communication logic away from application code
    - Improve software reuse
    - Provide a homogeneous communication interface

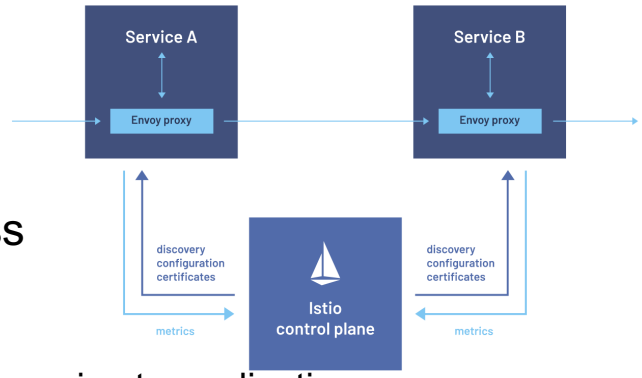


Service mesh

- A dedicated infrastructure layer for microservice apps that facilitates service-to-service communication
- Provided features (no need to embed them into application logic) https://servicemesh.es
  - Traffic management: service discovery, load balancing, routing
  - Security: authentication, encryption, and authorization between services
  - Resilience: circuit breaking, retries, timeouts, and fallbacks
  - Observability: monitoring, logging, and tracing service interactions
- Popular products:
  - Istio https://istio.io
  - Linkerd https://linkerd.io
  - Consul https://www.consul.io

Service mesh: architecture

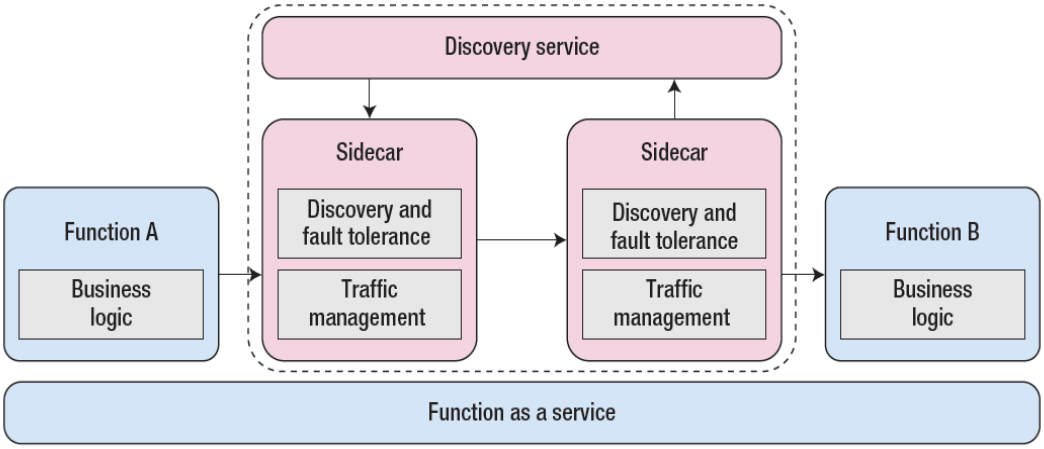
- Composed of data plane and control plane
- Data plane: decentralized
  - Sidecar proxies deployed alongside each microservice
  - Each proxy handles inbound and outbound traffic for the service it is attached to
- Control plane: centralized
  - Centralized configuration management for all proxies
- Recent approach: proxyless service mesh
  - Moves service-to-service communication from sidecar proxies to application or a lightweight, centralized proxy
  - Traffic management, security, and observability are still managed by the mesh



✓ Reduces resource overhead and simplifies deployment

Generations: serverless

- 4th generation: Function as a Service (FaaS) and serverless computing
  - Further simplify microservice development and delivery

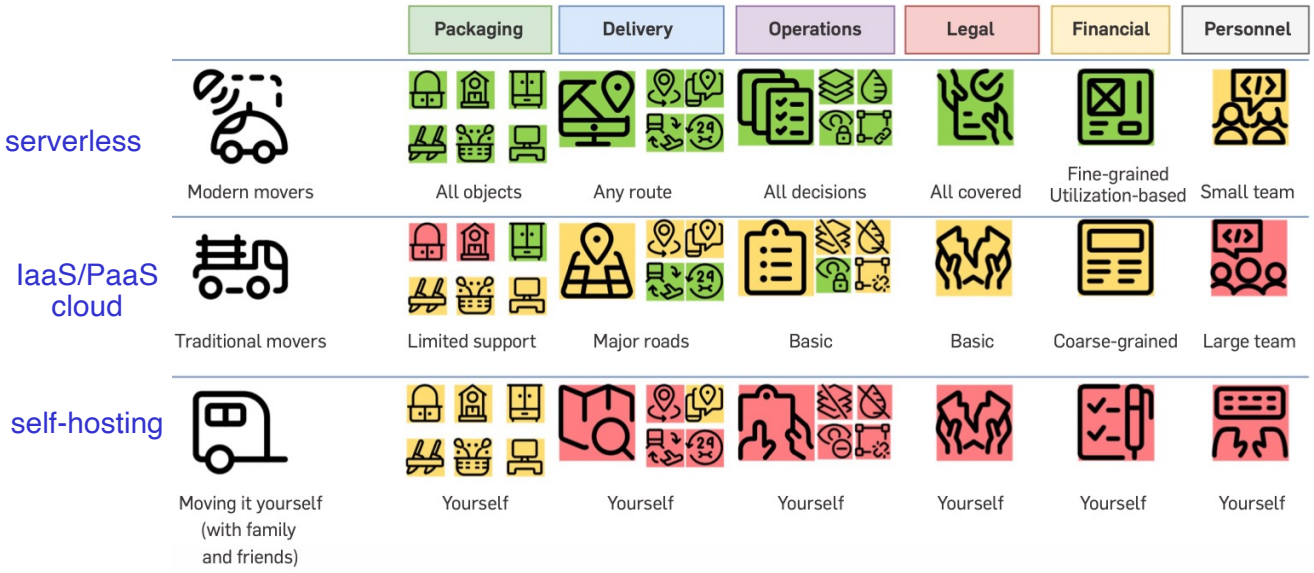


Serverless computing

- Cloud computing model that abstracts server management and low-level infrastructure decisions away from users through full automation
- Key characteristics in buona sostanza si gestisce SOLO il codice
  - Users develop, run, and manage application code (**functions**)
  - No need to provision, manage, or scale computing resources
  - The runtime environment is fully managed by the cloud (or private platform) provider
- Note: serverless does not mean no servers
  - Functions still run on servers, we simply do not manage or care about them

Serverless through an analogy

- Services for moving homes



Serverless: many definitions

van Eyk et al., Serverless is More: From PaaS to Present Cloud Computing (135 cit.), IEEE IC, May 2018 [47]:

"Serverless Computing is a form of cloud computing which allows users to run event-driven and granularly billed applications, without having to address the operational logic. Function-as-a-Service (FaaS) is a form of serverless computing where the cloud provider manages the resources, lifecycle, and event-driven execution of user-provided functions."

Hellerstein et al., Serverless Computing: One Step Forward, Two Steps Back (360 cit.). CIDR, Jan 2019 [23]:

"Serverless computing offers the attractive notion of a platform in the cloud where developers simply upload their code, and the platform executes it on their behalf as needed at any scale. Developers need not concern themselves with provisioning or operating servers, and they pay only for the compute resources used when their code is invoked... Serverless is not only FaaS. It is FaaS supported by a "standard library": the various multi-tenanted, autoscaling services provided by the vendor. In the case of AWS, this includes S3 (large object storage), DynamoDB (key-value storage), SQS (queuing services), SNS (notification services), and more."

Castro et al., The Rise of Serverless Computing (196 cit.), CACM, Dec 2019 [10]:

"Serverless computing is a platform that hides server usage from developers and runs code on-demand automatically scaled and billed only for the time the code is running. This definition captures the two key features of serverless computing: (a) Cost—billed only for what is running (pay-as-you-go)...; serverless essentially supports "scaling to zero" and avoids the need to pay for idle servers. (b) Elasticity—scaling from zero to "infinity." … The main differentiators of serverless platforms is transparent autoscaling and fine-grained resource charging only when code is running. Function-as-a-Service is a serverless computing platform where the unit of computation is a function that is executed in response to triggers such as events or HTTP requests. Mobile Backend as-a-Service (MBaaS) or more generalized Backend as-a-Service (BaaS) bears a close resemblance to serverless computing."

. Jonas,…, Patterson et al., Cloud Programming Simplified: A Berkeley View on Serverless Computing (485 cit.), arXiv, Feb 2019 [26], refined in a follow up publication by the same authors What Serverless Computing Is and Should Become: The Next Phase of Cloud Computing (75 cit.), CACM, May 2021 [41]

"In serverless computing, programmers create applications using high level abstractions offered by the cloud provider... They may also use serverless object storage, message queues, key-value store databases, mobile client data sync, and so on, a group of services offerings known collectively as Backend-as-a-Service (BaaS). Managed cloud function services are also called Function-as-a-Service (FaaS) and collectively Serverless Cloud Computing today = FaaS + BaaS. Three essential qualities of serverless computing are: 1. Providing an abstraction that hides the servers and the complexity of programming and operating them. 2. Offering a pay-as-you-go cost model instead of a reservation-based model, so there is no charge for idle resources. 3. Automatic, rapid, and unlimited scaling resources up and down to match demand closely, from zero to practically infinite."

Legend: 6x NoOps 6x Function-as-a-Service 5x pay-per-use 4x autoscaling/elasticity 3x Backend-as-a-Service 2x event-driven arch.

Serverless: many definitions

Kounev et al., Serverless Computing: What It Is, and What It Is Not?, Comm. ACM, 2023

Serverless computing is a cloud computing paradigm encompassing a class of cloud computing platforms that allow one to develop, deploy, and run applications (or components thereof) in the cloud without allocating and managing virtualized servers and resources or being concerned about other operational aspects.

The responsibility for operational aspects, such as fault tolerance or the elastic scaling of computing, storage, and communication resources to match varying application demands, is offloaded to the cloud provider.

Providers apply utilization-based billing: they charge cloud users with fine granularity, in proportion to the resources that applications actually consume from the cloud infrastructure, such as computing time, memory, and storage space.

Serverless, FaaS and BaaS

- Function as a Service (FaaS) and serverless often used interchangeably, some discussion on difference
- FaaS is the most prominent model of serverless computing
  - Can be defined as "a serverless computing platform where the unit of computation is a function that is executed in response to triggers such as events or HTTP requests" (Kounev et al.)
- Backend as a Service (BaaS)
  - Provides developers with backend functions via API, such as authentication, data storage, real-time messaging, and push notifications
  - Examples:
    - AWS DynamoDB (no-demand mode): pay-per-request pricing and automatic scaling
    - Google Cloud Firestore (NoSQL document DB) and Pub/Sub
    - AWS Amplify: serverless development platform that simplifies building, deploying, and managing web and mobile applications

Serverless: features

- Ephemeral compute resources
  - May exist only for the duration of a function invocation
  - ✗ Cold start: if no container or microVM is ready, the function waits until a new instance is launched
- Automated (i.e., zero configuration) elasticity
  - Compute resources auto-scale transparently from zero to peak load and back
- True pay-per-use: fine-grained and utilization-based
  - E.g., AWS Lambda pricing is based on the number of invocation requests and the duration it takes for code to run

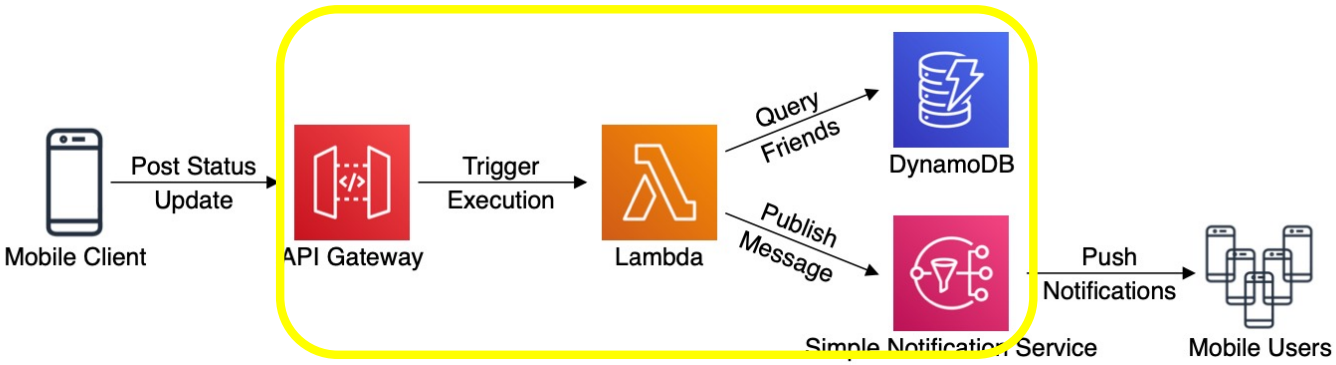
Architecture	Duration	Requests
x86 Price		
First 6 Billion GB-seconds / month	$0.0000166667 for every GB-second	$0.20 per 1M requests
Next 9 Billion GB-seconds / month	$0.000015 for every GB-second	$0.20 per 1M requests
Over 15 Billion GB-seconds / month	$0.0000133334 for every GB-second	$0.20 per 1M requests

Serverless: features

- Event-driven execution
  - Functions are triggered by events (e.g., file upload, message queue, HTTP request)
  - Infrastructure is dynamically allocated to execute the function code in response to the event
- NoOps (no operations)
  - Simplifies the deployment process: no need for developers to manage scaling, capacity planning, or infrastructure maintenance
  - Developers focus only on business logic
- Supports diverse applications
  - Use cases range from enterprise automation and real-time data analytics to scientific computing and ML inference

Serverless application: a first example

- Propagating updates in a social media app in a serverless fashion
  1. User creates and sends a status update
  2. Platform orchestrates operations needed to propagate the update inside the social media platform and to user's friends using serverless (AWS Lambda) and other cloud services (AWS DynamoDB and SNS)
  3. Friends receive the update

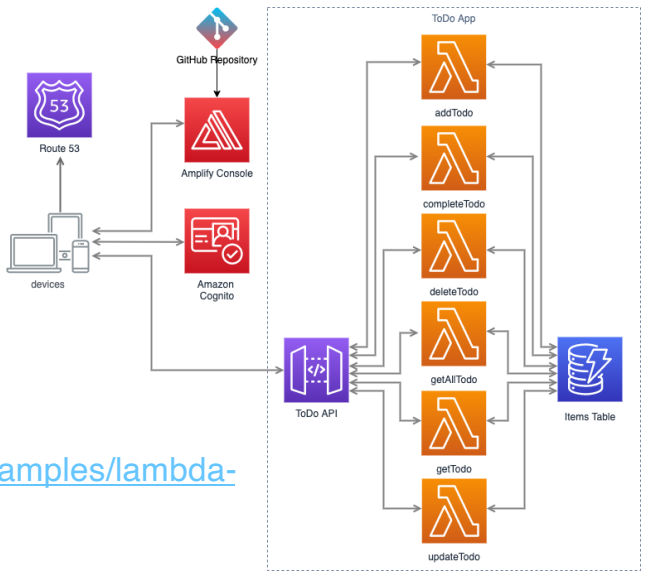


Serverless Cloud services

- Several Cloud providers offer serverless computing as fully managed service on their public clouds
  - AWS Lambda https://aws.amazon.com/lambda/
    - See hands-on course
    - Lambda@Edge: functions at the edge https://aws.amazon.com/lambda/edge/
  - Azure Functions https://azure.microsoft.com/products/functions
  - Google Cloud Run Functions https://cloud.google.com/functions
- User has limited knobs to control performance
  - Amount of memory allocated to function (CPU ~ memory)
  - Auto-scaling is handled automatically by the cloud provider
- Cloud platforms also offer supporting services to operate serverless ecosystems
  - E.g., event notifications, storage, message queues, DBs

Example: AWS reference Web app

- A simple “to-do list” web app that allows registered users to create, update, view, and delete to-do items
- Event-driven web app uses AWS Lambda and Amazon API Gateway for the business logic, DynamoDB as database, and Amplify Console to host static content



https://github.com/aws-samples/lambda-refarch-webapp

Example: Google Cloud Run Functions

- "Hello World" example from Google using Go
  - Basic web server that replies with "Hello, World!"

https://docs.cloud.google.com/run/docs/quickstarts/build-and-deploy/deploy-go-service

```go
// Sample run-helloworld is a minimal Cloud Run service.
package main

import (
        "fmt"
        "log"
        "net/http"
        "os"
)

func main() {
        log.Print("starting server...")
        http.HandleFunc("/", handler)

        // Determine port for HTTP service.
        port := os.Getenv("PORT")
        if port == "" {
                port = "8080"
                log.Printf("defaulting to port %s", port)
        }

        // Start HTTP server.
        log.Printf("listening on port %s", port)
        if err := http.ListenAndServe(":"+port, nil); err != nil {
                log.Fatal(err)
        }
}

func handler(w http.ResponseWriter, r *http.Request) {
        name := os.Getenv("NAME")
        if name == "" {
                name = "World"
        }
        fmt.Fprintf(w, "Hello %s!\n", name)
}
```


Serverless: state

- Stateless functions are easy to manage (horizontal scalability, fast recovery)
  - However, stateless functions are not enough for some applications (e.g., ML, streaming)
- How to support stateful computation?
  1. Externalize state (e.g., handed over to an external shared storage system), so functions themselves remain stateless
     - Requires efficient access to shared state, so to keep auto-scaling benefits
  2. Embed state into the serverless runtime: the serverless platform manages state transparently and colocates it with computation; the state is automatically persisted, replicated, and recovered
     - Examples: Azure Stateful Functions, Cloudflare Durable Objects
- What about transactions?

https://cacm.acm.org/practice/transactions-and-serverless-are-made-for-each-other/

Serverless: challenges and limitations

- Performance
  - Cold starts

    "Starting a new function instance involves loading the runtime and your code. Requests that include function instance startup, called *cold starts*, can be slower than requests routed to existing function instances." *Google Cloud Run Functions*
  - Autoscaling: can introduce performance variability, especially during traffic spikes
- Programming languages
  - Language support varies by cloud provider
  - Choice of language affects cold start times, function performance, and cost
- Security
  - E.g., more entry points, financial exhaustion attacks
- Resources
  - Limits on resource amount (e.g., in AWS Lambda per-function memory between 128 MB and 10 GB)
  - GPU support

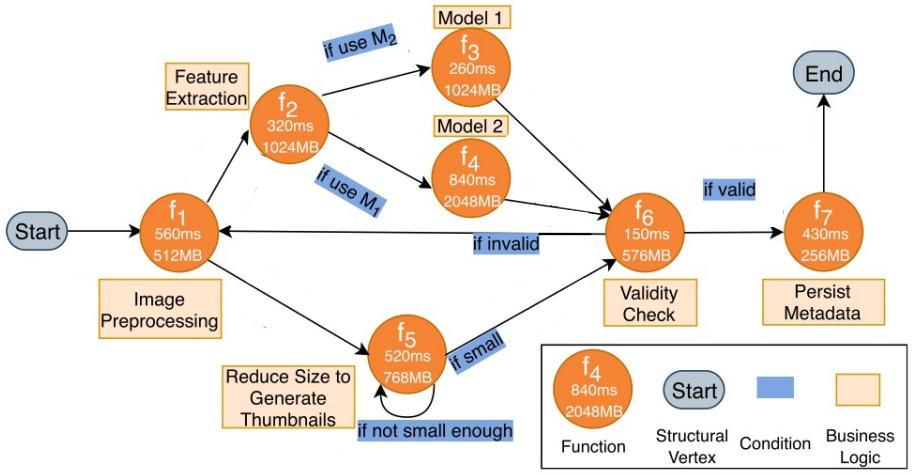
Lower flexibility

Serverless: challenges and limitations

- Cost-saving
  - Not always cost-saving for users, possibly leading to expense explosion
  - Not cost-saving: high, predictable traffic, long-running applications, complex applications, high memory/CPU needs
- Vendor lock-in
  - Serverless offerings are tied to specific cloud providers

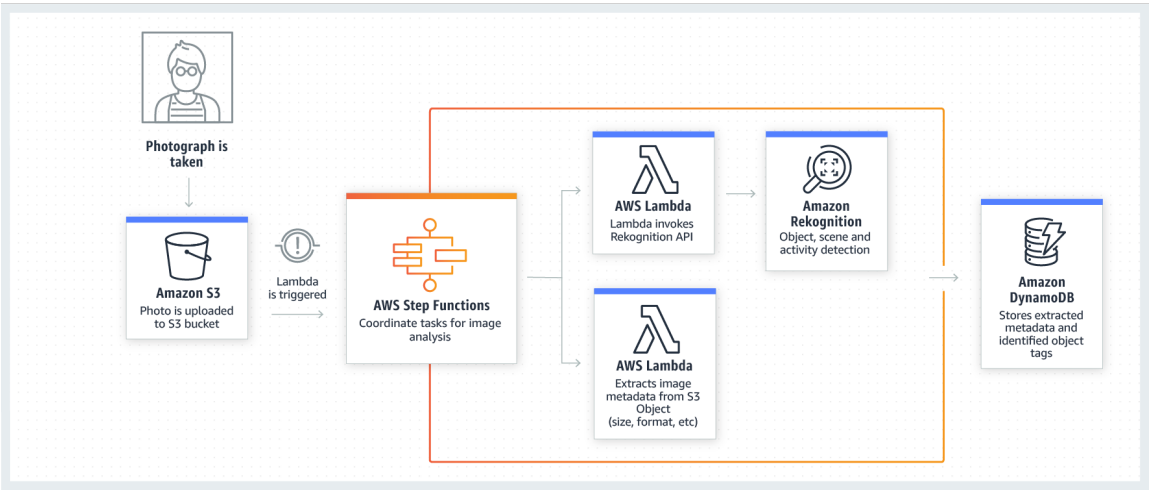
Composition of serverless functions

- Write small, simple, stateless functions
  - Complex functions are hard to understand, debug, and maintain
  - Separate code from data structures
- Compose multiple functions in a workflow



Example: AWS Step Functions

- AWS Step Functions: serverless orchestration service that allows developers to coordinate multiple Lambda functions into a single workflow
- Example: process photo after its upload to S3

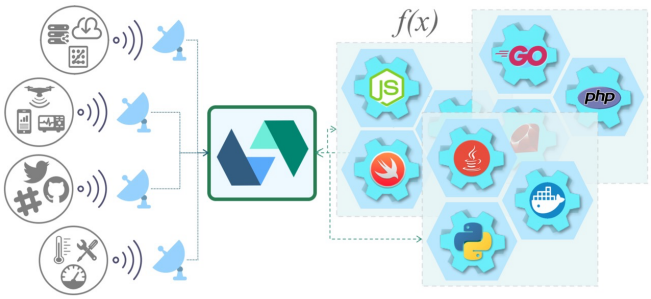


Open-source serverless platforms

- Can run on commodity hardware
- Popular serverless platforms
  - Apache OpenWhisk https://openwhisk.apache.org
  - OpenFaaS https://www.openfaas.com
  - Fission https://fission.io
  - Knative https://knative.dev
  - Nuclio https://nuclio.io
- Most platforms rely on Kubernetes for orchestration and management of serverless functions
  - Configuration and management of containers inside which functions run
  - Container scheduling and service discovery
  - Elasticity management

OpenWhisk

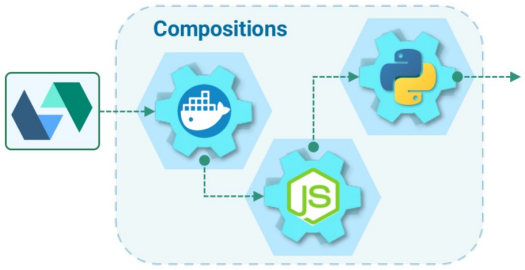
- Distributed serverless platform that executes functions in response to events
  https://openwhisk.apache.org



- Functions run inside Docker containers
- Support for multiple container orchestration frameworks

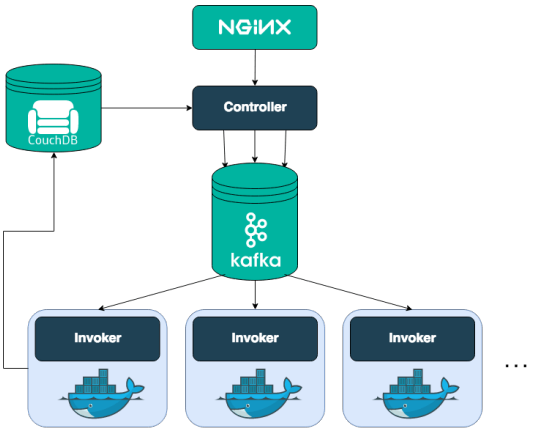
OpenWhisk

- Developers write functions, called actions
  - In any supported programming language
  - Actions are dynamically deployed, scheduled, and run in response to associated events (via triggers) from external sources (feeds) or from HTTP requests
- Functions can be combined into compositions



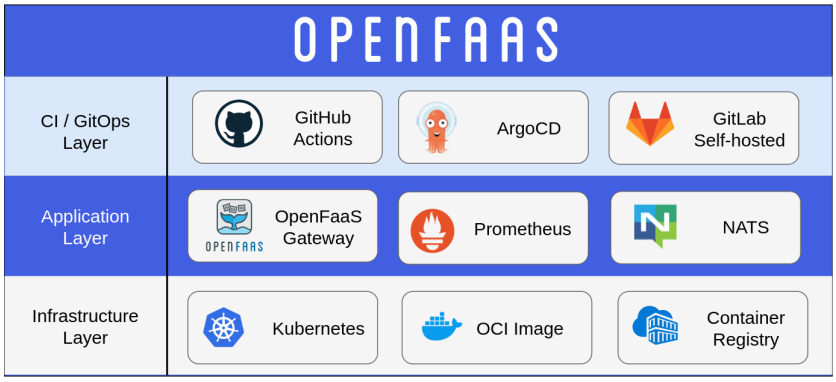
OpenWhisk: architecture

- Powered by multiple frameworks
  - NGINX: entry point that receives HTTP requests and forwards them to Controller
  - Controller: translates HTTP requests into invocations of the appropriate action; manages the lifecycle of actions and coordinates between other components
  - CouchDB (document-oriented NoSQL data store): stores authentication and authorization info, action code, ...
  - Kafka: mediates communication between Controller and Invokers
  - Docker: used by Invokers to execute action code



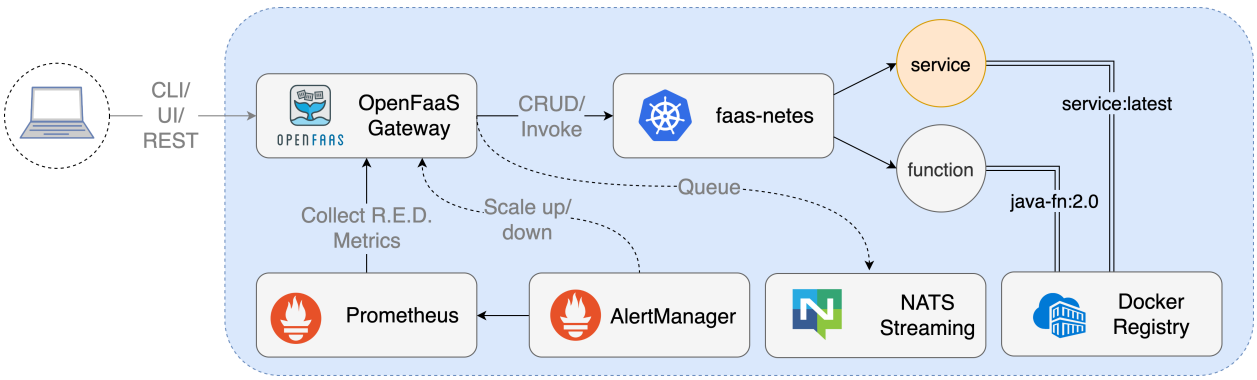
OpenFaaS

- Distributed serverless framework, built on top of Docker and Kubernetes https://www.openfaas.com
- Layered architecture
  - OpenFaaS gateway: provides REST API to manage and scale functions, record metrics
  - NATS: used for asynchronous function execution and queuing https://nats.io
  - Prometheus: provides metrics and enables auto-scaling https://prometheus.io



OpenFaaS

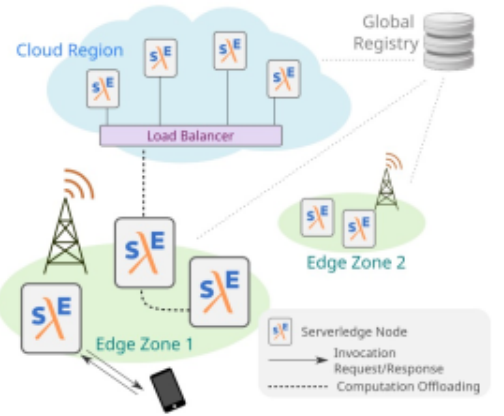
- Conceptual workflow
  - Gateway can be accessed through its REST API, CLI or UI
  - Prometheus collects metrics which are made available via gateway's API and are used for auto-scaling
  - NATS enables function invocations to run asynchronously



https://docs.openfaas.com/architecture/stack/

Serverless in the compute continuum

- Open-source serverless platforms typically rely on centralized components, which make them unsuitable for the compute continuum
- We are developing Serverledge, a decentralized FaaS framework: thesis opportunities!
  https://github.com/serverledge-faas/serverledge



Russo Russo et al., Decentralized Function-as-a-Service for the Edge-Cloud Continuum, Percom 2023 http://www.ce.uniroma2.it/publications/serverledgePerCom2023.pdf

Anche se la comunicazione è importante è solo un lato della medaglia, come i processi cooperano e sono sincronizzati è l'altro lato delle medaglia.
In un sistema centralizzato il tempo non è ambiguo. Quando un processo vuole conoscere il tempo fa una semplice chiamata al sistema operativo. Se un processo A chiede il tempo e dopo poco il processo B chiede anch'esso il tempo, il valore che B riceverà sarà certamente più alto o al limite uguale al valore ricevuto da A, non sarà di certo più basso.

Time in distributed systems

- In a DS, processes are executed on network-connected nodes that cooperate to complete a computation and communicate via message exchange
- Observations:
  - Algorithms require synchronization
  - Processes running on different nodes in the DS must have a common notion of time to perform synchronized actions based on time
  - Algorithms require event ordering, meaning determining which event occurred before another
  - In DS, time is important but problematic

All'interno di un sistema distribuito, i nodi devono essere d'accordo dal punto di vista temporale ma anche perché si eseguono algoritmi che richiedono l'ordinamento degli eventi del sistema.

Time in distributed systems

- In a centralized system, it is possible to determine the order in which events occurred come già visto
  - Shared memory and single clock
- In a DS, it is impossible to have a single physical clock common to all processes
- Yet, the global computation can be viewed as a total order of events, by considering the time at which events were generated
- For many distributed algorithms, it is crucial to determine this time or, at the very least, establish the ordering of events: **how?**

Time in distributed systems: solutions

- Solution 1: synchronizing **physical clocks**
  - A physical clock counts the number of seconds
  - Each node in the DS adjusts its physical clock to be consistent with the clocks of other nodes or with a reference clock

i clock fisici però non sono precisi per loro natura. Dietro c'è un grosso discorso che parte dagli orologi atomici al cesio 133 per arrivare al sistema Coordinated Universal Time (UTC) (vedi libro Tanenbaum)

- Solution 2: synchronizing **logical clocks**
  - A logical clock counts the number of events
  - In a DS, physical clock synchronization is not necessary; only the ordering of events is required (proposed by Leslie Lamport)

un logical clock altro non è che un numero intero

Rispetto al termine classico di nodi in un sistema distribuito, useremo il termine processo per indicare i partecipanti che si coordinano tra loro per portare a termine un algoritmo tramite scambio di messaggi.

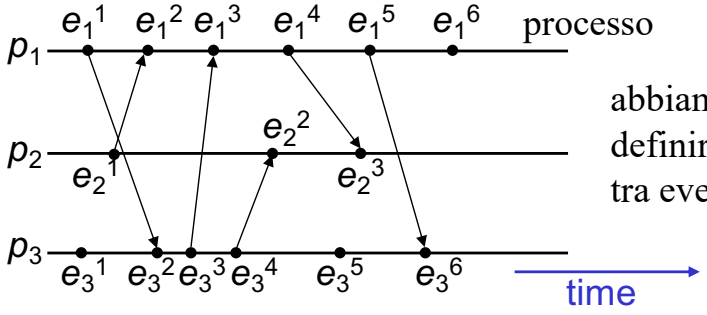
Model of distributed computation

- Components of DS: *N* processes and communication channels
- Each process p_i ($1 \le i \le N$) generates a sequence of events
  - e_i^k: *k*-th event generated by p_i
  - Internal events: state changes in the process
  - External events: send/receive messages
- Evolution of computation represented with space-time diagram

  $\rightarrow_i$: ordering relation between two events in p_i

  $e \rightarrow_i e'$ if and only if *e occurred before e'* in p_i ordinamento per eventi nello stesso processo

ovviamente ciascun processo conosce l'ordine dei propri eventi

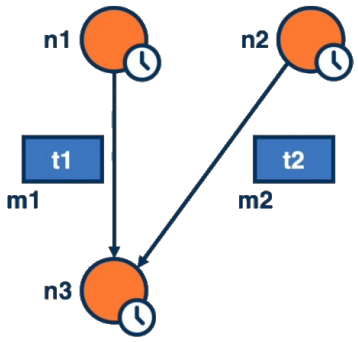


abbiamo come obiettivo definire l'ordinamento tra eventi di processi diversi

Timestamping

- Each process labels events with a timestamp
- Simple solution: each process labels events according to its own physical clock
- Does this work?
  - We can reconstruct the ordering of events on the same node
  - But what about the ordering of events on different nodes?
- In a distributed system, it is impossible to have a single physical clock shared by all processes

nella pratica il formato di un timestamp può essere in formato unix time



Synchronous vs. asynchronous DS

- Properties of a **synchronous DS** che devono essere soddisfatte!
  1. There are constraints on the execution speed of each process
     - The execution time of each step is limited, with both lower and upper bounds
  2. Each message transmitted on a communication channel is received within a limited time
  3. Each process has a physical clock with a known and limited clock drift rate from the real clock
     - Clock drift: deviation of clock from the true time over a period

esistono anche però, sistemi parzialmente sincroni

- In an **asynchronous DS**
  - *There are no constraints* on the execution speed of processes, the message transmission delay, or the clock drift rate

più flessibilità ma comunque serve ordine

Solutions to synchronize clocks

- First solution
  - Synchronize physical clocks of processes with some approximation using synchronization algorithms
  - Each process labels events with the value of its physical clock (which is synchronized with the other clocks with a certain degree of approximation)
  - Timestamping based on physical time (*physical clock*)

- Is it always possible to keep the approximation of physical clocks limited?
  - No, in an asynchronous DS
  - In an asynchronous DS, timestamping cannot be based on physical time; instead, it must be based on logical time (*logical clock*)

Physical clock

- At real-time instant t, the OS reads the time from the hardware clock $H_i(t)$ of the computer and generates the software clock $C_i(t) = aH_i(t) + b$ which approximately measures the physical time instant t for p_i b è l'istante iniziale mentre a è uno scalare che serve a convertire in ms
  - E.g., $C_i(t)$ is a 64-bit number that represents the number of nanoseconds elapsed from a reference instant until time t
  - In general, the clock is not perfectly accurate and may differ from t
  - If C_i behaves well enough, it can be used for event timestamping in p_i
- What should the clock resolution be in order to distinguish two events?
  - Smaller than the time difference ΔT between two relevant events la granularità dell'orologio è fondamentale per distinguere gli eventi

Physical clock in a DS

- In a SD physical clocks are different and can have different values
- *Skew*: instantaneous difference between the values of two clocks
- *Drift*: clocks measure time at different rates (due to physical variations), so over time they diverge from real time prese due macchine diverse, con il tempo i clock indicheranno tempi diversi
- *Drift rate*: difference per unit of time between a physical clock and an ideal clock
  - E.g., drift rate of 2 μsec/sec means that the clock increases by 1 sec + 2 msec every sec
  - Drift rate of standard quartz clocks: 10^{-6} s/s (~1 s in 11-12 days)
  - Drift rate of high-precision quartz clocks: 10^{-7} or 10^{-8} s/s

Universal Coordinated Time (UTC)

- International reference time standard
- Based on atomic time, occasionally corrected using astronomical time
  - 1 second is defined as the time is takes for a cesium-133 atom to complete 9,192,631,770 transitions
- Physical clocks using atomic oscillators are extremely accurate, with a drift rate as low as 10^{-13} s/s
- Atomic clock outputs are broadcast by terrestrial radio stations and satellites (e.g., GPS)
  - In Italy, Istituto Galileo Ferraris
- Nodes equipped with receivers can synchronize their clocks with these signals
  - Accuracy of signals from terrestrial radio stations: 1 to 10 ms
  - Accuracy of signals from satellites: 0.5 ms down to 50 ns

L'obiettivo degli algoritmi di sincronizzazione dei clock è di mantenere una deviazione tra i rispettivi clock tra due macchine qualunque in un sistema distribuito, all'intero di un limite detto precisione π :

$$\forall t, \forall p, q : |C_p(t) - C_q(t)| \leq \pi$$

Synchronizing physical clocks

- How to synchronize physical clocks with the atomic clock or with each other?
- **External** synchronization: the clocks C_i (i = 1, 2, … N) are synchronized with a time source S (UTC), such that, for a given real-time interval I

  $|S(t) - C_i(t)| \leq \alpha$ for $1 \leq i \leq N$ in the interval I

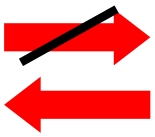
  - The clocks C_i have an **accuracy** of α, where $\alpha>0$
- **Internal** synchronization: two clocks C_i and C_j are synchronized with each other, so that

  $|C_i(t) - C_j(t)| \leq \pi$ for $1 \leq i,j \leq N$ in the interval I

  - The clocks C_i *and* C_j have a **precision** of π, where $\pi>0$

Synchronizing physical clocks

Internal synchronization



External synchronization

- Internally synchronized clocks are not necessarily synchronized externally
  - All clocks can collectively drift from an external source while still remaining synchronized with each other within a bound *D*

- If the set of processes is externally synchronized with accuracy of α, then it is also internally synchronized with a precision of 2α

dunque $\pi = 2\alpha$

Correctness of physical clocks

- A hardware clock *H is* correct if its drift rate is between $-\rho$ and $+\rho$ with $\rho > 0$
- If clock H is correct, the error made when measuring a real-time interval $[t, t']$ (with $t'>t$) is bounded:

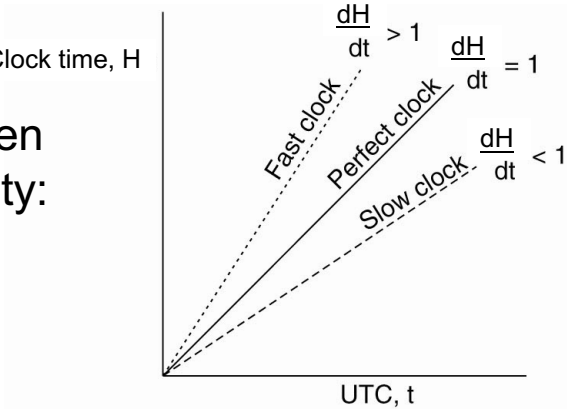
$$(1 - \rho)\,(t' - t) \leq H(t') - H(t) \leq (1 + \rho)\,(t' - t)$$

  - This avoids "jumps" in the clock value

- For a software clock C, it is often sufficient to require monotonicity:

$$t' > t \Rightarrow C(t') > C(t)$$

il tempo può soltanto crescere



When to synchronize physical clocks?

- Due to the drift rate, after a certain time, the clocks in a DS will become misaligned again → periodic synchronization is required to realign them
- Let's consider two clocks with the same maximum drift rate ρ from UTC
- Assume that after synchronization, the two clocks drift in opposite directions from UTC
  - After Δt from synchronization, they will have deviated by at most $2\rho\Delta t$
- To ensure that the difference between the two clocks remains within a desired tolerance δ, they must be resynchronized at least every $\delta/2\rho$

Internal synchronization in synchronous DS

- Internal synchronization algorithm between 2 processes in a synchronous DS
  - p_1 sends its local clock value t to p_2 via a message m, with transmission time T_{trasm}
  - p_2 receives m and sets its clock to $t + T_{trasm}$
    - T_{trasm} is unknown, but since the system is synchronous $T_{min} \leq T_{trasm} \leq T_{max}$
    - Let $u = (T_{max} - T_{min})$ be the transmission time uncertainty
  - Clock setting rule: if p_2 sets its clock to $t + (T_{max} + T_{min})/2$, the optimal lower bound on the clock skew is $u/2$
- Generalization: the algorithm can be extended to N processes
  - The optimal lower bound on the clock skew is $u(1 - 1/N)$
- Asynchronous DS
  - Message delays are unknown and unbounded: $T_{trasm} = T_{min} + x$, with $x \geq 0$ and unknown
  - Different physical clock synchronization algorithms are required

T_{trasm} non è noto a priori ma essendo sincrono il sistema abbiamo il lower e l'upper bound

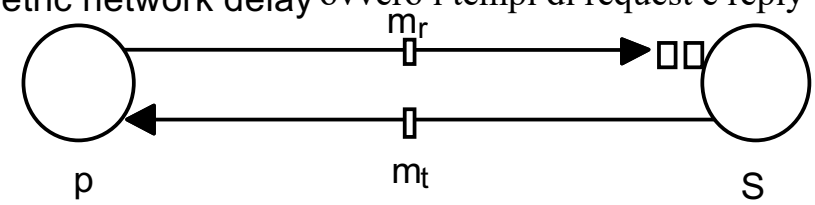
Physical synchronization via a time service

- A time service can provide the current time with a given precision
  - Equipped with a UTC receiver or a high-accuracy clock
- The group of processes that needs to synchronize uses a time service
  - Time service types: centralized or distributed
- Centralized time service
  - Request-driven: **Cristian's algorithm** (1989)
    - External synchronization
  - Broadcast-based: **Berkeley Unix algorithm** - Gusella & Zatti (1989)
    - Internal synchronization
- Distributed time service
  - **Network Time Protocol** (NTP) usato nella pratica oggi
    - External synchronization
    - Internet-scale

basato sulla stessa idea appena vista

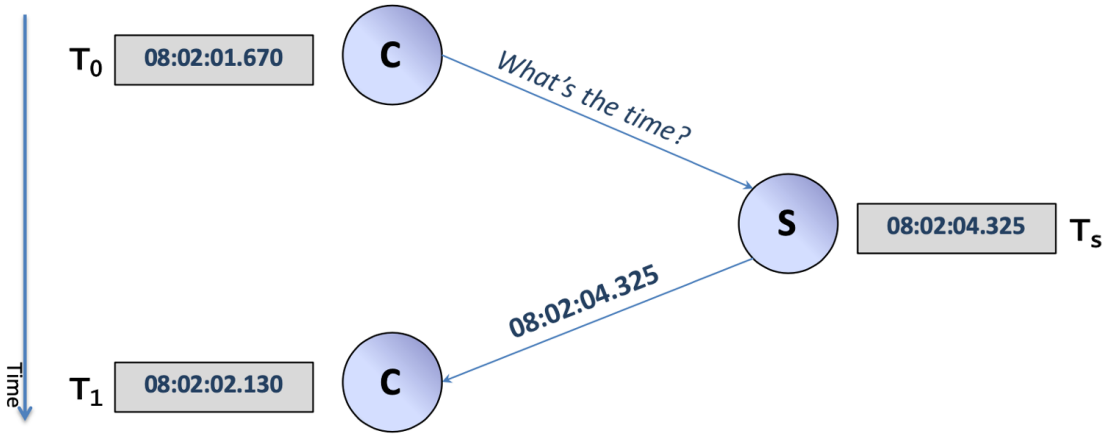
Cristian's algorithm

- A (passive) time server *S* receives the signal from a UTC source → external synchronization
- A process *p* requests the time by sending a request m_r and receives the time *t* in the reply m_t from *S*
- Process *p* sets its clock to $t + T_{round}/2$
  - T_{round} is the round-trip time measured by *p*, assume symmetric network delay ovvero i tempi di request e reply sono uguali



- Observations
  - A single time server may fail
    - Solution: use a group of synchronized time servers
  - Malicious time servers are not handled risolvibile tramite autenticazione
  - Reasonable accuracy only if T_{round} is small → suitable for low-latency LAN

Cristian's algorithm: example



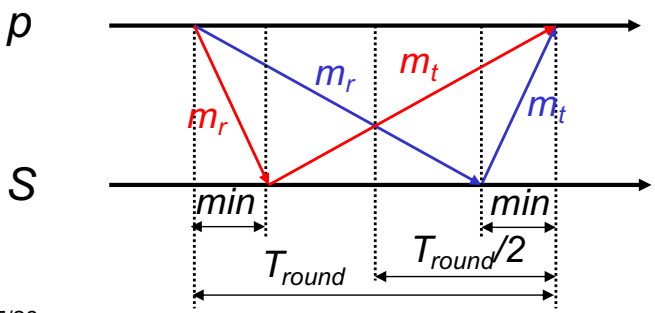
- $T_{round} = T_1 - T_0 = 460$ ms
- C sets its clock to $T_s + T_{round}/2$ = 08:02:04.325 + 230 ms = 08:02:04.555
- C clock increases by 2.425 s

Cristian's algorithm: accuracy

Case 1: *S* cannot insert *t* into m_t before *min* time units have elapsed since *p* sent m_r
- *min* is the minimum transmission time between *p* and *S*

Case 2: *S* cannot insert *t* into m_t after the instant when m_t arrives at *p* minus *min* tempo minimo di trasmissione tra client e server

- The time at *S* when m_t arrives at *p* lies in the interval $[t + min, t + T_{round} - min]$
  - Interval width: $T_{round} - 2\ min$
- Accuracy α: $\pm(T_{round}/2 - min)$



l'accuracy dipende da T_{round} dunque questo algoritmo non è adatto a reti sparse geograficamente

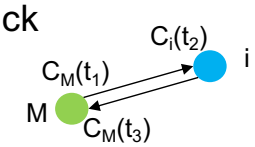
Berkeley algorithm

- Internal synchronization algorithm for a group of nodes in cui il master comunica il tempo ai worker
- The *master* node (*active* time server) broadcasts a request to all nodes, including itself, to get the clock values of the other nodes (*workers*)
- The master uses the RTTs to estimate the clocks of the workers
  - δ_i: difference between *M* clock and worker *i* clock (calculated similarly to Cristian's algorithm)

    $\delta_i = (C_M(t_1) + C_M(t_3))/2 - C_i(t_2)$

    $C_M(t_1)$ and $C_M(t_3)$: master's clock at sending and receiving

    $C_i(t_2)$: worker *i* clock when reply is sent

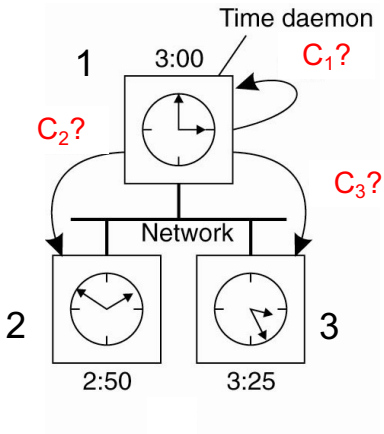


l'interazione tra master e worker è simile all'algoritmo di Cristian

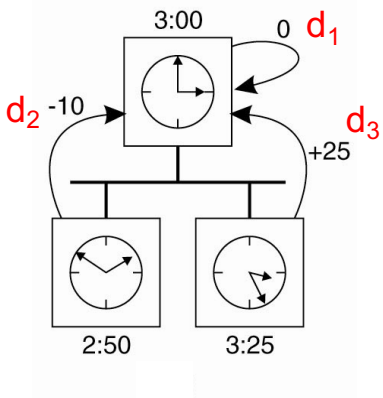
- Master calculates the average of of all δ_i (including its own) and sends correction value to workers
  - If backward adjustment needed: slow down clock

Berkeley algorithm: example

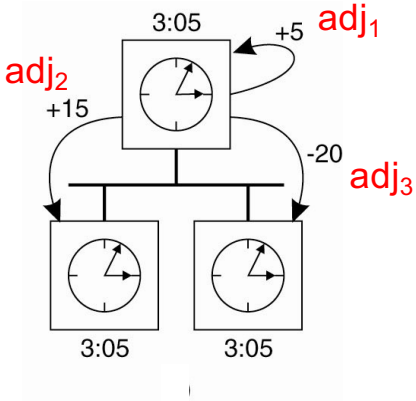
l'aggiustamento del tempo in avanti non porta problemi mentre il portare indietro il tempo viola il requisito della monoticità del tempo. In realtà il clock 3 rallenterà l'avanzata del tempo con interrupt, per colmare il divario. Il numero di interrupt è pari al tempo di solo down diviso il tempo di interrupt della CPU



Master broadcasts a request to all nodes



Master calculates the average of clock differences:
avg = (-10+25+0)/3 = 5



Master sends a correction value to each worker (adj_i = avg - d_i)

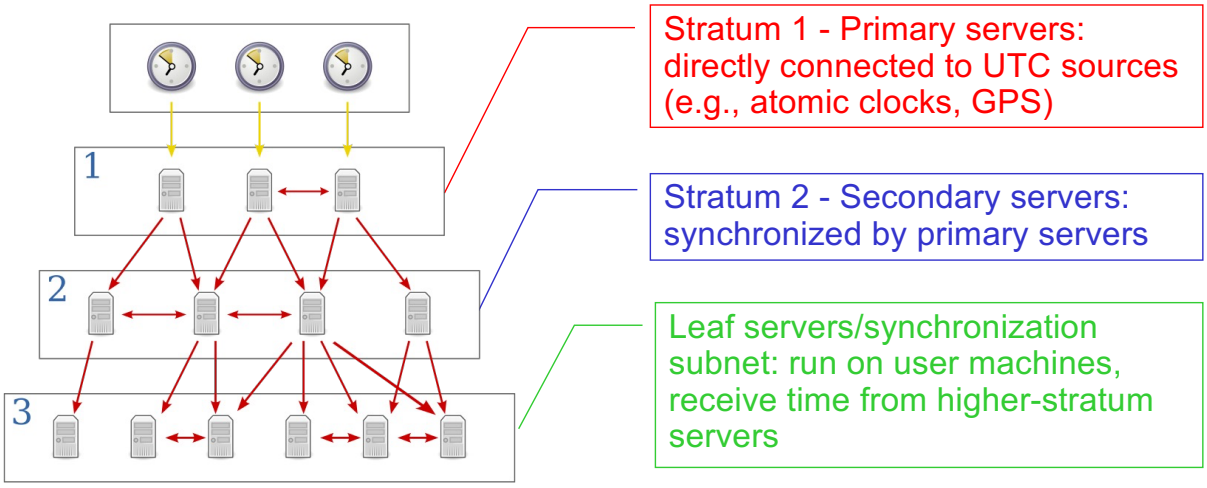
Berkeley algorithm: features

- Precision: depends on the maximum nominal RTT
  - The master ignores clock values associated with RTTs above this maximum
  - Outliers are discarded to improve accuracy

- Fault tolerance
  - Master failure: another node is elected as master using an election algorithm
  - Tolerance to arbitrary (Byzantine) behavior:
    - Workers sending incorrect clock values
    - The master only considers clock values that differ from each other by at most a specified threshold

Network Time Protocol (NTP)

- Time service for Internet (RFC 5905)
  - Provides external synchronization accurate wrt UTC
  - Configurable on multiple OSs
  - On GNU/Linux: `ntpd` daemon for automatic synchronization, `ntpdate` for manual synchronization
- Hierarchy of servers



da notare le interconnessioni esterne e interne
e come gli strati inferiori si sincronizzano con quelli superiori

NTP

- Architecture
  - Scalable and robust time service
  - Redundant time servers and network paths
  - Authentication of time sources to prevent

- NTP fault tolerance and reconfiguration
  - The synchronization subnet reconfigures automatically in case of failures
  - Primary server loses connection to UTC source: it becomes a secondary server
  - Secondary server loses connection to its primary (e.g., primary crashes): it can switch to another primary server

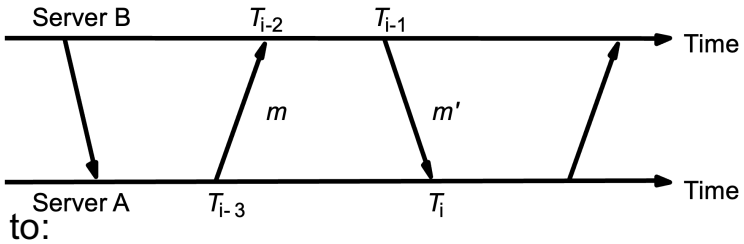
NTP: synchronization modes

1. Multicast mode
   - Server in a high-speed LAN multicasts its clock to others
   - Other servers set their clocks assuming a certain transmission delay
   - Accuracy: relatively low (no way to measure RTT)
2. Procedure call mode
   - Server responds to requests like in Cristian's algorithm non vedremo come
   - Accuracy: higher than multicast
   - Use case: when multicast is not available, makes sense for smaller networks
3. Symmetric mode
   - Server pairs exchange timing information
   - Accuracy: very high
   - Commonly used for higher-level hierarchical servers (Stratum 2 and above)

- All synchronization modes use UDP perché serve la velocità non fault tolerance

NTP: symmetric mode

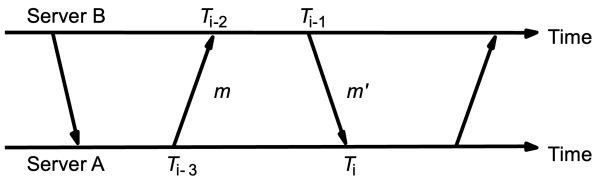
- Servers A and B exchange message pairs (*m*, *m'*) to improve synchronization accuracy
- Servers exchange messages with timestamps
  - A sends to B a message *m, which* contains:
    - Timestamp T_{i-3}: time when A sends *m*
  - B responds to A with message *m*', which contains:
    - Timestamp T_{i-2}: time when B receives *m*
    - Timestamp T_{i-1}: time when B sends *m'*
  - A records T_i when it receives *m'*
  - Time difference T_{i-1}-T_{i-2} may be non-negligible



- Timestamps allow servers to:
  - Estimate round-trip delay
  - Adjust their clocks more precisely

NTP: symmetric mode

- For each pair of messages *m* and *m'* exchanged between two servers, NTP estimates:
  - Offset o_i between the two clocks
  - Round-trip delay d_i, which is the total transmission time of *m* and *m'*



- Let's define:
  - *o*: real offset of clock B relative to clock A ($o = clock_B - clock_A$)
  - *t* and *t'*: actual transmission times of messages m and m', respectively
- For each pair of messages:

$$T_{i-2} = T_{i-3} + t + o \quad \text{and} \quad T_i = T_{i-1} + t' - o$$

$$\text{where } d_i = t + t' = T_{i-2} - T_{i-3} + T_i - T_{i-1}$$

- Subtracting the equation for T_i from the one for T_{i-2}, we solve for *o*:

$$o = [(T_{i-2} - T_{i-3}) + (T_{i-1} - T_i)]/2 + (t' - t)/2$$

Difference between the transmission times used to adjust the offset estimate

The estimated offset o_i is given by: $o_i = [(T_{i-2} - T_{i-3}) + (T_{i-1} - T_i)]/2$

The real offset *o* is: $o = o_i + (t' - t)/2$

NTP: symmetric mode

- The accuracy of the offset estimation is determined by the round-trip delay d_i
  - Since both t and t' are positive, the real offset o will lie within: $o_i - d_i/2 \leq o \leq o_i + d_i/2$
  - Thus, o_i is the estimated offset and its *accuracy* is $d_i/2$
- NTP servers apply a statistical filtering algorithm on the 8 most recent $<o_i, d_i>$ pairs, choosing as the estimate of o the o_j value corresponding to the minimum d_j
- They then apply a peer selection algorithm to possibly change the peer used for synchronization
  https://www.eecis.udel.edu/~mills/ntp/html/warp.html
- NTP accuracy:
  - 10 ms on the Internet
  - 1 ms on LAN

Perfect synchronization over networks is actually impossible

Google's TrueTime (TT)

- Distributed synchronized clock with bounded non-zero error
  - Designed by Google for Spanner, a global-scale, multi-version, distributed NewSQL database
  - Relies on a well-engineered tight clock synchronization available on all Google servers thanks to GPS and atomic clocks
  - Enables applications to generate monotonically increasing timestamps
  - *Cons*: TT requires special hardware and custom-build tight clock synchronization protocol, which is infeasible for many systems
    - Google also relies on its very high throughput, global fiber optic network linking its data centers

Time in asynchronous DS

- Physical clock synchronization algorithms
  - They (e.g., NTP) estimate transmission times to synchronize clocks between nodes
  - Accuracy depends on knowing the upper and lower bounds of transmission times
- Limitations in asynchronous DS
  - No constraints on transmission times → physical synchronization is used in asynchronous DS but with limited accuracy due to variable transmission times non si possono effettuare stime
  - We cannot use physical time to order events occurring on different nodes
- Logical time as a solution
  - What really matters is that processes agree on the order of events rather than the exact physical time at which events occurred
  - We use logical clocks (e.g., Lamport clocks) to maintain event ordering across nodes

Se due processi non interagiscono, non è necessario che siano sincronizzati perché la mancanza di sincronizzazione non sarebbe osservabile e non causerebbe alcun problema. Inoltre non tutti i processi devono concordare sul tempo, ma dovrebbero concordare sull'ordine in cui gli eventi occorrono.

Logical time

- Idea: order events based on two intuitive observations:
  1. Two events that occurred on the same process p_i happen exactly in the order observed by p_i
  2. When a message is sent from p_i to p_j, the send event happens before the receive event questo perché serve un tempo non nullo per ricevere un messaggio.
- Lamport (1978) and the **happened-before** relation
  - Leslie Lamport introduces the concept of the happened-before relation (also known as *causal ordering* or *precedence relation*)

  $\rightarrow_i$: ordering relation between two events that occurred on p_i

  $\rightarrow$: happened-before relation between any two events

L. Lamport, Time, Clocks and the Ordering of Events in Distributed Systems, 1978 https://lamport.azurewebsites.net/pubs/time-clocks.pdf

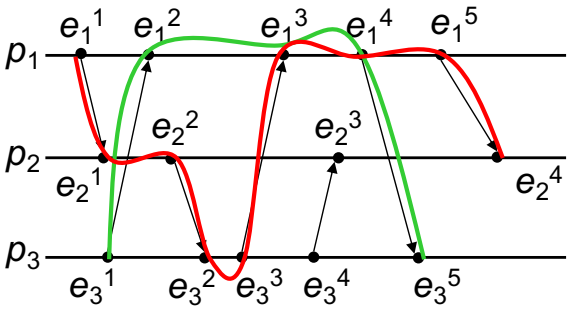
Happened-before relation

- Two events *e* and *e'* are in the happened-before relation (denoted by $e \rightarrow e'$) if one of the following is true:
  1. $\exists\ p_i \mid e \rightarrow_i e'$
  2. $e = send(m) \wedge e' = receive(m)$

     e is the event of sending message *m*, and *e'* is the corresponding receive event
  3. $\exists\ e, e', e'' \mid (e \rightarrow e'') \wedge (e'' \rightarrow e')$

     The happened-before relation is transitive
- By applying these rules, we can construct a sequence of events $e_1, e_2, \ldots, e_n$ that are causally ordered
- Observations
  - The happened-before relation defines a partial ordering (properties: *non-reflexive, anti-symmetric*, *transitive*)
  - The sequence $e_1, e_2, \ldots, e_n$ is not necessarily unique
  - For a pair of events, they are not always related by a happened-before relation. In such cases, the events are said to be concurrent (denoted by ||)

Happened-before relation: example



- Sequence $s_1 = e_1^1, e_2^1, e_2^2, e_3^2, e_3^3, e_1^3, e_1^4, e_1^5, e_2^4$
- Sequence $s_2 = e_3^1, e_1^2, e_1^3, e_1^4, e_3^5$
- Can you find another sequence?

- Events e_3^1 and e_2^1 are concurrent

  $e_3^1 \not\rightarrow e_2^1$ and $e_2^1 \not\rightarrow e_3^1$

Se due eventi avvengono in due processi diversi che non scambiano messaggi, tali eventi sono detti concorrenti. Ciò significa che semplicemente non si può dire nulla sul quando sono accaduti e l'ordine

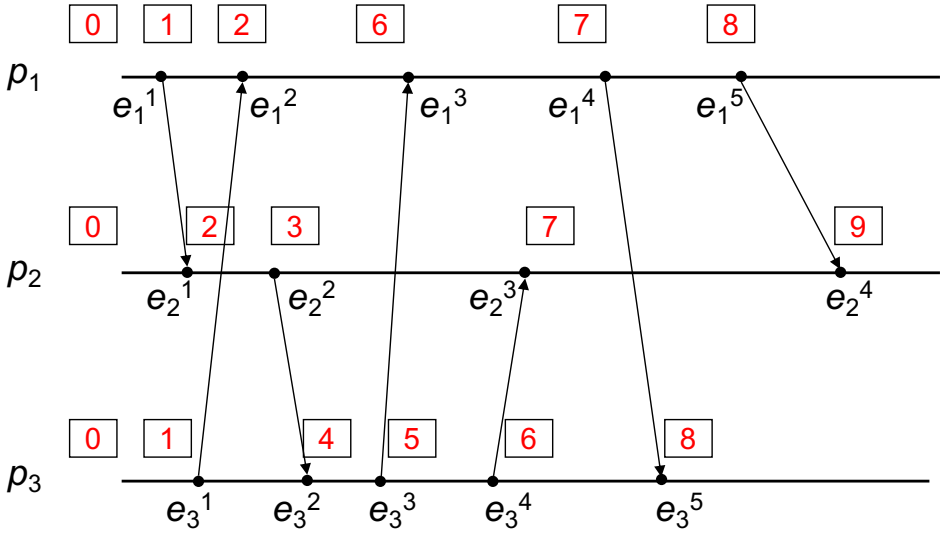
Scalar logical clock

- A monotonically increasing software counter (scalar value), used to order events in DS
  - Scalar logical clock (or Lamport clock): we call it "scalar" to distinguish from vector clock
- Does not rely on physical clock
- Each process p_i has its own scalar clock L_i, and it uses it to apply timestamps to events
- Denote $L_i(e)$ as the timestamp, based on the scalar clock, applied by process p_i to event e
- Key property: **if $e \rightarrow e'$ then $L(e) < L(e')$**
- Key observations:
  - L(e)<L(e') non implica $e \longrightarrow e'$
  - The happened-before relation introduces a partial ordering of events: in the case of concurrent events, it is impossible to determine which event actually happens first

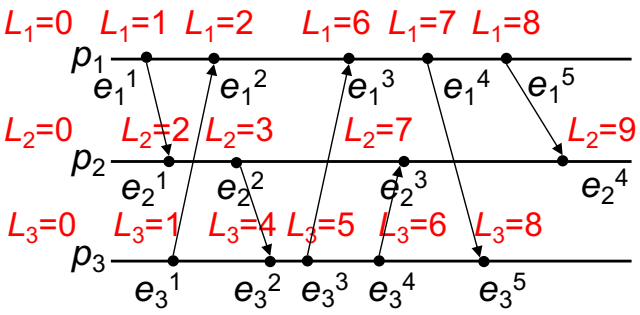
Scalar clock: update

- Scalar clock update algorithm by Lamport
- Initialization: each process p_i initializes its scalar clock L_i to 0 ($\forall\ i = 1, \ldots, N$)
- Internal event: before executing an internal event, p_i increments L_i by 1: $L_i = L_i + 1$
- Sending a message: when p_i sends message m to p_j
  - Increment the value of L_i: $L_i = L_i + 1$
  - Attach the timestamp $t = L_i$ to message m
  - Execute the *send*(m) event
- Receiving a message: when p_j receive message m with timestamp t
  - Update its scalar clock $L_j = max(t, L_j)$
  - Increment the value of L_j: $L_j = L_j + 1$
  - Execute the *receive*(m) event

Scalar clock: example



Scalar clock: example

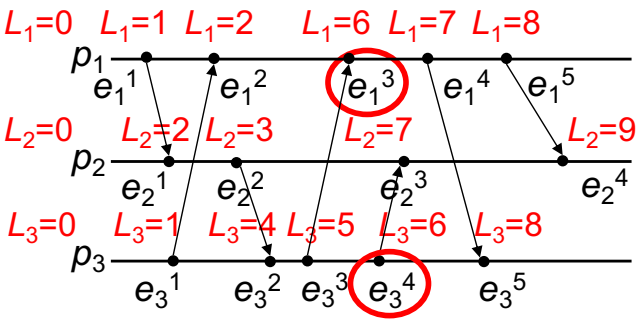


- Observations
  - $e_1^1 \rightarrow e_2^1$ and the respective timestamps reflect this causal relationship (L_1=1 and L_2=2); indeed, *if* $e_1^1 \rightarrow e_2^1$ *then* $L(e_1^1) < L(e_2^1)$
  - $e_1^1 \parallel e_3^1$ and the respective timestamps are equal (L_1=1 and L_3=1); indeed, *if* $L(e_1^1) \geq L(e_3^1)$ then $e_1^1 \not\rightarrow e_3^1$
  - $e_2^1 \parallel e_3^1$ and the respective timestamps differ (L_2=2 and L_3=1); indeed, *if* $L(e_2^1) \geq L(e_3^1)$ then $e_2^1 \not\rightarrow e_3^1$

Total ordering of events

- Problem: using a scalar clock, two or more events can have the same timestamp. How can we establish a total ordering of events to avoid two events occurring at the same logical time?
- Solution: in addition to the scalar clock, we use the process number on which the event occurred
  - We establish a total ordering ($\prec$) between the processes
- Total order relation between events, denoted by $e \Rightarrow e'$: if e is an event on process p_i and e' is an event on process p_j, then $e \Rightarrow e'$ if and only if:
  1. $L_i(e) < L_j(e')$ esempio: <40,2> e <40,3> allora si prende l'evento del processo con id 2. <39,2> e <38,3> si prende l'evento del processo con id 3.
  2. $L_i(e) = L_j(e') \wedge p_i \prec p_j$
- This relation is used in some distributed mutual exclusion algorithms to ensure a total ordering of events across processes

Total ordering of events: example



- e_1^3 and e_3^4 have the same scalar clock value: how to order them?
- $e_1^3 \Rightarrow e_3^4$ since $L_1(e_1^3) = L_3(e_3^4)$ and $p_1 \prec p_3$

Scalar clock and its limitation

- The scalar clock has the following property
  - If $e \rightarrow e'$, then $L(e) < L(e')$
- However, it is not possible to guarantee that:
  - If $L(e) < L(e')$, then $e \rightarrow e'$
    See slide 36: $L(e_3^1) < L(e_2^1)$ but $e_3^1 \,||\, e_2^1$
- Consequence: it is not possible to determine, by just looking at scalar clocks, whether two events are concurrent or not
- How to overcome this limitation?
- To solve this issue, vector clocks were introduced by Mattern (1989) and Fidge (1991)

Vector clock

- A vector clock for a system with N processes is a vector of N integers
- Each process p_i maintains its own vector clock V_i
- For process p_i, $V_i[\, i \,]$ is the local scalar clock
- Each process uses its own vector clock to assign timestamps to events
- Similar to Lamport's clock, the vector clock is attached to the message m, and the timestamp becomes vector-valued
- The vector clock captures the full characteristics of the happened-before relation

$$e \rightarrow e' \text{ if and only if } V(e) < V(e')$$

la condizione va verificate componente per componente nel vettore!

vettore di clock logici. Ad esempio per un sistema con 3 processi si ha un vettore di 3 componenti

Vector clock: meaning and comparison

- Given the vector clock V_i
  - $V_i[\,i\,]$ is the number of events generated by p_i
  - $V_i[\,j\,]$ with $i \neq j$ is the number of events that occurred at p_j, which p_i knows about

- Comparison of vector clocks
  - $V = V'$ if and only if $\forall\, j$: $V[\,j\,] = V'[\,j\,]$
  - $V \leq V'$ if and only if $\forall\, j$: $V[\,j\,] \leq V'[\,j\,]$
  - $V < V'$ (and thus the event associated with V *precedes* the event associated with V') if and only if
    - $\forall\, i \in [1, \ldots, N]$: $V[\,i\,] \leq V'[\,i\,]$

      and
    - $\exists\, j \in$ in $[1, \ldots, N]$: $V[\,j\,] < V'[\,j\,]$
  - $V \,||\, V'$ (and thus the event associated with V *is concurrent* to the event associated with V') se e solo se
    - $\neg(V < V')$ and $\neg(V' < V)$

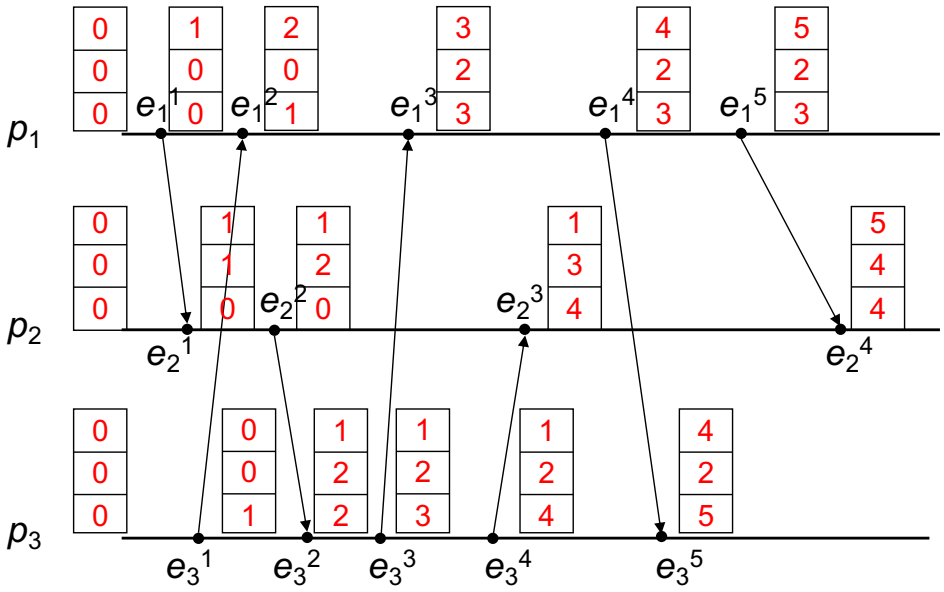
Vector clock: update

- Initialization: each process p_i initializes its own vector clock V_i: $V_i[k]=0 \;\; \forall\, k = 1, 2, \ldots, N$
- Internal event: before executing an internal event, p_i increments its own vector clock component $V_i[i]$ by 1: $V_i[i] = V_i[i] + 1$
- Sending a message: when p_i sends message m to p_j
  - Increment its own clock component $V_i[i]$ by 1: $V_i[i] = V_i[i] + 1$
  - Attach the vector timestamp $t = V_i$ to message m
  - Execute the *send(m)* event
- Receiving a message: when p_j receives message m with timestamp t:
  - Update its vector clock component for each process K: $V_j[k] = max(t[k], V_j[k]) \;\forall\, k = 1, 2, \ldots, N$
  - Increment its own clock component $V_j[j]$ by 1: $V_j[j] = V_j[j] + 1$
  - Execute the *receive*(m) event

non aumenta il valore di tutti i clock, ma solo quello associato a j

vedi gli esempi per capire meglio

Vector clock: example



Using vector clocks to compare events

- Let $V(e)$ and $V(e')$ be the vector timestamps of two events e and e'
- By comparing $V(e)$ and $V(e')$, we can determine if the events are in happened-before relationship or concurrent

1	1
2	2
0	2
V	V'

$V(e) < V'(e)$ and thus $e \rightarrow e'$

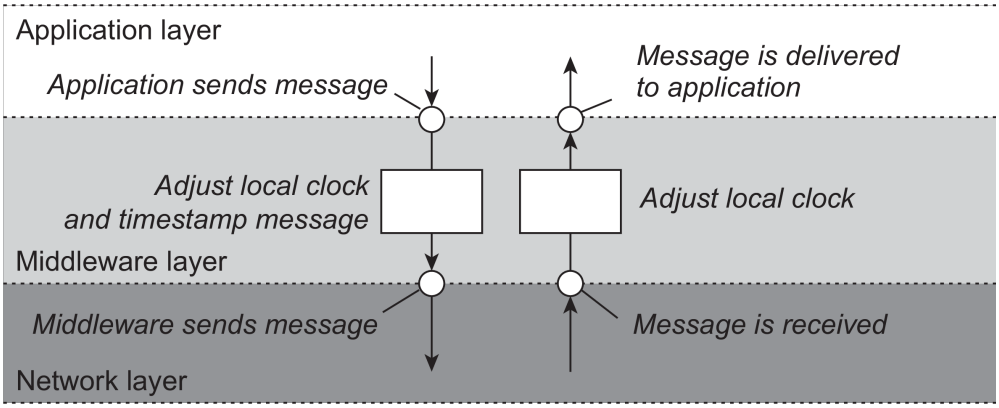
1	1
2	0
0	2
V	V'

$V(e) \neq V'(e)$ and thus $e \parallel e'$
nei componenti 2 e 3 sono diversi!

Examples of applications of logical time

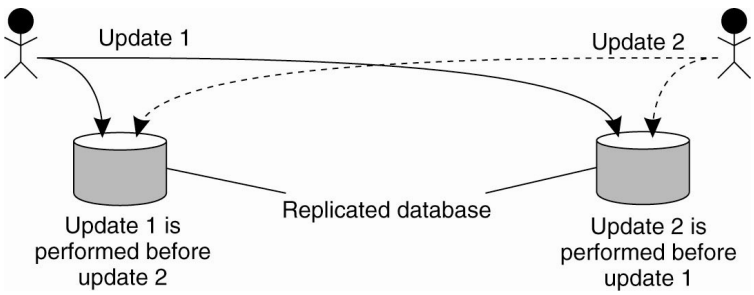
- We will examine two applications of logical time:

1. Scalar clock for totally ordered multicasting dove si usano clock scalari
2. Vector clock for causally ordered multicasting
   dove si usano clock vettoriali



Totally ordered multicasting: problem

- To ensure that concurrent updates on a replicated database are seen in the same order by every replica, you need to synchronize and order the updates across all replicas, ensuring consistency
  - p_1: add \$100 to a bank account (initial value: \$1000)
  - p_2: increment the account by 1%
  - There are two replicas and, without synchronization, each replica may apply the updates in a different order
    - Replica #1 ← 1111 (first p_1 and then p_2)
    - Replica #2 ← 1110 (first p_2 and then p_1)



Totally ordered multicasting

- A totally ordered multicast is a multicast operation in which all messages are delivered in the same order to every recipient
- Assumptions: abbastanza forti
  - Reliable communication: no message loss occurs
  - FIFO ordered communication: messages sent from p_i to p_j are received by p_j in the same order in which p_i sent them p_j

- Let's consider two solutions to implement totally ordered multicasting
  - Centralized
  - Distributed, using scalar clocks

Totally ordered multicasting: solutions

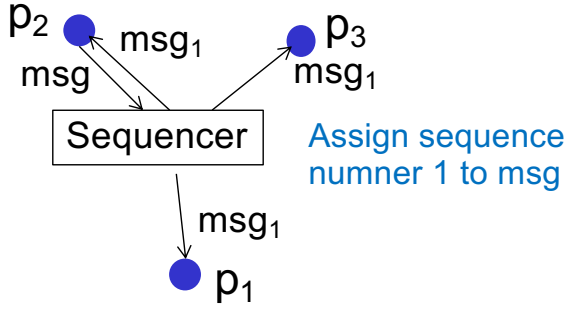
- Centralized solution: a single coordinator (*sequencer*) manages the order of messages)
- The sequencer ensures that messages are delivered to all recipients in the same order
  - Each process p_i sends its update message to the sequencer
  - The sequencer then assigns a unique sequence number to each update message and multicasts the message
  - Each process p_i executes the update in the order of the sequence numbers

✓ Simple to implement

✗ Single point of failure

✗ Potential bottleneck at the sequencer

ovviamente non è la soluzione ideale



Totally ordered multicasting: solutions

- Distributed algorithm using scalar clocks
  - Tag message with scalar clock: each update message msg_i sent by process p_i is tagged with p_i's scalar clock at the time of sending
  - Multicast the message: p_i multicasts to all processes (including itself)
  - Queue messages locally: each receiving process p_j places msg_i in a local queue $queue_j$, sorted by timestamp (scalar clock value)
  - Acknowledge receipt: p_j multicasts an ack for msg_i to all processes, indicating it has received the message
  - Deliver message to application: p_j delivers msg_i to the application only if all of the following conditions are met:
    1. msg_i is at the head of $queue_j$ (and all acks for msg_i have been received by p_j)
    2. for every other process p_k, there is a message msg_k in $queue_j$ with a timestamp greater than msg_i

    msg_i is delivered only when p_j knows that no other process can multicast a message with a smaller or equal timestamp than msg_i: this ensures that all processes deliver messages in the same total order

ricorda l'algoritmo di Berkeley un po'

attenzione a non confondersi, gli acks devono arrivare agli altri processi

ribadisce che msg_i debba essere in testa alla coda

50

Quando viene affidato all'applicazione il messaggio è rimosso dalla coda e gli ack rimossi

La soluzione funziona perché ogni processo ha la stessa copia della coda, quindi tutti i messaggi sono consegnati con lo stesso ordine ovunque.

Totally ordered multicasting: distributed

✓ Decentralized: no single point of failure and works better than centralized one with larger systems

✗ Slightly more complex than centralized solution

✗ Communication cost:

- Ack overhead: $O(N^2)$ acks for each message
  - Each process sends N-1 acks for each message it receives, and this happens for every process in the system

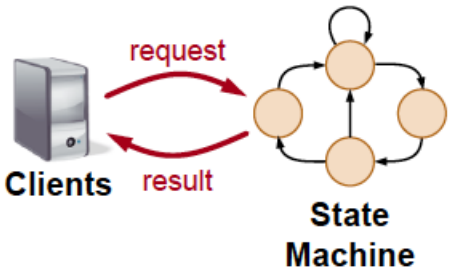
inoltre se c'è molta latenza oppure i processi falliscono la soluzione non è perfetta

- Mechanism designed by Lamport for **state-machine replication**

concetto importante che tornerà negli algoritmi di consenso

State machine replication

- Software replication technique applied to a service that can be implemented as a deterministic state machine
  - Used to replicate the state of a machine ("state" can be any data or process state) across multiple replicas
  - A state machine consists of a series of states and transitions between those states
  - Each operation in the system causes a transition between states: next state of service = f(current state, op)
  - Deterministic state machine: for every given state and operation, there is exactly one defined next state
- Each replica processes the same sequence of operations in the same order, ensuring that all replicas stay in sync (i.e., end up in the same state)



State machine replication

- To achieve fault tolerance, service is replicated on several nodes, all of them running a state machine replication (SMR) middleware
  - Set of replicas behaves as a "centralized" server
- Service makes progress as long as any majority of replicas are up
- Replicas can recover by reapplying the operations in the same order
- Used in systems that require strong consistency and fault tolerance, such as distributed databases, file systems, and distributed transaction systems

State machine replication

- Lamport's totally ordered multicast ensures consistency, but it may face scalability challenges
  - Particularly in systems with high latency or frequent failures appunto
- Other well-known solutions for ensuring state-machine replication we examine
  - Paxos consensus algorithm
  - Raft consensus algorithm

che vedremo entrambi

Causally ordered multicasting

- Ensures that a message is delivered only after all the messages that causally precede it (i.e., events that are causally related) have already been delivered
  - Causal relation between two events: the second event is *potentially* influenced by the first event, meaning there is a cause-effect relationship
  - Goal: deliver the cause before the effect to maintain the causal order of events
  - Weaker than totally ordered multicasting
    - We do not require all messages to be delivered in a global total order. Instead, we only require that causal relationships are respected
  - Example:
    - p_1 sends messages m_A and m_B
    - p_2 sends messages m_C and m_D
    - m_A causes m_C
    - Delivery sequences that are compatible with causal ordering (and FIFO ordering): $m_A\, m_B\, m_C\, m_D$ $m_A\, m_C\, m_B\, m_D$ $m_A\, m_C\, m_D\, m_B$ but $m_C\, m_A\, m_B\, m_D$ is not valid

come per la teoria del clock vettoriale, l'esempio fa capire meglio che le parole

Causally ordered multicasting

anche qui sono forti

- Assumptions: reliable and FIFO ordered communication
- Use vector clocks to solve the problem of causally ordered multicasting in a decentralized way
- Algorithm
  - Sending the message: process p_i sends the message m to all other processes, attaching a vector timestamp (ts_m). The timestamp is p_i's vector clock (V_i) at the time of sending the message
  - Receiving the message: *w*hen p_j receives m from p_i, p_j does not immediately deliver the message to the application. Instead, p_j places the message in a waiting queue
    - $V_j[i]$ counts the number of messages sent by p_i to p_j
  - Conditions for delivering the message: m will be delivered to the application only if both of the following conditions are met:
    1. $ts_m[i] = V_j[i] + 1$ — p_j has received the next message it expects from p_i
    2. $ts_m[k] \leq V_j[k] \ \forall \ k \neq i$ — p_j has seen at least all the messages that p_i has seen up to that point

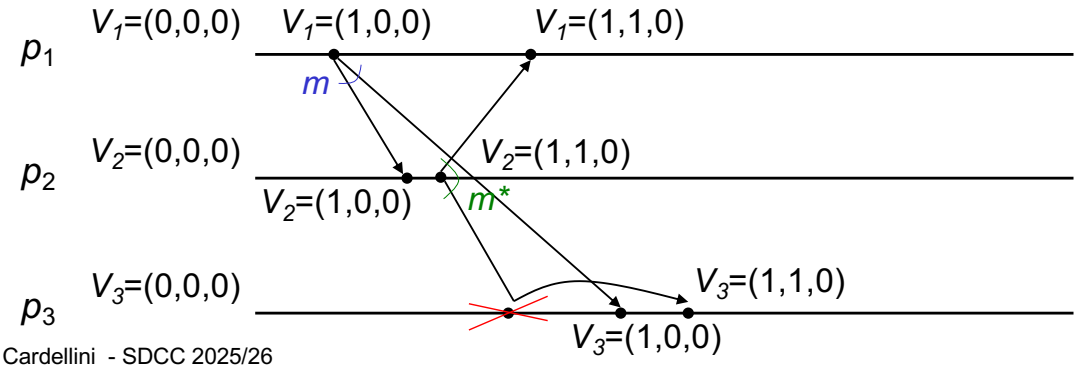
Causally ordered multicasting: example

- p_1 sends m to p_2 and p_3
- p_2 *receives* m and then sends m^* *to* p_1 and p_3
  - Cause-effect relationship between m and m^*
- Suppose that p_3 receives m^* before it receives m
  - The algorithm avoids the violation of causal ordering: p_3 will hold m^* in a waiting queue until m is delivered first

Clock update

p_i sends msg: $V_i[i] = V_i[i] + 1$

p_j receives msg with ts_{msg}: $V_j[k] = \max\{ V_j[k], ts_{msg}[k] \}$



viene rifiutato perché il clock vettoriale di p2 è (1,1,0) mentre quello di p3 è (0,0,0) quindi siccome ho che il messaggio di p1 non è stato ricevuto ovvero V3=(1,0,0) metto in coda il messaggio di p2!

Timestamps in practice

- Timestamps are useful for comparing events in DS, e.g.
  - Reconciling object updates in distributed storage system
  - Restoring system state after crash

    1. Checkpoint the system: a checkpoint is created, capturing the system's state at a specific point in time; 2 Record events: each event is recorded with a timestamp to track the order in which events occurred; 3 After a crash: the checkpoint is restored, and the system replays the events in order using the timestamps
- How to compare timestamps across different processes?

1. Physical timestamps: rely on synchronization of physical clocks across processes
   - Use case: Google Spanner uses TrueTime to maintain consistency
   - Limitation: require all clocks to be synchronized, which can be difficult in geographically distributed systems with high latencies quindi Spanner è utilizzato per sincronizzazione interna non esterna

Timestamps in practice

2. Logical timestamps: use scalar clocks to order events
   - Use case: Oracle DB use systems change numbers based on scalar clocks to track changes to data, ensuring that operations are applied in the correct order
   - Limitation: scalar clocks do not fully distinguish between causally related events and concurrent events che servono magari per sistemi critici con transazioni
3. Vector timestamps: extend logical timestamps by using vector clocks
   - Use case: Amazon DynamoDB uses vector clocks to determine which object version is the most recent by comparing vector timestamps

Main properties of algorithms for concurrent and distributed systems

- **Safety**: nothing bad will happen
  - Undesirable states will not occur
  - Example: If two processes attempt to write to the same resource at the same time, no data corruption or race conditions happen

- **Liveness**: something good will eventually happen
  - The system will make progress and avoid situations where it stops (deadlock or starvation)
  - Example: if a process requests a resource, it will eventually get it

tali proprietà vanno garantite ma come vedremo ciò non è sempre possibile

Mutual exclusion

- N processes want to access a shared resource
- Goal: ensure that each process can exclusively access a shared resource without interference from others
- Components of mutual exclusion algorithm:
  - Critical section (CS): the part of the code where a process accesses the shared resource
  - Trying protocol (TP): instructions that occur before entering the CS, ensuring no conflicts detta anche Trying section (TS)
  - Exit protocol (EP): instructions that occur after exiting the CS, releasing the resource for others

Properties of mutual exclusion algorithms

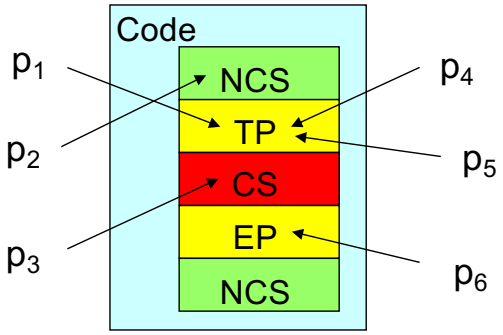
- **Mutual exclusion (ME)** or **safety**
  - At most one process at a time can execute in CS
- **No deadlock (ND)**
  - If one or more processes are blocked in their TS, at least one process will eventually enter and exit the CS
- **No starvation (NS)** or absence of indefinite postponement
  - No process remains blocked forever in the TS; every request to enter the CS is eventually satisfied
- Observations:
  - NS ⇒ ND, but ND ⇏ NS date magari priorità diverse per i processi.
  - NS and ND are liveness properties
  - NS is also a fairness property: every process eventually gets access
- **Ordering**
  - Requests to enter the CS are served in order of arrival (according to the happened-before relation)
  - Stronger fairness than NS: ordering ⇒ NS, but not viceversa

la mancanza di starvation quindi non implica ordinamento

Mutual exclusion in concurrent systems

- Mutual exclusion originated in concurrent systems
- Early mutual exclusion algorithms are based on the use of shared variables to coordinate *N* processes
- Dijkstra's algorithm (1965)
  - Designed for single-processor systems
  - Guarantees ME and ND, does not guarantee NS
- Lamport's bakery algorithm (1974)
  - Designed for shared-memory multiprocessor systems
  - Guarantees ME, ND, and NS

algoritmo che adattiamo a sistemi distribuiti



Lamport's bakery algorithm

- Solution inspired by a real-world situation
  - Waiting to be served at a bakery
- Assumptions on concurrent system model
  - Processes communicate by reading and writing to shared variables
  - Reading and writing to a shared variable are not atomic operations
    - A process may write while another is reading the same variable
  - Each shared variable is owned by a process:
    - Everyone can read it, but only the owner can write to it
    - No process can perform two writes simultaneously
  - The execution speeds of processes are not correlated

assunzioni importanti

- Shared variables:
  - `num[1,…,N]`: array of integers, initialized to `0` sarebbe il numero nella fila
  - `choosing[1,…,N]`: array of booleans, initialized to false
- Local variable: `j`: integer in the range `[1,…,N]`

Lamport's bakery algorithm

```
// non-critical section

// take a ticket                                          doorway
choosing[i] = true;          // start choosing a ticket
num[i] = 1 + max(num[x] : 1 ≤ x ≤ N);
choosing[i] = false;         // finish choosing the ticket
// wait until its number is called, comparing with others   bakery
for j = 1 to N do
    // busy waiting while process j is choosing
    while choosing[j] do NoOp();

    // busy waiting until pi has the smallest number
    // ties are broken by favoring the process with smaller id
    while num[j] ≠ 0 and {num[j], j} < {num[i], i} do NoOp();

// critical section

num[i] = 0;  // release the ticket
// end of critical section
```

può accedere che più processi prendano lo stesso biglietto. non si ha atomicità

ho preso un numero

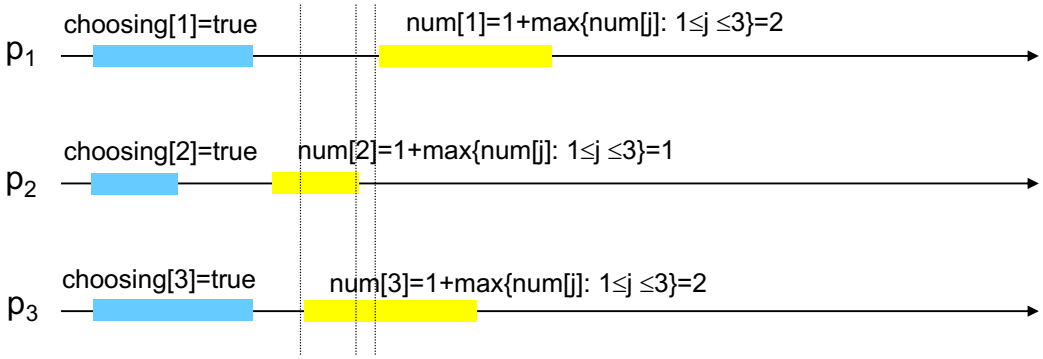
Precedence relation < on ordered integer pairs:
$\{a,b\} < \{c,d\}$ if $a < c$ OR if $a = c$ AND $b < d$

si usa l'id del processo ed il numero per garantire mutua esclusione, dato che due processi possono avere lo stessso numero

Lamport's bakery algorithm

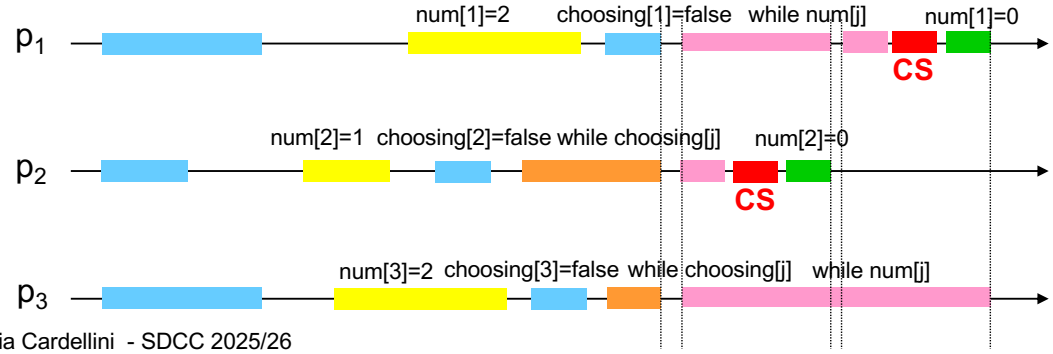
- Doorway
  - When p_i starts the EP, it notifies the other processes by setting `choosing[i]`
  - p_i takes a ticket number equal to one plus the maximum of the ticket numbers chosen by the other processes
  - Other processes may concurrently enter the doorway (ticket selection phase)
  - Example of doorway execution

tra p1 e p3 si ha concorrenza ma una volta arrivati a scegliere chi è primo, sarà p1 a entrare nella CS



Lamport's bakery algorithm

- Bakery
  - p_i must check that it is next in line among the processes waiting to enter the CS
  - The first while loop allows all processes in the doorway to finish choosing their ticket
  - The second while loop keeps p_i waiting until:
    - Its ticket number becomes the smallest among all waiting processes
    - Any process with the same ticket number has a larger process ID
  - Observation:
    - Cases where the same ticket number is chosen are resolved using the process ID as a tie-breaker
  - Example of bakery execution



Lamport's bakery algorithm: properties

si dimostra formalmente che l'algoritmo gode di mutua esclusione

- ME property
  - Ensured because if p_i is in the doorway and p_j is in the bakery, then *{num[j], j} < {num[i],i}*
- NS property
  - No process waits forever, because eventually its ticket number becomes the minimum
- Ordering
  - If p_i enters the bakery before p_j enters the doorway, then p_i will enter the CS before p_j

Distributed mutual exclusion: model

vediamo come cambia il modello nel caso distribuito

- Communication
  - Processes communicate via message passing; cannot directly access each other's variables non si accede direttamente come prima
  - A process sends a request and receives a reply with the value
  - Message transmission delays are unknown but finite
  - Reliable channels: messages are delivered correctly, FIFO-ordered, with no duplicates or spurious messages (received but never sent) assunzione forte
- System
  - N processes p_i (i = 1, …, N)
  - Asynchronous
  - Processes are fail-free (no crashes) altra assunzione forte
  - Each process spends a finite time in the CS
    non conosciamo il tempo ma è comunque finito

Adapting Lamport's bakery algorithm

- Adapting Lamport's bakery algorithm to DS

  Each process p_i acts as a server for its own local variables num[i] and choosing[i]

  Communication is via message passing: processes read other processes' local variables using request-reply messages
- Works correctly, but…
  - ✗ High communication cost to enter CS: 6N messages anche se poi è O(N)
    - Must read 3N variables (N for doorway, 2N for bakery) questo 6 è fastidioso
    - Each variable read requires 2 messages quindi 2*3N prima tutti i num e poi tutti i choosing e num
  - ✗ Latency depends on the slowest process-channel combination il peggiore influenza tutti
  - ✗ Efficiency and scalability are limited
    - No cooperation among processes

Distributed ME: considered algorithms

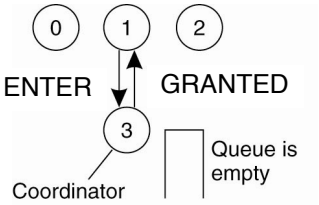
1. **Permission-based** algorithms
   - A process requesting access to the shared resource asks for permission, which can be managed:
     - Centrally (coordinator-based) efficiente in comunicazione ma SPoF e CdB
     - Distributed (Lamport's distributed algorithm, Ricart and Agrawala's algorithm)
2. **Token-based** algorithms
   - A special message, called a *token*, circulates among processes
     - The token is unique at any time
     - Only the process holding the token can access the shared resource
   - Token management can be distributed
3. **Quorum-based** (voting) algorithms
   - A process requests votes from a subset of processes before accessing the shared resource
   - Example: Maekawa's algorithm messaggi scambianti nell'ordine di $\sqrt{N}$

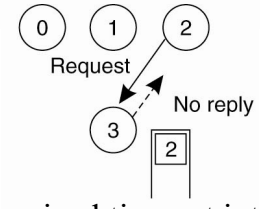
Distributed ME: performance metrics

- Main metrics to evaluate distributed ME algorithms
- Number of messages to enter and exit the CS
  - Indicates the network bandwidth consumption
- Number of messages to enter the CS
  - Indicates the waiting time before a process can enter the CS

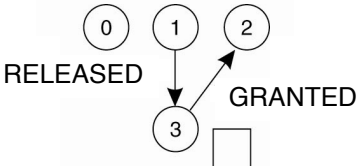
Permission-based: centralized algorithm

- Process p_i that requires access to the CS sends an access request (ENTER) to the central coordinator
- If the CS is free, the coordinator informs p_i that access is granted (GRANTED)
- Otherwise, the coordinator enqueues the request using a FIFO policy and informs p_i that access is denied (DENIED)
  - Alternatively, in case of synchronous DS, it does not reply (see figure)
- When p_i releases the resource, it notifies the coordinator (RELEASED)
- The coordinator removes the first pending request from the queue and sends GRANTED to its sender





se asincrono grazie al timeout intuisco che è DENIED



Centralized algorithm

- Properties
  - Guarantees safety and liveness
    - NS and ND: only if the coordinator and processes do not fail
  - Ordering
    - FIFO ordering is guaranteed according to the order in which requests arrive at the coordinator
    - Not according to the order in which requests are sent or the order in which processes request entry to the CSO
- Pros and cons
  - ✓ The simplest to implement
  - ✓ The most efficient in terms of number of messages nel caso migliore
    - Only 3 messages (ENTER, GRANTED, and RELEASE) are required to enter and exit the CS
  - ✗ The coordinator is a SPOF and a potential performance bottleneck
  - ✗ If a process fails while in the CS, RELEASE is lost

analogo al multicast totalmente ordinato con clock logico scalare

Permission-based: Lamport's distributed algorithm

- Each process p_i uses a scalar clock to timestamp messages (plus total ordering $\Rightarrow$) and maintains a local queue che deve essere sempre ordinata dunque
  - The queue stores CS access requests from other processes nella propria coda locale
  - The scalar clock is incremented before sending messages and after receiving messages

questo perché il sistema potrebbe essere asincrono e l'algoritmo è valido anche in quei sistemi

Lamport's distributed algorithm

- Algorithm rules: safety e ordering grazie alla coppia t_i, p_i liveness grazie alle release
  - Requesting CS access: p_i sends a request message REQ(t_i, p_i) with timestamp t_i (its scalar clock) to all other processes and inserts REQ(t_i, p_i) into its local queue
  - Receiving a request: when p_j receives a request from p_i it inserts REQ(t_i, p_i) into its queue and sends an ACK message to p_i
  - Entering the CS: p_i enters the CS if and only if:
    - REQ(t_i, p_i) precedes all other requests in the queue (i.e., {t_i, p_i} is the minimum according to $\Rightarrow$) non sul tempo fisico!
    - p_i has received from every other process a message (ACK or REQ) with a timestamp greater than t_i (according to $\Rightarrow$)
  - Exiting the CS: p_i removes REQ(t_i, p_i) from its queue and sends a RELEASE message to all other processes
  - Receiving a RELEASE: when p_j receives a RELEASE from p_i it removes REQ(t_i, p_i) from its queue

Lamport's distributed algorithm

- Similar to the distributed algorithm for totally ordered multicast 🤔
- Guarantees safety, liveness, and ordering
  - Ordering: requests are served according to their timestamp, based on scalar clock, and process id

ricorda: ordering $\Longrightarrow NS \Longrightarrow ND$

- Performance: requires 3(*N*−1) messages to enter and exit the CS:
  - *N*−1 REQUEST messages
  - *N*−1 ACK messages
  - *N*−1 RELEASE messages

Permission-based: Ricart and Agrawala algorithm

- Optimization of Lamport's distributed algorithm
  - Scalar clock (and ⇒) and local queue

- A process that wants to enter the CS sends a REQUEST message to all other processes containing:
  - its own process ID
  - timestamp based on scalar clock
- It then waits for a reply from all other processes
- After receiving all REPLY messages, it enters the CS
- Upon exiting the CS, it sends a REPLY message to the processes with queued requests

- A process that receives a REQUEST message may:
  - Not be in the CS and not want to enter it → sends a REPLY to the sender
  - Be in the CS → does not reply and places the request in its local queue
  - Want to enter the CS → compares its own timestamp and ID with the received timestamp and ID. The smaller pair wins: if the other process wins → sends a REPLY; if it wins → does not reply and places the request in its local queue

si risparmiano diversi messaggi in questo modo

vedi esempio per capire meglio

Ricart and Agrawala's algorithm

- Local variables for each process
  - #replies: number of REPLY messages received (initialized to 0)
  - State ∈ {Requesting, CS, NCS} (initialized to NCS)
  - Q: queue of pending requests (initially empty)
  - Last_Req: timestamp of the REQUEST message (initialized to 0)
  - Num: scalar clock (initialized to 0)
- Each process p_i executes the following algorithm

Begin

1. State = Requesting;
2. Num = Num+1; Last_Req = Num;
3. for j=1 to N-1 send REQUEST to pj;
4. Wait until #replies=N-1;
5. State = CS;
6. CS
7. ∀ r∈Q send REPLY to r
8. Q=∅; State=NCS; #replies=0;

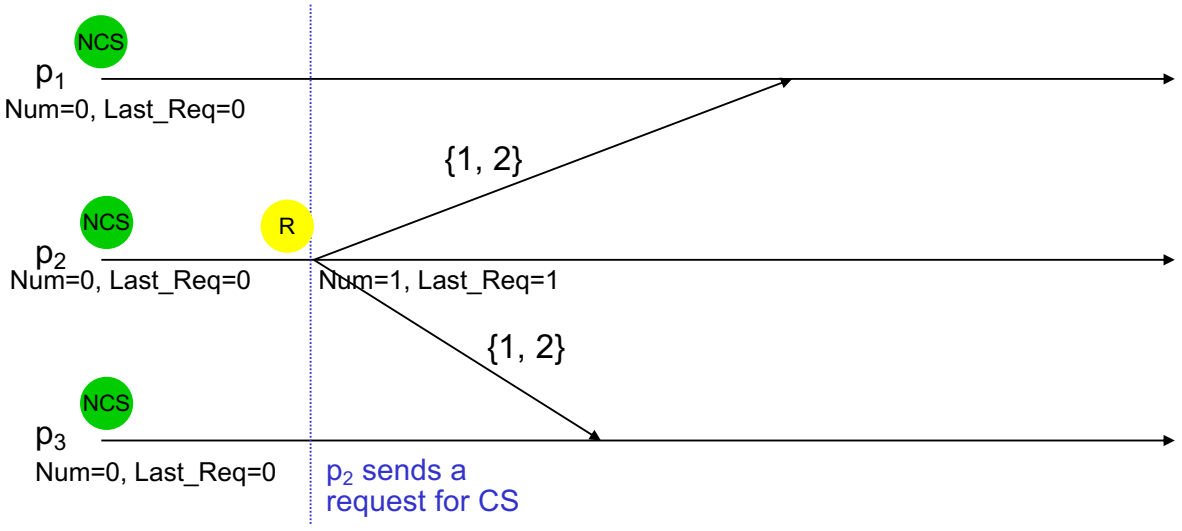
Upon receipt of REQUEST(t) from pj

1. if State=CS or (State=Requesting and {Last_Req, i} < {t, j})
2. then insert {t, j} in Q
3. else send REPLY to pj
4. Num = max(t, Num)

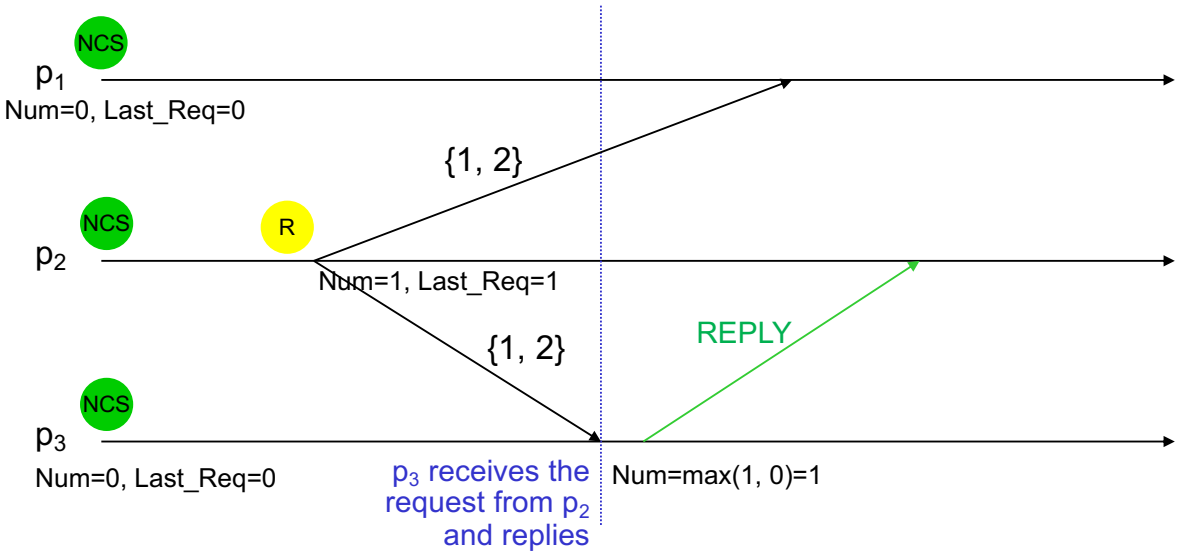
Upon receipt of REPLY from pj

1. #replies = #replies+1

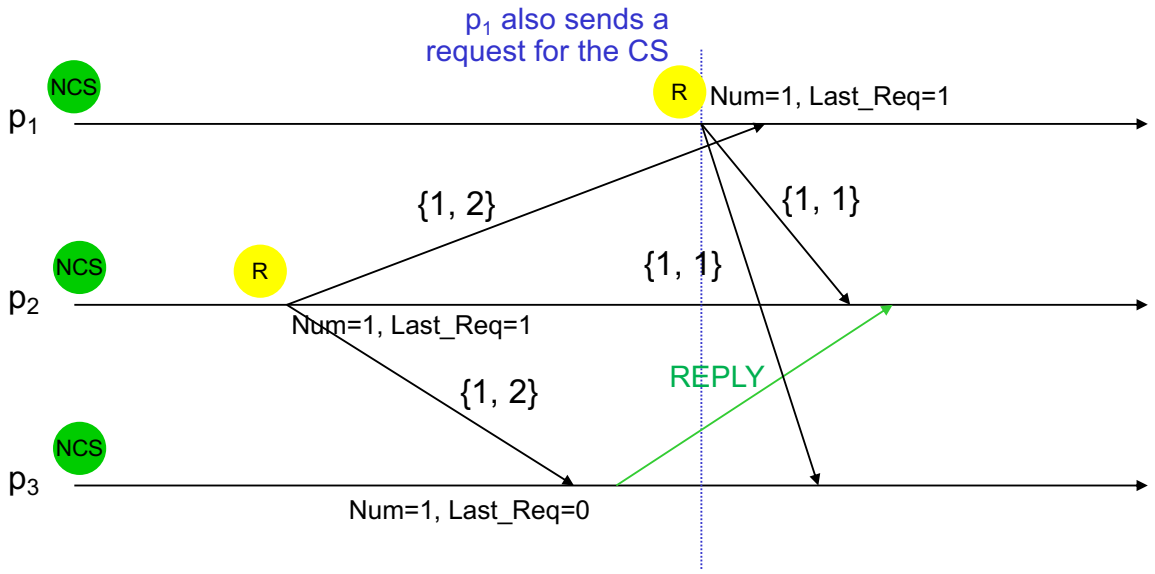
Ricart and Agrawala's algorithm: example



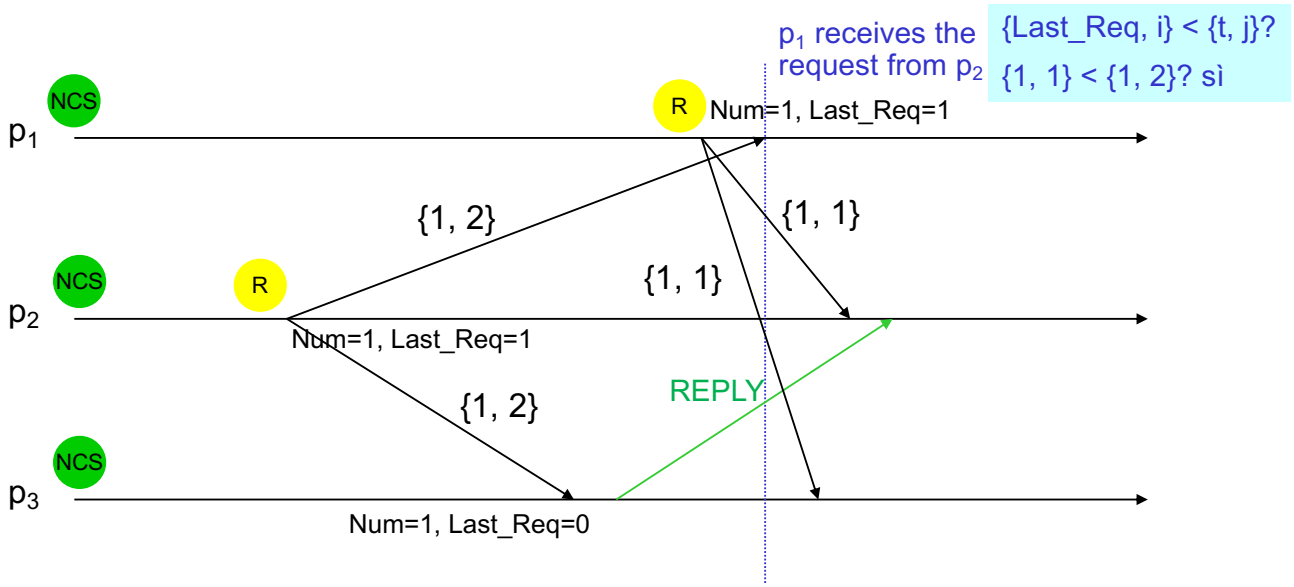
Ricart and Agrawala's algorithm: example



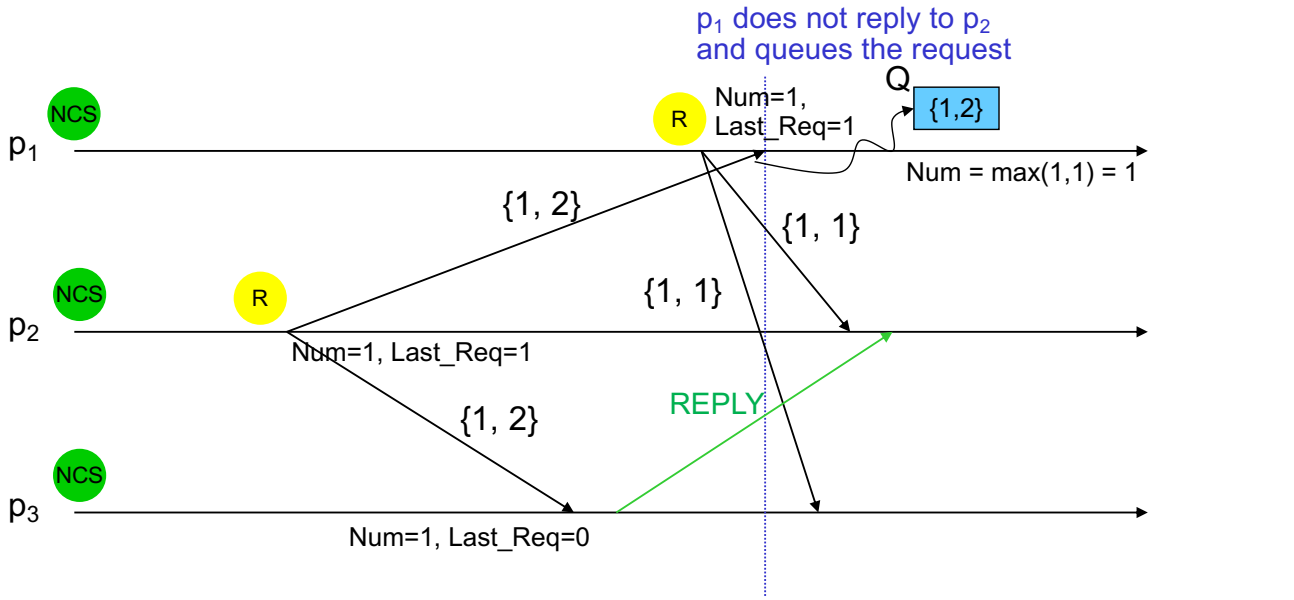
Ricart and Agrawala's algorithm: example



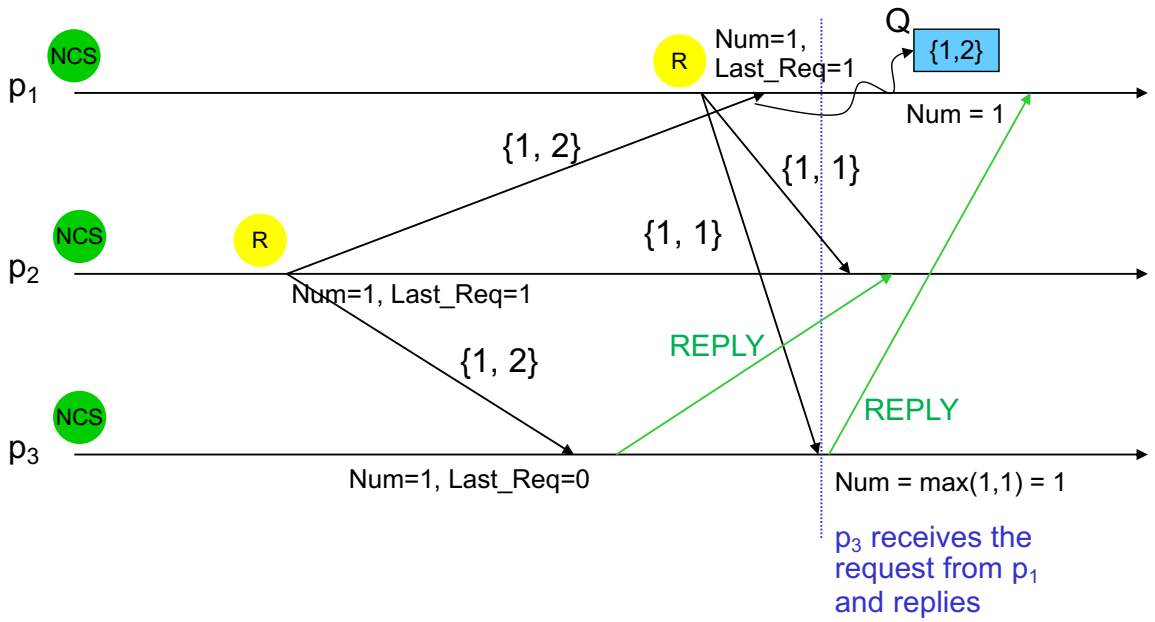
Ricart and Agrawala's algorithm: example



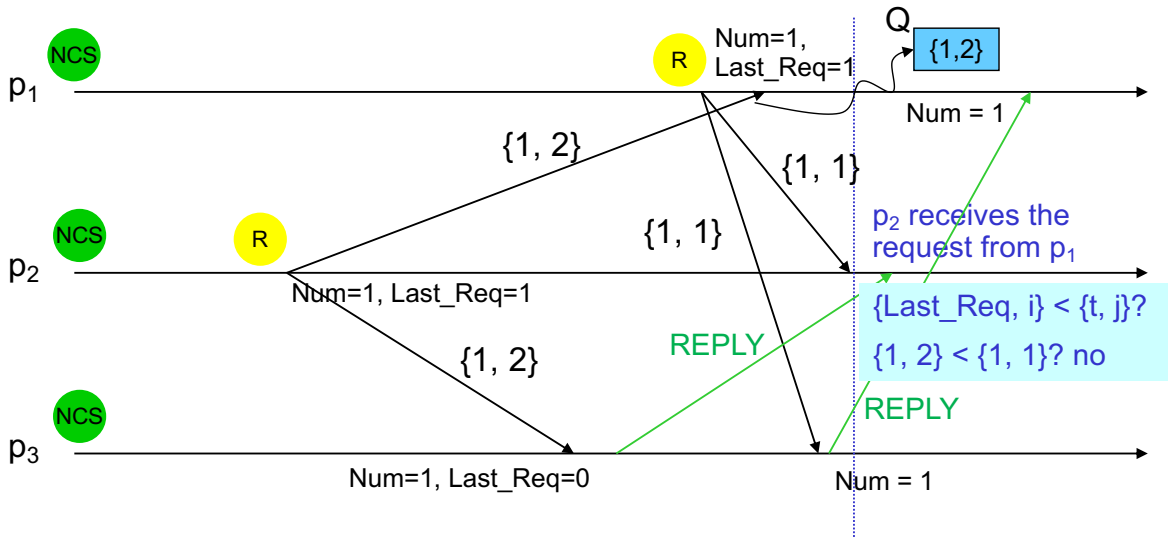
Ricart and Agrawala's algorithm: example



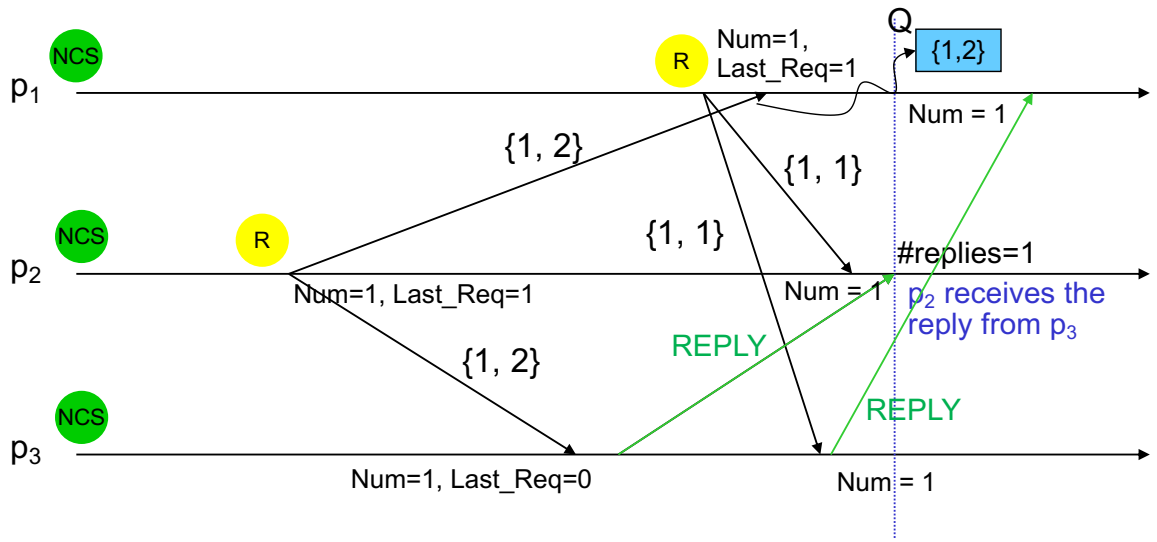
Ricart and Agrawala's algorithm: example



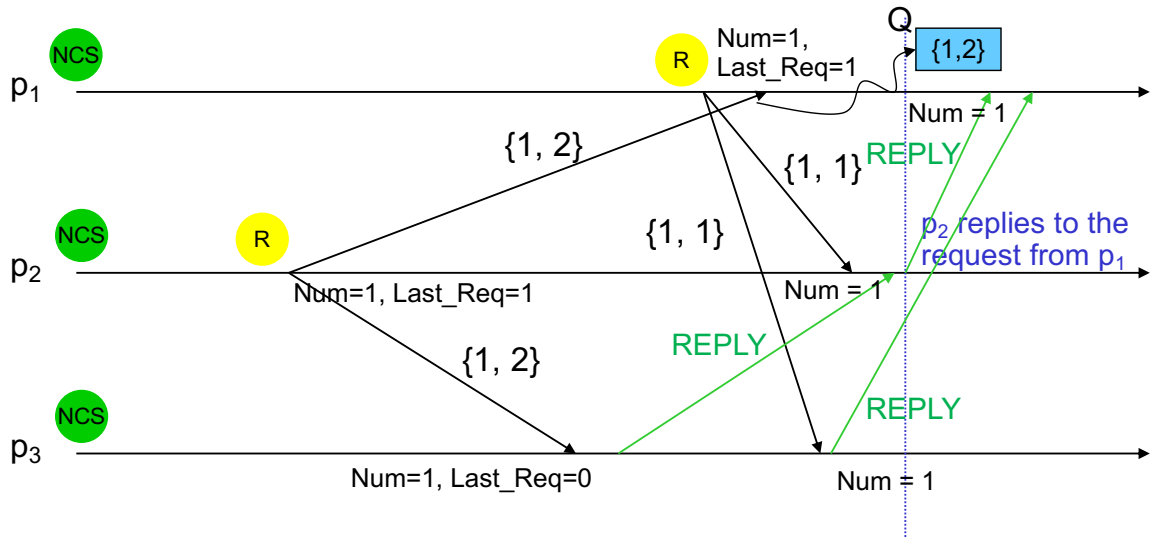
Ricart and Agrawala's algorithm: example



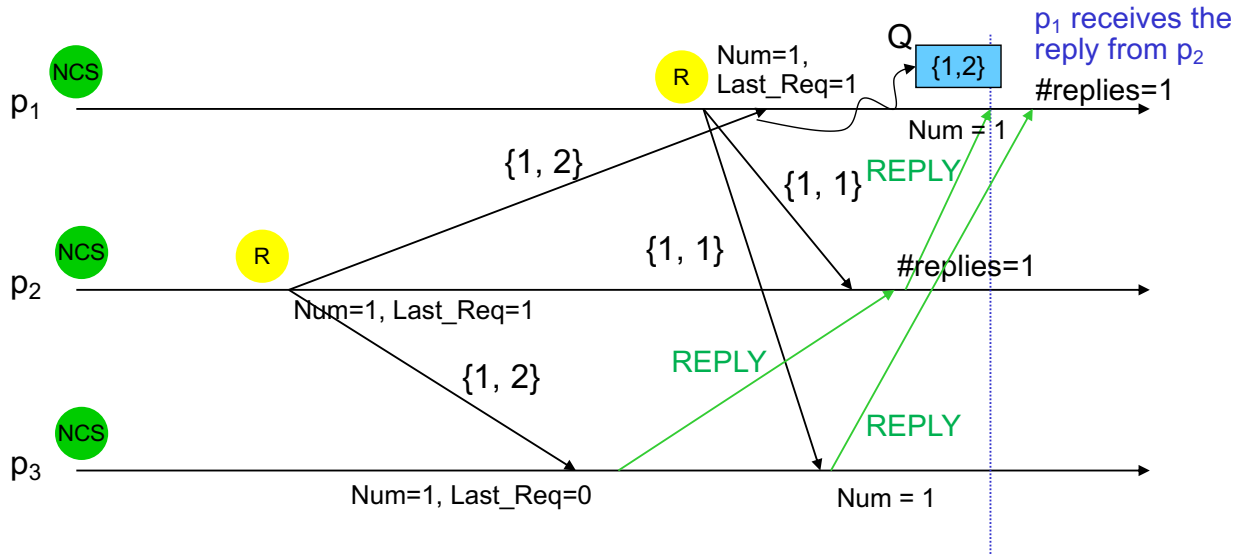
Ricart and Agrawala's algorithm: example



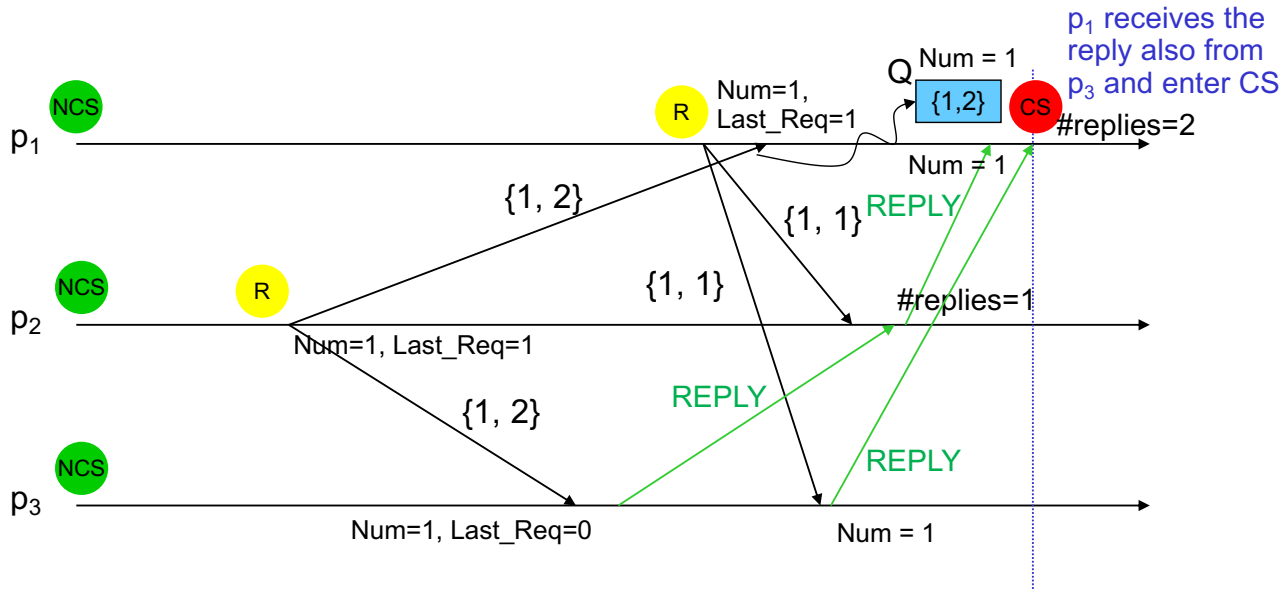
Ricart and Agrawala's algorithm: example



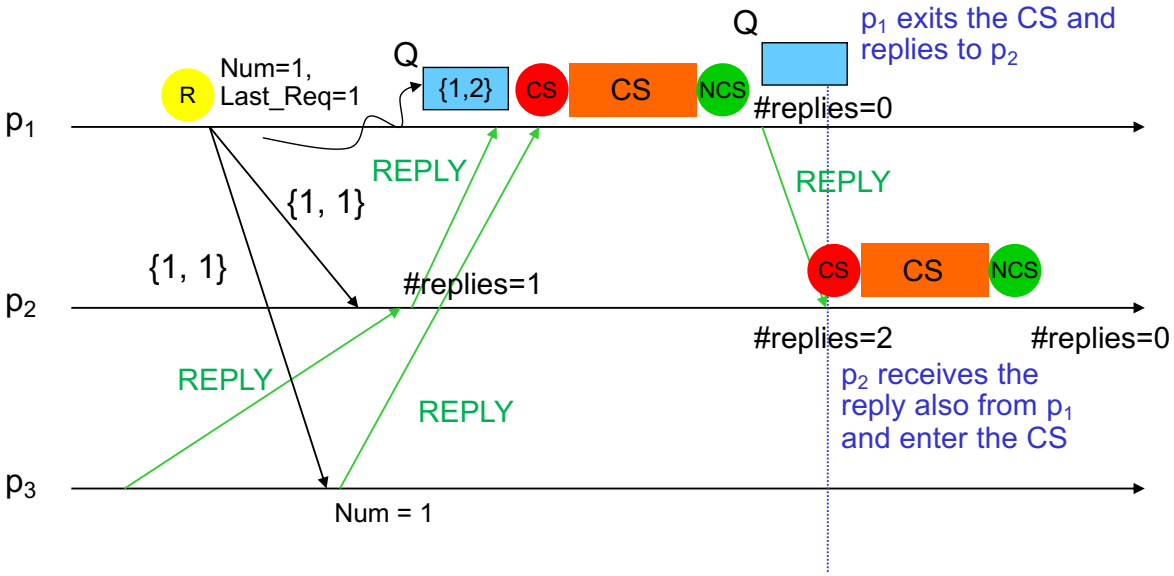
Ricart and Agrawala's algorithm: example



Ricart and Agrawala's algorithm: example



Ricart and Agrawala's algorithm: example



Ricart and Agrawala's algorithm: example

- Pros
  - ✓ Fully distributed, like Lamport's algorithm
    - No central coordinator
  - ✓ Fewer messages than Lamport's algorithm
    - No RELEASE message; ACKs are deferred until exit from CS
    - Only 2(*N*−1) messages per CS execution: *N*−1 REQUEST messages, *N*−1 REPLY messages
  - ✓ Literature reports further optimizations (not covered) reducing messages to *N*
- Cons (like Lamport's algorithm)
  - ✗ If any process fails, no one can enter the CS → requires a *failure detection* mechanism
  - ✗ Every process can become a bottleneck
    - Every process participates in every decision
  - ✗ Must know the membership of the multicast group

la failure detection implica un sistema sincrono parzialmente sincrono

serve service discovery

Token-based algorithms

- An auxiliary resource called a **token** is used
  - Other distributed algorithms also use tokens (e.g., leader election)
- The algorithm must define:
  - How token requests are made
  - How the token is maintained and granted
- In a token-based algorithm, at any time there is exactly one token holder
  - This guarantees safety (mutual exclusion)
- Many token-based ME algorithms exist in the literature; we analyze:
  - Decentralized (or *perpetuum mobile*): token management is decentralized and the token moves through the system

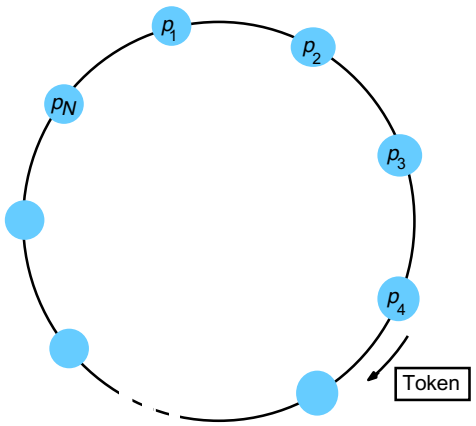
Token-based: decentralized algorithm

- Processes are logically organized in a (unidirectional) ring
  - No relation between the ring topology and the physical interconnection of nodes

non è possibile avere un ring bidirezionale altrimenti si avrebbe starvation!

- The token travels from one process to the next
  - Passes from p_i to $p_{(i+1) \bmod N}$
- The process holding the token can enter the CS
- If a process receives the token but does not want to enter the CS, it passes the token to the next process

importante è mantenere la struttura ad anello



Token-based: decentralized algorithm

- ✓ Safety: guaranteed come per Ricart
- ✓ NS: guaranteed if the ring is unidirectional
- ✓ ND: guaranteed if the token is not lost
- Ordering? non c'è. se p6 richiede il token deve aspettare che gli arrivi anche se la richiesta è prima di quella di un altro processo che la fa dopo ma ottiene il token prima
- ✗ Network bandwidth is consumed transmitting the token even when no process wants to enter the CS
- ✗ Token loss requires token regeneration ed invalidare il vecchio token
- ✗ Temporary failures may lead to multiple tokens
- Crash of individual processes:
  - Ring must be reconfigured if a process fails
  - If the token holder fails, the token must be regenerated and the next token owner elected

Quorum-based algorithms

- Idea: to enter the CS, a process only needs to collect votes from a subset of processes (*quorum*), not from all processes
- Voting within the subset: hanno tutti la stessa cardinalità
  - The processes vote to determine which process is authorized to enter the CS
  - A process can vote for only one process per turn
- Voting set V_i: subset of $\{p_1, \ldots, p_N\}$, associated with each process p_i

- A process p_i to enter the CS
  - Sends a *request* to all other members of V_i
  - Waits for a *reply* from all members of V_i
  - Upon receiving all the replies from V_i members, it enters CS
  - Upon exiting the CS, it sends a *release* to all members of V_i

- A process p_j in V_i that receives a *request*
  - If the process is in CS or has already replied after receiving the last release, it does not reply and queues the request
  - Otherwise, it replies immediately with a reply

- A process that receives a release:
  - Extracts one request from the queue and sends a reply

Maekawa's algorithm

- Each process p_i executes the following algorithm:

Initialization

```
state = RELEASED;
voted = FALSE;
```

CS entry section for p_i

```
state = WANTED;
multicast request to all processes in V_i (including itself);
wait until (number of replies received = K);   // K = |V_i|
state = HELD;
```

Upon receiving a request from p_j ($i \neq j$)

```
if (state = HELD or voted = TRUE) then
        queue request from p_j without replying;
else
        send reply to p_j;     // vote in favour of p_j
        voted = TRUE;
end if
```


Maekawa's algorithm

Exit protocol from CS for p_i

```
state = RELEASED;
multicast release to all processes in V_i;
if (queue of requests is non-empty) then
        remove head of queue – from p_k say;
        send reply to p_k;      // vote in favour of p_k
        voted = TRUE;
else
        voted = FALSE;
end if
```

On receipt of a release from p_j ($j \neq i$)

```
if (queue of requests is non-empty) then
        remove head of queue – from p_k say;
        send reply to p_k;      // vote in favour of p_k
        voted = TRUE;
else
        voted = FALSE;
end if
```

da K dipende il numero di messaggi e l'efficienza dell'algoritmo

Maekawa's algorithm: voting set

- How is the voting set V_i defined for p_i?

1. $V_i \cap V_j \neq \emptyset \quad \forall\, i, j$
   - Every pair of voting has non-null intersection: why?
2. $|V_i| = K \quad \forall\, i$
   - All processes have voting sets with the same cardinality K (same *effort* for each process)
3. Each process p_i belongs exactly to K voting sets
   - Equal *responsibility* for every process

si ha così carico di lavoro distribuito e stessa responsabilità

4. $p_i \in V_i$
   - To reduce the numer of transmitted messages

- The optimal solution that minimizes K is $K = \lceil \sqrt{N} \rceil$

per garantire la mutua esclusione si ha intersezione non nulla. in questo modo se il processo ha votato in un insieme non può votare nell'altro insieme a cui appartiene

N=3

V_1={1,2}
V_3={1,3}
V_2={2,3}

N=7

V_1={1,2,3}
V_4={1,4,5}
V_6={1,6,7}
V_2={2,4,6}
V_5={2,5,7}
V_7={3,4,7}
V_3={3,5,6}

il 3 può votare per 2 o 1 una volta votato non può più votare

Maekawa's algorithm: properties and performance

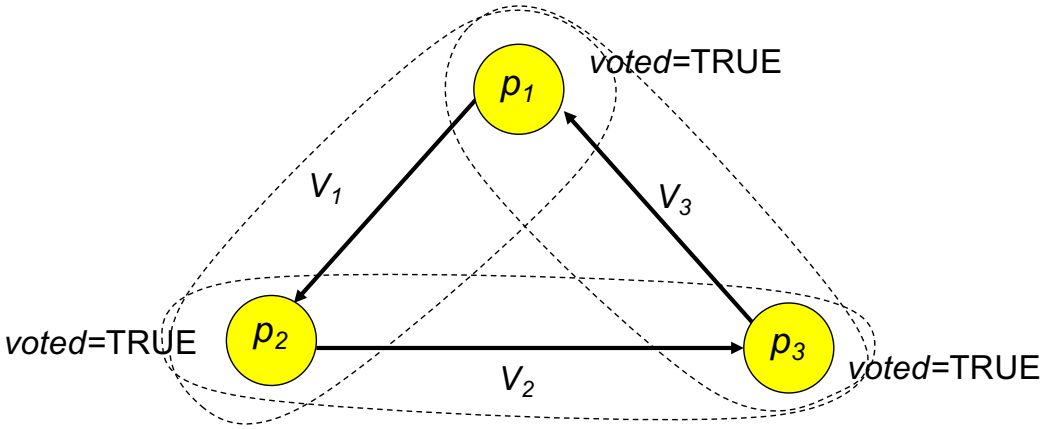
- Safety is guaranteed
  - Voting sets are constructed to ensure they have a non-empty intersection
  - If a quorum grants access to the CS for a process, no other quorum can grant the same permission
- Liveness is not guaranteed
  - Deadlock can occur
  - The algorithm can be made deadlock-free with additional messages
- Performance
  - To enter and exit the CS, $3\sqrt{N}$ messages are required ($2\sqrt{N}$ to enter and $\sqrt{N}$ to exit)
    - More efficient than Ricart and Agrawala for large-scale systems, since $3\sqrt{N} < 2(N-1)$ for $N > 4$

Maekawa algorithm: deadlock example

$P = \{p_1, p_2, p_3\}$
$V_1 = \{p_1, p_2\}$, $V_2 = \{p_2, p_3\}$, $V_3 = \{p_3, p_1\}$

Deadlock situation

- p_1, p_2, and p_3 simultaneously request entry to the CS
- p_1, p_2, and p_3 each set voted=TRUE and wait for a response from the other processes
- There is a circular waiting that causes the deadlock



Comparison of distributed ME algorithms

Algorithm	#msg to enter and exit the CS	#msg to enter the CS	Issues
Permission-based centralized	3	2	Coordinator crash
Ricart Agrawala	2(*N*-1)	2(*N*-1)	Crash of any process
Token-based decentralized	Da 1 a ∞ (se anello bidirezionale)	Da 0 a *N*-1	Token loss Crash of any process
Maekawa	3√*N*	2√*N*	Possible deadlock

Distributed election algorithms

- Many distributed algorithms require a *coordinator* (or *leader*), e.g.,
  - Sequencer in totally ordered multicast
  - Coordinator in mutual exclusion
- Problem: how to elect the coordinator at runtime?
  - The existing coordinator can crash
  - Election requires reaching distributed consensus
- Two classic election algorithms
  - **Bully algorithm**
  - **Ring election algorithm** (Fredrickson & Lynch)

Distributed election: model

entrambi hanno delle assunzioni, come gli algoritmi di mutua esclusione.

- System with N processes p_i, $i = 1, \ldots, N$
- Processes may crash
- Reliable communication: messages are neither lost, corrupted, nor duplicated
- Each process can hold at most one election at a time
- Each process has a unique ID and the non-faulty process with the highest ID is elected
- Processes can crash, but will eventually recover

a differenza degli algoritmi di mutua esclusione.
Inoltre, i crash non sono di tipo bizantino

ma può partecipare a più elezioni

quindi si elege come leader quello con ID più alto e non crashato

Distributed election: properties

anche in questo caso ritornano le proprietà viste negli algoritmi di mutua esclusione

- *Safety*: only the non-faulty process with the highest ID is elected as leader
  - The election result does not depend on which process started the election
  - If multiple processes start an election at the same time, a single winner is eventually announced

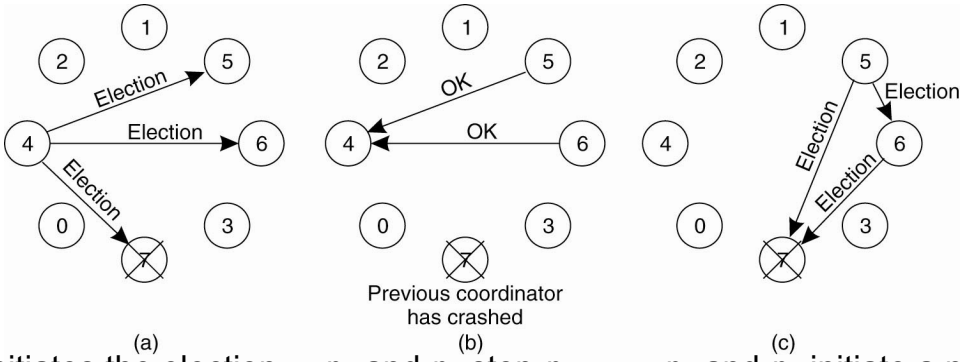
- *Liveness*: at any time, some process is eventually elected as leader
  l'elezione ha sempre successo

Bully algorithm (Garcia-Molina)

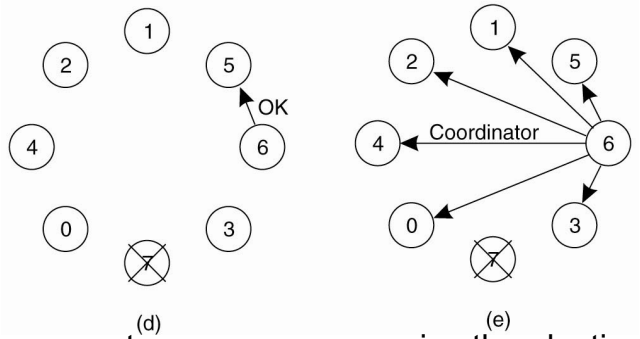
- "Node with highest ID bullies its way into leadership" appunto
- Steps
  - Detection: p_i notices that the leader is not responding and initiates an election
  - Election message: p_i sends an ELECTION message to all processes with higher IDs (p_{i+1}, p_{i+2}, …, p_N) rispetto al suo ID
  - If no one responds, p_i becomes the new leader and announces victory to all processes sending a COORDINATOR message
  - If p_k ($k>i$) receives an ELECTION message from p_i, it replies OK, takes over and starts a new election
  - If p_i receives an OK, it sits back si suppone perciò un timeout per l'OK
- Outcome: the non-faulty process with the highest ID is elected as leader
- Note: a new or restarted process that does not know the leader can trigger a new election

da notare che si manda anche al vecchio leader in quanto potrebbe essere tornato operativo!

Bully algorithm: example



p_4 initiates the election — p_5 and p_6 stop p_4 — p_5 and p_6 initiate a new election



p_6 stops p_5 — p_6 wins the election and becomes the new leader

Which is a strong assumption?

Synchronous DS (or at least partially sync with no unbounded delay), so that we can use some timeout mechanism to identify faulty leader

Bully algorithm: communication cost

- Communication cost = how many messages

ricordando che N è il numero di processi

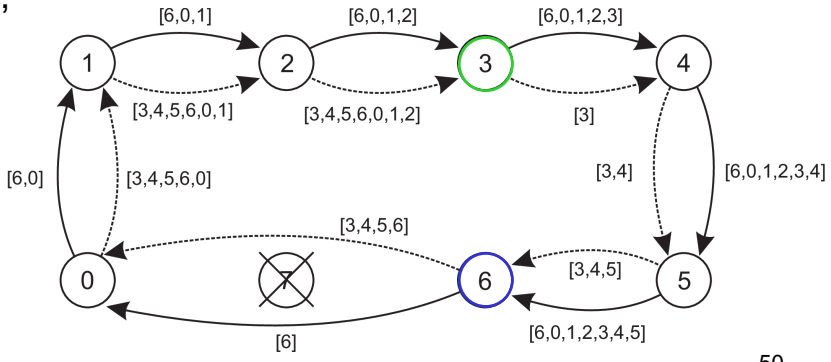
- Best case: the process with the second highest identifier notices leader's failure
  - It can immediately select itself as leader and then send N-2 COORDINATOR messages ⇒ O(N) messages
- Worst case (assuming no process fails during election): the process with the lowest id initiates the election
  - It sends N-1 ELECTION messages to the other processes, which themselves initiate each one an election
    $((N\text{-}1) + (N\text{-}2) + \ldots + 1) + N\text{-}1 \Rightarrow O(N^2)$ messages

messaggi di coordinamento

Ring algorithm (Fredrickson and Lynch)

- Processes are organized in a logical ring (unidirectional)
  - Each process knows at least its successor
- p_i notices that the leader is failed and initiates election
  - p_i sends ELECTION message to $p_{(i+1) mod\ N}$ with its own id
  - If $p_{(i+1) mod\ N}$ *is* faulty, p_i skips over it and goes to the next process along the ring, until a non-faulty process is located
  - At each step, the receiver adds its own id to the list in ELECTION message and forwards the message to the next process
  - Eventually, ELECTION message gets back to p_i, which identifies the highest id in the list and circulates COORDINATOR message to inform everyone else about the new leader

algoritmo adatto per media piccola scala, dato che bisogna conoscore maggiori informazioni, per maggiori meccanismi ad esempio il successore del successore



il caso peggiore è quando il processo con ID più alto deve aspettare che il messaggio ritorni indietro. Il caso migliore è quando si parte da un altro ID, appena si arriva al processo con ID più alto si elege il leader.

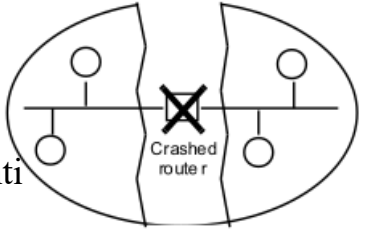
Ring algorithm: communication cost

- Requires 2N messages
  - *N* for ELECTION message, *N* for COORDINATOR message
- But messages are larger than in Bully algorithm

Election algorithms: properties

- Both algorithms assume reliable communication
- Ring election:
  - Works for synchronous and asynchronous systems
  - Works for any *N* and does not require any process to know how many processes are in the ring
- Fault tolerance with respect to process failure
  - What happens if a process crashes during election? It depends on algorithm and crashed process
  - Additional mechanisms may be needed, e.g., ring reconfiguration
- Something to consider:
  - What happens in case of network partition?

    Multiple new leaders, one per partition



una partizione di rete avviene quando tutti i processi sono funzionanti ma sono divisi in due sottoinsiemi che non comunicano tra di loro.
nella figura si vede che crasha il nodo route che collega i due insiemi

prima di trattare gli algoritmi di consenso occorre studiare nel dettaglio i fallimenti

Tipi di failure

- In che modo possono fallire i componenti di un SD?
- Diversi tipi di failure:
  - Crash: il componente si arresta, ma aveva funzionato correttamente fino a quel momento
  - Omissione: il componente non risponde ad una richiesta
  - Fallimento nella temporizzazione: il componente risponde ma il tempo di risposta supera l'intervallo specificato
  - Fallimento nella risposta: la risposta del componente non è corretta
    - Fallimento nel valore
    - Fallimento nella transizione di stato
  - Fallimento arbitrario (o *bizantino*): il componente può produrre una risposta arbitraria con tempi arbitrari i più difficili da identificare
    - Guasto bizantino: sintomi diversi ad osservatori diversi
- I crash sono i fallimenti più innocui, quelli bizantini i più gravi e richiedono maggior ridondanza

In maniera bruta, la tecnica principe per gestire la tolleranza ai guasti è la ridondanza.

Modelli di failure

- *Problema nei SD*: distinguere tra componente che ha subito un crash ed uno che è solo troppo lento
  - Esempio: il processo P attende dal processo Q una risposta, che tarda ad arrivare
    - Q è soggetto ad un fallimento nella temporizzazione o ad una omissione?
    - Il canale di comunicazione tra P e Q è soggetto ad un guasto?
- Modelli di failure: dal meno al più grave
  - Fallimento **fail-stop**: Q ha subito un crash e P può scoprire il fallimento (tramite timeout o preannuncio)
  - Fallimento **fail-silent**: Q ha subito un crash o un'omissione, ma P non può distinguerli
  - Fallimento **fail-safe**: Q ha subito un fallimento arbitrario, ma senza conseguenze
  - Fallimento **fail-arbitrary**: Q ha subito un fallimento arbitrario non osservabile

Rilevare i fallimenti

- Per mascherare i fallimenti, bisogna innanzitutto rilevarli
- **Failure detection** per rilevare il fallimento di un processo
  1. *Attiva*: invio di un messaggio e timeout per rilevare se un processo è fallito
     - Soluzione più usata, adatta per fallimento fail-stop
  2. *Passiva*: attesa di ricevere un messaggio raft usa un meccanismo passivo
  3. *Proattiva:* come effetto collaterale dello scambio di informazioni tra vicini (ad es. disseminazione delle informazioni basata su gossiping)
- Difficoltà con timeout
  - Come impostare il valore del timeout? Ok nei SD sincroni, ma in quelli asincroni? come in quelli semi sincroni non si conosce il tempo massimo di trasmissione e si ha difficoltà a impostare il timeout
  - Inoltre, timeout dipende anche dall'applicazione
  - Come distinguere tra fallimenti dei processi o della rete?

Practical failure detection

- How can we reliably detect that a process has actually crashed?
- General model
  - Each process is equipped with a failure detection module
  - A process P probes another process Q for a reaction
  - If Q reacts: Q is considered to be alive (by P)
  - If Q does not react within *t* time units: Q is suspected to have crashed (if the system is synchronous: suspected = sure)
- Practical implementation di failure detection attiva
  - If P did not receive heartbeat from Q within timeout *t*: P suspects Q
  - If Q later sends a message (which is received by P):
    - P stops suspecting Q
    - P increases *t*

    meccanismo visto nel RPC con backoff esponenziale
  - Note: if Q did crash, P will keep suspecting Q

Resilienza dei processi

- In ingegneria: resilienza = capacità di un materiale di resistere a forze di rottura
- Nei SD: capacità del SD di fornire e mantenere un livello di servizio accettabile in presenza di guasti e minacce alla normale operatività

- Come mascherare in un SD il guasto di un processo?

  Replicando e distribuendo la computazione in un gruppo di processi

  appunto con tecniche di ridondanza e distribuzione del carico

Gruppi e mascheramento dei guasti

- Consideriamo un gruppo di processi
- Un gruppo composto da *N* processi è ***k*-fault tolerant** se può mascherare *k* guasti concorrenti ovvero nello stesso intervallo di tempo
  - *k* è detto grado di tolleranza ai guasti

- Quanto deve essere grande un gruppo *k*-fault tolerant?
  - **Dipende** dal modello di failure

Gruppi e mascheramento dei guasti

- Quanto deve essere grande un gruppo *k*-fault tolerant? ricordando che k è il numero di guasti concorrenti
  - Fallimento fail-stop o fail-silent → $N >= k+1$ processi
    - Nessun processo del gruppo produrrà un risultato errato, quindi è sufficiente il risultato di un solo processo non guasto
  - Fallimento arbitrario e il risultato del gruppo è definito tramite un meccanismo di voto → $N >= 2k+1$ processi come vedremo in Paxos e Raft
    - Abbiamo bisogno di $k+1$ processi non guasti in modo che il risultato corretto possa essere ottenuto con un voto a maggioranza
- *Assunzioni* importanti:
  1. I processi sono identici
  2. I processi eseguono i comandi nello stesso ordine
  - Per essere certi che tutti i processi facciano esattamente la stessa cosa quindi non c'è bisogno di consenso, sono tutte copie!

Consenso nei sistemi distribuiti

- *Assunzione*: consideriamo ora che i processi del gruppo *non* siano identici, ovvero che ci sia una computazione distribuita
- *Obiettivo*: i processi non guasti del gruppo devono raggiungere un **consenso** (accordo) unanime su uno stesso valore (es. il prossimo comando da eseguire) in un numero finito di passi, nonostante la presenza di processi guasti
  - Esempi di consenso: elezione di un coordinatore, mutua esclusione, commit di una transazione
- Che tipo di guasti?
  - Guasti non bizantini (es. crash, omissioni)
  - Guasti bizantini

nel corso useremo consensus e agreement come sinonimi. ma in teoria sono 2 cose totalmente diversi

Consensus = agreement?

- In the course we treat these terms as synonyms
- But for the theoretical DS community the two terms refer to very similar but not identical problems
  - Agreement problem: a single process has the initial value
  - Consensus problem: all processes have an initial value

differenza imporante

- We'll consider 3 consensus algorithms
  1. **Paxos** (only overview)
  2. **Raft**

  } algoritmi di consensus
  3. **Byzantine generals** algoritmo di agreement
  - The first two focuns on consensus, the last on agreement (see original algorithm by Lamport)
- Let us first examine the necessary conditions for consensus and the FLP impossibility result

Consenso distribuito: quando può essere raggiunto

- Consideriamo le diverse tipologie di:
  - processi
  - ritardi di comunicazione
  - ordinamento dei messaggi
  - trasmissione dei messaggi
- Processi: *sincroni* o *asincroni*?
  - Processi sincroni: operano in modalità *lock-step*, ovvero esiste *c* >= 1 tale che se un processo ha eseguito *c*+1 passi, ogni altro processo ha eseguito almeno 1 passo
- Ritardo nella comunicazione: *limitato* o *illimitato*?
- Ordinamento dei messaggi: messaggi consegnati nello stesso *ordine* in cui sono stati inviati oppure *senza ordine*?
- Trasmissione dei messaggi: *unicast* o *multicast*?

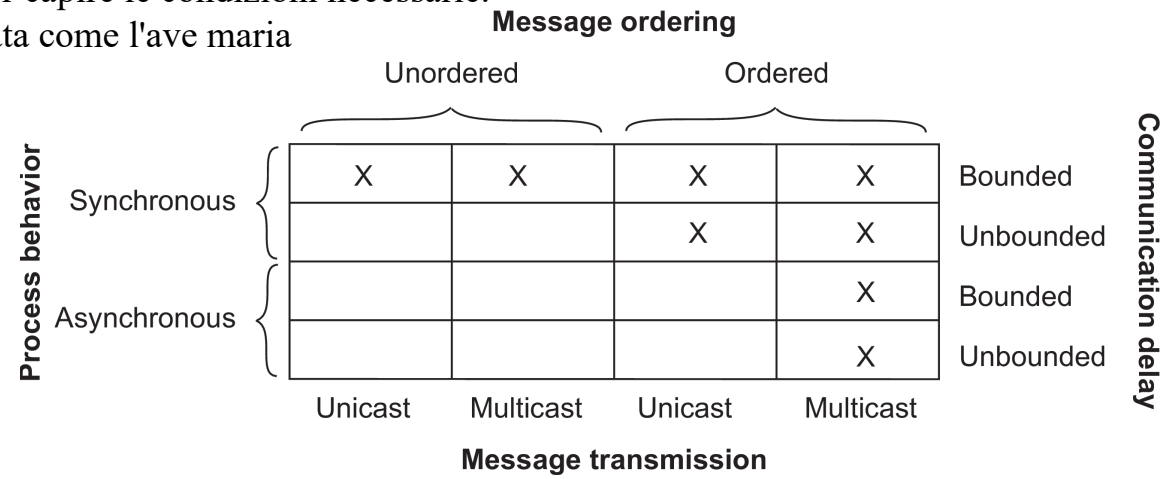
che saranno le condizioni necessarie per raggiungere il consenso

al più a distanza di c passi tra di loro. dunque possono avere velocità di esecuzione diverse ma comunque si ha un bound sulla divergenza

Consenso distribuito: quando può essere raggiunto

- Quali sono le condizioni necessarie per poter raggiungere il consenso in un SD soggetto a guasti?

tabella utili per capire le condizioni necessarie.
non va imparata come l'ave maria



da notare nel caso asincrono solo con comunicazione multicast ordinata si hanno raggiungono condizioni necessarie. ricordiamo necessarie e non sufficienti!

FLP impossibility result

- FLP impossibility result:

Fischer, Lynch and Patterson, "Impossibility of distributed consensus with one faulty process", 1985

  - In un modello di sistema distribuito asincrono, in cui abbiamo un ritardo nei messaggi finito ma non noto, in cui anche un solo processo può subire un crash, non esiste un algoritmo distribuito che risolve il problema del consenso
  - They prove that no asynchronous algorithm for agreeing on a one-bit value can guarantee that it will terminate in the presence of crash faults
    - And this is true even if no crash actually occurs!

non si garantisce la liveness, però queste esecuzioni sono estremamente rare!

- The good news: impossibility means "is not always possible"
  - FLP proves that any fault-tolerant algorithm solving consensus has runs that never terminate, but these runs are extremely unlikely ("probability is zero")

FLP impossibility result

chi sono i processi coinvolti

- What makes consensus hard? *Membership* in an asynchronous environment:
  1) we can't detect failures reliably because process speeds and channel delays are not bounded
  2) a faulty process stops sending messages but a "slow" message might confuse us
- Are distributed and Cloud systems asynchronous?
  - Not in the sense of the definition used in the FLP result, where asynchronous systems have no common clocks and make no use of time, and networks never lose messages but can delay them arbitrarily long
  - In practice we have partially synchronous systems: most of the time, we can assume the system to be synchronous, yet there is no bound on the time that a system is asynchronous
    - And we can reliably detect crash failures (see slide 4)

Paxos è un fault-tolerant consensus protocol in cui girano N processi (dove N >= 2k+1) che tollerano fino a k guasti in un sistema asincrono. Alcune varianti sono cheap-Paxos o fast-Paxos.

Paxos: distributed system model

bisogna prima fare delle assunzioni

Assumptions (rather weak and realistic)

- Partially synchronous system (it may even be asynchronous) with non-arbitrary failures
- Processes communicate with one another by sending messages
- Communication between processes may be unreliable, indeed messages:
  - Take arbitrarily long to be delivered
  - May be duplicated or lost
  - Corrupted messages can be detected and thus subsequently ignored

Paxos può essere utilizzato per problemi di mutua esclusione o di elezione

Paxos: distributed system model

Assumptions (rather weak and realistic)

- Processes:
  - Set of processes that run Paxos is known a-priori
  - Operate at arbitrary speed
  - Are fail-stop: may exhibit crash failures but **not** arbitrary failures
  - Can be restarted and rejoin if they fail
  - Can remember some information if restarted after failure (accessing a persistent storage that survives crashes)
  - Do not collude (i.e., do not team up to produce a wrong result)

Paxos: a quorum-based protocol

- Problem: get a set of processes to reach consensus on a single proposed value ad esempio hash in blockchain
  - no value proposed → no value chosen
  - value chosen → processes should learn the chosen value

- Main idea: a **quorum-based** protocol
  - Proposals are associated with a *unique sequence number*
  - Processes vote on each proposal
  - A proposal approved by *a majority will get passed*
  - Size of majority is "well known" because potential membership of system was known a-priori
  - A process considering two proposals approves the one with the larger sequence number

che in realtà è un problema separato da risolvere

Paxos requirements

- Safety requirements
  - Only a value that has been proposed may be chosen
  - Only a single value is chosen
  - A process never learns that a value has been chosen unless it really *has* been chosen
- Don't care *what value is chosen, just as long* as it satisfies those three requirements

- Liveness requirements
  - Some proposed value is eventually chosen
  - If a value has been chosen, a process can learn the value

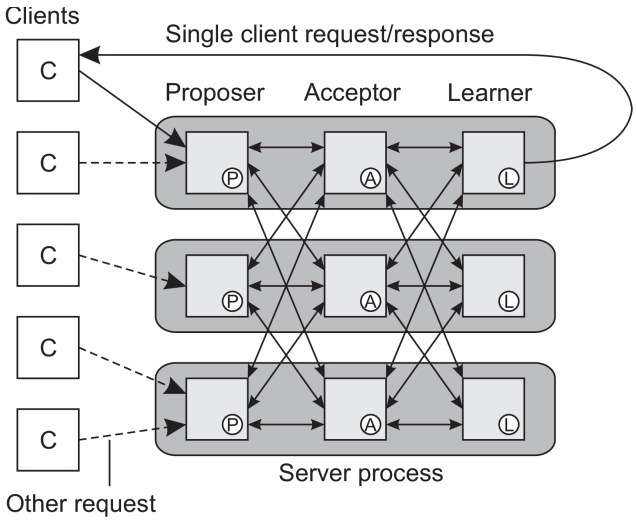
Paxos compromise

- Remember FLP impossibility result
  - No asynchronous consensus algorithm can guarantee both liveness and safety
- However, asynchrony of FLP is *overly pessimistic*
- Real systems are usually partially synchronous
  - System behavior is close to synchronous “most of the time”, sometimes goes asynchronous
- Practical compromise
  - Compromise liveness when system behaves asynchronously
  - But never safety
- Therefore
  - Paxos **does not guarantee liveness**
  - Might never terminate, but in practice it does terminate

in definitiva Paxos garantisce la safety ma non sempre garantisce liveness

Paxos roles

- A process may play 3 different roles:
  - *Proposer*, *acceptor*, *learner*
- Proposers *propose* a value to be chosen on behalf of clients
- Acceptors (i.e., voters) *decide* which value to choose
- Learners *learn* which value was chosen and report final decision back to clients
- Processes can play 1, 2, or all 3 roles
  - Thinking of these roles as being separate makes Paxos easier to understand



Paxos: some ideas

1. One acceptor makes things really easy:
   - But this isn't distributed at all: if acceptor fails, game over!
   - So, let's have multiple acceptors, each of which can accept a proposed value
2. A value is chosen once a simple majority of acceptors accepts it
   - If *m* acceptors, then > *m*/2 need to accept

o meglio servono m/2+1 voti per la maggioranza.

   - Why does this work?
     - Any two majorities of acceptors must have at least one acceptor in common
     - An acceptor can accept only one value at a time
     - Therefore, any two majorities that choose a value must choose the same value

se ho due sottoinsiemi di m/2 acceptor avrò per forza la stessa maggioranza data l'intersezione non nulla e il fatto che un acceptor vota solo una volta

   - Just need to make sure acceptors do not accept something else once a value is chosen

Paxos: some ideas

3. Acceptors need a way to distinguish one proposal from another
   - Proposers assign a unique sequence number to each proposal they make
   - A proposal has two parts:
     - Proposal number (i.e., the unique identifier)
     - Proposed value (could be a decision value or some other information, such as "Frank's new salary" or "Position of Air France flight 21")
   - There can be *multiple distinct* proposals for the same value
     - But they differ by the proposal number !
4. Stable storage, preserved during failures, is used to maintain information that must be remembered in case of failure

Paxos: some ideas

5. Paxos use a multiple-round approach
   - Once a decision on a value is reached in a round, decisions in all subsequent rounds must agree
     - Once decision is reached on a value, Paxos must force future proposers to select that same value
   - *Within each round*, finding consensus is a two-phase process, where each phase consists of:
     - a request sent from a proposer to a group of acceptor
     - a reply from the acceptors to the proposer
   - The two phases are:

     1) **Prepare**

     2) **Accept**

si avranno diversi round fino a che non si raggiunge il consenso

Paxos: the two phases

- At high level:

  1) **Prepare**

  - A proposer asks a majority of acceptors whether anyone already received a proposal
  - If the answer is no, propose a value

  2) **Accept**

  - If a majority of acceptors agree to this value, then that is our consensus
- Goals of the two phases:

  Prepare

  - Proposers check whether a value has already been chosen se la risposta è no si propone un altro valore
  - Older proposals that have not yet completed are blocked con sequence number inferiore

  Accept

  - Ask acceptors to accept a specific value

Paxos algorithm: phase 1

- Phase 1 (**prepare**):
  a) A proposer selects a proposal number *n* and sends a *prepare request* with number *n* to a majority of acceptors

  b) If an acceptor receives a prepare request with number *n*:
     - If *n* is greater than the proposal number of *any* prepare request the acceptor has responded to, the acceptor promises not to accept any lower-numbered proposals and replies with the highest-numbered proposal and the proposed value the acceptor has accepted, if any priorità alla "nuova" proposta
     - Otherwise the acceptor does not respond (or responds with a negative ack) il numero della proposta è fondamentale per scegliere la value

l'esempio fa capire meglio

Paxos algorithm: phase 2

- Phase 2 (**accept**):
  a) If the proposer receives a response to its prepare requests for the proposal numbered *n* from a majority of acceptors, it sends an *accept request* to each of those acceptors for a proposal numbered *n* with a value *v* which is the value of the highest-numbered proposal among the responses of phase 1 (if no acceptor had accepted a proposal up to this point, then the proposer may choose any value for its proposal) se riceve una reply propone una value qualunque altrimenti propone la value associata alla proposta con numero più alto!

  b) If an acceptor receives an accept request for *n*, it accepts the proposal unless it has already responded to a prepare request having a number greater than *n*

attenzione: la value v NON è uguale al numero di proposta!

- Definition of chosen
  – A value is chosen at proposal number *n* *iff* majority of acceptors accept that value in phase 2 of the proposal number

Paxos properties

- Any proposal number is unique
- Any two sets of acceptors have at least one acceptor in common
- The value sent out in the accept phase is the value of the highest-numbered proposal of all the responses received in the prepare phase

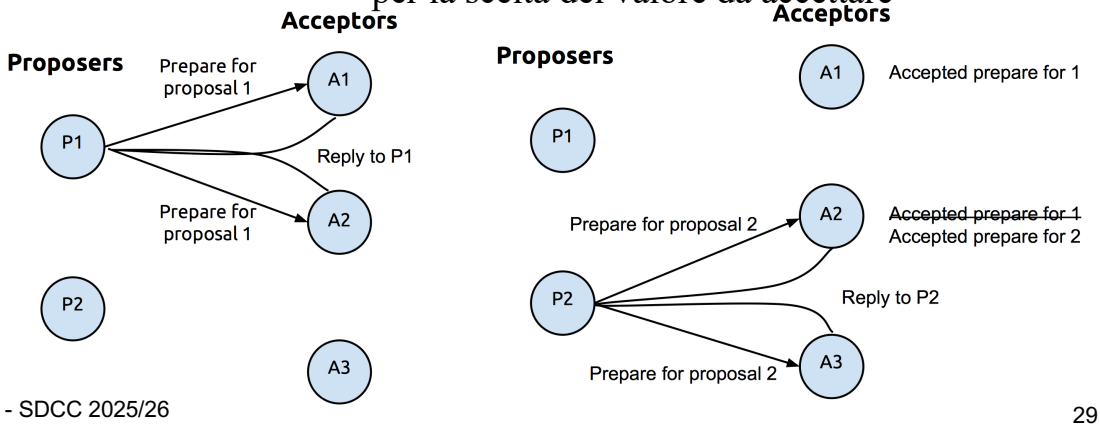
Paxos: example (without failures)

- Proposers are p_1 and p_2 e manderanno come value un gusto per la pizza
- Acceptors are a_1, a_2, and a_3 senza fallimenti

1° round, prepare phase

- p_1 sends prepare request for proposal 1 to a_1 and a_2
- a_1 and a_2 reply to p_1
- p_2 sends prepare request for proposal 2 to a_2 and a_3
- a_2 and a_3 reply to p_2

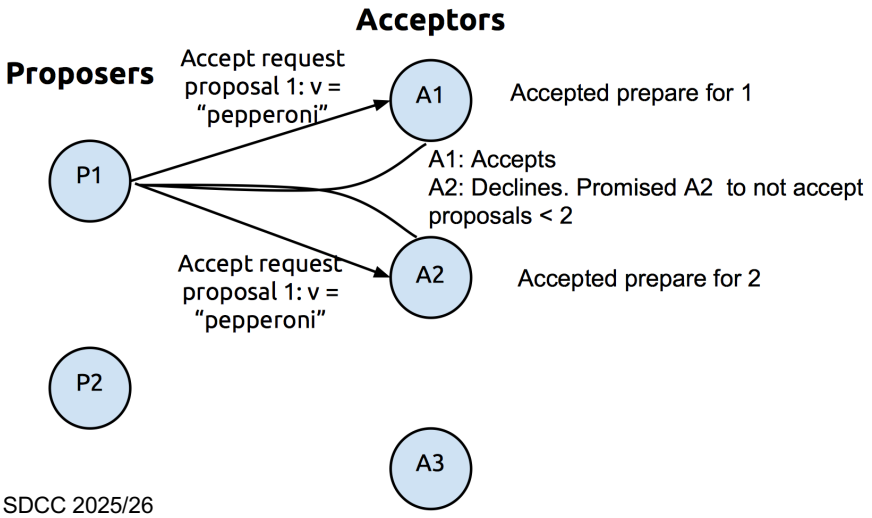
A2 è il nodo di intersezione, determinante per la scelta del valore da accettare



Paxos: example (without failures)

1° round, accept phase

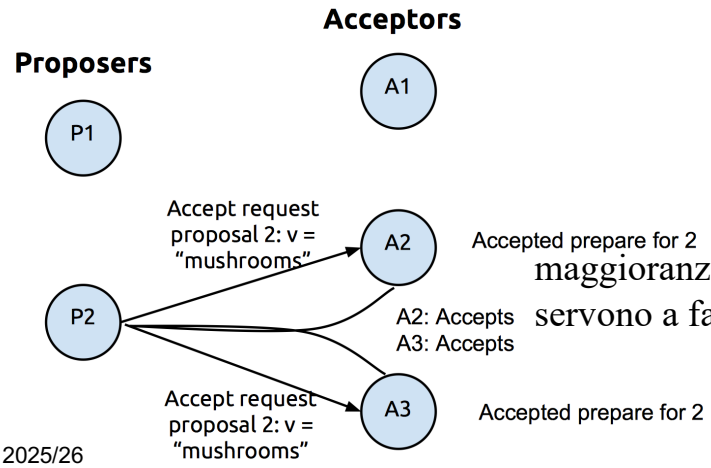
- p_1 sends accept request to a_1 and a_2 for proposal 1 with value "pepperoni"
  - p_1 got to select which value to propose
- a_1 accepts proposal 1
- a_2 does not accept proposal 1 (the older proposal is blocked)
  - a_2 promised p_2 it would not accept proposals < 2



Paxos: example (without failures)

1° round, accept phase (continued)

- p_2 sends accept request to a_2 and a_3 for proposal 2 with value "mushrooms"
  - p_2 also got to select which value to propose
- a_2 accepts proposal 2
- a_3 accepts proposal 2
- $\{a_2, a_3\}$ is a majority of acceptors, so proposal 2 is chosen
  - The chosen value is "mushrooms"



maggioranza raggiunta, i prossimi round servono a far convergere gli altri acceptor.

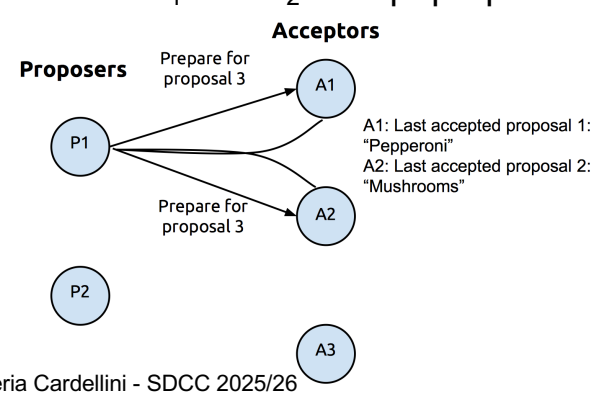
Paxos: example (without failures)

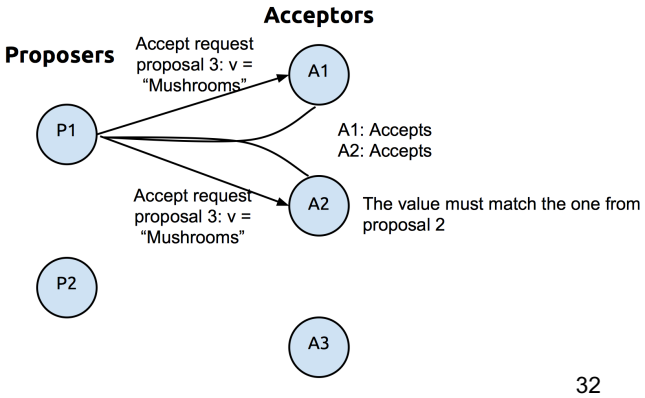
2° round, prepare phase

- p_1 sends prepare request for proposal 3 to a_1 and a_2
- a_1 replies; it last accepted proposal 1 for “pepperoni”
- a_2 replies; it last accepted proposal 2 for “mushrooms”

2° round, accept phase

- p_1 sends accept request to a_1 and a_2 for proposal 3 with value “mushrooms”
  - Value must match the one from proposal 2 perché numero di proposta più alto!
- a_1 and a_2 accept proposal 3





Paxos: what about learners?

- There are some options to learn a chosen value:

a) Each acceptor, whenever it accepts a proposal, informs all the learners
   ✗ Lots of messages to be sent

b) Acceptors inform a distinguished learner (usually the proposer) and let the distinguished learner broadcast the result
   ✗ Single point of failure

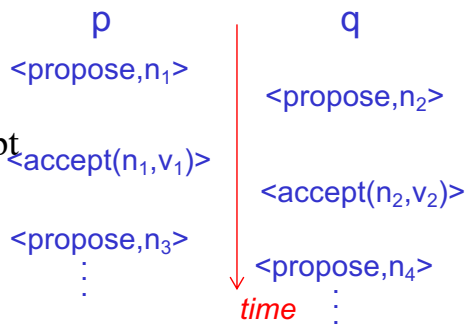
c) Compromise with a set of distinguished learners?
   ✓ Limits number of messages needed
   ✓ All distinguished learners need to fail to cause a problem

nel esempio eravamo nel caso C. se ho ad esempio 3 proposer e 4 acceptor, si avranno 2 learner cosi che si ha sia una ridondanza minima, sia il minor numero di messaggi in transito nella rete

Paxos: distinguished proposer (or leader)

- Multiple dueling proposers that propose conflicting values may stall the protocol (because of FLP result)

ogni volta una fase di propose termina prima della fase di accept del altro processo

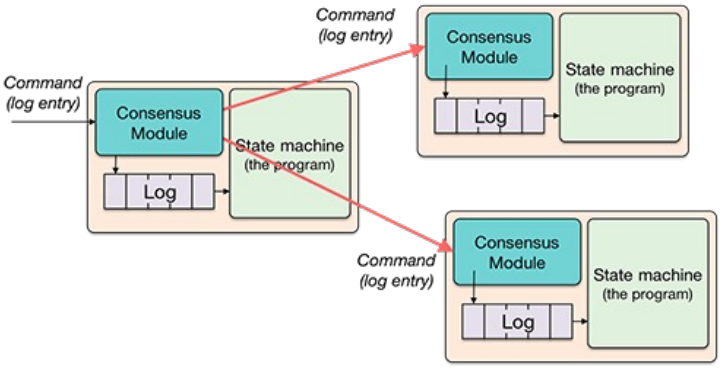


p completes phase 1 for proposal number n_1. Another proposer q then completes phase 1 for proposal number $n_2 > n_1$. p's phase 2 accept requests for proposal numbered n_1 are ignored because at least one acceptor has promised not to accept any new proposal numbered less than n_2. So, p then begins and completes phase 1 for new proposal number $n_3 > n_2$, causing the second phase 2 accept requests of q to be ignored. And so on.

ricorda vale FLP!

- Paxos guarantees progress (i.e., liveness) **if** only one of the proposers is eventually chosen as *leader*
- Therefore, in many Paxos implementations there is only one active proposer (i.e., leader)
  - Other proposers send proposals only when the current leader fails, and a new one needs to be elected

State machine replication and consensus protocols



- State machine replication (SMR): general approach to build fault-tolerant systems based on replicated servers
  - Each replica has a state machine (SM) and we want to make it fault-tolerant
  - A log is a list of commands that are received and stored; this list is read sequentially and used as input by SM
  - Using a consensus protocol, each SM processes the same list of commands in the log and thus produces the same series of results and arrives at the same series of states

SMR and Paxos

- Paxos is applied to achieve SMR
  - SM commands and their sequencing (the order in which they appear) are the values to agree
  - But requires one instance of Paxos per command: many instances of Paxos are executed simultaneously! non efficiente
- Multi-Paxos is a more efficient solution to reduce the number of messages
  - Why multi? Multiple rounds from a stable leader
  - Prepare phase only in first round, then only accept phase in next rounds
    - After first round, leader enters into to a galloping mode where it sends successive accept messages when it receives a majority of acks for previous accept request
    - Galloping mode may be interrupted by leader crashing, in that case new leader must be elected

Paxos: other common use patterns

- Besides SMR, there are other common use patterns of Paxos, including:
- Log replication
  - To duplicate data across different nodes (different from SMR whose goal is to make copies of server state)
- Synchronization service
  - To control concurrent access to shared data
- Configuration management
  - Leader election, group membership, service discovery, and metadata management

Paxos in practice

- Some DSs that use Paxos
  - The first ones: Petal (distributed virtual disks) and Frangipani (scalable distributed file system)
  - Chubby: Google's distributed lock service used in BigTable, Google Analytics and other Google products
    - Zookeeper uses a Paxos-variant protocol called Zab
  - Spanner: Google's globally distributed NewSQL database
  - XtreemFS: fault-tolerant distributed file system for WANs
  - Mesos: uses Paxos to manage its replicated log
  - LibPaxos: implementations for your app
- However, getting Paxos right in practice is hard
  - E.g., how to implement a globally unique proposal number
  - See Paxos made live paper by Google researchers

esiste ad oggi però un RFC che standardizza la procedura per creare id univoci che sono problematici per l'implementazione di Paxos

> While Paxos can be described with a page of pseudo-code, our complete implementation contains several thousand lines of C++ code. The blow-up is not due simply to the fact that we used C++ instead of pseudo notation, nor because our code style may have been verbose. Converting the algorithm into a practical, production-ready system involved implementing many features and optimizations – some published in the literature and some not.

Raft

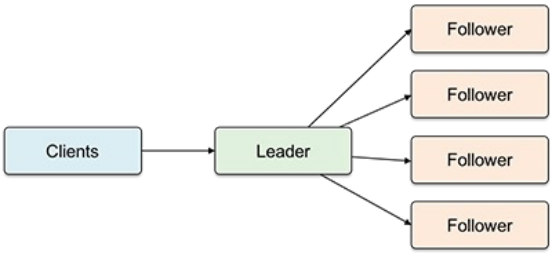
- Consensus algorithm developed in 2014 at Stanford University https://raft.github.io
  - RAFT: Replicated And Fault Tolerant
- Goals:
  - Designed to be easier to understand, implement and validate than Paxos
  - Complete foundation for implementation
    - Paxos not complete enough for real implementations, the algorithm is specified in a way that is detached from real-world implementation issues and use cases
    - Paxos implementations need extensive proofs and verification of their own, detaching them further from the original theoretical results
  - Equivalent and as efficient as Paxos for log replication

Raft: features

- System model similar to Paxos' one
  - Delayed/lost messages, fail stop (not Byzantine)
- Equivalent to (multi-)Paxos in fault-tolerance and performance
- Differences from Paxos si usa un approccio divede-et-impera
  - Problem decomposed into relatively independent sub-problems (leader election and log replication)
  - Addresses all major pieces needed for practical systems
- Rapid and widespread adoption
  - Many implementations currently listed on Raft home page, >10 versions in production
  - Some systems using Raft: etcd, CockroachDB, Consul, Hazelcast

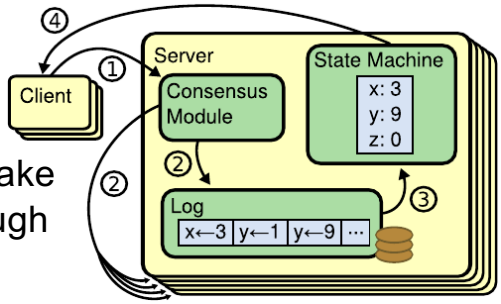
Raft: overview

- Raft is implemented on a cluster of servers, each of which hosts:
  - State machine (provided service)
  - Log that contains inputs fed into state machine
  - Raft protocol
- One of the servers is *elected* to be the *leader*, the others function as *followers*
- Clients send commands only to the leader, who forwards them to followers
- Each of the servers stores received commands in a log



Raft: overview

- Raft is a *state machine replication* protocol
  - Each server has a state machine and a log: state machine is what we want to make fault-tolerant (e.g., key-value pairs) through replication



  - The state machine is a deterministic program that specifies the desired behavior of the cluster as a whole
  - The state machine processes a sequence of *commands*, given by external clients; they interact with the system as if it were a single node running a single copy of the state machine
  - Each server simulates a copy of the state machine
  - Protocol goal: to maintain consistency across the copies of the state machine by ensuring proper log replication

Raft: overview

- Two major components of Raft

1. Leader election

  - Select one of the servers to act as leader
  - Leader is responsible for log replication to followers
  - In case of leader crash, choose new leader

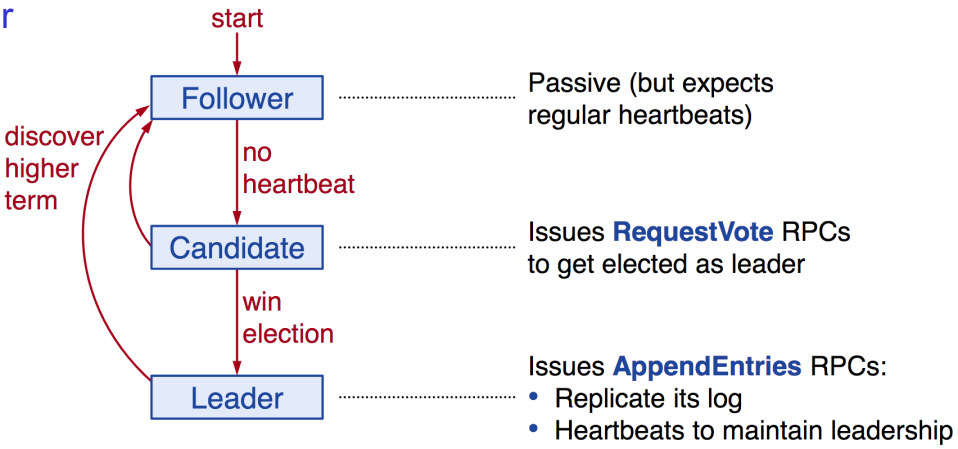
2. Log replication (normal operation)

  - Goal: make sure that replicated state machine is up to date across a majority of servers in the cluster
  - Leader accepts commands from clients, appends them to its log (note that log is append-only)
  - Leader replicates its log to other servers (overwriting inconsistencies)

- Let's examine Raft using http://thesecretlivesofdata.com/raft/ animazione molto utile per capire

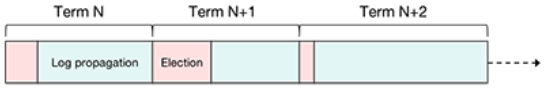
Raft: server state

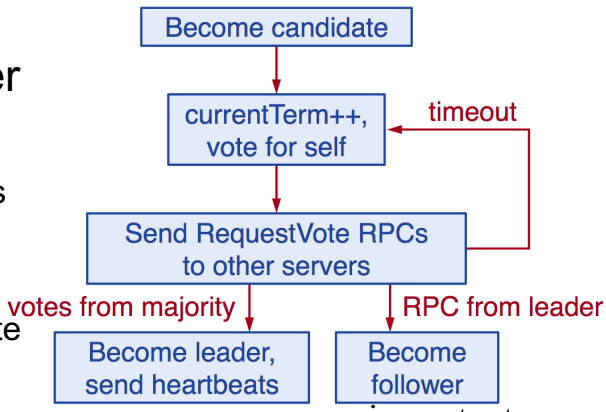
- A server can be in 1 of 3 states:
  - Leader (at most one leader per term)
  - Candidate
    - If followers don't hear from a leader (**heartbeat**) by their own **election timeout** then they can become a candidate
    - Request votes from other nodes
    - Can become the leader if gets majority vote (**leader election**)
  - Follower



Raft: leader election

- At most one leader per term un term dura fino a quando il leader non crasha
  - Some term has no leader because of failed election (*split vote*)
- Each server maintains *currentTerm* value (no global view)
  - Each server has its own local view of time that is represented by its *currentTerm*; it increases monotonically over time
- **Election timeout**
  - Follower waits until become candidate
- **Election term** starts if follower doesn't see a leader
  - Candidate votes for self, and sends out *RequestVote* RPCs to all the other servers
  - Receiving servers vote on candidate iff they haven't voted yet this term
  - Election timeout reset





quando si ha una differenza nel term, il leader con term più alto resta tale e chi era indietro si riallinea con il log del leader.

caso importante per gestire la consistenza del sistema

Raft: log replication

- Client sends command to leader
- Leader appends command to its log
- Leader sends *AppendEntries* RPCs to all followers in order to replicate all changes to all servers
  - *AppendEntries* message sent by leader to followers at periodic intervals specified by **heartbeat timeout** fondamentale
  - Followers acknowledge *AppendEntries* message
  - Once the leader receives acks from a majority of servers, it executes the command on its state machine and returns result to client, and the log entry is considered *committed*

solo dopo la commit si rende l'append al log effettiva. e dopo la commit del leader, si fa quella dei follower

Per l'elezione si hanno due timeout l'election timeout e l'heartbeat timeout. L'election timeout è un valore random tra 150ms e 300ms in modo che non ci siano candidati in concorrenza. L'heartbeat timeout deve essere inferiore al valore minimo dell'election timeout altrimenti ci sarebbero ogni volta nuovi candidati. Quindi se scade l'heartbeat parte l'election timeout e quando scade si manda la richiesta come candidato agli altri nodi.

Raft: log replication

- Once new log entry is committed:
  - Leader notifies followers of committed entries in subsequent *AppendEntries* RPCs per impedire anche lo scadere dell'heartbeat tiemout
  - Followers execute committed commands in their state machines
- Election term will continue until a follower stops receiving heartbeats and becomes a candidate

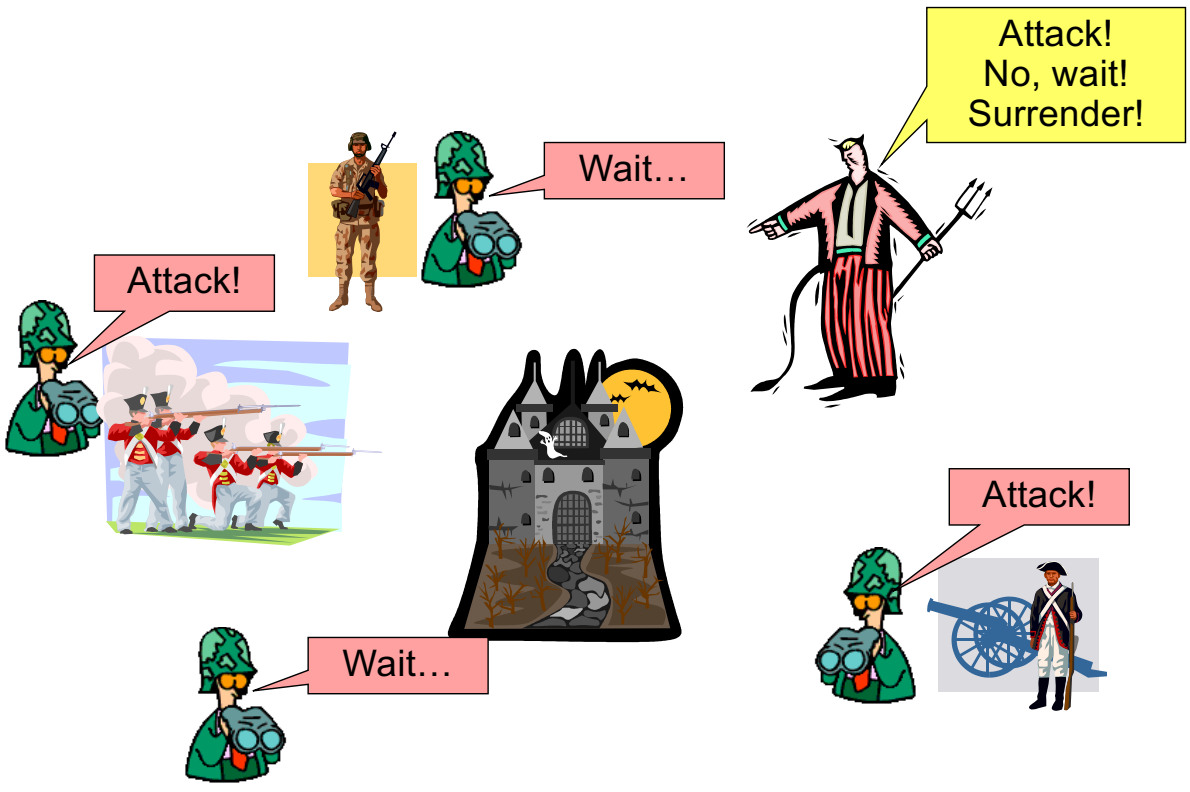
Raft: properties

- Raft guarantees safety
  - At most one leader per term anche in presenza di partizioni di rete!
  - Logs are kept consistent
  - Only a node with an up-to-date log can become a leader

- Raft also provides a liveness guarantee
  - If there are “sufficiently few failures”, then the system will eventually process and respond to all client commands

- Raft is tolerant to network partitions
  - Log uncommitted so long as no majority
  - Majority as seen in face of partition, e.g. 2+3 partition
  - Recovery
    - Old leader steps down when sees higher term
    - Rolls backs uncommitted entries and matches new leader’s log

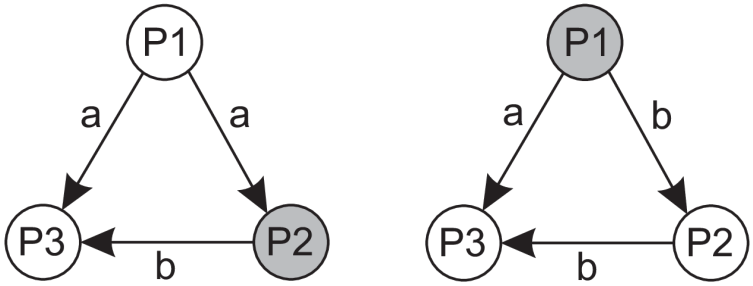
Byzantine scenario



Accordo in presenza di fallimenti bizantini

- In presenza di fallimenti bizantini, per sopravvivere ad attacchi di *k* processi guasti e raggiungere l'accordo distribuito occorre avere N >= 3*k*+1 processi
  - E' il problema dei generali bizantini (definito da Lamport)
  - *Idea*: si vuole raggiungere l'accordo se attaccare una città oppure ritirarsi tra un gruppo di generali fedeli, essendoci *k* generali traditori → occorrono 2*k*+1 generali fedeli
    - Se non ci sono più dei 2/3 di generali fedeli, non è possibile raggiungere l'accordo
    - P3 non riesce a capire chi sia il traditore tra P1 e P2

maggiore di 2k+1, numero necessario in Paxos e Raft



Problema dell'accordo bizantino

- *Assunzioni*:
  - Processi sincroni
  - Comunicazione unicast
  - Ordinamento dei messaggi
  - Ritardi limitati

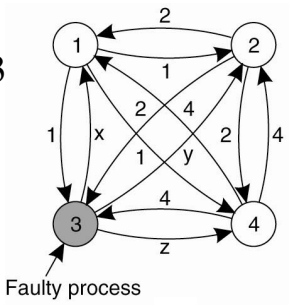
assunzioni tutto sommato forti

- Ci sono *N* generali (ovvero processi) ed ogni processo *i* fornisce un valore v_i agli altri processi
  - v_i rappresenta la forza della truppa del generale *i*
- *Obiettivo*: far costruire ad ogni processo un vettore *V* di dimensione *N* tale che se il processo *i* è non guasto allora *V[i]* = v_i, altrimenti *V[i]* è non definito

Problema dell'accordo bizantino

- Come raggiungere l'accordo nel caso di 3 processi che funzionano correttamente ed 1 fraudolento (N=4, k=1)?
  - Per semplicità assumiamo $v_i = i$

in questo esempio per ipotesi, il nodo 3 subisce un guasto bizantino ed invia agli altri nodi valori non significativi



Passo 1: ogni processo P_i invia il suo valore v_i agli altri

Passo 3: ogni processo invia il proprio vettore agli altri

Passo 2: i risultati del passo 1 sono riuniti in vettori V

1 Got(1, 2, x, 4)
2 Got(1, 2, y, 4)
3 Got(1, 2, 3, 4)
4 Got(1, 2, z, 4)

1 Got	2 Got	4 Got
(1, 2, y, 4)	(1, 2, x, 4)	(1, 2, x, 4)
(a, b, c, d)	(e, f, g, h)	(1, 2, y, 4)
(1, 2, z, 4)	(1, 2, z, 4)	(i, j, k, l)

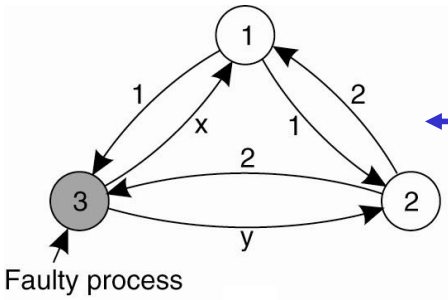
Passo 4: voto di maggioranza sui valori ricevuti: se P_i funziona correttamente, $V[i] = v_i$

1 2 ? 4

applicando la maggioranza su ogni colonna, la terza colonna non è definita

Problema dell'accordo bizantino

- Perché con 2 processi che funzionano correttamente ed 1 processo fraudolento (N=3, k=1) non si riesce a raggiungere l'accordo?



Passo 1: ogni processo invia il suo valore agli altri

Passo 3: ogni processo invia il proprio vettore agli altri

Passo 2: i risultati del passo 1 sono riuniti in vettori

1 Got(1, 2, x)
2 Got(1, 2, y)
3 Got(1, 2, 3)

1 Got	2 Got
(1, 2, y)	(1, 2, x)
(a, b, c)	(d, e, f)

Passo 4: voto di maggioranza sui valori ricevuti

? ? ?

Lamport's algorithm for Byzantine agreement (oral message)

Algorithm OM(0)

1. The commander sends his value to every lieutenant
2. Each lieutenant uses the value he receives from the commander, or uses the value RETREAT if he receives no value

Algorithm OM(k), k>0 guarda l'esempio per capire meglio

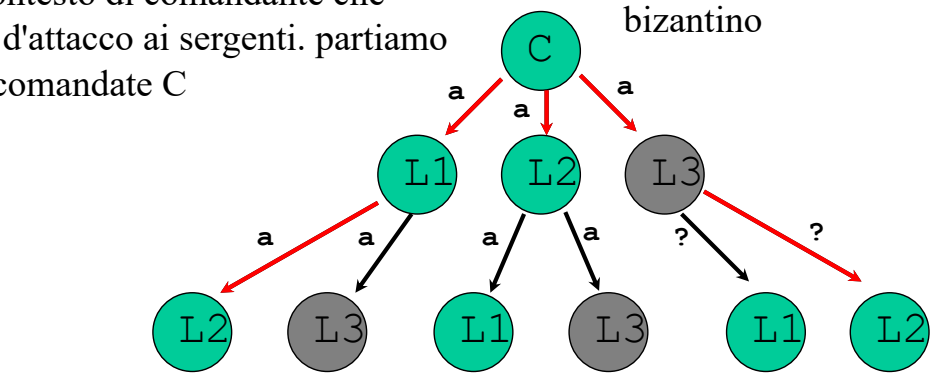
1. The commander sends his value to every lieutenant
2. For each i, let v_i be the value Lieutenant i receives from the commander, or else be RETREAT if he receives no value. Lieutenant i acts as the commander in Algorithm OM(k-1) to send the value v_i to each of the N-2 other lieutenants
3. For each i and each $j \neq i$, let v_j be the value Lieutenant i received from Lieutenant j in step 2 (using Algorithm OM(k-1)), or else RETREAT if he received no such value. Lieutenant i uses the value majority (v_1, …, v_{N-1})

Lamport et al, The Byzantine Generals Problem, ACM Transactions on Programming Languages and Systems, 1982

Demo: OM(1), L3 as traitor

restando nel contesto di comandante che invia comandi d'attacco ai sergenti. partiamo dalla radice o comandate C

in grigio ci sono i nodi con fallimento bizantino



L2 → **L1** said **C** said **'a'**

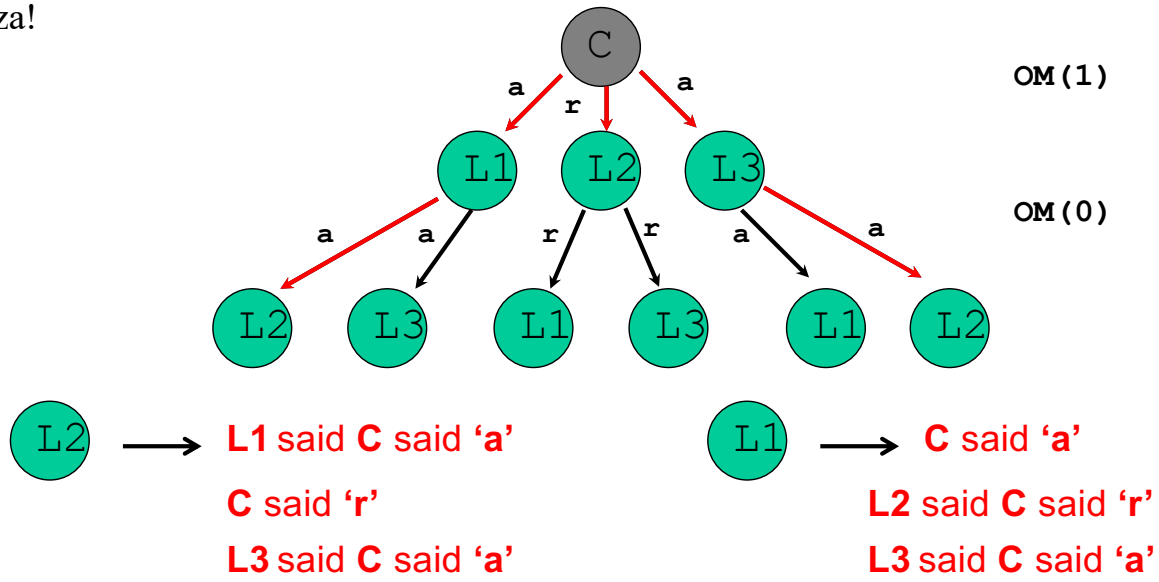
C said **'a'**

L3 said **C** said **'?'**

Result: majority(a, a, ?) = a

Demo: OM(1), C as traitor

anche se il traditore è il comandante si ha ha tolleranza!



L2 result: majority(a, r, a) = a; L1 result: majority (a, r, a) = a

Byzantine fault tolerance (BFT) in practice

- What about the performance of Byzantine generals algorithm?
  - k+1 synchronous rounds: quite slow
  - $O(N^k)$ messages: high traffic

velocità algoritmo: n° di round pari al numero di guasti più 1

da notare come con k>>1 esplode il n° di messaggi

- BFT protocols were long considered too expensive to be practical
- In 1999 Practical Byzantine Fault Tolerance (PBFT) algorithm was proposed
  - Thousands of requests per second with only sub-millisecond increases in latency
  - PBFT triggered a renaissance in BFT research

Byzantine fault tolerance and blockchains

- Blockchain system require BFT
- For example, Bitcoin uses Nakamoto Consensus protocol, which elects the leader through a Proof of Work algorithm
  - Participating nodes compete to solve a puzzle, and the node that solves the puzzle first becomes the leader
  - The leader has the right to generate a new block and append it to the blockchain

Oggi giorno si usa anche la Proof of Stake. Ad esempio Ethereum utilizza la proof of Stake che ha un consumo energetico di molto inferiore a quella del Proof of Work

il PoW soffre però di un alto consumo energetico

References

- Sections 8.1, 8.2, 8.3 and 5.4.4 of van Steen & Tanenbaum book

- Lamport, Paxos made simple, *ACM SIGACT News*, 2001.
- The Paper Trail, Consensus Protocols: Paxos, 2009. (html)

- References on Raft
  - Raft paper: In Search of an Understandable Consensus Algorithm (extended version), 2014
  - Talk on Raft by John Ousterhout, 2016 https://youtu.be/vYp4LYbnnW8
  - Another Raft visualization: https://raft.github.io/raftscope/

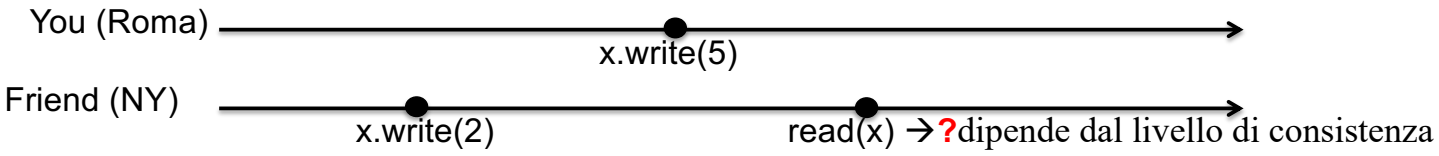
- Lamport et al, The Byzantine Generals Problem, ACM Transactions on Programming Languages and Systems, 1982

Why replicate data?

- To increase DS availability when servers fail or network is partitioned
  - p = probability that 1 server fails
  - p^n = probability that n servers fail
  - $1-p^n$ = availability of service/system with n servers
    - p=5% and n=1 => service is available 95% of time
    - p=5% and n=3 => service is available 99.9875% of time
- To increase DS fault tolerance
  - Under *fail-stop model*, if up to k of k+1 servers crash, at least one is alive and can be used
    - Fail-stop: failed component simply stops functioning without any additional erroneous behavior
  - Protect against corrupted data
- To improve DS performance through scalability
  - Scale with size and geographical areas

Replication cons

- What does data replication entail?
  - Having multiple copies of same data
- We need to keep replicas **consistent**
  - When one copy is updated we need to ensure that the other copies are updated as well; otherwise the replicas will no longer be the same



Data center in North Carolina

Data center in Ireland

Consistency: the fundamental issue

- Consistency maintenance is itself an issue
- How and when to update replicas?
- How to avoid significant performance loss due to consistency, especially in large scale DS?
  - Latency is non-negligible
    - Inter-data center latency: from 10 ms to 250 ms
    - Even inside data center: ~1 ms
  - and impacts performance
    - Amazon said: *just an extra one tenth of second (i.e., 100 ms) on the response times will cost 1% in sales*
    - Google said: *a half a second (i.e., 500 ms) increase in latency will cause traffic to drop by a fifth*

Consistency: what we need

- To keep replicas consistent, we generally need to ensure that all conflicting operations on the same data are done in the the same order everywhere
- Conflicting operations (from transactions world):
  - Read-write conflict: a read operation and a write operation act concurrently
  - Write-write conflict: two concurrent write operations
- Guaranteeing global ordering on conflicting operations may be too costly (requires global synchronization), thus downgrading scalability
- *Solution*: weaken consistency requirements so that global synchronization can be avoided and we get a "consistent" and efficient system

l'operazione che porta a conflitti e inconsistenza è la write

l'utilizzo di un clock fisico sincronizzato come visto è nella pratica irrealizzabile nei SD

nascono dunque i modelli di consistenza

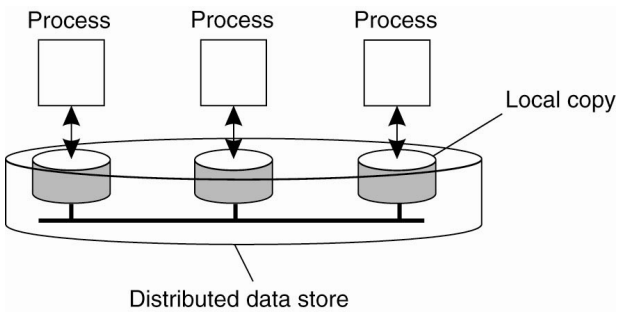
➡ **Different consistency models**

introduciamo i modelli di consistenza

Consistency models

- **Distributed data store**: distributed collection of storage, physically distributed and replicated across multiple processes
  - E.g., distributed database, distributed file system, Cloud storage

ipotiziamo che nel sistema ciascun processo effettua le operazioni sulla propria copia locale



- **Consistency model** (or consistency semantics)
  - Contract between a distributed data store and processes, in which the data store specifies precisely what the results of read and write operations are in presence of concurrency

ad esempio il contratto specifica che la read restituisca il valore dell'ultima write effettuata.

Consistency models

- All consistency models try to return *the last write* operation on the data as a result of data read operation
- Consistency models differ in *how* the last write operation is determined/defined and with respect *to whom*
- **Data-centric** consistency models più accurati
  - Goal: provide a system-wide view of a consistent data store
- **Client-centric** consistency models che non vedremo
  - Goal: provide a view of a consistent data store at a single client level
  - Faster but less accurate consistency management than data-centric consistency

Choosing a consistency model

- No right or wrong consistency model
  - There is no unique general solution (i.e., consistency model that fits well all situations) but rather multiple solutions, that are suitable to applications with different consistency requirements
- Non-trivial trade-off among easy of programmability, cost/efficiency, consistency, and availability
  - Low consistency is cheaper but it might result in higher operational cost (e.g., overselling of products in a Web shop)
- Not all data need to be treated at the same level of consistency
  - Consider a Web shop: credit card and account balance information require higher consistency levels, whereas user preferences (e.g., “users who bought this item also bought… ”) can be handled at lower consistency levels

Data-centric consistency models

- Consistency models describe *how* and *when* different data store replicas see operations order
  - Replicas must agree on the global ordering of operations before making them persistent

un esempio di data store è Cassandra che permette di scegliere il livello di consistenza sulle operazioni oppure AWS DynamoDB che a seconda del livello aumenta costi e costo computazionale

Data-centric consistency models we study

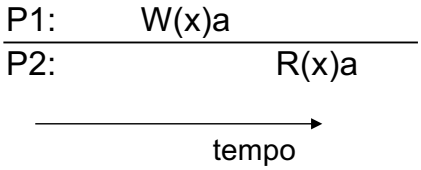
- Main consistency models based on ordering of read and write operations on shared and replicated data

l'intervallo di tempo detto finestra di inconsistenza è quello in cui due dispositivi leggono due valori diversi prima che la consistenza sia aggiornata

- Strict consistency: strongest model
- Linearizability, sequential, causal and eventual consistency: progressive weakening of strict consistency

Modelli di consistenza: notazione

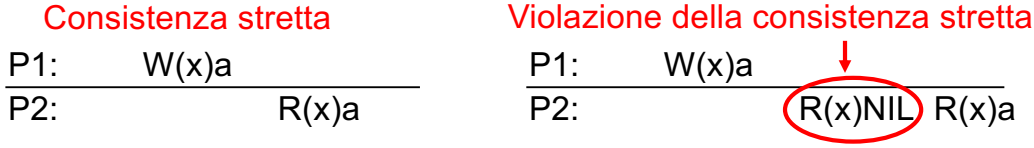
- Rappresentiamo il comportamento dei processi che eseguono operazioni di lettura o scrittura sui dati condivisi
  - $W_i(x)a$: operazione di scrittura da parte del processo P_i sul dato x con valore scritto a
  - $R_i(x)b$: operazione di lettura da parte del processo P_i sul dato x con valore letto b



Consistenza stretta: il modello ideale

riassumibile come

Qualsiasi read su un dato x ritorna un valore corrispondente al risultato più recente della write su x



- Write eseguita su tutte le repliche come singola operazione atomica
  - E' come se ci fosse una copia unica, ovvero la write è vista istantaneamente da tutti i processi

richiede però

- Ordinamento temporale assoluto delle operazioni: richiede un clock fisico globale
  - No ambiguità su "più recente"

Implementing strict consistency

P1:	W(x)a	
P2:		R(x)a

- To achieve it, one would need to ensure:
  - Each read must be aware of, and wait for, each write
    - $R_2(x)a$ aware of $W_1(x)a$
    - Clocks must be strictly synchronized
- But time between instructions << communication time: strict consistency is tough to implement efficiently a meno di bloccare le operazioni, come l'atm di una banca
- Solution: linearizability and sequential consistency
  - Slightly weaker models than strict consistency
  - Still provide the illusion of single copy
    - From the outside observer, the system should (almost) behave as if there's only a single copy

il rilassamento consiste nel non avere un ordinamento totale nel sistema

Linearizability più utilizzato nella pratica come modello

ogni operazione viene vista come avere un effetto istantaneo in un qualche momento compreso tra l'inizio e la conclusione dell'operazione, come se ci fosse un clock fisico sincronizzato che ci permette di attribuire un valore globale alle operazioni. se due operazioni sono concorrenti dato che non ho sufficiente risoluzione dell'orologio, tutti devono vedere lo stesso ordine

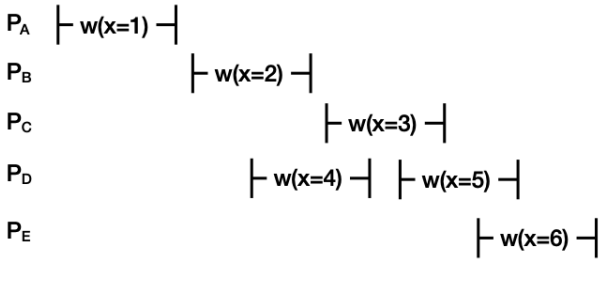
- Operations (op = read, write) receive global timestamp using synchronized clock (e.g., NTP) sometime during their execution la risoluzione ricordiamo è l'intervallo minimo tra due istanti che l'orologio può rappresentare come diversi.
- All replicas execute operations in some total order
- That total order preserves the real-time ordering between operations (and each process' own local ordering) non si ha ordine temporale assoluto
  - If op A completes before op B begins, then A is ordered before B in real-time
  - If neither A nor B completes before the other begins, then there is no real-time order. But there must be some total order (i.e., same order for A and B)

tutti vedono lo stesso ordine anche se non è assolutamente corretto nel tempo, ma è corretto dal punto di vista di tempo reale. vedi esempio per capire meglio

Linearizability: properties

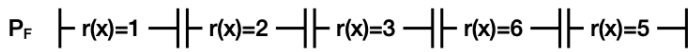
- Weaker model than strict consistency
- Still provides single-client, single-copy semantics
  - That is, read/write behave as if there were
    - a single client making all the requests in a given order
    - over a single copy
- A read op returns the most recent write, regardless of the clients
- All subsequent reads should return the same result until the next write, regardless of the clients
- Does not mandate any particular order for *overlapping operations*
  - The system needs to provide an ordering of ops
  - The ordering should give an illusion that there is a single copy

Linearizability: example

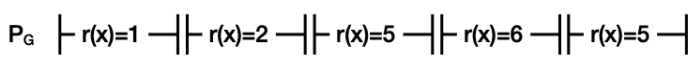


in questo esempio abbiamo che il processo B e C ma anche C e D effettuano delle operazioni che sono parzialmente sovrapposte

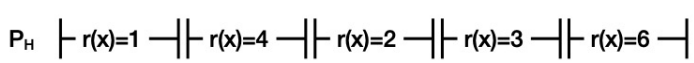
se le operazioni non sono sovrapposte devo vederle nel ordine temporale assoluto



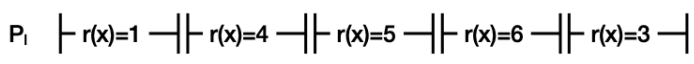
✓



X qui si ha doppia lettura di 5 ma così si torna indietro nel tempo

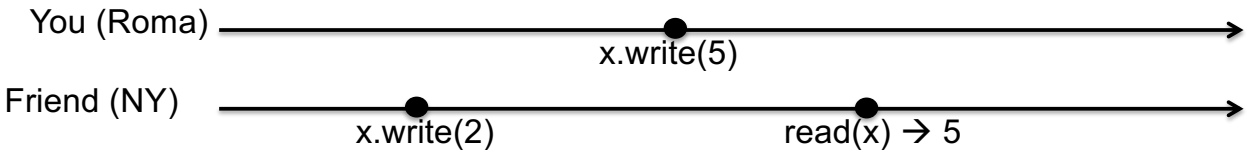


✓



X 6 e 3 non sono sovrapposte, anche qui si torna indietro!

Linearizability: performance



- How to implement linearizability? (see slide 2)
  - Clients send read/write requests to Ireland DC (primary)
  - Ireland datacenter propagates write to North Carolina DC
  - Read never returns until propagation is done
  - Linearizability? Yes
  - Performance? No, because of WAN latencies
- Linearizability requires *complete* synchronization of multiple replicas before write returns
- It makes less sense in global setting, but still makes sense in local setting (e.g., within a single data center)

Consistenza sequenziale

Il risultato di una qualunque esecuzione è uguale a quello ottenuto se le operazioni (di read e write) da parte di tutti i processi sull'archivio di dati fossero eseguite

- *secondo un* ***ordine sequenziale*** che non è quello temporale
- ***e*** *le operazioni di ogni singolo processo apparissero in questa sequenza* ***nell'ordine specificato dal suo programma***

vedi esempio per capire

- Quando i processi sono in esecuzione concorrente, qualunque alternanza (*interleaving*) di operazioni è accettabile (purché rispetti l'ordine di programma), ma *tutti i processi vedono la* ***stessa alternanza di operazioni***

Sequential consistency: properties

- Weaker model than linearizability
- Still provides single-client, single-copy semantics
- All replicas execute operations in some total order
- That total order preserves the program order of each process between operations
  - If process P *issues* op A before op B, then A is ordered before B by P's program order (i.e., preserves local ordering)
  - If ops A and B and done by different processes then there is no program order between them. But there must be *some* total order

Sequential consistency: example

- Sequentially consistent data store

P1:	W(x)a			
P2:		W(x)b		
P3:			R(x)b	R(x)a
P4:			R(x)b	R(x)a

anche in questo caso tutte le read devono leggere lo stesso valore finché non avviene una nuova write!

- Operation interleavings that satisfy program order of each process (∃ total order + process' program ordering) in questo esempio 3 e 4 devono leggere prima B che A

  $W_2(x)b\ R_3(x)b\ R_4(x)b\ W_1(x)a\ R_4(x)a\ R_3(x)a$

  $W_2(x)b\ R_4(x)b\ R_3(x)b\ W_1(x)a\ R_4(x)a\ R_3(x)a$

  $W_2(x)b\ R_3(x)b\ R_4(x)b\ W_1(x)a\ R_3(x)a\ R_4(x)a$

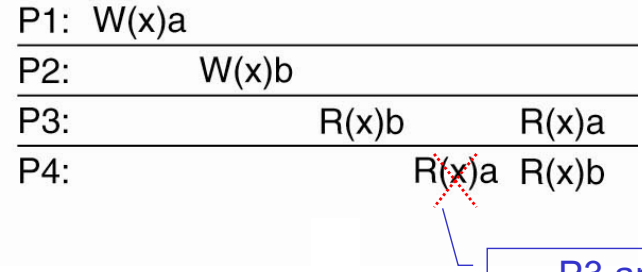
  $W_2(x)b\ R_4(x)b\ R_3(x)b\ W_1(x)a\ R_3(x)a\ R_4(x)a$

tutti vedono che prima si scrive b e poi a anche se non sono in ordine temporale

una sequenza del tipo: W2(x)bR3(x)aR3(x)b...
non è ammissibile dato che si legge prima a che b e non si rispetta l'ordine di programma del processo 3!

Sequential consistency: example

- Data store that is not sequentially consistent



- We cannot find any interleaving that satisfies the program order of each process, e.g.,
  - $W_1(x)a\ R_4(x)a\ R_3(x)a\ W_2(x)b\ R_3(x)b\ R_4(x)b$ violates P3 program order
  - $W_2(x)b\ R_3(x)b\ R_4(x)b\ W_1(x)a\ R_3(x)a\ R_4(x)a$ violates P4 program order

Sequential consistency: performance

- Sequential consistency is *programmer-friendly*, but hard to implement efficiently
  - Writes should be applied in the same order across different copies to keep the single-copy illusion

- How to implement sequential consistency?
  - Use a global sequencer (centralized), or
  - a totally ordered multicast protocol (decentralized)

un idea per raggiungere il consenso nell'ordinamento è utilizzare raft

Linearizability vs sequential consistency

- Linearizability is stronger than sequential consistency
- Both provide single-client, single-copy semantics
- With linearizability: interleaving across all processes is pretty much determined on the basis of time
- With sequential consistency: freedom to interleave operations coming from different processes, as long as ordering from each process is preserved

- In a nutshell:
  - Linearizability: ∃ total order + real-time ordering
  - Sequential: ∃ total order + process' program ordering

Casual and eventual consistency

- More relaxed consistency models are often used to increase performance and availability and lower cost
  - **Causal** consistency
  - **Eventual** consistency
- But we lose the single-copy illusion
- Causal consistency
  - We care about ordering causally-related write operations correctly (e.g., Facebook post-like pairs)
- Eventual consistency detta consistenza finale in italiano
  - As long as we can say all replicas converge to the same copy eventually, we're fine

si ha una finestra di inconsistenza appunto un intervallo di tempo in cui la consistenza non è garantita

Casual consistency: informal example

- Consider these posts on a social network:
  1. Oh no! My cat just jumped out the window.
  2. [a few minutes later] Whew, the catnip plant broke her fall.
  3. [reply from a friend] I love when that happens to cats!

a seconda dell'ordine causale le informazioni sono diverse

- Causality violation could result someone else reads:
  1. Oh no! My cat just jumped out the window.
  2. [reply from a friend] I love when that happens to cats!
  3. Whew, the catnip plant broke her fall.

Consistenza causale

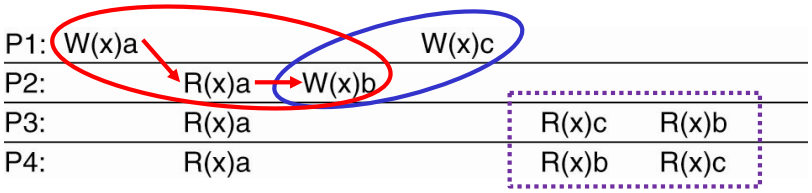
Operazioni di write che sono potenzialmente in relazione di causa/effetto devono essere viste da tutti processi nello stesso ordine. Operazioni di write concorrenti possono essere viste in ordine differente da processi differenti

- In relazione di causa/effetto:
  - read seguita da write sullo stesso processo: write è (potenzialmente) causalmente correlata con read
  - write di un dato seguita da read dello stesso dato su processi diversi: read è (potenzialmente) causalmente correlata con write
  - Si applica la proprietà transitiva: se P1 scrive x e P2 legge x e usa il valore letto per scrivere y, la lettura di x e la scrittura di y sono causalmente correlate
- Se due processi scrivono simultaneamente, le due write non sono causalmente correlate (*write concorrenti*)

- Indebolimento della consistenza sequenziale
  - Distingue tra operazioni che sono potenzialmente in relazione di causa/effetto e quelle che non lo sono

Consistenza causale: esempi

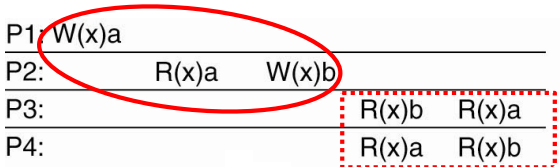
- Esempio di sequenza valida in un archivio di dati causalmente consistente, ma non in un archivio sequenzialmente consistente
  - $W_2(x)b$ e $W_1(x)c$ sono write concorrenti: possono essere viste dai processi in ordine differente
  - $W_1(x)a$ e $W_2(x)b$ sono write in relazione di causa/effetto



No consistenza sequenziale

Causal consistency: examples

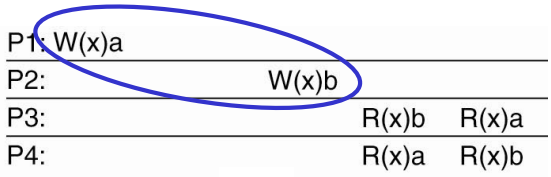
- Example 1: sequence of operations which is not valid in a causally consistent data store
  - $W_1(x)a$ and $W_2(x)b$ are causally related: must be seen in same order by all processes



Different order

le sequenze: W1(x)aR(x)aW(x)bR3(x)bR4(x)a come W1(x)aR(x)aW(x)bR4(x)aR3(x)b non sono valide, perché non si può leggere a se l'ultimo valore di x è b!

- Example 2: sequence of operations which is valid in a causally consistent data store
  - $W_1(x)a$ and $W_2(x)b$ are concurrent: can be seen in different order
  - But not valid in a sequentially consistent data store



esempio: W1(x)aR3(x)aR4(x)aW2(x)bR3(x)bR4(x)b è valida causally ma non sequentially

Implementing causal consistency

- No longer single-copy illusion
  - Concurrent writes can be applied in different orders across copies
  - Causally-related writes do need to be applied in the same order for all copies
- Thanks to relaxed requirement on writes, latency is less problematic
- However, we need a mechanism to keep track of causally-related writes (i.e., which processes have seen which writes)
  - Build and maintain a dependency graph showing which operations depend on which other operations
  - Or use vector clocks: more amenable for computation

Eventual consistency

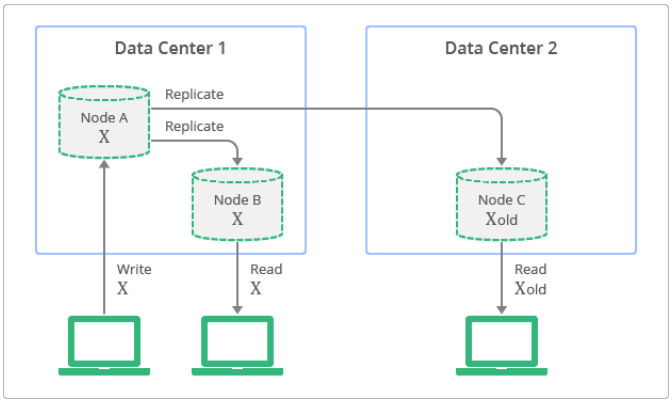
Ipotiziamo di avere dei data store in cui non esistono conflitti di tipo write-write o sono molto facili da risolvere, perché si assume che solo un piccolo gruppo di processi può effettuare le write. Dunque nella pratica, spesso in caso di conflitti, solo una specifica write viene effettuata andando così a sovrascrivere gli effetti di ogni altra write in conflitto. Avremo perciò:

- Strong prevalence of reads compared to writes (i.e., mostly read)

- we can adopt a relaxed consistency model, called eventual consistency (in Italian: eventual=*finale*)
- What it guarantees: if no new updates occur, *eventually* all reads will return the last updated value
  - That is, all replicas gradually become consistent within a time window (called *inconsistency window*)
  - Without failures, length of inconsistency window depends on: communication latency, number of replicas, system load
- Model used in some Cloud storage services and NoSQL data stores

Dunque nel eventual consistency, tutte le repliche convergeranno in copie identiche. Essenzialmente si richiede solo che la propogazione degli update sulle repliche sia garantita.

Eventual consistency vs. strong consistency

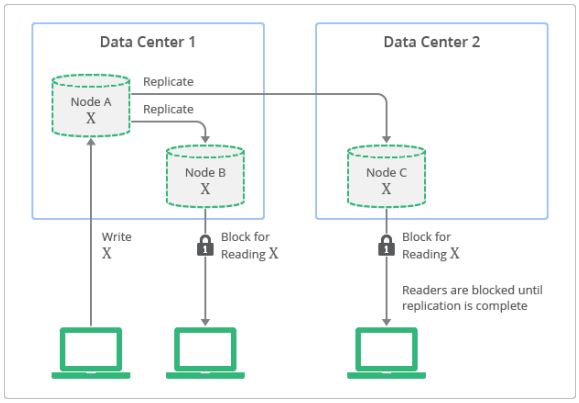


Eventual consistency

- Replicas are always available to read
- But some replica (e.g., C) may be inconsistent with the latest write

Strong consistency (e.g., linearizability)

- Replicas are always consistent
- But replicas are not available until update completes



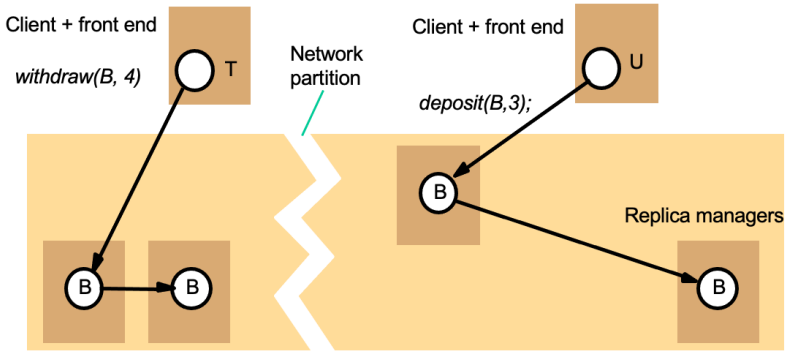
Eventual consistency: pros and cons

- Pros
  - ✓ Simple and inexpensive to implement
  - ✓ Fast reads and writes on local replica
  - – E.g., used by DNS: when authoritative name server updates a resource record, other name servers store it for TTL period
    ad esempio youtube ha un TTL di 300 secondi
- Cons
  - ✗ No illusion of single copy
  - ✗ Possible data inconsistency (*staleness*) caused by conflicting writes: conflicts must be resolved by means of reconciliation strategy
  - ✗ Can make applications more complex to write: providing stronger consistency falls on developer's shoulder
    - Developer must know which consistency model is provided by data store
    - If read does not return the value of the most recent write, developer must decide whether such inconsistency is acceptable to application

Eventual consistency: reconciliation

- *How* to reconcile conflicting versions of replicas that have diverged due to concurrent updates?
  - – Popular strategy: last write wins
    - Tag data with vector clock as timestamp and use vector clock to capture causality between different versions of data
    - Popular solution in many systems (e.g., Cassandra)
  - – Alternatively, push conflict resolution to application which invokes a user-specified conflict handler (e.g., done by Amazon Dynamo)
- *When* to reconcile?
  - – Usually on read (e.g., Amazon Dynamo) so to provide an "always-writable" experience (but slows down reads)
  - – Alternatives are: on write (reconcile during writes, slowing down them) and asynchronous repair (correction is not part of read or write ops)

Consistency and network partitions



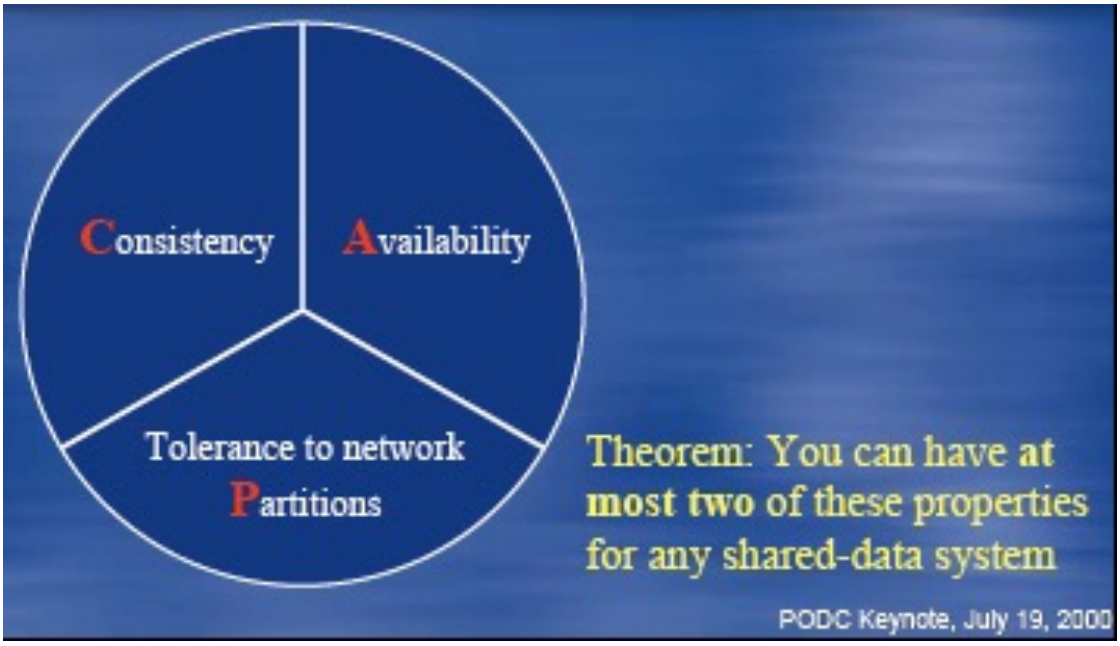
- Dilemma with network partitions
  - To keep replicas consistent, you need to block waiting for replicas update
    - To outside observer, system appears to be unavailable
  - If you don't block and still serve requests from the two partitions, then replicas will diverge
    - System is available, but weaker consistency
- Which choice? **CAP theorem** explains this dilemma

CAP theorem

- CAP theorem
  - Conjecture proposed by E. Brewer in 2000 and formally proved by S. Gilbert and N. Lynch in 2002 under certain conditions
  - One of the most important findings for distributed data stores
- Any networked shared-data system can have **at most two** of the three desirable properties at any given time:
  - **Consistency** (***C***): have a single up-to-date copy of data
    dalla sequenziale in su
    "All the clients see the same view, even in presence of updates."
  - **Availability** (***A***) of that data (for updates)
    "All clients can find some replica of data, even in presence of failure."
  - **Partition tolerance** (***P***)
    "The system property holds even if the system is partitioned."

Brewer's slides at PODC 2000 https://people.eecs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf

CAP theorem



Partition tolerance: why do we care?

- Network partitions can occur across data centers when Internet gets disconnected
  - Internet router outages
  - Under-sea cables cut
  - DNS not working

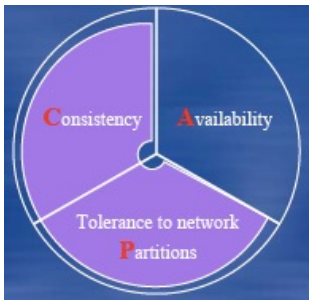
As result of partition, network can lose arbitrarily many messages sent from one node to another

- Network partitions can also occur within a datacenter (e.g., rack switch outage), but less frequently
- We still desire DS to continue functioning normally under network partitions → fix **P**
- According to CAP, consistency and availability cannot be achieved at the same time when partition occurs
- Which one to give up? **C**onsistency or **A**vailability?

It's a design choice

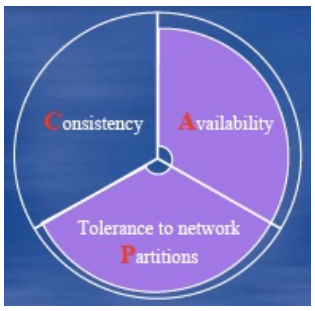
CAP and network partitions

- If consistency is priority, forfeit availability: **CP** system



- If availability is priority, forfeit consistency: **AP** system
  - Relaxed consistency model: **eventual consistency**

i primi sistemi cloud erano sistemi AP

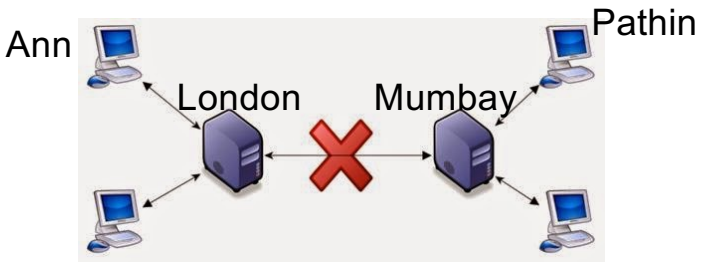


CAP and network partitions

- When using CP and AP systems, developer needs to be aware of what system offers
- CP system: may not be available to take a write
  - If write fails because of system unavailability, developer has to decide what to do with data to be written esempio ATM banca
- AP system: may always accept a write, but under certain conditions a read will not reflect the result of a recently completed write esempio google drive: mi interessa avere il file non magari l'ultimo aggiornamento della cartella
  - Developer has to decide whether application requires access to the absolute latest update all the time
- Take-away message: CAP choice depends on application requirements
  - Blog different from financial exchange or shopping cart

CAP: example

- Booking system of Ace Hotel in New York uses a replicated database with master server located in Mumbai and one replica server in London
- Ann is trying to book a room on replica server
- Pathin is trying to do the same on master
- There is only a room available and the network link between the two servers breaks



CAP: example

- CP system:
  - Pathin can book the room soluzione reale
  - Ann can see the room information but cannot book it
- AP system: both servers accept the room booking
  - Overbooking!
- CA system: neither user can book any hotel room
  - No tolerance to network partitions

ACID vs BASE

- ACID and BASE: contrasting approaches to achieving data consistency in DS
- **ACID: Atomicity, Consistency, Isolation, Durability**
  - Pessimistic approach: prevent conflicts from occurring
  - Standard for relational DBMSs: Postgres and MySQL are CA!
- **BASE: Basically Available, Soft state, Eventual consistency**
  - Optimistic approach: let conflicts occur, but detect them and take action to sort them out
  - *Basically available*: system is available most of the time and there could exist a subsystem temporarily unavailable
  - *Soft state*: data is not durable in the sense that its persistence is left to developer that must take care of it
    - Data is durable if its changes survive failures and recoveries
  - *Eventually consistent*: system eventually converges to a consistent state

ACID vs BASE

- Consistency
  - ACID provides strong consistency, ensuring that data is always in a consistent state un DB SQL
  - BASE provides eventual consistency, allowing temporary inconsistencies but ensuring convergence to a consistent state over time uno storage online
- Availability
  - ACID system may experience limited availability during certain operations or under system failures, as it prioritizes consistency
  - BASE system prioritizes availability and strives to remain accessible even during failures or network partitions

ACID vs BASE

- Performance
  - ACID system may incur higher latency and performance overhead, requiring synchronous replication and strict consistency enforcement
  - BASE systems can achieve higher throughput and lower latency due to their asynchronous replication and relaxed consistency
- Use cases
  - ACID is well-suited for applications that require strong data integrity and consistency, e.g., financial systems or transactional applications
  - BASE is often used in large-scale DS, NoSQL data stores, and web applications where high availability and horizontal scalability are more critical than strict consistency

Protocolli di consistenza data-centrica

- Protocollo di consistenza: implementazione di uno specifico modello di consistenza
- Analizziamo protocolli di consistenza data-centrica linearizzabile e sequenziale
  - Protocolli **primary-based**
    - Operazioni di scrittura eseguite su una sola replica (quella primaria), che successivamente assicura che gli aggiornamenti siano opportunamente ordinati ed inoltrati alle altre repliche
  - Protocolli **replicated-write**
    - Operazioni di scrittura eseguite su molteplici repliche

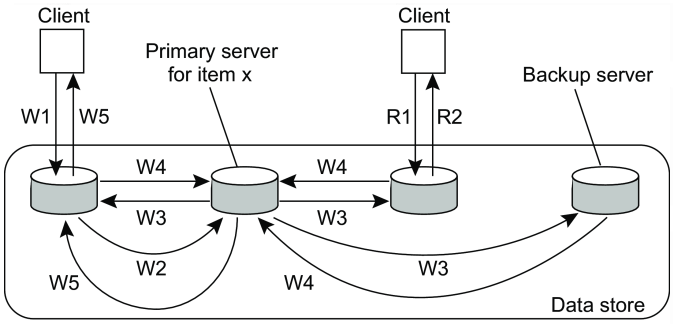
Protocolli primary-based

- Anche detti protocolli primary-backup o di replicazione passiva (o leader-based replication)
  - Ad ogni dato *x* è associata una replica primaria (*leader*) che ha il compito di coordinare le operazioni di *scrittura* di *x* sulle repliche secondarie (*follower*)
  - L'operazione di *lettura* di *x* può essere eseguita su ogni replica (ad es. replica locale al client)

ad esempio raft dove la write sul leader coordina i follower

- Protocolli primary-based di tipo remote-write
  - L'operazione di scrittura di *x* è inviata alla replica primaria (eventualmente remota), che poi la inoltra alle repliche secondarie coordinandone l'aggiornamento
- Protocolli primary-based di tipo local-write
  - La copia primaria di *x migra* verso la replica locale rispetto al client per l'operazione di scrittura; la replica locale inoltra l'operazione di scrittura alle altre repliche
  - Non esaminato

Primary-based: protocolli remote-write



W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

R1. Read request
R2. Response to read

il primary server è quello che riceve la richiesta di write INOLTRATA dal backup server a cui il client ha fatto una richiesta di write

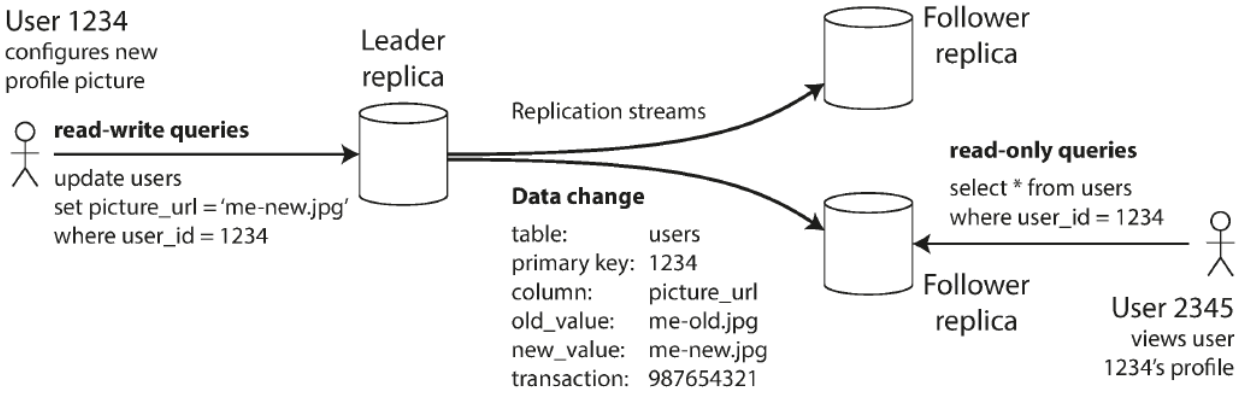
In figura: protocollo remote-write *bloccante*

- Tipicamente usati nei DB distribuiti (e.g., MySQL, PostgreSQL), in alcuni data store NoSQL (e.g., MongoDB), nei MQS (Kafka) e file system distribuiti, ovvero quando si richiede un elevato grado di tolleranza ai guasti
- Svantaggi
  - Lentezza in caso di repliche distribuite geograficamente
  - Scarsa scalabilità all'aumentare del numero di repliche

il processo del client che chiede la write viene implementato come processo bloccante. ma se fosse non bloccante?

Primary-based remote-write: example

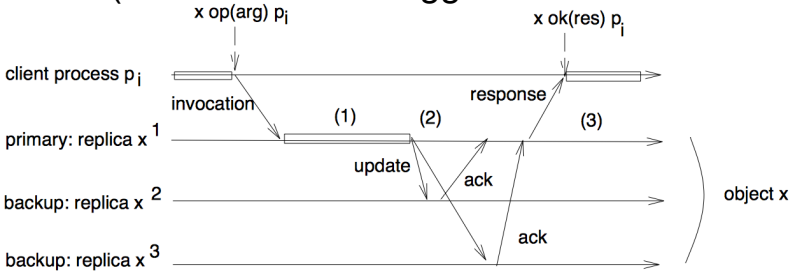
simile a quanto visto in kafka



Primary-based: bloccante

- Aggiornamento delle repliche da parte della replica primaria tramite *log shipping*
- Aggiornamento delle repliche in modo *bloccante* o *non bloccante* per il client appunto come detto

1. Bloccante (o replicazione sincrona):
   - La replica primaria notifica al client che la scrittura è stata completata su tutte le repliche il client perciò vedrà sempre l'effetto dell'ultima write
   - Modello di consistenza: linearizzabilità
   - Vantaggi: maggiore tolleranza a guasti (repliche sincronizzate), incluso crash della replica primaria
   - Svantaggi: lentezza (il client attende l'aggiornamento di tutte le repliche)



Primary-based: non bloccante

2. Non bloccante (o replicazione asincrona):
   - La replica primaria notifica al client che la scrittura è stata completata solo su di essa
   - Modello di consistenza: sequenziale
   - Vantaggi: minore attesa per il client, più adatto per repliche in numero elevato e distribuite geograficamente
   - Svantaggi: minore tolleranza ai guasti e perdita della linearizzabilità

il client non è detto che vedrà sempre l'effetto dell'ultima write

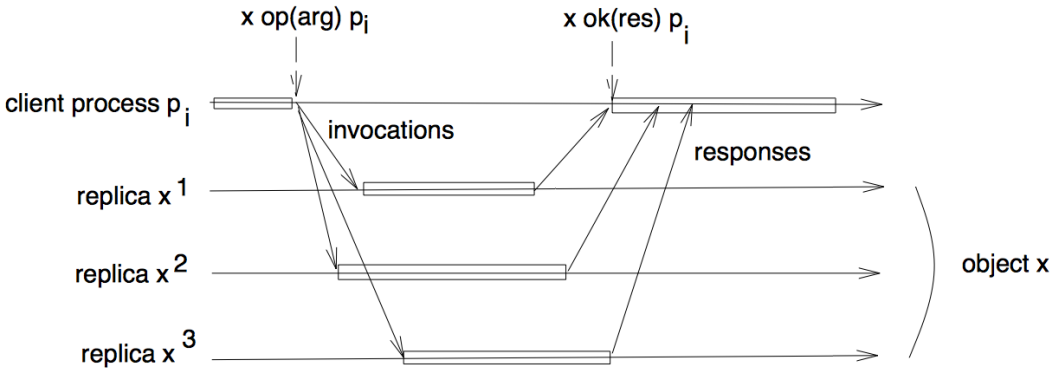
Protocolli replicated-write

- Rispetto ai protocolli primary-based:
  - No controllo centralizzato delle scritture da parte della replica primaria
  - Scritture eseguite su molteplici repliche
- Approcci
  - **Replicazione attiva** (o multi-leader replication)
  - **Protocolli basati su quorum**

Replicated-write: replicazione attiva

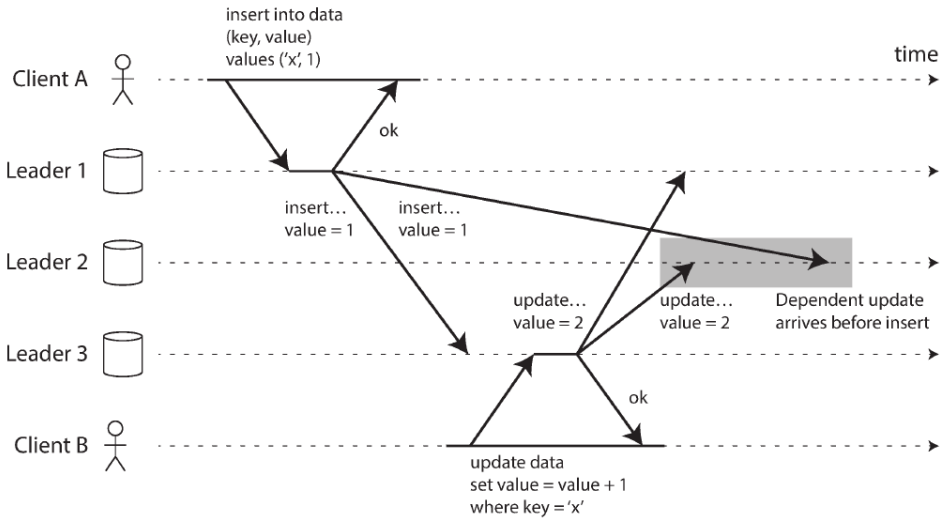
- Replicazione attiva
  - Lettura su replica locale
  - Scrittura su ogni replica
- Soluzione già esaminata (state-machine replication): perché?
  - Per mantenere la consistenza delle repliche, occorre inviare in multicast la scrittura a tutte le repliche

Nella replicazione attiva, ogni replica ha un processo associato che si occupa di propagare le operazioni di update



Replicated-write: replicazione attiva

- Qual è il problema da risolvere?
  - Scritture in ordine diverso sulle repliche
    problema di totally ordered multicast



Replicated-write: replicazione attiva

- Occorre eseguire le operazioni di scrittura nello stesso ordine su tutte le repliche
- Come? Multicasting totalmente ordinato
  - Centralizzato tramite sequencer
    - Scarsa scalabilità e single point of failure
  - Decentralizzato usando clock scalare
    - Scalabilità limitata in sistemi a larga scala a causa di elevato numero di messaggi
  - Decentralizzato usando un protocollo di consenso distribuito
    - E.g., Raft
- Modello di consistenza data-centrica supportato
  - Consistenza sequenziale
  - Consistenza linearizzabile

Replicated-write: protocolli quorum-based

L'idea è di chiede il permesso a più repliche per operazioni (read o write) su un dato replicato

- Votazione attuata da un *sottoinsieme* di repliche
- Consideriamo *N* repliche di un dato x
  - Quindi un totale di *N* voti
- Ad ogni dato è associato un numero di versione
  - Ad ogni operazione di scrittura, il numero di versione viene incrementato
- L'operazione di lettura di x richiede un quorum per la lettura N_R per garantire che venga letta l'ultima versione di x
- L'operazione di scrittura su x richiede un quorum per la scrittura N_W per assegnare il numero di versione

Gifford, Weighted voting for replicated data, *Proc. ACM* SOSP 1979
https://dl.acm.org/doi/pdf/10.1145/800215.806583

Protocolli quorum-based: condizioni

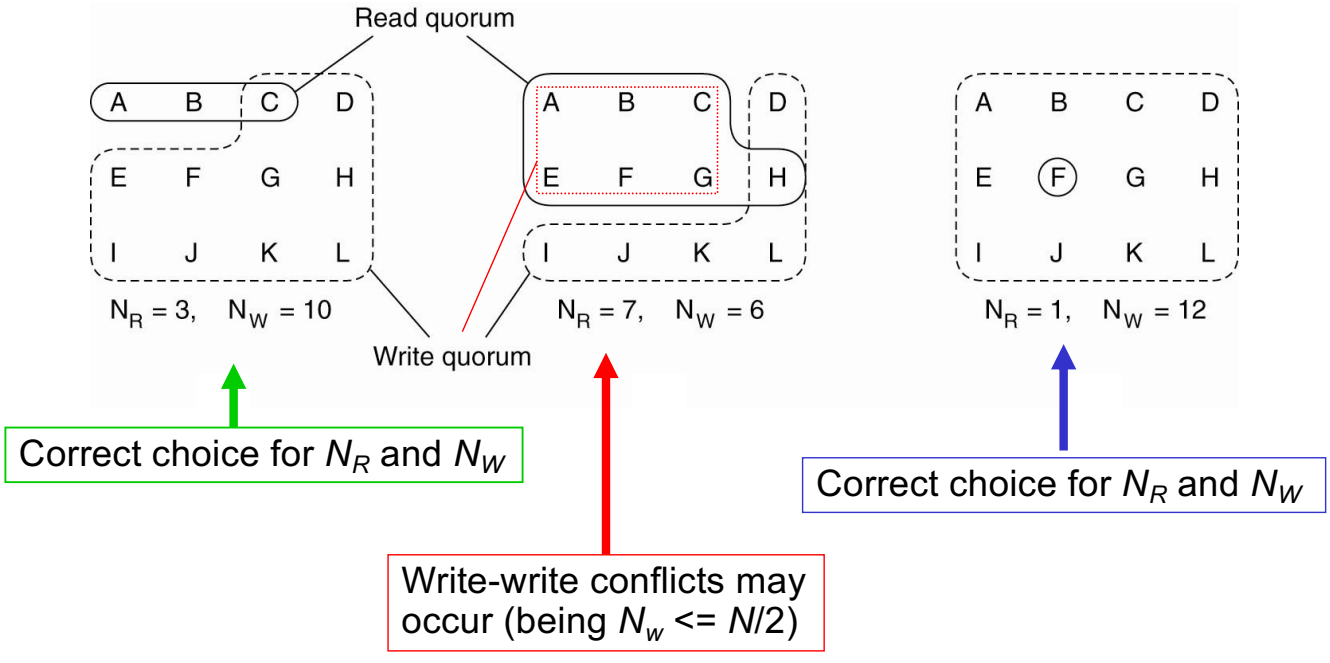
- Per N_R e N_W valgono le condizioni

a) $N_R + N_W > N$
b) $N_W > N/2$

a) per impedire conflitti lettura-scrittura
b) per impedire conflitti scrittura-scrittura (un solo scrittore alla volta può ottenere il quorum per la scrittura)

Se a) e b) sono entrambe soddisfatte, si garantisce la consistenza sequenziale

Setting read and write quorums

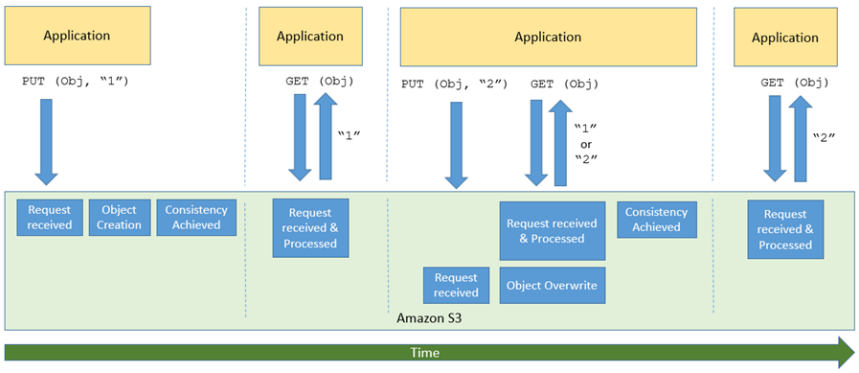


Setting read and write quorums

- Some specific settings for N_R and N_W
  1. $N_R=1$ e $N_W=N$
     - Called *ROWA* (Read Once Write All)
     - Fast reads but slow writes
  2. $N_W=1$ e $N_R=N$
     - Called *RAWO* (Read All Write Once)
     - Fast writes but slow reads
     - Be careful: conflicting writes may occur (being $N_W <= N/2$)
  3. $N_W= N_R=N/2 +1$
     - Called *Majority*
     - Both reads and writes are relatively slow, but high availability
- *Practical use*: quorum-based storage systems allow users to choose between strong and eventual consistency by selecting different read and write quorums (e.g., Cassandra)

Cloud storage and consistency

- Cloud storage services have often adopted weak consistency models
  - Eventual consistency: most popular for a long time
  - New trend towards strong consistency, see AWS S3
- AWS S3 (until 2020)
  - Eventual consistency: after a PUT call, inconsistency window where data has been accepted and durably stored, but not yet visible to all GET or LIST requests



Cloud storage and consistency

- Amazon S3 consistency model (now)
  https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html#ConsistencyModel

  "Amazon S3 provides strong read-after-write consistency for PUT and DELETE requests of objects in your Amazon S3 bucket in all AWS Regions. This behavior applies to both writes to new objects as well as PUT requests that overwrite existing objects and DELETE requests"

  But…

  "Amazon S3 does not support object locking for concurrent writers. If two PUT requests are simultaneously made to the same key, the request with the latest timestamp wins. If this is an issue, you will need to build an object-locking mechanism into your application"

  "Bucket configurations have an eventual consistency model. This means that if you delete a bucket and immediately list all buckets, the deleted bucket might still appear in the list."

Data store systems and consistency

- Some NoSQL data stores offer tunable consistency: user can tradeoff between consistency and latency
- Amazon's DynamoDB: user can choose eventually consistent reads or strongly consistent reads
  https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html
  - Strongly consistent reads experience higher read latency, twofold reduction in read throughput and cost more
- Similarly for Google's Cloud Datastore
  https://cloud.google.com/datastore/docs/articles/balancing-strong-and-eventual-consistency-with-google-cloud-datastore
- Cassandra provides quorum-based consistency, where quorums are configurable
  https://cassandra.apache.org/doc/5.0/cassandra/architecture/dynamo.html#tunable-consistency
  - N+W > R and W >= N/2 +1: strong consistency but higher latency